Python Handbook — Vol 2

decorators, type hints, standard library, testing, concurrency, internals, data stack, coding interviews, idioms, cheat sheet, question bank (Ch 12–22)

11 parts · one place to study: DesignGurus courses + classic books + the web, combined, beginner to intermediate

what it is · problem · how it works · trade-offs · real world · interview · self-check · TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 12 — Decorators, Context Managers and Metaprogramming**
   - 12.1 Decorators
   - 12.2 Everyday built-in decorators
   - 12.3 Context managers
   - 12.4 Descriptors — how properties and methods really work
   - 12.5 `__getattr__` vs `__getattribute__`
   - 12.6 Metaclasses and `__init_subclass__`
   - 12.7 Monkey patching — caveats
   - 12.8 Introspection and dynamic code (brief)
   - 12.9 Chapter self-check
   - Chapter TL;DR
2. **Chapter 13 — Type Hints and Static Typing**
   - 13.1 Why typing?
   - 13.2 Annotation syntax
   - 13.3 Built-in generics and the abstract collection types
   - 13.4 `Optional`, `Union` and `X | None`
   - 13.5 `Any` (and `object`)
   - 13.6 `TypeVar` and generics
   - 13.7 `Callable` and `ParamSpec`
   - 13.8 `Protocol` — structural typing
   - 13.9 `TypedDict`
   - 13.10 `Literal`, `Final`, `NewType`
   - 13.11 dataclasses and typing
   - 13.12 Type aliases and the `type` statement
   - 13.13 Running mypy and pyright; common errors
   - 13.14 Runtime vs static — and pydantic
   - 13.15 Chapter self-check
   - Chapter TL;DR
3. **Chapter 14 — A Tour of the Standard Library**
   - 14.1 `datetime` and `zoneinfo` — naive vs aware
   - 14.2 `time` — timestamps and timers
   - 14.3 `re` — regular expressions, deeper
   - 14.4 `logging` — levels, handlers, formatters, best practice
   - 14.5 `argparse` — command-line interfaces
   - 14.6 `os`, `sys`, `subprocess`, `shutil` (+ `pathlib` recap)
   - 14.7 `random` vs `secrets`
   - 14.8 `math`, `statistics`, `decimal`, `fractions`
   - 14.9 A toolbox of small modules: `copy`, `pprint`, `enum`, `uuid`, `hashlib`, `json`, `csv`
   - 14.10 Pointer: `collections`, `itertools`, `functools`
   - 14.11 `sqlite3` — an embedded SQL database
   - 14.12 `urllib` and `http` (brief)
   - 14.13 When to reach for third-party packages
   - 14.14 Chapter self-check
   - Chapter TL;DR
4. **Chapter 15 — Testing and Debugging**
   - 15.1 Why write tests?
   - 15.2 The module under test
   - 15.3 `unittest` basics
   - 15.4 pytest
   - 15.5 Test doubles and `unittest.mock`
   - 15.6 Coverage
   - 15.7 doctest — examples that are tests
   - 15.8 The TDD loop (red, green, refactor)
   - 15.9 Property-based testing with Hypothesis (brief)
   - 15.10 Reading tracebacks
   - 15.11 Debugging with `pdb` and `breakpoint()`
   - 15.12 Logging vs `print` for debugging
   - 15.13 Common bugs checklist
   - 15.14 Chapter self-check
   - Chapter TL;DR
5. **Chapter 16 — Concurrency and Async**
   - 16.1 Concurrency vs parallelism
   - 16.2 I/O-bound vs CPU-bound
   - 16.3 Threads with `threading`
   - 16.4 Race conditions and locks
   - 16.5 Producer–consumer with `queue.Queue`
   - 16.6 The GIL (Global Interpreter Lock)
   - 16.7 `multiprocessing`
   - 16.8 `concurrent.futures` — executors
   - 16.9 `asyncio` — async/await
   - 16.10 Choosing the right tool
   - 16.11 Common pitfalls
   - 16.12 Chapter self-check
   - Chapter TL;DR
6. **Chapter 17 — Performance, Memory and CPython Internals**
   - 17.1 The CPython object model
   - 17.2 Memory management and garbage collection
   - 17.3 Small-int caching and string interning
   - 17.4 Mutability and copying (recap)
   - 17.5 Big-O of built-in operations
   - 17.6 Profiling — measure before you optimise
   - 17.7 Optimisation techniques
   - 17.8 Chapter self-check
   - Chapter TL;DR
7. **Chapter 18 — The Data Stack: numpy, pandas, matplotlib**
   - 18.1 numpy arrays
   - 18.2 Broadcasting
   - 18.3 Linear algebra and random numbers
   - 18.4 pandas: Series and DataFrame

- 18.5 Selecting data: `loc`, `iloc` and filters
- 18.6 Cleaning: missing data and new columns
- 18.7 `groupby`: split–apply–combine
- 18.8 Combining tables: `merge`, `join`, `concat`
- 18.9 Reshaping: `pivot_table` and `melt`
- 18.10 Dates and times
- 18.11 `apply` vs vectorised, and performance tips
- 18.12 matplotlib: Figure, Axes and the four workhorse charts
- 18.13 Mini end-to-end analysis
- 18.14 Chapter self-check
- Chapter TL;DR

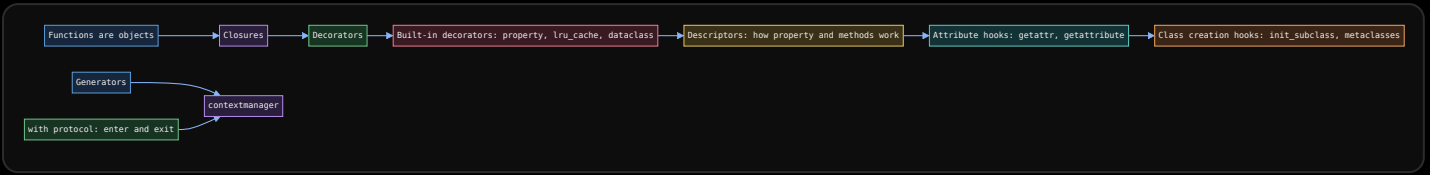
8. **Chapter 19 — Python for Coding Interviews**
   - 19.1 Big-O and the built-ins cheat table
   - 19.2 Arrays and strings: two pointers, sliding window, prefix sums
   - 19.3 Hashing: `dict`, `set`, `Counter`, `defaultdict`
   - 19.4 Stacks and queues: `list` and `deque`
   - 19.5 Heaps with `heapq`
   - 19.6 Sorting with keys; binary search and `bisect`
   - 19.7 Linked lists and trees in Python
   - 19.8 Graphs: adjacency lists, BFS, DFS, Dijkstra, topological sort
   - 19.9 Union-Find (disjoint set union)
   - 19.10 Dynamic programming: `@cache` and tabulation
   - 19.11 Backtracking
   - 19.12 Bit tricks and math
   - 19.13 Tries (prefix trees)
   - 19.14 Greedy
   - 19.15 Intervals
   - 19.16 Fast & slow pointers
   - 19.17 Sort & search: quicksort, quickselect, Dutch flag, merge sort on a list
   - 19.18 Math & geometry
   - 19.19 Common pitfalls
   - 19.20 How to talk through a solution
   - Chapter TL;DR
9. **Chapter 20 — Pythonic Idioms, Gotchas and Interview Q&A**
   - 20.1 Pythonic idioms
   - 20.2 Top gotchas — demo and fix
   - 20.3 Python interview Q&A
   - Chapter TL;DR
10. **Chapter 21 — Cheat Sheets and Glossary**
    - 21.1 Syntax and control flow (§1.9–1.11, Ch 3)
    - 21.2 Types, numbers and operators (Ch 2)
    - 21.3 Data structures and big-O (§5.3, §6.2, §6.12, §17.5, §19.1)
    - 21.4 String methods and formatting (Ch 4)
    - 21.5 List, tuple, dict and set methods (Ch 5, 6)
    - 21.6 Comprehensions, iterators and generators (§5.6, §6.5, Ch 11)
    - 21.7 Functions and arguments (Ch 7)
    - 21.8 OOP and dunder methods (Ch 10, §12.4–12.6)
    - 21.9 Exceptions (Ch 9)
    - 21.10 Files, paths and data formats (§9.8–9.11, §14.6)
    - 21.11 `itertools`, `functools`, `collections` (§6.9, §7.6, §11.5, §12.2)
    - 21.12 Decorators, context managers, metaprogramming (Ch 12)
    - 21.13 Type hints (Ch 13)
    - 21.14 Standard-library quick picks (Ch 14)
    - 21.15 Modules, packages and environments (Ch 8)
    - 21.16 Testing and debugging (Ch 15)
    - 21.17 Concurrency: which tool? (Ch 16)
    - 21.18 Performance and internals (Ch 17)
    - 21.19 numpy, pandas, matplotlib essentials (Ch 18)
    - 21.20 Interview patterns and gotchas (Ch 19, 20)
    - 21.21 Which tool for which job
    - 21.22 Glossary (A–Z)
    - Chapter TL;DR
11. **Chapter 22 — Interview Question Bank**
    - How to use this bank
    - 1. Basics and how Python runs
    - 2. Data types, mutability and strings
    - 3. Collections: lists, tuples, dicts, sets and `collections`
    - 4. Functions, arguments, scope and closures
    - 5. OOP, dunder methods and the MRO
    - 6. Iterators, generators and comprehensions
    - 7. Decorators, context managers and metaprogramming
    - 8. Exceptions and files
    - 9. Modules, packaging, environments and type hints
    - 10. Memory model, internals and performance
    - 11. Concurrency: GIL, free-threading, threads, processes and asyncio
    - 12. Standard library, testing and debugging
    - 13. "What does this print?" puzzles
    - 14. Common coding tasks in Python rounds
    - 15. NumPy and pandas basics
    - Sources of the question lists

Chapter 12 — Decorators, Context Managers and Metaprogramming

What you will learn. Decorators (plain, with arguments, stacked, class decorators, class-based decorators, `functools.wraps`) and the built-in ones you use daily (`@property`, `@classmethod`, `@staticmethod`, `@lru_cache` / `@cache`, `@dataclass`). Context managers: the `with` protocol, class-based managers, `@contextlib.contextmanager`, `suppress`, `ExitStack`, async and reentrant managers. Metaprogramming — code that manipulates code: descriptors, `__getattr__` vs `__getattribute__`, metaclasses, `__init_subclass__`, monkey patching, introspection.

Prerequisites. Chapter 10 (classes, dunders) and Chapter 11 (generators — `@contextmanager` is built on them). Key idea: **functions and classes are objects** you can pass around, wrap and create at run time.

Code. `chapters/12-decorators-context-managers-metaprogramming/code/` — `decorators.py`, `context_managers.py`, `descriptors_attrs.py`, `metaprogramming.py`. Timings vary by machine.



12.1 Decorators

What it is

A **decorator** is a callable (a function, a class, or an object with `__call__`) that takes a function (or class) and returns a replacement — usually a **wrapper** that adds behaviour around the original. `@` is shorthand:

```
@timed
def slow_add(a, b): ...
# is exactly
def slow_add(a, b): ...
slow_add = timed(slow_add)
```

It relies on **first-class functions** (functions can be passed and returned) and **closures** (the inner `wrapper` remembers `func` after the outer function returns). Basics are in Chapter 7 §7.4–7.6.

Why it matters

Decorators separate **cross-cutting concerns** (needs that recur across unrelated functions — logging, timing, caching, retries, authentication, registration) from business logic and apply them in one line. This is Python's native Decorator pattern (LLD Handbook ch. 9) and a light form of aspect-oriented programming.

How it works

1. At **definition time** (usually import time) Python creates the function, calls `decorator(func)`, and binds the name to the result.
2. At **call time**, callers call the wrapper, which calls the original via the closure.
3. The wrapper takes `*args, **kwargs` and **returns** the original's result, so it fits any signature.

```python
def timed(func):
    @functools.wraps(func)                        # copy name, docstring, etc.
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)          # call the original, return its result
        finally:
            elapsed = (time.perf_counter() - start) * 1000
            print(f"  [timed] {func.__name__} took {elapsed:.1f} ms")
    return wrapper

@timed
def slow_add(a, b):
    """Add slowly."""
    time.sleep(0.02)
    return a + b
```

Output:

```
by hand: HELLO ANN
without wraps, name is: wrapper
  [timed] slow_add took 21.3 ms
slow_add: 5 | name: slow_add | doc: Add slowly. | original: slow_add
```

`functools.wraps`

Without it, the decorated function's `__name__` is `"wrapper"` and its docstring is gone — breaking debugging, logs, `help()`, doc tools and frameworks that look functions up by name (e.g. Flask endpoints). `@functools.wraps(func)` copies `__module__`, `__name__`, `__qualname__`, `__doc__`, `__annotations__` (and more), merges `__dict__`, and sets `wrapper.__wrapped__ = func` so the original stays reachable. **Always use it.**

Decorators with arguments — three levels

`@retry(times=3)` first *calls* `retry(times=3)`, which must *return* the real decorator: factory → decorator → wrapper.

```python
def retry(times: int = 3, exceptions=(Exception,)):   # 1. factory: receives the options
    def decorator(func):                               # 2. decorator: receives the function
        @functools.wraps(func)
        def wrapper(*args, **kwargs):                  # 3. wrapper: runs on every call
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except exceptions as e:
                    print(f"  [retry] attempt {attempt} failed: {e}")
                    if attempt == times:
                        raise
        return wrapper
    return decorator

@retry(times=3, exceptions=(ConnectionError,))
def flaky_fetch(): ...        # fails twice, then returns "data"
```

Output:

```
  [retry] attempt 1 failed: timeout
  [retry] attempt 2 failed: timeout
flaky_fetch -> data
```

Real retry code also waits between attempts with **exponential backoff** (1 s, 2 s, 4 s …) plus **jitter** (random extra delay so clients don't retry in lock-step) — libraries such as `tenacity` do this.

Stacking

```
@tag("b")
@tag("i")
def text(): return "hi"
# == text = tag("b")(tag("i")(text))
```

Decorators are **applied bottom-up** and their wrappers **run top-down**. Output: `stacked: <b><i>hi</i></b>`. Order matters: put `@app.route` outermost and the auth check inside it, or the route registers the unprotected function.

Stateful and registering decorators

A decorator may simply **register** the function and return it unchanged — how many frameworks work:

```
REGISTRY = {}
def register(func):
    REGISTRY[func.__name__] = func       # side effect at definition time
    return func

def count_calls(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        wrapper.calls += 1               # state stored as a function attribute
        return func(*args, **kwargs)
    wrapper.calls = 0
    return wrapper
```

Output: `registry: ['export_csv', 'export_json']` and `ping.calls: 2`.

Class decorators and classes as decorators

- A **class decorator** receives a *class* and returns a class (often the same one, modified). `@dataclass` is one.
- A **class used as a decorator** works because instances implement `__call__`; call `functools.update_wrapper(self, func)` in `__init__`.

```
def add_repr(cls):
    def __repr__(self):
        fields = ", ".join(f"{k}={v!r}" for k, v in vars(self).items())
        return f"{cls.__name__}({fields})"
    cls.__repr__ = __repr__
    return cls

class CountCalls:
    def __init__(self, func):
        functools.update_wrapper(self, func)
        self.func, self.calls = func, 0
    def __call__(self, *args, **kwargs):
        self.calls += 1
        return self.func(*args, **kwargs)
```

Output: `class decorator: Point(x=1, y=2)` and `CountCalls: 2 | name: square`.

Caveat: a class-based decorator on a *method* doesn't bind `self` (the instance isn't a descriptor unless you implement `__get__`, §12.4). Prefer function-based decorators.

Trade-offs

Approach	Pros	Cons
Function decorator	Simple, works on methods	State via closure/attributes
Decorator factory (with args)	Configurable	Three levels of nesting; easy to forget the call parentheses
Class-based decorator	Natural place for state	Needs `__get__` to work on methods
Class decorator	Modify/register whole classes	Can surprise readers; metaclass-like power

Common mistakes

- Forgetting `return` in the wrapper → the decorated function returns `None`.
- Forgetting `@functools.wraps`.
- Factory without the call: `@retry` instead of `@retry()` → `retry` receives the function as `times`.
- Heavy work at decoration time (it runs on import).
- Overuse: logic used once, surprising signature/return changes, five stacked decorators nobody can follow.
- Wrong stacking order (e.g. caching outside an auth check caches responses across users).

Real-world examples

Flask `@app.route("/")` / FastAPI `@app.get("/")` register view functions; pytest `@pytest.fixture`, `@pytest.mark.parametrize`; Click `@click.command()` / `@click.option()`; Celery (background-job queue) `@app.task`; Django `@login_required`; `@functools.singledispatch` (overloading by argument type); `@contextlib.contextmanager`; `@unittest.mock.patch`.

In the interview

> **30-second answer:** "A decorator is a callable that takes a function and returns a replacement, usually a closure wrapper that runs code around the original; `@d` above `def f` is `f = d(f)` at definition time. I always use `functools.wraps` to keep the name, docstring and `__wrapped__`. A decorator with arguments is a factory returning a decorator — three levels. Stacked decorators apply bottom-up and run top-down."

- *When does a decorator run?* — Once, at definition (typically import); the wrapper runs on each call.
- *Write a decorator that times a function.* — `timed` above (`perf_counter`, `try/finally`, `wraps`).
- *How do you decorate a class?* — A function that takes and returns a class, e.g. `@dataclass`.

12.2 Everyday built-in decorators

Decorator	What it does	Where covered
`@property` (+ `.setter`, `.deleter`)	Method accessed like an attribute; a descriptor	Ch. 10 §10.1, §12.4
`@classmethod`	Passes the class as `cls` (alternative constructors)	Ch. 10 §10.1
`@staticmethod`	No implicit first argument	Ch. 10 §10.1
`@abstractmethod`	Marks methods an ABC subclass must implement	Ch. 10 §10.7
`@dataclass`	Class decorator generating `__init__` / `__repr__` / `__eq__` …	Ch. 10 §10.10
`@functools.total_ordering`	Class decorator filling in comparison methods	Ch. 10 §10.2
`@functools.lru_cache(maxsize=128)` / `@functools.cache`	Memoisation (below)	here
`@functools.cached_property`	Compute once per instance, store in the instance	§12.4
`@functools.wraps`	Copies metadata onto wrappers	§12.1
`@contextlib.contextmanager`	Generator → context manager	§12.3
`@typing.override` (3.12)	Marks a method as overriding a parent's so type checkers flag typos	—

`@lru_cache` and `@cache` — memoisation

Memoisation stores a call's result keyed by its arguments so a repeat call returns instantly. `lru_cache(maxsize=N)` keeps the N most recent results and evicts the **least recently used** (LRU) when full; `maxsize=None` or `@functools.cache` (3.9) is unbounded. Default `maxsize` is 128.

```
@functools.lru_cache(maxsize=None)
def fib(n: int) -> int:
    return n if n < 2 else fib(n - 1) + fib(n - 2)
```

Output:

```
fib(25) plain 6 ms vs cached 0.01 ms -> cached faster: True
cache_info: CacheInfo(hits=23, misses=26, maxsize=None, currsize=26)
unhashable arg: unhashable type: 'list'
```

Worked numbers: plain recursive `fib(25)` makes 242,785 calls; cached, each `fib(k)` for k = 0…25 is computed once → **26 misses**, and each of `fib(3)…fib(25)` finds its second sub-call `fib(k−2)` cached → **23 hits**.

Rules and caveats:

1. Arguments must be **hashable** (the cache is a dict) — no lists/dicts.
2. Only for **pure** functions (same input → same output, no side effects). Don't cache `now()` or reads that must be fresh.
3. Returned objects are shared — mutating a cached list corrupts the cache.
4. On **methods**, the cache holds a reference to `self`, keeping instances alive (a memory leak) and sharing one cache across instances; prefer `cached_property` or a per-instance cache.
5. `fib.cache_clear()` empties it; `typed=True` caches `f(3)` and `f(3.0)` separately.

Real world: parsed config, compiled regexes (`re` also has its own cache), expensive pure computations, dynamic-programming solutions in interviews.

12.3 Context managers

What it is

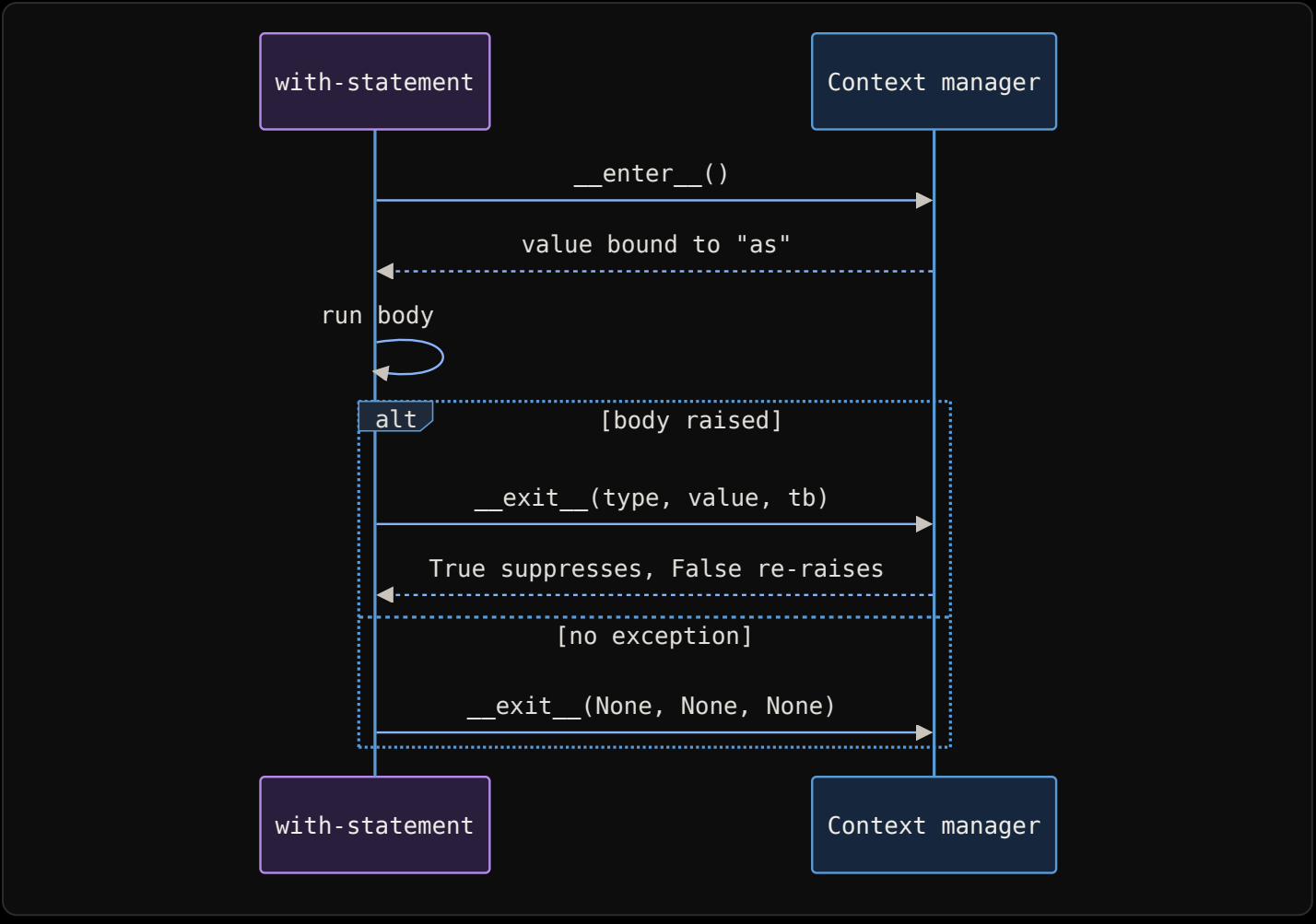
A **context manager** sets something up at the start of a `with` block and **guarantees** tear-down at the end — even if the block raises or `return`s. It implements `__enter__` and `__exit__`.

Why it matters

Leaked resources (files, sockets — network connections, locks, DB transactions, temporary state) are a top source of bugs. `with` replaces fragile `try/finally` boilerplate and makes clean-up impossible to forget.

How it works — `with EXPR as VAR: BODY`

1. Evaluate `EXPR` to get the manager `m`.
2. Call `m.__enter__()`; bind its **return value** to `VAR` (not necessarily `m` — `open()` returns the file, a lock returns `True`).
3. Run `BODY`.
4. Always call `m.__exit__(exc_type, exc_value, traceback)` — all three `None` if no exception.
5. If `__exit__` returns a **truthy** value the exception is **suppressed**; otherwise it propagates. Return `False` / `None` unless suppressing is the point.



Class-based

```
class Timer:
    def __enter__(self):
        self.start = time.perf_counter()
        return self
    def __exit__(self, exc_type, exc, tb):
        self.elapsed = time.perf_counter() - self.start
        return False                    # propagate exceptions
```

`context_managers.py` also has `Suppress` — a home-made `contextlib.suppress` whose `__exit__` returns `exc_type is not None and issubclass(exc_type, self.excs)`.

Generator-based: `@contextlib.contextmanager`

Write a generator that **yields exactly once**: code before `yield` is `__enter__`, the yielded value goes to `as`, code after is `__exit__`. If the body raises, the exception is **re-raised at the `yield`** — so wrap it in `try/finally` (or `except`) or clean-up is skipped.

```
@contextlib.contextmanager
def cd(path):                           # same idea as contextlib.chdir (3.11+)
    old = os.getcwd()
    os.chdir(path)
    try:
        yield path                      # the with-body runs here
    finally:
        os.chdir(old)                   # runs even if the body raises

@contextlib.contextmanager
def transaction(log):
    log.append("BEGIN")
    try:
        yield
    except Exception:
        log.append("ROLLBACK")
        raise                           # re-raise: don't hide the error
    else:
        log.append("COMMIT")
```

`ExitStack` — a dynamic number of managers

`contextlib.ExitStack` enters an unknown number of managers (e.g. N files from a list) and registers arbitrary clean-up callbacks. On exit it unwinds them in **reverse (LIFO)** order; if entering the k-th fails, the k−1 already entered are still closed. `stack.pop_all()` transfers ownership out once setup succeeded (useful in constructors).

```
with contextlib.ExitStack() as stack:
    files = [stack.enter_context(Resource(n, log)) for n in names]
    stack.callback(log.append, "callback ran")
    log.append(f"using {len(files)} resources")
```

Output (`context_managers.py`):

```
Timer: elapsed > 0: True
Suppress: survived ZeroDivisionError
contextlib.suppress: survived FileNotFoundError
cd restored: True | was inside tmp: True
transaction log: ['BEGIN', 'insert row', 'COMMIT', 'BEGIN', 'bad insert', 'ROLLBACK']
ExitStack: ['open a.txt', 'open b.txt', 'open c.txt', 'using 3 resources', 'callback ran', 'close c.txt', 'close
b.txt', 'close a.txt']
ExitStack partial failure: ['open ok1', 'close ok1'] | cannot open
redirect_stdout captured: 'captured!\n'
nullcontext gives: x
two managers: ['open r1', 'open r2', 'close r2', 'close r1']
```

The `contextlib` toolbox

Tool	Use
`contextmanager` / `asynccontextmanager`	Generator (or async generator) → context manager
`suppress(*excs)`	Ignore specific exceptions (`with suppress(FileNotFoundError): os.remove(p)`)
`ExitStack` / `AsyncExitStack`	Dynamic number of managers + callbacks
`closing(obj)`	Call `obj.close()` on exit for objects without `__exit__`
`redirect_stdout` / `redirect_stderr`	Temporarily capture output
`nullcontext(x)`	Do-nothing manager for "optional" context (`with lock if threaded else nullcontext():`)
`chdir(path)` (3.11)	Temporarily change the working directory
`AbstractContextManager`	ABC base giving a default `__enter__` that returns `self`

Several managers: `with A() as a, B() as b:` (parenthesised across lines since 3.10); exited in reverse order ("two managers" above).

Async context managers. In `async def` code (Chapter 16) use `async with`; the protocol is the coroutines `__aenter__` / `__aexit__`, so setup and clean-up can `await` I/O (opening a connection, returning a pooled DB connection). `@contextlib.asynccontextmanager` builds one from an `async def` generator. Examples: `async with aiohttp.ClientSession() as s:`, `async with asyncio.TaskGroup() as tg:`, `async with lock:` for an `asyncio.Lock`.

Reusable and reentrant managers. A *single-use* manager (a `@contextmanager` object, an open file) can be entered once. A *reusable* one can be entered again after it exits. A *reentrant* one can be entered again **while already active** (`with m:` inside `with m:`) — `threading.RLock` and `contextlib.redirect_stdout` are; a plain `threading.Lock` is not (re-acquiring it from the same thread deadlocks).

Trade-offs

	Class-based	`@contextmanager`
Code size	More	Less
Reusable object with state/methods	Yes	No (one-shot generator)
Can be re-entered / reused	If you design it so	No — each call makes a new one
Exception handling	Explicit `__exit__` args	`try/except/finally` around `yield`
Also usable as a decorator	Via `ContextDecorator`	Yes

Common mistakes

- `@contextmanager` without `try/finally` around `yield` → clean-up skipped on error.
- Yielding more than once → `RuntimeError: generator didn't stop`.
- Accidentally returning `True` from `__exit__` → silently swallows all exceptions.
- Using the resource after the block (`f.read()` after `with open(...)` → `ValueError: I/O operation on closed file`).

Real-world examples

`open()`, `threading.Lock`, `sqlite3.connect()` (commits or rolls back — it does *not* close the connection), `requests.Session`, `tempfile.TemporaryDirectory`, `decimal.localcontext`, `unittest.mock.patch`, `pytest.raises`, PyTorch `torch.no_grad()`, `ThreadPoolExecutor` (waits for tasks on exit).

In the interview

> **30-second answer:** "A context manager implements `__enter__` and `__exit__`; `with` calls enter, binds its return value to `as`, runs the body, and always calls exit with the exception info, which can suppress it by returning true. For simple cases I write a generator with `@contextmanager` — setup, `yield`, clean-up in `finally`. For a variable number of resources I use `ExitStack`, which unwinds in reverse order even if one fails to open."

- *How do you suppress an exception?* — Return a truthy value from `__exit__`, or use `contextlib.suppress`.
- *What happens to an exception inside a `@contextmanager` body?* — It's thrown into the generator at the `yield`.

12.4 Descriptors — how properties and methods really work

What it is

A **descriptor** is an object stored as a **class attribute** that defines at least one of `__get__(self, obj, objtype=None)` (read), `__set__(self, obj, value)` (assign), `__delete__(self, obj)` (`del`). Optionally `__set_name__(self, owner, name)` (3.6) runs at class creation so it learns its attribute name.

- **Data descriptor** — defines `__set__` or `__delete__`. `property` is one.
- **Non-data descriptor** — only `__get__`. Plain functions and `functools.cached_property` are.

Why it matters

Descriptors power `property`, `classmethod`, `staticmethod`, bound methods, `__slots__` and ORM fields (Django/SQLAlchemy columns). Your own give **reusable** attribute logic — one `Positive` validator for many fields instead of copy-pasted property pairs.

How it works — attribute lookup precedence for `obj.attr`

1. Find `attr` on `type(obj)` along the MRO. If it's a **data descriptor**, call its `__get__` — it **wins over the instance dict**.
2. Else, if `attr` is in `obj.__dict__`, return that.
3. Else, if the class attribute is a **non-data descriptor**, call its `__get__`; if a plain class attribute, return it.
4. Else call `__getattr__` if defined; otherwise raise `AttributeError`.

(This is `object.__getattribute__`.) `obj.attr = v` calls a data descriptor's `__set__` if there is one; otherwise it writes `obj.__dict__`.

Consequences:

- **Methods:** a function is a non-data descriptor; `Greeter.__dict__["hello"].__get__(g)` returns the bound method — what `g.hello` does.
- `property` always defines `__set__` (raising `AttributeError` without a setter), so an instance-dict entry can't shadow it.
- `cached_property` is non-data: the first access stores the value in `obj.__dict__`; later accesses hit step 2 and never call the descriptor.

```python
class Positive:
    def __set_name__(self, owner, name):
        self.private = "_" + name                # e.g. "_qty"
    def __get__(self, obj, objtype=None):
        if obj is None:                          # accessed on the class
            return self
        return getattr(obj, self.private)
    def __set__(self, obj, value):
        if value <= 0:
            raise ValueError(f"{self.private[1:]} must be > 0, got {value}")
        setattr(obj, self.private, value)

class Order:
    qty = Positive()                             # one reusable validator...
    price = Positive()                           # ...for many fields
    def __init__(self, qty, price):
        self.qty = qty                           # runs Positive.__set__
        self.price = price
```

`descriptors_attrs.py` also re-implements `property` as `my_property`: `__get__` returns `self.fget(obj)` (or `self` on the class), `__set__` raises `AttributeError("read-only")` when there is no setter. Output (`descriptors_attrs.py`, first part):

```
Order: 2 9.99 | instance dict: {'_qty': 2, '_price': 9.99}
descriptor refused: qty must be > 0, got 0
on the class: Positive object (the descriptor itself)
my_property: 212.0
AttributeError: read-only
  (computing data once)
first: [1, 2, 3]
second: [1, 2, 3] | stored in instance dict: True
function has __get__: True | f.__get__(g)(): hello
data descriptor? True | cached_property has __set__? False
```

Common mistakes

- Storing the value on the descriptor (`self.value = v`) — one descriptor is shared by **all instances**. Store per-instance data on `obj`.
- Creating a descriptor in `__init__` (`self.qty = Positive()`) — descriptors only work as **class** attributes.
- Forgetting the `obj is None` case (class access).

In the interview

30-second answer: "A descriptor is a class attribute with `__get__`, and optionally `__set__` / `__delete__`. Data descriptors take priority over the instance dict; non-data ones don't. `property`, `classmethod`, `staticmethod` and bound methods are built on this — a function's `__get__` returns a bound method. I'd write one for reusable validation across fields, like ORM columns."

- *Why can `cached_property` be overwritten by the instance?* — It's non-data, so the instance dict wins.

12.5 `__getattr__` vs `__getattribute__`

What it is

Hook	Called when	Typical use
`__getattr__(self, name)`	Only when **normal lookup fails**	Delegation/proxies (objects that forward requests to a wrapped target), lazy attributes, defaults, deprecation shims (keep old names working with a warning)
`__getattribute__(self, name)`	On **every** attribute access (it *is* normal lookup)	Rare: tracing, access control, special proxies
`__setattr__` / `__delattr__`	Every assignment / deletion	Validation, immutability, change tracking
Module-level `__getattr__(name)` (PEP 562, 3.7)	Missing attribute on a *module*	Lazy imports, deprecation warnings

```
class Proxy:
    def __init__(self, target):
        self._target = target
    def __getattr__(self, name):                  # only for names not found normally
        print(f"  __getattr__({name!r}) -> delegating")
        return getattr(self._target, name)

class Logged:
    def __init__(self):
        self.x = 1
    def __getattribute__(self, name):             # every access
        if not name.startswith("__"):
            print(f"  __getattribute__({name!r})")
        return super().__getattribute__(name)     # MUST delegate via super()/object
```

Output:

```
  __getattr__('upper') -> delegating
proxy.upper(): HELLO WORLD
proxy._target found normally (no __getattr__ call): hello world
  __getattribute__('x')
value: 1
```

Common mistakes

- `self.anything` inside `__getattribute__` → infinite recursion. Use `super().__getattribute__(name)` or `object.__getattribute__(self, name)`.
- In `__getattr__`, touching an attribute that may not exist yet (`self._target` during unpickling or copying, before `__init__` ran) → infinite recursion. Guard or raise `AttributeError`.
- Returning `None` for unknown names instead of raising `AttributeError` → breaks `hasattr`, `getattr(obj, n, default)`, copy/pickle.
- Implicit special-method lookup (`len(p)`, `p + 1`) **bypasses** both hooks — dunders are looked up on the type. A proxy must define them explicitly.

Real-world examples

`unittest.mock.Mock` creates child mocks in `__getattr__`; RPC/SDK clients (remote-procedure-call / service client libraries) build dynamic endpoints (`client.users.list`); pandas column access `df.price`; lazy-loading packages use module `__getattr__`.

12.6 Metaclasses and `__init_subclass__`

What it is

A **metaclass** is "the class of a class". The default is **`type`**: `type(42)` is `int`, `type(int)` is `type`, `type(type)` is `type`. A `class` statement is roughly `Name = type("Name", bases, namespace)`.

```
def speak(self): return f"{self.name} speaks"
Dog = type("Dog", (object,), {"name": "Rex", "speak": speak})   # a class, built by a call
```

How a class is created

1. The class body executes into a namespace dict.
2. The metaclass is determined (explicit `metaclass=`, or the most derived metaclass of the bases, default `type`).
3. `Meta(name, bases, namespace)` → `Meta.__new__` builds the class, `Meta.__init__` initialises it.
4. Inside `type.__new__` (so before `Meta.__init__`), descriptors' `__set_name__` hooks run, then the parent's `__init_subclass__`.
5. Class decorators run last, on the finished class.

A custom metaclass subclasses `type` and overrides `__new__`/`__init__` (or `__call__`, to control **instance** creation — e.g. a Singleton metaclass, allowing only one instance).

`__init_subclass__` — the simpler hook

PEP 487 (3.6) added `__init_subclass__(cls, **kwargs)`: an implicit classmethod on a parent that runs **every time a subclass is created**, receiving keyword arguments from the class line (`class Home(Handler, route="/")`). It covers the common metaclass uses — registries, validation, configuration — with no metaclass conflicts.

```
class RegistryMeta(type):                          # metaclass version
    registry: dict[str, type] = {}
    def __new__(mcls, name, bases, namespace):
        cls = super().__new__(mcls, name, bases, namespace)
        if bases:
            mcls.registry[name.lower()] = cls
        return cls

class Plugin(metaclass=RegistryMeta): pass
class CsvExporter(Plugin): ...

class Handler:                                     # __init_subclass__ version (preferred)
    handlers: dict[str, type] = {}
    def __init_subclass__(cls, /, route: str, **kwargs):
        super().__init_subclass__(**kwargs)
        Handler.handlers[route] = cls

class Home(Handler, route="/"): ...

class RequiresRun:                                 # enforce a rule at class-creation time
    def __init_subclass__(cls, **kwargs):
        super().__init_subclass__(**kwargs)
        if not callable(cls.__dict__.get("run")):
            raise TypeError(f"{cls.__name__} must define run()")
```

Output (`metaprogramming.py`):

```
Rex speaks | type(Dog): type | type(type): type
type(42): int | type(int): type
metaclass registry: {'csvexporter': <class '__main__.CsvExporter'>, 'jsonexporter': <class
'__main__.JsonExporter'>}
type(CsvExporter): RegistryMeta
__init_subclass__ routes: {'/': 'Home', '/about': 'About'}
GoodJob: ran
TypeError: BadJob must define run()
```

`type(CsvExporter)` is `RegistryMeta` — the metaclass is inherited by every subclass.

When to use what (least power first)

Need	Use
Add/modify methods of one class	Class decorator
Reusable attribute behaviour	Descriptor
React to / register / validate every subclass	`__init_subclass__`
Control the class namespace, customise `isinstance`, change instance creation, framework-level DSLs (domain-specific mini-languages)	Metaclass

Tim Peters: metaclasses are deeper magic than 99% of users should worry about — and since 3.6 `__init_subclass__` and `__set_name__` removed most remaining reasons.

Common mistakes

- **Metaclass conflict:** two bases with unrelated metaclasses → `TypeError: metaclass conflict` (a reason libraries avoid custom metaclasses).
- Forgetting `super().__init_subclass__(**kwargs)` — breaks cooperative inheritance.
- A metaclass for something a decorator could do.

Real-world examples

`abc.ABCMeta` (abstract methods), `enum.EnumType` (formerly `EnumMeta`), Django's `ModelBase` (fields → model with manager and metadata), Pydantic's `ModelMetaclass` (collects fields, builds validators). Plugin systems use `__init_subclass__` registries.

In the interview

> **30-second answer:** "A metaclass is the class of a class — `type` by default — and runs when a class statement executes, so it can inspect or change the new class. Django's ORM and `abc` use them. In application code I almost never need one: class decorators, descriptors and `__init_subclass__` cover registration and validation with less magic and no metaclass conflicts."

- *What's `type(type)`?* — `type`.
- *What is `__init_subclass__` for?* — A parent hook run for each new subclass (registries, validation, per-subclass config).

12.7 Monkey patching — caveats

What it is

Monkey patching replaces attributes (functions, methods, classes) of modules or classes **at run time** from outside, e.g. `Greeter.hello = lambda self: "patched!"`. It always works because classes and modules are mutable — which is the danger.

Output (`descriptors_attrs.py`, last part):

```
monkey patched: patched! | existing instance too: patched!
restored: hello
```

Patching a class affects **every existing instance**, since methods are looked up on the class.

Legitimate uses

- **Tests** — replace slow/external dependencies (network, time, randomness) with `unittest.mock.patch` or pytest's `monkeypatch`, which **undo the change automatically**.
- **Hot-fixing a third-party bug** until the upstream fix ships (document it, pin the version).
- Framework instrumentation: gevent's `monkey.patch_all()` makes the standard library cooperative; APM (application performance monitoring)/tracing agents (OpenTelemetry auto-instrumentation) that time each step of a request wrap library functions.

Caveats

1. **Global and invisible** — every user of the module/class sees it, including other libraries and threads.
2. **Patch where it's looked up, not where it's defined** — if `app.py` did `from utils import send_email`, patch `app.send_email`; patching `utils.send_email` doesn't change the name already bound in `app`.
3. **Order-dependent** — code that imported or cached the original first keeps it.
4. **Fragile across upgrades** — patched private internals change without notice.
5. **Always restore** (try/finally or auto-restoring tools), or test pollution (state left behind by one test) breaks unrelated tests.
6. C-implemented built-ins (`str`, `int`, `list`) can't be patched: `str.upper = ...` raises `TypeError`.

In the interview

"Monkey patching is modifying classes or modules at run time. It's fine in tests via `mock.patch` or pytest's `monkeypatch`, which restore automatically and must target the name where it's used. In production I avoid it — global, order-dependent, breaks on upgrades — and prefer dependency injection (passing collaborators such as a DB client or clock in as arguments), subclassing or a wrapper."

12.8 Introspection and dynamic code (brief)

Introspection — a program examining its own objects at run time — is the everyday side of metaprogramming:

Tool	Does	Example
`getattr(obj, name, default)`, `setattr`, `hasattr`, `delattr`	Read/write attributes named by strings	`getattr(handler, f"do_{verb}")` to dispatch by name
`vars(obj)`, `dir(obj)`	Instance `__dict__`; all attribute names	Debugging, generic `__repr__`
`type(x)`, `isinstance`, `issubclass`, `callable`	Types and capabilities	
`inspect.signature(f)`	Parameter names, defaults, annotations	FastAPI and pytest read signatures to inject arguments/fixtures
`inspect.getsource(f)`, `inspect.getmembers(obj)`	Source text; all members	Documentation tools
`eval(expr)`, `exec(code)`	Run a string as Python	Almost never needed; **never on untrusted input** — it runs any code
`ast.literal_eval(s)`	Safely parse a literal (`"[1, 2, {'a': 3}]"`)	The safe replacement for `eval` on data

`eval` / `exec` on user-influenced input is a security hole, like unpickling untrusted data (Chapter 9).

12.9 Chapter self-check

1. What does `@d` above `def f` do, and when? → `f = d(f)`, once at definition time. And `@a @b def f`? → `f = a(b(f))`.
2. Why `functools.wraps`? → Keeps `__name__`, `__doc__`, etc. and sets `__wrapped__` (reach the original).
3. `@retry` vs `@retry()` for a factory decorator? → The factory must be called: `@retry()`. Three levels: options → function → per-call wrapper.
4. Order of stacked decorators? → Applied bottom-up, run top-down.
5. What if the wrapper doesn't `return func(...)`? → Callers get `None`.
6. What's required of `lru_cache` arguments? → Hashable.
7. What gets bound by `as`, and what are `__exit__`'s arguments? → `__enter__`'s return value; exception type, value, traceback (all `None` if no error).
8. How does `__exit__` suppress an exception? → By returning a truthy value.
9. What must surround `yield` in a `@contextmanager`? → `try/finally`, so clean-up runs on errors.
10. Exit order in `with A(), B():`, and why `ExitStack`? → B then A; for an unknown or conditional number of managers with guaranteed LIFO clean-up.
11. Descriptor precedence? → Data descriptor > instance dict > non-data descriptor.
12. When is `__getattr__` called? → Only when normal lookup fails.
13. Simplest way to register every subclass? → `__init_subclass__`.

Chapter TL;DR

- **Decorators:** `f = d(f)` at definition time; `*args, **kwargs`, return the result, `@functools.wraps`. Factories for arguments; stacks apply bottom-up, run top-down; decorators may just register.
- **Built-ins:** `property`, `classmethod`, `staticmethod`, `dataclass`, `total_ordering`, `lru_cache` / `cache` (hashable args, pure functions, beware methods), `cached_property`.
- **Context managers:** `__enter__` / `__exit__` guarantee clean-up; truthy `__exit__` suppresses. `@contextmanager` = setup, `yield` in `try`, `finally`. `ExitStack` for dynamic resources (LIFO).
- **Descriptors** power properties and methods; data descriptor > instance dict > non-data.
- **Attribute hooks:** `__getattr__` for missing names, `__getattribute__` for everything (rare).
- **Class creation:** prefer decorators → descriptors → `__init_subclass__` → metaclasses.
- **Monkey patching:** tests only (auto-restore, patch where used).

Chapter 13 — Type Hints and Static Typing

What you will learn. Why a dynamic language added optional types and how to write them: annotation syntax, built-in generics (`list[int]`), `Optional` / `Union` / `X | None` , `Any` , generics with `TypeVar` and the 3.12 bracket syntax, `Callable` and `ParamSpec` , `Protocol` , `TypedDict` , `Literal` , `Final` , `NewType` ; dataclasses, aliases and the `type` statement; running **mypy**/**pyright** and reading their errors; what hints do (and don't do) at runtime; and how **pydantic** turns hints into runtime validation.

Prerequisites. Functions (Chapter 7), classes and dataclasses (Chapter 10), decorators (Chapter 12).

Code. `chapters/13-type-hints/code/` , Python 3.12+. `04_run_mypy.py` needs `pip install mypy` :

```
python chapters/13-type-hints/code/02_generics_protocols.py
mypy --strict chapters/13-type-hints/code/03_special_forms.py
```



13.1 Why typing?

What it is

A **type hint** (annotation) says what kind of value a variable, parameter or return should hold: `def area(w: float, h: float) -> float` . Python **does not enforce** it. **Static type checkers** (mypy, pyright) read the source *without running it* and report contradictions.

- **Dynamic typing** — types belong to *values* and are checked as the program runs (`"a" + 1` fails only when that line executes). Python is dynamic.
- **Static typing** — types are checked *before* running, from the source; Java, Go and Rust do it in the compiler.

Python offers **gradual typing**: annotate as much or as little as you like; un-annotated code is treated leniently.

Why it matters

1. **Bugs found early** — `None` where a `str` is expected, misspelled attributes, wrong argument order — before tests or production.
2. **Documentation that can't go stale** — `def fetch(ids: list[int]) -> dict[int, User]` explains itself, and the checker complains if code drifts.
3. **Editor power** — autocomplete, go-to-definition, safe renames (VS Code's Pylance is built on pyright).
4. **Safer refactoring** at scale — change a signature and the checker lists every caller to fix.

How it works

1. Annotations arrived in 3.0 (PEP 3107) as free-form metadata; PEP 484 (3.5) gave them standard meaning and created `typing` .
2. At runtime they're stored (`func.__annotations__`) and otherwise ignored.
3. A checker parses the code, infers the type of every expression, and compares it with the annotations.
4. Un-annotated code is `Any` (§13.5), so you can adopt typing one file at a time.

Trade-offs

Benefit	Cost
Catches whole classes of bugs before running	Extra syntax to learn and write
Self-documenting APIs, better IDE (integrated development environment — VS Code, PyCharm) support	Some dynamic patterns are awkward to type
Refactoring confidence at scale	False sense of safety if `Any` leaks everywhere
Zero runtime cost (hints are not checked)	…which also means no runtime protection

When not to use it

Throw-away scripts, exploratory notebooks and tiny programs gain little. Types pay off in long-lived, shared code and libraries.

Real-world examples

Dropbox type-checked about 4 million lines with mypy (Guido van Rossum worked on mypy there). Meta built Pyre; Microsoft built pyright. **FastAPI** and **pydantic** read hints at runtime to validate requests and generate API docs. Standard-library and popular-package types live in **typeshed**, used by every checker.

In the interview

"Python is dynamically typed, but since PEP 484 we can add optional type hints. They're ignored at runtime; a static checker like mypy or pyright verifies them before the code runs. It's gradual, so you can type a codebase incrementally. I use them on anything long-lived because they catch `None` bugs and make refactors safe."

- *Do hints make code faster?* No — CPython ignores them. (mypyc can compile typed code to C extensions, but that's a separate compiler.)
- *Does* `x: int = "a"` *raise?* No; only the checker complains.
- *Checkers?* mypy, pyright; also Pyre and newer Rust-based ones (Astral's ty, Meta's Pyrefly).

13.2 Annotation syntax

```python
def greet(name: str, times: int = 1) -> str:   # parameters and return
    return ", ".join([f"hi {name}"] * times)

port: int = 8080                               # variable with value
scores: dict[str, list[int]] = {"asha": [90]}  # nested types
total: int                                     # bare annotation: declares, creates nothing

class User:
    name: str            # class-level annotation (dataclasses use these as fields)
    def rename(self, new: str) -> None: ...    # self is not annotated
```

How it works

1. `param: Type`, with a default `param: Type = default` (spaces around `=`, unlike un-annotated `param=default` — PEP 8).
2. `-> Type` for the return; returns nothing → `-> None`.
3. `name: Type = value` annotates a variable (PEP 526). A bare `name: Type` creates **no** variable — reading `total` raises `NameError`.
4. Stored in `__annotations__`; `typing.get_type_hints(obj)` returns them resolved (string forward references → real classes).

5. **Forward references** — a type not yet defined (a class referring to itself) is written as a string: `-> "Node"`. Since 3.11, `typing.Self` means "the current class".
6. **Evaluation time** — up to 3.13, annotations are evaluated when the `def` runs (unless `from __future__ import annotations` stores them as strings). **Python 3.14** (PEP 649/749) evaluates them *lazily*, only when requested, so forward references usually need no quotes; `annotationlib` exposes the details.

Script `01_annotations_basics.py`:

```
print("greet annotations:", greet.__annotations__)
print("get_type_hints    :", get_type_hints(greet))
print("greet(123, 2)     :", greet(123, 2))   # int passed for str - runs fine
```

Output:

```
greet annotations: {'name': <class 'str'>, 'times': <class 'int'>, 'return': <class 'str'>}
get_type_hints    : {'name': <class 'str'>, 'times': <class 'int'>, 'return': <class 'str'>}
greet(123, 2)     : hi 123, hi 123
```

Python ran `greet(123, 2)`; mypy reports it:

```
01_annotations_basics.py:34: error: Argument 1 to "greet" has incompatible type "int"; expected "str"  [arg-type]
```

What to annotate

- **Always:** parameters and return types of public functions.
- **Rarely:** locals — the checker **infers** `x = 5` as `int`. Annotate only when inference can't, e.g. an empty container: `seen: set[str] = set()`.

Common mistakes

- Forgetting `-> None`: by default mypy doesn't check inside a function with *no* annotations — `-> None` opts it in.
- `def f(x: int=5)` — works, but style is `x: int = 5`.
- Expecting `total: int` to create `total`.

13.3 Built-in generics and the abstract collection types

What it is

A **generic type** is a container type with a parameter for what it holds: `list[int]`. Since 3.9 (PEP 585) built-ins accept brackets directly; `typing.List`, `typing.Dict` etc. are deprecated aliases.

Type	Meaning
`list[int]`	list of ints
`dict[str, float]`	keys `str`, values `float`
`set[str]`, `frozenset[str]`	set of strings
`tuple[int, str]`	**exactly** two items: an int then a str
`tuple[int, ...]`	any length, all ints
`tuple[()]`	the empty tuple
`type[User]`	the class `User` itself (or a subclass), not an instance

Accept abstract, return concrete

`collections.abc` has parameterisable *abstract* types describing what a collection supports:

Abstract type	What it promises	Accepts
`Iterable[T]`	can be looped over	list, tuple, set, dict, generator, file…
`Sequence[T]`	indexable, has `len`, read-only	list, tuple, str, range
`Mapping[K, V]`	read-only dict-like	dict, `MappingProxyType`, custom
`MutableSequence[T]`, `MutableMapping[K, V]`	can be modified	list / dict
`Iterator[T]`, `Generator[Y, S, R]`	produced by generators	generator functions

Rule: **parameters ask for the least they need; return values say exactly what they give.**

```
from collections.abc import Iterable

def total(prices: Iterable[float]) -> float:   # works with list, tuple, generator
    return sum(prices)

def tokens(text: str) -> list[str]:            # caller knows they get a real list
    return text.split()
```

Variance in one paragraph

Is `list[int]` a `list[float]`? **No** — a function taking `list[float]` could append `2.5`, corrupting your `list[int]`. Mutable containers are **invariant** (must match exactly). Read-only `Sequence` is **covariant**: `Sequence[int]` is accepted where `Sequence[float]` is expected. (Separately, a plain `int` may be passed where `float` is expected — "numeric promotion".) Another reason to prefer `Sequence` / `Iterable` parameters.

Common mistakes

- `def f(xs: list)` — bare `list` is `list[Any]`; items go unchecked.
- `tuple[int]` is a one-element tuple; "tuple of ints" is `tuple[int, ...]`.
- `List`, `Dict` from `typing` in new code (works, old style).

13.4 `Optional`, `Union` and `X | None`

What it is

- A **union** — "one of these": `int | str` (3.10+, PEP 604) or `Union[int, str]`.
- `X | None` — "an `X` or `None`". `Optional[X]` is identical; it does **not** mean "optional argument", it means "may be None".

```
def find_user(user_id: int) -> str | None:
    users = {1: "asha", 2: "ben"}
    return users.get(user_id)
```

Why it matters

`'NoneType' object has no attribute ...` is among the most common production errors (Tony Hoare called null references his "billion-dollar mistake"). Putting `None` in the type forces every caller to handle it.

How it works — narrowing

After a check, the checker knows a more specific type in that branch (**narrowing**):

```python
def f(x: int | str | None) -> None:
    if x is None:
        return
    reveal_type(x)          # int | str
    if isinstance(x, int):
        reveal_type(x)      # int
    else:
        reveal_type(x)      # str
```

`reveal_type(...)` (importable from `typing` since 3.11) makes mypy print the inferred type:

```
note: Revealed type is "int | str"
note: Revealed type is "int"
note: Revealed type is "str"
```

Narrowing constructs: `is None` / `is not None`, `isinstance`, truthiness (`if x:` — also removes `""` and `0`), `==` against a literal, `match`, early `return`/`raise`, and your own functions returning `TypeGuard[T]` or (3.13+) `TypeIs[T]`. Unions work in `isinstance` at runtime: `isinstance(5, int | str)` → `True`.

Common mistakes

- `def f(x: str = None)` — must be `str | None`; mypy rejects "implicit Optional" since 0.990.
- `if x:` when `0` or `""` are valid — use `if x is not None:`.
- `X | None` everywhere "just in case" — every caller must then check. Raise when absence is an error.

In the interview

"`Optional[X]` is just `X | None`. The checker forces callers to narrow with `is not None` before use, which eliminates most `NoneType has no attribute` errors."

- *Is `Optional` about default arguments?* No — a parameter can have a default and not be Optional, and vice versa.

13.5 `Any` (and `object`)

`Any` is the escape hatch: compatible with **every** type in both directions, every operation allowed, checking off. Un-annotated parameters are implicitly `Any`. `object` is the opposite: the top of the hierarchy — **anything can be passed in**, but you can only do what every object supports (`print`, `==`, `hash`) until you narrow.

	Accepts anything?	Can call any method on it?	Safety
`Any`	yes	yes	none
`object`	yes	no (must narrow with `isinstance`)	full

```python
def load(raw: Any) -> None:
    raw.whatever()          # checker says nothing - could crash at runtime

def show(x: object) -> None:
    x.upper()               # error: "object" has no attribute "upper"
    if isinstance(x, str):
        x.upper()           # fine after narrowing
```

When to use `Any`: truly dynamic data at a boundary (parsed JSON before validation), untyped third-party libraries, temporarily during migration.

Mistake: `Any` is **contagious** — `y = raw.field` is `Any`, and so is everything derived. A few `Any`s at the edges silently disable checking deep inside. Prefer `object` for "I accept anything"; convert `Any` data into real types early (§13.14). `mypy --strict` warns about `Any` leaking from untyped code.

13.6 `TypeVar` and generics

What it is

A **type variable** stands for "some type, the same one each time it appears", so one function works for many types *without* losing precision. `def first(items: list[Any]) -> Any` misses `first([1, 2]) + "x"`; with a type variable, the checker knows `first([1, 2])` is `int`.

How it works

Old style (3.5–3.11), still common:

```
from typing import TypeVar
T = TypeVar("T")

def first_old(items: Sequence[T]) -> T:
    return items[0]
```

New style (3.12+, PEP 695) — type parameters in square brackets:

```
def first[T](items: Sequence[T]) -> T:
    return items[0]

def biggest[N: (int, float)](a: N, b: N) -> N:   # constrained: N is int or float
    return a if a > b else b

class Stack[T]:
    def __init__(self) -> None:
        self._items: list[T] = []
    def push(self, item: T) -> None:
        self._items.append(item)
    def pop(self) -> T:
        return self._items.pop()
```

1. At each call the checker **solves** `T`: `first("xyz")` → `T = str`.
2. **Bound** `[T: Hashable]` (old: `TypeVar("T", bound=Hashable)`) — any subtype of `Hashable` (a type usable wherever `Hashable` is expected, e.g. a subclass).
3. **Constraints** `[N: (int, float)]` — solved to exactly `int` or exactly `float`, never a subclass or mix: `biggest(2, 7)` → `int`, `biggest(2, 7.0)` → `float` (numeric promotion, §13.3). Classic: `AnyStr = (str, bytes)` rejects `concat("a", b"b")`.
4. **Generic class** — `Stack[int]()` or `s: Stack[int] = Stack()` fixes `T`; `s.push("a")` is then an error. Old style: `class Stack(Generic[T]): ...`.
5. Since 3.13 (PEP 696) type parameters can have **defaults**: `class Box[T = int]: ...`.

Output of `02_generics_protocols.py` (generic part):

```
first_old([3,4]) : 3
first('xyz')     : x
biggest(2, 7)    : 7
Stack pop        : 2 | len 1
type params      : (T,) (T,)
```

The 3.12 syntax records parameters at runtime in `__type_params__`.

Trade-offs

	Old `TypeVar`	3.12 `[T]` syntax
Works on	3.5+	3.12+ only
Scope	module-global name, easy to misuse	scoped to the function/class
Variance	declared manually (`covariant=True`)	inferred automatically

Common mistakes

- A type variable used once in a signature (`def f(x: T) -> None`) adds nothing — use `object`.
- Reusing one old-style `T` across unrelated functions and getting confused — each solves it independently; the 3.12 syntax makes that obvious.

Real-world examples

`dict.get`, `list.__getitem__`, `max`, `sorted`, `lru_cache`, `asyncio.Queue[T]` are generic in typeshed; SQLAlchemy 2.0's `Mapped[int]` and Django-stubs give ORM columns precise types.

In the interview

"A TypeVar ties input and output types together, so `first(list_of_ints)` returns `int` instead of `Any`. Since 3.12 I write `def first[T](xs: Sequence[T]) -> T` and `class Stack[T]`."

- *Bound vs constrained?* Bound: any subtype of X. Constrained: exactly one of the listed types.
- *Why is `list[int]` not a `list[float]`?* Invariance — the callee could insert a float.

13.7 `Callable` and `ParamSpec`

`Callable[[Arg1, Arg2], Return]` types "something callable with these arguments returning this". Import from `collections.abc` (the `typing` one is a deprecated alias).

```
from collections.abc import Callable, Iterable

def apply_all(fn: Callable[[int], int], xs: Iterable[int]) -> list[int]:
    return [fn(x) for x in xs]
```

- `Callable[..., int]` — any arguments, returns int.
- `Callable[[], None]` — no-argument callback.
- Keyword arguments, overloads or defaults can't be expressed — use a `Protocol` with `__call__` (§13.8).

ParamSpec — typing decorators (brief)

A decorator wrapping *any* function should keep its exact parameters. Before `ParamSpec` (3.10, PEP 612) people wrote `Callable[..., Any]` and lost all argument checking.

```
P = ParamSpec("P")
R = TypeVar("R")

def timed(fn: Callable[P, R]) -> Callable[P, R]:
    @wraps(fn)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        start = time.perf_counter()
        try:
            return fn(*args, **kwargs)
        finally:
            print(f"  {fn.__name__} took {time.perf_counter() - start:.4f}s")
    return wrapper

@timed
def add(a: int, b: int) -> int:
    return a + b
```

1. `P` captures `fn`'s parameter list; `R` its return type.
2. `*args: P.args, **kwargs: P.kwargs` — "the wrapper takes exactly what `fn` takes", so `add("x", 1)` is still flagged after decoration.
3. 3.12 syntax: `def timed[**P, R](fn: Callable[P, R]) -> Callable[P, R]:`.
4. `Concatenate[Request, P]` types decorators that add or remove a leading argument.

Output:

```
apply_all       : [1, 4, 9]
add(2, 3) ->
  add took 0.0000s
  result 5
```

13.8 `Protocol` — structural typing

What it is

Nominal typing: B is a subtype of A because it *inherits* from A. **Structural typing**: B is compatible with A because it *has the right methods*. A `Protocol` (PEP 544, 3.8) declares such a shape — duck typing made checkable.

Why it matters

- Type code against classes you don't control (a third-party class can't inherit your ABC but may already have `close()`).
- Decoupling: the consumer defines the tiny interface it needs; producers needn't import it.

How it works

```python
from typing import Protocol, runtime_checkable

@runtime_checkable
class SupportsClose(Protocol):
    def close(self) -> None: ...

class DbConnection:              # does NOT inherit from SupportsClose
    def close(self) -> None:
        print("  db closed")

def close_all(things: Iterable[SupportsClose]) -> None:
    for t in things:
        t.close()
```

1. The Protocol body lists required members with `...` bodies.
2. Any class whose members match (names **and** compatible signatures) satisfies it — checked statically.
3. `@runtime_checkable` also allows `isinstance(x, SupportsClose)`, which checks only that **names exist**, not signatures.
4. Protocols can be generic: `class SupportsLessThan[T](Protocol): def __lt__(self, other: T) -> bool: ...`.

Output:

```
close_all ->
  db closed
  temp file removed
isinstance(DbConnection(), SupportsClose): True
isinstance(42, SupportsClose)            : False
```

Trade-offs — Protocol vs ABC

	`Protocol`	`abc.ABC`
Relationship	structural (shape)	nominal (must inherit / register)
Checked	by the type checker	at instantiation (`TypeError` if abstract methods missing)
Shares implementation	not intended	yes, concrete helper methods
Works with third-party classes	yes	only with `register()`

Common mistakes

- Trusting runtime-checkable `isinstance` to verify signatures — it doesn't.
- Big protocols. Keep them to one to three methods, like `Iterable`, `Sized`, `Hashable`, `SupportsInt`, `SupportsAbs`.

Real-world examples

`typing.SupportsIndex`, `collections.abc.Iterable`, typeshed's `SupportsRead[str]` ("anything with `.read()`", used by `json.load`).

In the interview

"A Protocol is a statically checked duck type: any class with matching methods satisfies it, no inheritance. I use it for the minimal interface a function needs, like `SupportsClose`, which also makes testing with fakes (simple stand-in objects) easy."

- *Protocol or ABC?* ABC to share code and force subclasses to implement methods; Protocol to describe a shape only.

13.9 `TypedDict`

A **`TypedDict`** describes a dict with fixed string keys, each with its own value type — the shape of JSON objects and config.

```
from typing import NotRequired, TypedDict

class Movie(TypedDict):
    title: str
    year: int
    rating: NotRequired[float]      # key may be absent (3.11+)

m: Movie = {"title": "Arrival", "year": 2016}
```

1. The checker verifies keys and value types: `m["yeer"]` and `m["year"] = "2016"` are errors.
2. Keys are required by default. `NotRequired[...]` per key, or `total=False` for all (then `Required[...]` for exceptions). `ReadOnly[...]` (3.13) forbids writes.
3. **At runtime it is a plain `dict`** — no validation; `isinstance(x, Movie)` raises `TypeError`.

Output:

```
TypedDict value  : {'title': 'Arrival', 'year': 2016} | type at runtime: dict
Movie keys       : frozenset({'title', 'year'}) frozenset({'rating'})
```

TypedDict vs dataclass

	TypedDict	dataclass
Runtime object	plain `dict`	instance of your class
Access	`m["title"]`	`m.title`
Methods	none	yes
Best for	JSON in/out, `**kwargs` typing (`Unpack[Opts]`)	domain objects (`Order`) in your code

---

13.10 `Literal`, `Final`, `NewType`

`Literal` — only these exact values

```
Mode = Literal["r", "w", "a"]
def open_log(path: str, mode: Mode = "r") -> str: ...
open_log("app.log", "x")      # error: expected "Literal['r', 'w', 'a']"
```

For string/int flags when an `Enum` feels heavy. `typing.get_args(Mode)` → `('r', 'w', 'a')` at runtime (useful for validation); `match` / `==` narrows it. typeshed types `open()`'s `mode` with literals, so `open(p, "rb")` is known to return a binary file.

`Final` — must not be reassigned

```
MAX_RETRIES: Final = 3        # inferred as int, and constant
MAX_RETRIES = 4               # error: Cannot assign to final name "MAX_RETRIES"
```

`@final` on a method (no overriding) or class (no subclassing). Not enforced at runtime. `@typing.override` (3.12) marks a method that must override a parent's, catching name typos.

`NewType` — a distinct name for the same runtime type

```
UserId = NewType("UserId", int)
OrderId = NewType("OrderId", int)

def get_user(uid: UserId) -> str: ...
get_user(UserId(7))   # OK
get_user(42)          # error: expected "UserId"
```

1. For the checker `UserId` is a *subtype* of `int`: a `UserId` goes where an `int` is expected, not the reverse.
2. At runtime `UserId(7)` returns `7` — `type(UserId(7)) is int`. Zero cost.
3. Stops mixing up IDs, units (`Meters` vs `Feet`), raw vs sanitised strings (`SafeHtml`).

Output of `03_special_forms.py`:

```
Literal args     : ('r', 'w', 'a')
open app.log in 'a'
NewType          : user#7 | UserId(7) is int: True
```

Common mistakes

- `UserId = int` is an alias (interchangeable, no protection); `NewType` is distinct.
- Expecting `Final` or `Literal` to raise at runtime.

Other special forms you will meet

Form	Meaning	Example
`Annotated[T, meta]` (3.9)	`T` plus metadata the checker ignores but libraries read	FastAPI/pydantic: `age: Annotated[int, Field(gt=0)]`
`Self` (3.11)	An instance of the current class — fluent (chainable) methods, alternative constructors	`def with_name(self, n: str) -> Self:`
`NoReturn` / `Never` (3.11)	Never returns normally (always raises or exits)	`def fail(msg: str) -> NoReturn: raise SystemExit(msg)`
`ClassVar[T]`	Class-level attribute, not a per-instance field	`count: ClassVar[int] = 0`
`@overload`	Several signatures when the return type depends on the arguments	`open(p, "rb")` → binary file, `open(p, "r")` → text file
`TypeGuard[T]` / `TypeIs[T]` (3.10 / 3.13)	User function that narrows its argument when it returns `True`	`def is_str_list(x: list[object]) -> TypeIs[list[str]]:`
`LiteralString` (3.11)	Only strings written in source, not user input	SQL builders defending against injection
`Unpack[TD]` (3.11/3.12)	Type `**kwargs` with a `TypedDict`	`def f(**kw: Unpack[Opts]):`

13.11 dataclasses and typing

`@dataclass` (Chapter 10) reads the **class-level annotations** to decide which fields exist and their order, and generates `__init__`, `__repr__`, `__eq__`. It's the best-known place annotations change runtime behaviour (others: `typing.NamedTuple`, `TypedDict`, `functools.singledispatch`'s `register`, which can dispatch on an annotation) — but only as *field declarations*; no type validation.

```
@dataclass(frozen=True, slots=True)
class Point:
    x: float
    y: float
    tags: tuple[str, ...] = ()

@dataclass
class Cart:
    owner: str
    items: list[str] = field(default_factory=list)
```

1. Each annotated class attribute becomes a field; the generated `__init__` is typed, so checkers verify `Point("a", 2)`.
2. `ClassVar[int]` attributes are **not** fields; `InitVar[str]` is an `__init__`-only parameter passed to `__post_init__`.
3. Mutable defaults need `field(default_factory=list)` — plain `= []` raises `ValueError` at class creation.
4. `frozen=True` → immutable (checkers flag `p.x = 5`; at runtime `FrozenInstanceError`).
5. `Point(x="oops", y=2)` **runs** — no type checks.

Output:

```
dataclass        : Point(x=1.0, y=2.0, tags=()) | fields: ['x', 'y', 'tags']
default_factory  : ['pen'] []
frozen           : FrozenInstanceError
```

Related typed record types

Tool	Mutable	Validates types	Notes
`@dataclass`	yes (or `frozen`)	no	the default choice
`typing.NamedTuple`	no	no	a real tuple (unpacks, indexes)
`TypedDict`	yes	no	plain dict
`attrs` (third-party)	configurable	optional validators	dataclasses' ancestor
`pydantic.BaseModel`	yes	**yes**	for untrusted input (§13.14)

13.12 Type aliases and the `type` statement

A **type alias** is a short name for a long type.

```
Vector = list[float]                                  # implicit alias (any version)
from typing import TypeAlias
Headers: TypeAlias = dict[str, str]                   # explicit (3.10+, now deprecated style)
type JSON = dict[str, JSON] | list[JSON] | str | int | float | bool | None   # 3.12+
```

1. The `type` **statement** (3.12, PEP 695) creates a `TypeAliasType`, evaluated **lazily** — so it can be recursive (`JSON`) or reference later names without quotes.
2. Aliases can be generic: `type Pair[T] = tuple[T, T]`, then `Pair[int]`.
3. An alias is interchangeable with what it names — unlike `NewType`, no extra safety.
4. A `type` alias is **not** a class: `isinstance(x, JSON)` fails, it can't be called. `JSON.__value__` shows the underlying type.

Output:

```
type alias       : JSON -> TypeAliasType
alias value      : dict[str, 'JSON'] | list['JSON'] | str | int | float | bool | None
scale            : [3.0, 6.0]
```

(The script quotes `"JSON"` only for display; with `type` quotes are optional.)

13.13 Running mypy and pyright; common errors

What it is

mypy is the original checker (PEP 484's reference implementation, compiled with mypyc). **pyright** (Microsoft, TypeScript) is faster, powers Pylance, and infers more in un-annotated code. Both use the same annotations and typeshed stubs; they occasionally disagree on edge cases.

How it works — using mypy

```
pip install mypy
mypy src/                       # check a package
mypy --strict src/              # turn on all optional strictness flags
```

Configure once in `pyproject.toml`:

```toml
[tool.mypy]
python_version = "3.12"
strict = true                       # new code: go strict
warn_unused_ignores = true

[[tool.mypy.overrides]]
module = ["legacy.*"]
ignore_errors = true                # old code: switch off per module while migrating
```

pyright: `pip install pyright` (or `npm i -g pyright`), `pyright src/`, configured in `[tool.pyright]` with `typeCheckingMode = "strict"`.

Gradual adoption plan (as Dropbox and others describe):

1. Run the checker in CI with defaults on everything; fix or silence the first errors.
2. Annotate the most-used modules and public functions first.
3. Turn on `strict` per module as it becomes fully typed.
4. Forbid new untyped functions (`disallow_untyped_defs`).
5. Install third-party stubs: `pip install types-requests` etc. (mypy suggests the name).

Reading real errors

`bad_types.py` has seven deliberate mistakes; `04_run_mypy.py` runs mypy via `mypy.api.run`:

```python
if __name__ == "__main__":
    area("3", 4)                    # 1 wrong argument type
    name = find(1)                  # returns str | None
    print(name.upper())             # 2 Optional not narrowed
    get_user(42)                    # 3 plain int is not a UserId
    mode("x")                       # 4 not an allowed literal
    LIMIT = 11                      # 5 reassigning a Final
    count: int = "three"            # 6 wrong assignment
    shout(name)                     # 7 str | None passed where str needed
```

Output (real mypy 2.x):

```
bad_types.py:29: error: Argument 1 to "area" has incompatible type "str"; expected "float"  [arg-type]
bad_types.py:31: error: Item "None" of "str | None" has no attribute "upper"  [union-attr]
bad_types.py:32: error: Argument 1 to "get_user" has incompatible type "int"; expected "UserId"  [arg-type]
bad_types.py:33: error: Argument 1 to "mode" has incompatible type "Literal['x']"; expected "Literal['r', 'w']"
[arg-type]
bad_types.py:34: error: Cannot assign to final name "LIMIT"  [misc]
bad_types.py:35: error: Incompatible types in assignment (expression has type "str", variable has type "int")
[assignment]
bad_types.py:36: error: Argument 1 to "shout" has incompatible type "str | None"; expected "str"  [arg-type]
exit status: 1
```

Format: `file:line: error: message [error-code]`; the bracketed code is what a targeted ignore names.

Common errors and fixes

Error (code)	Usual cause	Fix
`Incompatible types in assignment` `[assignment]`	variable reused for a different type	new variable name, or a union type
`Argument 1 has incompatible type` `[arg-type]`	wrong type passed	convert (`float(s)`) or fix the signature
`Item "None" of "X | None" has no attribute` `[union-attr]`	Optional not narrowed	`if x is not None:` / early return / raise
`Need type annotation for "d"` `[var-annotated]`	empty container `d = {}`	`d: dict[str, int] = {}`
`Missing return statement` `[return]`	a branch falls off the end	return or raise in every branch
`Function is missing a type annotation` `[no-untyped-def]`	strict mode, un-annotated def	annotate it
`Library stubs not installed for "requests"` `[import-untyped]`	no type info for a package	`pip install types-requests`
`Returning Any from function declared to return "int"` `[no-any-return]`	value came from untyped code	`cast(int, ...)` after validating, or validate

Escape hatches (use sparingly)

- `# type: ignore[arg-type]` — always with the code, so unrelated errors aren't hidden; `warn_unused_ignores` finds stale ones.
- `typing.cast(T, value)` — "trust me" for the checker; no-op at runtime.
- `if TYPE_CHECKING:` — imports only for the checker (avoids runtime import cycles).
- `.pyi` **stub files** — type info kept separate from code (how typeshed works).
- `@overload` — precise signatures when the return type depends on arguments.

Common mistakes

- Blanket `# type: ignore` without a code.
- Assuming un-annotated functions are checked — mypy skips their bodies unless `check_untyped_defs = true` (in `--strict`).
- `cast` instead of real validation of external data.

In the interview

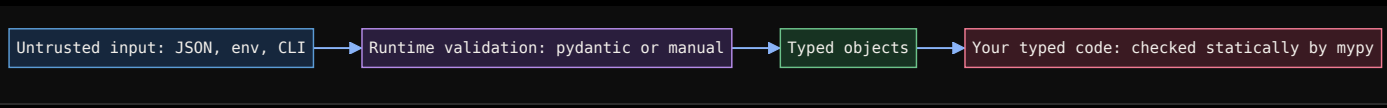
"I run mypy or pyright in CI and pre-commit (git hooks run before each commit), strict for new code with per-module overrides for legacy code. Most errors are unhandled `None`, empty containers needing annotations, and missing stubs. I debug inference with `reveal_type` and always put an error code on `type: ignore`."

13.14 Runtime vs static — and pydantic

What it is

- **Static** checking reasons about **code** before it runs; it knows nothing about real data.
- **Runtime validation** checks **data** as it arrives — essential for anything external: HTTP requests, files, environment variables, user input, LLM output.

A fully mypy-typed program still crashes on a JSON payload with `"age": "thirty"`, because `json.loads` returns `Any`.



What Python does at runtime with hints

Feature	Runtime effect
Parameter/variable annotations	stored only; never checked
`@dataclass`	fields from annotations; **no** type check
`NewType`, `cast`, `Final`, `Literal`, `TypedDict`	no checks (NewType returns its argument)
`isinstance(x, int | str)`	works
`isinstance(x, list[int])`	`TypeError` — parameterised generics can't be checked with isinstance
`@runtime_checkable` Protocol	`isinstance` checks method names only
`get_type_hints`, `get_origin`, `get_args`	let libraries read hints and act on them

How runtime validation works — a tiny version

`05_runtime_validation.py` builds a mini validator from `get_type_hints` + `get_origin`/`get_args` — the idea behind pydantic:

```
def check(value: object, tp: object) -> bool:
    origin = get_origin(tp)
    if tp is NoneType or tp is None:
        return value is None
    if origin is UnionType:                      # X | Y
        return any(check(value, arg) for arg in get_args(tp))
    if origin is list:                           # list[T]
        (item_tp,) = get_args(tp)
        return isinstance(value, list) and all(check(v, item_tp) for v in value)
    if isinstance(tp, type):                     # plain class
        return isinstance(value, tp) and not (tp is int and isinstance(value, bool))
    raise TypeError(f"unsupported type {tp!r}")
```

Output:

```
Signup(email='a@x.io', age=30, referrer=None, tags=['beta'])
rejected: age='30' is not <class 'int'>
rejected: tags=['ok', 5] is not list[str] | None
```

(`bool` is special-cased because it subclasses `int`: `isinstance(True, int)` is `True`.)

pydantic (brief, real-world)

pydantic is the most widely used runtime-validation library; v2's core (`pydantic-core`) is written in Rust. Declare a model with ordinary hints; it validates, converts and serialises. (`pip install pydantic` — not run here.)

```python
from pydantic import BaseModel, ValidationError

class Signup(BaseModel):
    email: str
    age: int
    referrer: str | None = None

s = Signup(email="a@x.io", age="30")      # "lax" mode converts "30" -> 30
print(s.age + 1)                          # 31
print(s.model_dump())                     # {'email': 'a@x.io', 'age': 30, 'referrer': None}

try:
    Signup(email="b@x.io", age="thirty")
except ValidationError as e:
    print(e.error_count(), "error")       # 1 error: age is not a valid integer

s2 = Signup.model_validate_json('{"email": "c@x.io", "age": 41}')
```

1. **Lax** by default — converts compatible input (`"30"` → `30`); `strict=True` (model or field) rejects that.
2. All errors are collected in one `ValidationError`, each with its location.
3. `model_dump()` / `model_dump_json()` serialise; `model_json_schema()` emits JSON Schema (a standard machine-readable description of valid JSON).
4. Used by **FastAPI** (request validation, OpenAPI docs), `pydantic-settings` (typed config from env vars), LLM SDKs (Anthropic's, OpenAI's) for responses and "structured output" validation.
5. Alternatives: `attrs` + `cattrs`, `msgspec` (very fast), `marshmallow` (schemas separate from classes), `typeguard` / `beartype` (check hints at call time).

When not to use it

Don't re-validate internal data already validated and covered by static types. Validate **once, at the boundary**, then trust the types ("parse, don't validate").

In the interview

"Type hints are static; Python never enforces them. For untrusted data I validate at the boundary with pydantic, which uses the same annotations to parse JSON into typed objects; mypy then guarantees the rest of the code uses them correctly."

- *Would mypy catch a bad API payload?* No — it never sees runtime data.
- *Why is pydantic v2 fast?* Its validation core is Rust.

13.15 Chapter self-check

1. Does `def f(x: int)` stop `f("a")` from running? → No; only a checker flags it.
2. What does "gradual typing" mean? → Typed and untyped code coexist; un-annotated parts are `Any`.
3. `Optional[str]` equals? → `str | None`.
4. Parameter type for "anything I can loop over once"? → `Iterable[T]`.
5. Why is `list[int]` not a subtype of `list[float]`? → Lists are mutable, so invariant.
6. `Any` vs `object`? → `Any` disables checks; `object` accepts all but allows only universal operations.
7. 3.12 syntax for a generic function / class? → `def first[T](xs: Sequence[T]) -> T:` / `class Stack[T]:`.
8. When is a TypeVar pointless? → When it appears only once in the signature.
9. How do you keep a decorated function's signature? → `ParamSpec` (`Callable[P, R]`).
10. Class with `close()` but no inheritance — matches `SupportsClose`? → Yes, structurally.
11. What is a TypedDict at runtime? → A plain dict.
12. `UserId = int` vs `NewType("UserId", int)`? → Alias = interchangeable; NewType = distinct for the checker.
13. Recursive JSON type in 3.12? → `type JSON = dict[str, JSON] | list[JSON] | str | int | float | bool | None`.

14. How do you see what mypy inferred? → `reveal_type(expr)`.
15. `Need type annotation for "d"` — fix? → `d: dict[str, int] = {}`. `Library stubs not installed`? → install `types-…`.
16. What does `cast` do at runtime? Why a code in `# type: ignore[...]`? → Nothing; to suppress only that error and let `warn_unused_ignores` catch stale ones.
17. Does `@dataclass` check `x: int`? Can you `isinstance(x, list[int])`? → No; no (`TypeError`).
18. Where do you validate API data, and pydantic lax vs strict? → Once at the boundary; lax converts `"30"`→30, strict rejects.

Chapter TL;DR

- **Hints are optional and unenforced.** Checkers (mypy, pyright) read them before runtime; typing is gradual.
- **Syntax:** `param: T`, `-> T`, `var: T = v`. Annotate signatures; let inference handle locals. 3.14 evaluates annotations lazily.
- **Generics:** `list[int]`, `dict[str, int]`, `tuple[int, ...]`; parameters take `Iterable`/`Sequence`/`Mapping`; mutable containers are invariant.
- **Unions:** `A | B`; `X | None` = `Optional[X]`; narrow with `is not None`/`isinstance`. `Any` switches checking off, `object` doesn't.
- **Type variables:** `def f[T](...)`, `class C[T]` (3.12), bounds and constraints. `Callable` for function values, `ParamSpec` for decorators.
- `Protocol` = structural interface. `TypedDict` = typed dict. `Literal` = fixed values. `Final` = constant. `NewType` = zero-cost distinct type.
- **dataclasses** use annotations as fields but never validate. **Aliases** via `type X = ...`.
- **Tooling:** `mypy --strict` / pyright in CI, config in `pyproject.toml`, `reveal_type`, `# type: ignore[code]`, stubs (`types-*`).
- **Runtime vs static:** validate untrusted data once at the boundary (pydantic), trust the types inside.

Chapter 14 — A Tour of the Standard Library

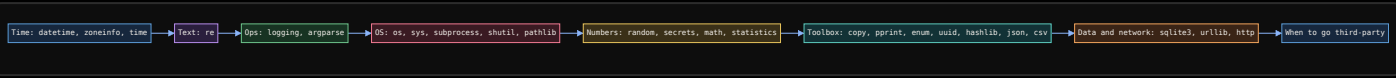
What you will learn. The modules you reach for every week, and the traps in each: dates and time zones (`datetime` , `zoneinfo` , `time`), regular expressions in depth (`re`), `logging` , `argparse` , the OS (`os` , `sys` , `subprocess` , `shutil` , `pathlib`), `random` vs `secrets` , numbers (`math` , `statistics` , `decimal` / `fractions`), small modules (`copy` , `pprint` , `enum` , `uuid` , `hashlib` , `json` , `csv`), the embedded database `sqlite3` , basic HTTP (`urllib` , `http` — see System Design Fundamentals), and when a third-party package is the better choice.

Prerequisites. Chapters 1–12. `collections` , `itertools` and `functools` are in Chapters 6, 7 and 11 — pointer in §14.10.

Code. `chapters/14-standard-library-tour/code/` , stdlib only, no internet (the HTTP demo uses a server it starts on `127.0.0.1`), Python 3.12+:

```
python chapters/14-standard-library-tour/code/01_datetime_time.py
```

"Batteries included": the standard library ships a few hundred modules. You need to know which one to reach for and its known traps.



14.1 `datetime` and `zoneinfo` — naive vs aware

What it is

Type	Holds	Example
`date`	year, month, day	`date(2026, 9, 28)`
`time`	hour … microsecond (no date)	`time(14, 30)`
`datetime`	date + time (+ optional time zone)	`datetime(2026, 9, 28, 14, 30)`
`timedelta`	a duration	`timedelta(days=2, hours=3)`

A datetime is **naive** if `tzinfo is None` — "14:30 on some wall clock, somewhere" — and **aware** if it carries a `tzinfo` , so it names one exact instant. `zoneinfo.ZoneInfo("Asia/Kolkata")` (3.9+) supplies real rules from the IANA time-zone database, including **DST** (daylight saving time). `datetime.UTC` (3.11+, alias of `timezone.utc`) is UTC, the reference clock with no DST.

Why it matters

Classic time bugs: meetings an hour off after a DST change, reports missing a day because the server runs in UTC and the user in IST (UTC+5:30), crashes from comparing naive and aware values.

How it works

1. **Store and compute in UTC; convert to local time only for display.**
2. Create aware values: `datetime.now(UTC)` or `datetime(..., tzinfo=ZoneInfo("Europe/London"))` . Avoid `datetime.utcnow()` — it returns a *naive* value and is deprecated since 3.12.
3. Convert with `.astimezone(ZoneInfo(...))` — same instant, different wall clock.
4. Parse/format: `fromisoformat("2026-09-28T09:15:00+05:30")` / `.isoformat()` for ISO 8601 machine formats; `strptime(text, fmt)` / `strftime(fmt)` for custom ones (`%Y-%m-%d %H:%M` , `%A` weekday, `%I %p` 12-hour clock, `%Z` zone name).

5. Arithmetic: `datetime ± timedelta`, `datetime − datetime → timedelta`. **Caution:** adding a `timedelta` to an aware datetime is *wall-clock* arithmetic in its zone — no DST adjustment. For "exactly N hours later", convert to UTC, add, convert back.
6. Naive and aware values can't be compared or subtracted (`TypeError`).
7. On Windows (no system zone database) install `tzdata` so `zoneinfo` works.

Script `01_datetime_time.py` (key parts):

```
naive = datetime(2026, 3, 8, 1, 30)
aware = datetime(2026, 3, 8, 1, 30, tzinfo=ZoneInfo("America/New_York"))
naive < aware                                  # TypeError

later = aware + timedelta(hours=1)             # wall-clock arithmetic
real = (aware.astimezone(UTC) + timedelta(hours=1)).astimezone(aware.tzinfo)

meeting_utc = datetime(2026, 9, 28, 14, 0, tzinfo=UTC)
for tz in ["Asia/Kolkata", "Europe/London", "America/Los_Angeles"]:
    print(f"  {tz:<20}", meeting_utc.astimezone(ZoneInfo(tz)).strftime("%a %H:%M %Z"))
```

Output:

```
naive tzinfo      : None
aware             : 2026-03-08T01:30:00-05:00
compare naive/aware: can't compare offset-naive and offset-aware datetimes
aware + 1h (wall) : 2026-03-08T02:30:00-05:00
aware + 1h (real) : 2026-03-08T03:30:00-04:00
  Asia/Kolkata         Mon 19:30 IST
  Europe/London        Mon 15:00 BST
  America/Los_Angeles  Mon 07:00 PDT
fromisoformat     : 2026-09-28 09:15:00+05:30 | utcoffset 5:30:00
strptime          : 2026-09-28 18:05:00 | strftime: 2026-09-28 06:05 PM
date + 30 days    : 2026-03-02 | weekday Monday
days until 2027   : 95
```

On 8 March 2026 New York clocks jump from 02:00 to 03:00. The "wall" result, 02:30, never existed on a New York clock; the UTC round-trip gives the correct 03:30 EDT.

Common mistakes

- Naive datetimes in a server with users in several zones.
- Hard-coded offsets (`+05:30`) instead of zone names — offsets change with DST and politics; zone rules get updated.
- `pytz` in new code — its `localize()` API is error-prone; `zoneinfo` replaced it.
- Expecting month arithmetic — `timedelta` has no months. Use `dateutil.relativedelta` (third-party).

Real-world examples

PostgreSQL stores `timestamptz` as UTC; Django (`USE_TZ = True`) and most APIs exchange ISO 8601 in UTC; calendar apps store UTC plus the zone name so recurring meetings follow DST.

In the interview

"I keep all datetimes aware and in UTC internally, convert to the user's `ZoneInfo` zone only at display time, and serialise as ISO 8601. Naive datetimes are ambiguous and can't be compared with aware ones."

- *Why not `utcnow()`?* It returns a naive value that looks local; use `datetime.now(UTC)`.

14.2 `time` — timestamps and timers

Function	Returns	Use for
`time.time()`	seconds since the Unix **epoch** (1970-01-01 UTC), float	timestamps (jumps if the system clock is changed)
`time.perf_counter()`	highest-resolution counter, arbitrary start	measuring short durations
`time.monotonic()`	never goes backwards, arbitrary start	timeouts, rate limiters (cap actions per second)
`time.sleep(s)`	—	pause the thread
`time.time_ns()`, `perf_counter_ns()`	integer nanoseconds	avoid float precision loss

Timestamp → datetime: `datetime.fromtimestamp(ts, tz=UTC)` — `1_790_000_000` → `2026-09-21 14:13:20+00:00`.

Mistake: timing with `time.time()` — if NTP (network time sync) adjusts the clock mid-measurement you get wrong or negative durations. Use `perf_counter()` / `monotonic()`; for micro-benchmarks use `timeit`, which repeats the code and disables garbage collection during timing.

14.3 `re` — regular expressions, deeper

Chapter 4 introduced `re` (`search`, `match`, `findall`, basic patterns). This adds what real parsing needs. `re` compiles a pattern into a matcher run by a **backtracking** engine (on failure it goes back and tries the next alternative).

How it works — the essentials

Feature	Syntax	Meaning
Raw strings	`r"\d+"`	always use them so `\` reaches `re` unchanged
Group	`( … )`	capture a part; `m.group(1)` or `m[1]`
Named group	`(?P<year>\d{4})`	`m["year"]`, `m.groupdict()`
Non-capturing	`(?: … )`	group without capturing
Alternation	`cat|dog`	either
Quantifiers	`* + ? {m,n}`	how many times; greedy (as much as possible) by default
Lazy	`*? +? ??`	as little as possible
Anchors	`^ $ \b \A \Z`	line start/end, word boundary, string start/end
Lookahead	`(?=…)`, `(?!…)`	require / forbid what follows, without consuming
Lookbehind	`(?<=…)`, `(?<!…)`	same, for what precedes (fixed width)
Backreference	`\1`, `(?P=name)`	the same text again

Functions: `search` (anywhere), `match` (start only), `fullmatch` (whole string — for validation), `findall` (strings, or tuples if there are groups), `finditer` (match objects — memory-friendly, gives positions), `sub(pattern, repl, s)` (repl is a string with `\1` or a **function** of the match), `split`.

Flags: `re.I` (IGNORECASE), `re.MULTILINE` (`^` / `$` at each line), `re.DOTALL` (`.` matches newline), `re.X` (VERBOSE: whitespace and `#` comments in the pattern).

Compile once at module level (`LINE = re.compile(...)`); `re` also caches recent patterns, so this is mostly for readability.

Script `02_regex_deeper.py`:

```
LINE = re.compile(r"""
    ^(?P<date>\d{4}-\d{2}-\d{2})\s     # date
    (?P<time>[\d:]{8})\s+
    (?P<level>[A-Z]+)\s+
    \[(?P<svc>\w+)\]\s                 # service in brackets
    (?P<msg>.*?)\s                     # lazy: stop before the last field
    user=(?P<user>\d+)$
""", re.VERBOSE | re.MULTILINE)

for m in LINE.finditer(LOG):
    if m["level"] == "ERROR":
        print("error:", m.group("svc", "user"), "|", m["msg"])

re.sub(r"\d{12}(\d{4})", lambda m: "*" * 12 + m[1], card)   # function replacement
re.sub(r"(\w+), (\w+)", r"\2 \1", "Lovelace, Ada")            # backreferences
strong = re.compile(r"(?=.*\d)(?=.*[A-Z]).{8,}")              # two lookaheads
```

Output:

```
error: ('payments', '42') | card declined
error: ('auth', '7') | bad password
findall groups  : ['42', '7', '7']
greedy          : <a><b>
lazy            : <a>
masked          : pay ************1111 now
swap            : Ada Lovelace
strong('password'  ): False
strong('Passw0rdX' ): True
IGNORECASE      : ['ERROR', 'ERROR']
split multi     : ['a', 'b', 'c ', 'd']
match vs search : None 1
```

Trade-offs / when not to use it

- **Structured formats have parsers**: JSON → `json`, CSV → `csv`, HTML → BeautifulSoup/lxml, URLs → `urllib.parse`, emails → a validation library.
- Simple checks read better as string methods: `s.startswith("http")`, `"@" in s`, `s.isdigit()`.
- **Catastrophic backtracking**: nested quantifiers like `(a+)+$` on a long non-matching input take exponential time — real outages at Stack Overflow (2016) and Cloudflare (2019). Avoid ambiguous quantifiers on untrusted input; since 3.11 atomic groups `(?>…)` and possessive quantifiers `*+` ("never backtrack into this") prevent it.

Common mistakes

- No `r` prefix (`"\b"` is a backspace, not a word boundary).
- `match` when you meant `search` (or `fullmatch` for validation).
- Greedy `.*` swallowing too much — use `.*?` or a negated class `[^\]]*`.
- `findall` with groups returns only the groups.

In the interview

"I compile patterns with named groups and `re.VERBOSE`, use `finditer` for large inputs and `fullmatch` for validation, avoid regex for structured formats, and watch nested quantifiers because of catastrophic backtracking on untrusted input."

14.4 `logging` — levels, handlers, formatters, best practice

What it is

Part	Job	Example
Logger	the object your code calls; named hierarchically with dots	`logging.getLogger("shop.orders")`
Handler	sends records (one per logged event) somewhere	`StreamHandler` (console), `FileHandler`, `RotatingFileHandler`, `TimedRotatingFileHandler`, `SysLogHandler`, `QueueHandler`
Formatter	turns a record into text	`"%(asctime)s %(levelname)s %(name)s %(message)s"`
Filter	drops or edits records	add a request ID to each record

Levels: `DEBUG` (10) developer detail · `INFO` (20) normal events · `WARNING` (30) unexpected but working · `ERROR` (40) an operation failed · `CRITICAL` (50) the program may not continue.

Why it matters (logging vs print)

`print` has no levels, timestamps or destinations and can't be silenced per module. With logging the **same code** is silent in tests, shows INFO on the console in development, and ships JSON to a log collector in production — only configuration changes.

How it works

1. Each module: `log = logging.getLogger(__name__)`. Names form a tree: `shop.orders` → `shop` → root.
2. `log.info("order placed qty=%d", qty)` creates a **LogRecord** if the logger's effective level allows INFO.
3. The record goes to the logger's handlers, then **propagates** to parent handlers up to the root.
4. Each handler has its own level and formatter — one record can reach the console (INFO+) and a file (DEBUG+) in different formats.
5. With no configuration, WARNING+ goes to stderr via a "last resort" handler; DEBUG/INFO are dropped (root default level is WARNING).
6. Configure **once, at program start**: `logging.basicConfig(level=..., format=...)` for simple apps, `logging.config.dictConfig({...})` for real ones. `basicConfig` does nothing if the root already has handlers, unless `force=True`.

Script `03_logging_demo.py`:

```python
log = logging.getLogger("shop.orders")      # normally getLogger(__name__)

def place_order(qty: int) -> None:
    log.debug("validating qty=%s", qty)
    if qty <= 0:
        log.warning("rejected order with qty=%d", qty)
        return
    try:
        1 / (qty - 3)
    except ZeroDivisionError:
        log.exception("pricing failed for qty=%d", qty)   # ERROR + traceback
        return
    log.info("order placed qty=%d", qty)

logging.config.dictConfig({
    "version": 1,
    "disable_existing_loggers": False,
    "formatters": {"short": {"format": "%(levelname)-7s %(name)s: %(message)s"},
                   "full":  {"format": "%(asctime)s %(levelname)s %(name)s "
                                       "%(filename)s:%(lineno)d %(message)s"}},
    "handlers": {"console": {"class": "logging.StreamHandler", "level": "INFO",
                             "formatter": "short", "stream": "ext://sys.stdout"},
                 "file": {"class": "logging.StreamHandler", "level": "DEBUG",
                          "formatter": "full", "stream": file_like}},
    "loggers": {"shop": {"level": "DEBUG", "handlers": ["console", "file"]}},
})
```

Output (console handler; path shortened):

```
INFO    shop.orders: order placed qty=2
WARNING shop.orders: rejected order with qty=0
ERROR   shop.orders: pricing failed for qty=3
Traceback (most recent call last):
  File ".../03_logging_demo.py", line 15, in place_order
    1 / (qty - 3)
    ~~^~~~~~~~~~
ZeroDivisionError: division by zero
--- file handler got 11 lines; DEBUG line present: True
logger chain: shop.orders -> shop -> root
effective level: DEBUG
```

The console got INFO+; the "file" handler (a `StringIO`) also got DEBUG.

Best practice

1. `getLogger(__name__)` per module; libraries never log on the root logger.
2. **Libraries never configure logging** — at most add `logging.NullHandler()`; the application decides destinations.
3. Pass values as arguments — `log.info("user %s", uid)`, not f-strings — so formatting is skipped when disabled and aggregators can group identical templates.
4. In `except`, use `log.exception(...)` (ERROR + traceback) or `exc_info=True`.
5. Never log secrets or personal data.
6. Production: **structured** (JSON) logs with context fields (request ID, user ID), rotated files or stdout collected by the platform ("12-factor": log to stdout, let Docker/Kubernetes collect). Helpers: `structlog`, `python-json-logger`.
7. High-throughput or async apps: `QueueHandler` + `QueueListener` moves slow I/O off the hot path.

Common mistakes

- `basicConfig` in an imported module (silently wins or silently does nothing).
- Adding handlers in a repeatedly called function → duplicated messages. Configure once.
- Messages twice: handlers on both a child and the root with `propagate` True.
- `dictConfig`'s `disable_existing_loggers` defaults to **True**, silencing loggers created earlier — set `False`.

Real-world examples

Django (`LOGGING` setting) and Flask configure via `dictConfig`; `uvicorn`, `requests`/`urllib3`, `boto3` log through named loggers you can tune: `logging.getLogger("urllib3").setLevel(logging.WARNING)`.

In the interview

"Each module gets `getLogger(__name__)`; the application configures handlers and formatters once with `dictConfig`; libraries don't configure logging. I use lazy `%s` arguments, `log.exception` in except blocks, and structured JSON logs to stdout in containers."

- *Why not print?* No levels, routing, per-module control, timestamps or context.
- *Why are my debug logs missing?* Logger or handler level above DEBUG — root defaults to WARNING.

14.5 `argparse` — command-line interfaces

`argparse` turns `sys.argv` (the words typed after the program name) into a validated object, generates `--help`, and prints usage errors.

1. `ArgumentParser(prog=..., description=...)`.
2. `add_argument("title")` — a **positional** (required, identified by position).
3. `add_argument("--priority", type=int, choices=..., default=...)` — an **option**; `type` converts, `choices` restricts.
4. `action="store_true"` for flags, `action="count"` for `-vv`, `action="append"` for repeatable options, `nargs="+"` for one-or-more, `required=True` to force an option.
5. `add_subparsers()` for git-style subcommands (`todo add`, `todo list`).
6. `parse_args()` returns a `Namespace`; on invalid input it prints usage to stderr and exits with status **2** (0 = success, non-zero = failure). `-h/--help` is automatic.
7. `main(argv=None)` accepting a list lets tests call it without a real command line.

Script `04_argparse_cli.py`:

```
p = argparse.ArgumentParser(prog="todo", description="A tiny todo CLI.")
p.add_argument("-v", "--verbose", action="count", default=0)
sub = p.add_subparsers(dest="command", required=True)

add = sub.add_parser("add", help="add a task")
add.add_argument("title")
add.add_argument("--priority", type=int, choices=range(1, 4), default=2)
add.add_argument("--tag", action="append", default=[])

ls = sub.add_parser("list", help="list tasks")
ls.add_argument("--done", action="store_true")
ls.add_argument("--limit", type=int, default=10)
```

Output:

```
$ todo add Buy milk --priority 1 --tag home --tag shop
  parsed: {'verbose': 0, 'command': 'add', 'title': 'Buy milk', 'priority': 1, 'tag': ['home', 'shop']}
$ todo -vv list --done
  parsed: {'verbose': 2, 'command': 'list', 'done': True, 'limit': 10}
$ todo add x --priority 9
usage: todo add [-h] [--priority {1,2,3}] [--tag TAG] title
todo add: error: argument --priority: invalid choice: '9' (choose from '1', '2', '3')
  exit code: 2
usage: todo [-h] [-v] {add,list} ...
```

(`"Buy milk"` was one list item; in a real shell you'd quote it.)

Making it runnable: a `[project.scripts]` entry in `pyproject.toml` (`todo = "todo.cli:main"`) makes `pip install` create a `todo` command, or use `python -m todo` with a `__main__.py` (Chapter 8). End with `raise SystemExit(main())` so the return code reaches the shell.

Library	Style	Good for
`argparse` (stdlib)	imperative, no dependency	scripts, tools that must not need installs
`click`	decorators, composable command groups	larger CLIs (Flask's `flask` command uses it)
`typer`	builds a click CLI from type hints	modern typed CLIs
`sys.argv` directly	manual	only for one or two trivial arguments

Common mistakes: `type=bool` (`bool("False")` is `True` — use `store_true`); hand-parsing `sys.argv` for anything non-trivial; `parse_args()` at import time (breaks tests).

14.6 `os`, `sys`, `subprocess`, `shutil` (+ `pathlib` recap)

Module	Area	Most-used parts
`os`	the operating system	`os.environ`, `os.getenv`, `os.cpu_count()`, `os.getpid()`, `os.walk`, `os.sep`, `os.replace` (atomic rename)
`sys`	the running interpreter	`sys.argv`, `sys.exit()`, `sys.path`, `sys.version_info`, `sys.platform`, `sys.executable`, `sys.stdin/stdout/stderr`, `sys.modules`
`subprocess`	running other programs	`subprocess.run([...], capture_output=True, text=True, check=True, timeout=...)`
`shutil`	high-level file operations	`copy2`, `copytree`, `move`, `rmtree`, `make_archive`, `which`, `disk_usage`
`pathlib`	paths as objects (Chapter 9)	`Path`, `/`, `.read_text()`, `.glob()`, `.rglob()`, `.mkdir(parents=True)`
`tempfile`	throw-away files/dirs	`TemporaryDirectory()`, `NamedTemporaryFile()`

Key rules

1. **Config from the environment**: `os.environ.get("APP_ENV", "dev")` (`os.environ["X"]` raises `KeyError` if missing). Values are always strings.
2. **Paths**: `pathlib.Path` in new code; `os.path` still works.
3. **`subprocess.run`**:
   - Pass a **list** of arguments; never `shell=True` with user text (shell injection: `"file.txt; rm -rf ~"`).
   - `capture_output=True, text=True` → `result.stdout` as `str`.
   - `check=True` raises `CalledProcessError` on non-zero exit; `timeout=` raises `TimeoutExpired`.
   - `sys.executable` = "the same Python that runs me".
   - `os.system` / `os.popen` are older, less safe.
4. `shutil.rmtree` is permanent — double-check the path. `os.replace(tmp, final)` gives atomic "write then swap" updates.
5. `sys.exit(0)` success, non-zero failure (it raises `SystemExit`, so `finally` still runs).

Script `05_os_sys_subprocess_shutil.py` (key parts):

```
with tempfile.TemporaryDirectory() as tmp:
    root = Path(tmp)
    (root / "src" / "pkg").mkdir(parents=True)
    (root / "src" / "pkg" / "a.py").write_text("print('hi')\n")
    shutil.copytree(root / "src", root / "backup")
    archive = shutil.make_archive(str(root / "bundle"), "zip", root / "src")
    shutil.rmtree(root / "backup")
    r = subprocess.run([sys.executable, str(root / "src" / "pkg" / "a.py")],
                       capture_output=True, text=True, check=True, timeout=10)
```

Output:

```
python      : (3, 14) | platform: darwin
argv[0]     : 05_os_sys_subprocess_shutil.py
HOME set    : True | APP_ENV = dev (default)
cpu_count>0 : True
copied      : ['backup/notes.txt', 'backup/pkg', 'backup/pkg/a.py']
py files    : ['a.py', 'a.py']
archive     : bundle.zip True
after rmtree: ['bundle.zip', 'src']
child stdout: hi | returncode 0
failed child: returncode 3
which python: True
disk free GB: True
```

Common mistakes

- `subprocess.run("ls " + user_input, shell=True)` — injection.
- Ignoring the return code (no `check=True`).
- `os.chdir` in library code — changes the working directory for the whole process.
- Concatenating paths with `"/"` instead of `Path` / `os.path.join`.

In the interview

"`subprocess.run` with a list, `check=True`, a timeout, never `shell=True` on user input. Config from `os.environ.get` with defaults, paths as `pathlib.Path`, scratch work in `tempfile.TemporaryDirectory`."

14.7 `random` vs `secrets`

- `random` — **pseudo-random** (deterministic formula, Mersenne Twister): fast, reproducible with a **seed**, statistically good, but **predictable** after enough outputs.
- `secrets` (3.6+) — the OS's cryptographically secure source (`os.urandom`): unpredictable, not reproducible.

Rule: `random` for simulations, games, sampling, tests; `secrets` for passwords, tokens, keys, reset links.

`random`: `random.Random(42)` (own seeded generator — other code can't disturb it, unlike global `random.seed`), `randint(a, b)` (both ends inclusive), `random()` in [0, 1), `uniform`, `choice`, `choices(pop, weights=, k=)` (with replacement), `sample(pop, k)` (without), `shuffle` (in place), `gauss(mu, sigma)`.

`secrets`: `token_bytes(n)`, `token_hex(n)` (2n hex chars), `token_urlsafe(n)` (≈1.3 × n URL-safe chars), `choice(seq)`, `randbelow(n)`, and `compare_digest(a, b)` — constant-time comparison against **timing attacks** (`==` stops at the first wrong character, leaking how many were right).

Output of `06_random_secrets_math_stats.py` (first part):

```
randint      : [6, 1, 1, 6, 3]
choice       : red
sample       : [28, 17, 94]
shuffle      : [7, 2, 5, 6, 0, 3, 4, 9, 8, 1]
weighted     : ['A', 'A', 'A', 'B', 'B', 'A', 'A', 'A']
gauss        : 1.1491
token_urlsafe: 22 chars
token_hex(8) : 16 hex chars
compare_digest: True
```

Mistakes: `random` for security tokens; `==` on secrets (use `compare_digest`). For NumPy use `numpy.random.default_rng(seed)` (ML handbook).

14.8 `math`, `statistics`, `decimal`, `fractions`

`math` = fast C functions on single numbers. `statistics` = descriptive statistics on small-to-medium lists — correct and readable, slower than NumPy/pandas on big arrays.

Need	Function	Note
Compare floats	`math.isclose(a, b)`	default relative tolerance 1e-9; `0.1 + 0.2 == 0.3` is `False`
Rounding	`math.floor`, `math.ceil`, `math.trunc`, `round`	`round` uses **banker's rounding** (halves to even): `round(2.5) == 2`, `round(3.5) == 4`
Roots, distances	`sqrt`, `isqrt` (exact int), `hypot`, `dist`	
Counting	`comb(n, k)`, `perm(n, k)`, `factorial`	5 choose 2 = 10
Number theory	`gcd`, `lcm`	`lcm` since 3.9
Logs, constants	`log(x, base)`, `log2`, `log10`, `exp`, `pi`, `e`, `inf`, `nan`	
Accurate sums	`math.fsum`	exact float sum
Averages	`statistics.mean`, `fmean`, `median`, `mode`, `multimode`	
Spread	`pstdev` / `pvariance` (population — all values), `stdev` / `variance` (sample; divides by n−1)	
Quartiles	`statistics.quantiles(data, n=4)`	
Relations	`correlation`, `covariance`, `linear_regression`	3.10+

Output (second part):

```
isclose      : False True
floor/ceil   : -3 -2 2 4
sqrt, hypot  : 4.0 5.0
comb, perm   : 10 20 120
gcd, lcm     : 6 12
log          : 3.0 3.0 2.71828
fsum vs sum  : 1.0 1.0
mean/median  : 5 4.5
mode         : 4
pstdev/stdev : 2.0 2.1381
quantiles    : [4.0, 4.5, 6.5]
correlation  : 0.7746
linear_reg   : LinearRegression(slope=0.6, intercept=2.2)
```

Worked check: `[2, 4, 4, 4, 5, 5, 7, 9]` has mean 5; squared deviations sum to 9+1+1+1+0+0+4+16 = 32. Population variance 32/8 = 4 → `pstdev` 2.0; sample variance 32/7 ≈ 4.571 → `stdev` ≈ 2.1381.

`fsum vs sum`: since 3.12 built-in `sum()` of floats uses compensated (Neumaier) summation, so `sum([0.1] * 10)` is `1.0`; on 3.11 and earlier it was `0.9999999999999999`. `math.fsum` remains the exactly-rounded option.

`decimal` and `fractions` — exact arithmetic

Floats are binary, so `0.1` isn't exact and money sums drift:

```
from decimal import Decimal, ROUND_HALF_UP
from fractions import Fraction

Decimal("0.1") + Decimal("0.2")                                  # Decimal('0.3') - exact
Decimal(0.1)                                                     # 0.1000000000000000055511... (built from the
float!)
Decimal("2.675").quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)   # Decimal('2.68')
round(2.675, 2)                                                  # 2.67 - the float is really 2.67499999...
Fraction(1, 3) + Fraction(1, 6)                                  # Fraction(1, 2)
```

- **`Decimal`**: base-10, configurable precision (28 significant digits default), explicit rounding modes. Use for money; build from **strings**, not floats.
- **`Fraction`**: exact numerator/denominator — exact ratios, slow if numbers grow.
- Both are much slower than `float`; use only where exactness matters.

Common mistakes

- `math.floor(-2.5)` is `-3` (towards −∞); `int(-2.5)` is `-2` (towards zero).
- `==` on floats — use `isclose`, or `Decimal` for money.
- Mixing up `stdev` (sample) and `pstdev` (population).

14.9 A toolbox of small modules: `copy`, `pprint`, `enum`, `uuid`, `hashlib`, `json`, `csv`

`copy` — shallow vs deep

- `copy.copy(x)` — **shallow**: new outer container, shared inner objects.
- `copy.deepcopy(x)` — recursive (handles cycles with a memo dict); slower; customise with `__copy__` / `__deepcopy__`.
- `copy.replace(obj, **changes)` (3.13+) — modified copy of a dataclass, namedtuple, datetime, etc.

```
shallow sees change: ['a', 'b', 'c'] | deep: ['a', 'b']
```

The shallow copy shares the `hosts` list, so appending through `cfg` shows in it.

`pprint` — readable nested data

`pprint.pp(obj, width=50)` (3.8+) prints nested structures across lines in insertion order (`pprint.pprint` sorts dict keys by default); `pprint.pformat` returns the string.

```
{'service': 'api',
 'replicas': 3,
 'env': {'DB_URL': 'postgres://db:5432/app',
         'DEBUG': False},
 'ports': [80, 443],
 'owners': ['platform-team', 'sre']}
```

`enum` — named constants

See Chapter 10. `Enum` for a closed set, `StrEnum` (3.11+) whose members *are* strings (great for JSON/DBs), `IntEnum`, `Flag` for combinable bits, `auto()` (for `StrEnum`, the lower-cased member name):

```
StrEnum: active BANNED | Enum: Color.GREEN ['RED', 'GREEN']
```

`uuid` — unique identifiers

A **UUID** is a 128-bit, 36-character identifier generated anywhere without coordination.

Function	Based on	Use
`uuid4()`	random	the default for IDs
`uuid5(namespace, name)` / `uuid3`	SHA-1 / MD5 hash of a name	same input → same ID
`uuid1()`	time + MAC address	leaks the machine's MAC; avoid
`uuid7()` (3.14+)	Unix time + random	time-ordered, so database indexes stay compact

```
uuid4 version: 4 | uuid5 is deterministic: 6639460f-3425-5329-8097-a58f06127860
```

`hashlib` — cryptographic hashes

A **hash function** maps any bytes to a fixed-size **digest** (a short fingerprint); same input → same digest, and a good one makes collisions infeasible to find.

```python
hashlib.sha256(b"hello").hexdigest()      # '2cf24dba5fb0a30e...'
h = hashlib.sha256()
for chunk in (b"hel", b"lo"):              # hash large files chunk by chunk
    h.update(chunk)
key = hashlib.pbkdf2_hmac("sha256", b"s3cret", salt, 100_000)
```

Output:

```
sha256('hello'): 2cf24dba5fb0a30e ...
streamed equal : True
pbkdf2 key len : 32
```

- SHA-256 (or BLAKE2, `hashlib.blake2b`) for integrity checks, cache keys, content addressing (Git uses SHA-1 and is adding SHA-256).
- MD5 and SHA-1 are broken for security; OK only for non-security checksums (detecting accidental corruption).
- **Never store passwords with a fast hash.** Use a slow, salted **KDF** (key-derivation function; a **salt** is per-user random data so equal passwords hash differently): `hashlib.scrypt` or `pbkdf2_hmac` with `secrets.token_bytes(16)` salt and many iterations — or `argon2-cffi` / `bcrypt`. (The demo's fixed salt is for reproducibility only.)
- Message authentication (proving a message came from a holder of a shared key, unaltered): `hmac.new(key, msg, "sha256")`, compared with `hmac.compare_digest`.
- Built-in `hash()` is **not** stable — string hashes are randomised per process.

`json` and `csv` — recap (Chapter 9)

- `json.dumps(obj, default=str, sort_keys=True, indent=2)` — `default` handles unsupported types (dates, sets, Decimal); tuples become lists; `json.loads` returns plain dicts/lists.
- `csv.DictWriter` / `DictReader` handle quoting (`"Li, Wei"` stays one field). Open files with `newline=""`.

```
json: {"id": 7, "tags": ["a", "b"], "when": "2026-09-28"} | back: ['a', 'b']
csv text: ['name,city', 'Asha,Pune', '"Li, Wei",Oslo']
csv read: {'name': 'Li, Wei', 'city': 'Oslo'}
```

More small modules worth knowing

Module	Does	Example
`tomllib` (3.11)	Read TOML config (read-only)	`tomllib.load(open("pyproject.toml", "rb"))`
`zipfile`, `tarfile`, `gzip`	Archives and compressed files	`gzip.open("log.gz", "rt")` streams compressed text
`base64`	Encode bytes as ASCII text	`base64.b64encode(b"hi")` → `b'aGk='`
`textwrap`	Wrap/indent/dedent paragraphs	`textwrap.dedent(help_text)`
`shlex`	Split a command line like a shell	`shlex.split('git commit -m "fix bug"')` → `['git', 'commit', '-m', 'fix bug']`
`difflib`	Diffs and "did you mean" matches	`difflib.get_close_matches("pyhton", ["python", "java"])`
`getpass`	Read a password without echo	`getpass.getpass()`
`string`	Constants and simple templates	`string.ascii_lowercase`, `string.Template("$name")`
`timeit`, `cProfile`, `tracemalloc`	Timing, profiling (where time is spent), memory tracing	Chapter 17

14.10 Pointer: `collections`, `itertools`, `functools`

Module	Highlights	Chapter
`collections`	`Counter`, `defaultdict`, `deque`, `namedtuple`, `OrderedDict`, `ChainMap`	6
`itertools`	`chain`, `islice`, `groupby`, `product`, `permutations`, `combinations`, `accumulate`, `batched` (3.12), `pairwise`	11
`functools`	`lru_cache` / `cache`, `partial`, `reduce`, `wraps`, `total_ordering`, `singledispatch`, `cached_property`	7, 11, 12
`heapq`, `bisect`	priority queues, binary search in sorted lists	5, 6
`dataclasses`, `abc`, `contextlib`	records, abstract classes, context managers	10, 12

14.11 `sqlite3` — an embedded SQL database

SQLite is a full SQL database in a single file (or memory), running inside your process — no server. `sqlite3` is the built-in driver (DB-API 2.0, PEP 249, the same interface as PostgreSQL/MySQL drivers).

1. `con = sqlite3.connect("app.db")` (or `":memory:"`).
2. `con.execute(sql, params)`; `executemany` per row.
3. **Always use placeholders** (`?` or `:name`) — the driver escapes values, preventing **SQL injection**. Never build SQL with f-strings from user input.
4. `with con:` wraps a **transaction**: commit on success, rollback if an exception escapes (it does *not* close the connection — call `con.close()`).
5. `con.row_factory = sqlite3.Row` → columns by name.
6. Results: `.fetchone()`, `.fetchall()`, or iterate the cursor.

Script `08_sqlite_demo.py`:

```
con = sqlite3.connect(":memory:")
con.row_factory = sqlite3.Row
con.execute("""CREATE TABLE expense (
                id INTEGER PRIMARY KEY,
                category TEXT NOT NULL,
                amount REAL NOT NULL CHECK (amount > 0))""")
with con:
    con.executemany("INSERT INTO expense (category, amount) VALUES (?, ?)", rows)

total = con.execute("SELECT SUM(amount) AS s FROM expense WHERE category = ?",
                    (cat,)).fetchone()["s"]

try:
    with con:                          # both inserts, or neither
        con.execute("INSERT INTO expense (category, amount) VALUES ('gift', 5)")
        con.execute("INSERT INTO expense (category, amount) VALUES ('bad', -1)")
except sqlite3.IntegrityError as e:
    print("rolled back:", e)
```

Output:

```
food total: 19.75
  travel  n=1 sum=40.0
  food    n=2 sum=19.75
  books   n=1 sum=18.0
rolled back: CHECK constraint failed: amount > 0
rows now: 4
```

The valid `gift` insert rolled back with the invalid one — atomicity.

When not to use it: many concurrent writers (SQLite allows one writer at a time; WAL — write-ahead log — mode helps readers) or a database shared by several servers — use PostgreSQL/MySQL. Excellent for apps, tests, caches, CLIs, prototypes, small-to-medium sites. It ships in every Android/iOS device, Firefox and Chrome, many desktop apps, and is Django's default dev database.

Common mistakes: f-string SQL (injection); forgetting to commit; `(cat)` instead of `(cat,)` — parameters must be a sequence.

14.12 `urllib` and `http` (brief)

- `urllib.parse` — split, build, encode URLs: `urlparse`, `parse_qs`, `urlencode`, `urljoin`, `quote`.
- `urllib.request` — basic HTTP client: `urlopen(Request(url, headers=..., data=...), timeout=...)`.
- `urllib.error` — `HTTPError` (server replied 4xx client error / 5xx server error) and `URLError` (couldn't connect).
- `http.HTTPStatus` — named codes (`HTTPStatus.NOT_FOUND == 404`).
- `http.server` — minimal server (`python -m http.server 8000` shares a folder); **not for production**.

`09_urllib_http.py` starts a local server on `127.0.0.1` (a port is the numbered endpoint a service listens on; port 0 = any free port) in a background thread, then calls it:

```
u = urlparse("https://api.example.com:8443/v1/users?id=7&sort=name#top")
print("parsed :", u.scheme, u.hostname, u.port, u.path, parse_qs(u.query))

req = Request(f"{base}/greet?{urlencode({'name': 'Asha'})}",
              headers={"Accept": "application/json"})
with urlopen(req, timeout=5) as resp:
    print("GET    :", resp.status, resp.headers["Content-Type"], json.load(resp))
```

Output:

```
parsed : https api.example.com 8443 /v1/users {'id': ['7'], 'sort': ['name']}
encode : q=type+hints&page=2
urljoin: https://x.org/docs/b
GET    : 200 application/json {'message': 'hello Asha'}
error  : 404 Not Found
```

When not to use it: real clients need connection pooling (reusing open connections across requests), JSON/auth/retry helpers and async, which `urllib.request` lacks — use **requests** (synchronous, most popular) or **httpx** (sync + async, HTTP/2). Servers: a framework (FastAPI, Flask, Django) behind uvicorn/gunicorn.

Mistakes: no `timeout=` (a stuck server hangs you forever); hand-built query strings instead of `urlencode`.

14.13 When to reach for third-party packages

How to decide

1. **Is the stdlib good enough?** For scripts and tools that must run anywhere without installs (CI helpers, system scripts), prefer it.
2. **Is there a clear community standard** — de-facto, well maintained, widely used? Then the dependency is usually worth it.
3. **Cost:** install size, security updates, licence, supply-chain risk (compromised or typo-squatted packages). Pin versions with a lock file (Chapter 8).

Need	stdlib	Common third-party choice	Why switch
HTTP client	`urllib.request`	**requests**, **httpx**	sessions/pooling, JSON, auth, retries, async (httpx)
CLI	`argparse`	**click**, **typer**	composable commands, less boilerplate, type-hint driven (typer)
Terminal output	`print`, `logging`	**rich**	colours, tables, progress bars, pretty tracebacks and logging handler
Dates	`datetime`, `zoneinfo`	`python-dateutil`, `pendulum`, `arrow`	month arithmetic, fuzzy parsing
Validation	dataclasses	**pydantic**	runtime validation (Chapter 13)
Data tables	`csv`, `statistics`	**pandas**, **polars**	vectorised analysis at scale
Numerics	`math`, `random`	**NumPy**	fast arrays
Testing	`unittest`	**pytest**	simpler tests, fixtures, plugins (Chapter 15)
Env config	`os.environ`	`python-dotenv`, `pydantic-settings`	`.env` files, typed settings
Passwords	`hashlib.scrypt`	`argon2-cffi`, `bcrypt`	modern password hashing
Database	`sqlite3`	**SQLAlchemy**, `psycopg`	ORM / PostgreSQL

A taste (needs `pip install requests rich click` — not run here):

```python
import requests
r = requests.get("https://api.github.com/repos/python/cpython", timeout=10)
r.raise_for_status()                # HTTPError on 4xx/5xx
print(r.json()["stargazers_count"])

from rich.console import Console
from rich.table import Table
t = Table("service", "status"); t.add_row("api", "[green]up[/]")
Console().print(t)                  # a coloured, boxed table

import click
@click.command()
@click.argument("name")
@click.option("--count", default=1, type=int)
def hello(name: str, count: int) -> None:
    for _ in range(count):
        click.echo(f"Hello {name}")
```

14.14 Chapter self-check

1. Naive vs aware datetime? → Aware has `tzinfo`; store aware UTC, convert with `astimezone(ZoneInfo(...))` for display.
2. Does `aware + timedelta(hours=1)` account for DST jumps? → No, wall-clock arithmetic; go via UTC.
3. Timer for measuring code? → `time.perf_counter()`.
4. Regex function for validating a whole string? → `re.fullmatch`.
5. Why `log.info("x=%s", x)` instead of an f-string? → Formatting skipped when disabled; aggregators group by template.
6. Root logger default level; which call logs a traceback; why duplicates? → WARNING; `log.exception` in `except`; handlers on several loggers in a propagating chain.
7. What should a library do about logging configuration? → Nothing beyond `getLogger(__name__)` (optionally `NullHandler`).
8. argparse error exit code? → 2.
9. Safest way to run a command with user input? → `subprocess.run([...list...], check=True, timeout=...)`, no `shell=True`.
10. `random` or `secrets` for a password-reset token? → `secrets`.
11. `round(2.5)`? → 2 (banker's rounding).
12. Shallow copy of `{"a": [1]}`, append to original's list — does the copy change? → Yes.
13. How to store passwords? → Salted slow KDF (scrypt/pbkdf2/argon2), never plain SHA-256.
14. How to pass values into SQL safely? → `?` placeholders with a parameters tuple.
15. Why prefer requests/httpx over urllib.request? → Pooling, JSON/auth/retries, async (httpx), nicer API.

Chapter TL;DR

- **Time:** aware datetimes in UTC, `ZoneInfo` for display, ISO 8601 on the wire; `timedelta` has no months; `perf_counter` for durations, `monotonic` for timeouts.
- **re:** raw strings, named groups, `VERBOSE`, `finditer`, `fullmatch`, `sub` with a function; beware catastrophic backtracking; don't regex JSON/HTML/CSV.
- **logging:** `getLogger(__name__)`, configure once with `dictConfig`, lazy args, `log.exception`, libraries don't configure, structured logs in production.
- **argparse:** positionals/options/subcommands, `main(argv=None)`, exit code 2 on errors.
- **OS:** `os.environ.get`, `sys.argv` / `sys.exit`, `subprocess.run([...], check=True, timeout=...)`, `shutil` for trees, `pathlib` for paths, `tempfile` for scratch space.
- **Numbers:** `random` (seeded `Random`) vs `secrets`; `math.isclose`, banker's `round`; `statistics`; `Decimal` (from strings) for money.

- **Toolbox:** `deepcopy`, `pprint.pp`, `StrEnum`, `uuid4`/`uuid7`, `hashlib` (SHA-256; KDFs for passwords), `json` `default=`, `csv.Dict*`.
- **sqlite3:** placeholders, `with con:` transactions, `sqlite3.Row`.
- **HTTP:** `urllib.parse` always; `urllib.request` for quick scripts; requests/httpx for real work.
- **Third-party** when a community standard clearly beats the stdlib — then pin it.

Chapter 15 — Testing and Debugging

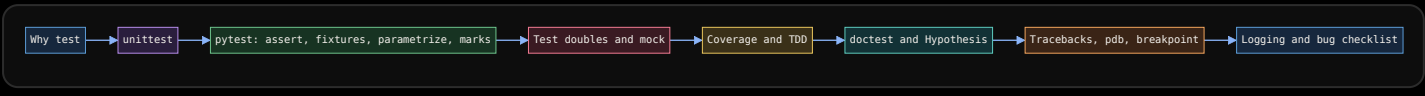
What you will learn. Why automated tests pay for themselves and how to write them: `unittest`, then **pytest** — plain `assert`, fixtures, `parametrize`, `tmp_path`, marks, `conftest.py`. Isolating code from slow or external dependencies with **test doubles** and `unittest.mock` (`Mock`, `patch`, `side_effect`, autospec). **Coverage**, the red-green-refactor **TDD** loop, **doctest**, and **property-based testing** with Hypothesis. Then debugging: tracebacks, `pdb` and `breakpoint()`, logging instead of `print`, and a checklist of the most common Python bugs.

Prerequisites. Functions, classes, exceptions (Chapters 7, 9, 10). Logging basics: Chapter 14.

Code. `chapters/15-testing-and-debugging/code/` : a small module `inventory.py` plus its tests. Install once, run from that folder:

```
pip install pytest pytest-cov hypothesis
cd chapters/15-testing-and-debugging/code
pytest -q                      # 31 passed, 1 skipped, 1 xfailed
python debugging_demo.py
```

("xfailed" = a test *expected* to fail did fail — §15.4.5.)



15.1 Why write tests?

What it is

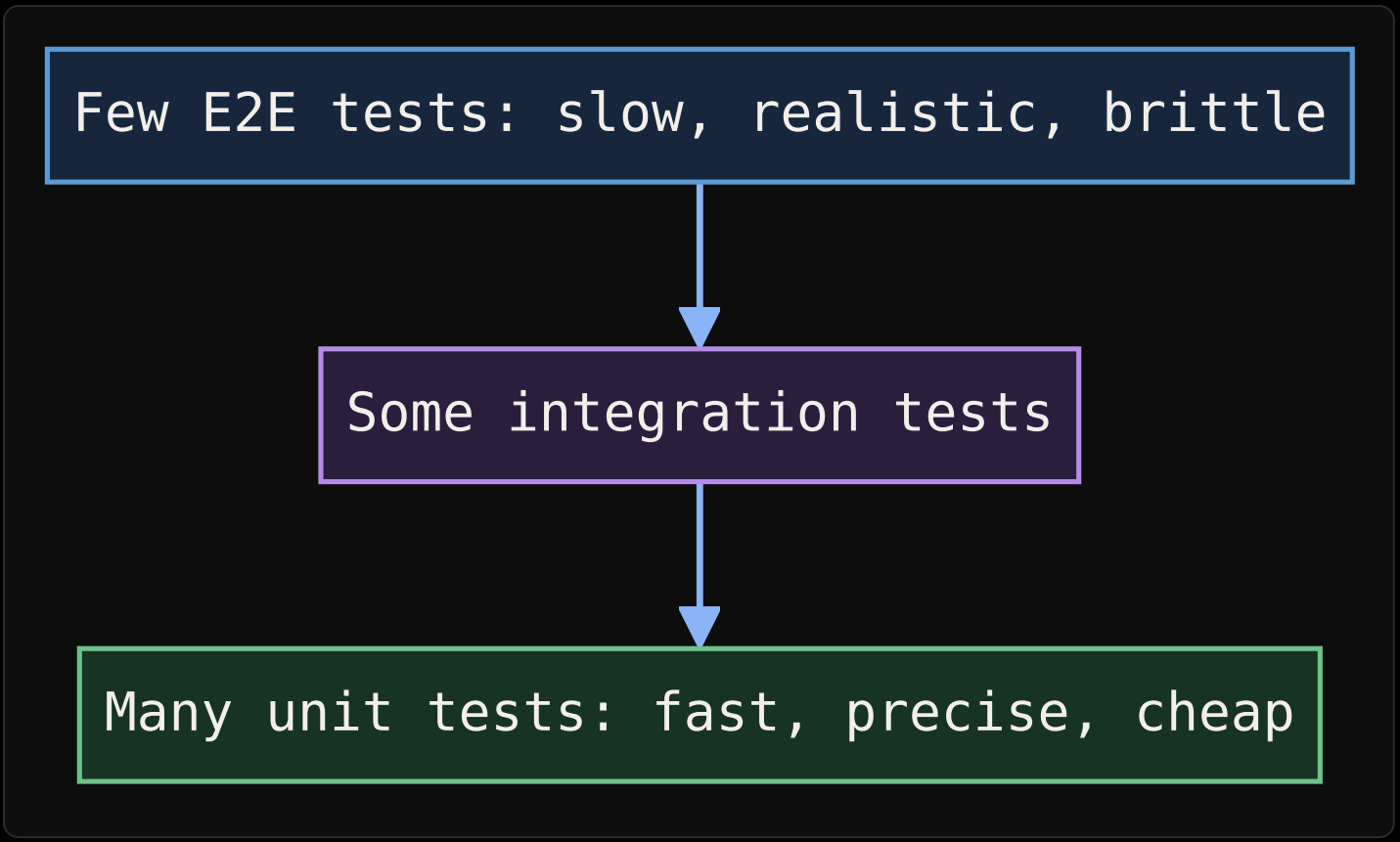
An **automated test** runs your code with known inputs and checks the result.

Term	Meaning
Unit test	tests one small piece (a function, a class) in isolation, in milliseconds
Integration test	tests several pieces together (your code + a real database or file system)
End-to-end (E2E) test	drives the whole system like a user (HTTP calls, browser)
Test suite	all the tests of a project
Regression	a bug that comes back, or a change that breaks something that used to work
SUT	*system under test* — the code being tested
Arrange–Act–Assert	the usual three-part shape of a test: set up, do the thing, check the outcome

Why it matters

1. **Confidence to change code** — refactoring without tests is guessing; with a good suite you know in seconds if anything broke.
2. **Bugs are cheaper earlier** — minutes on your laptop vs incidents, data fixes and lost trust in production.
3. **Executable documentation** — tests show exactly how code is meant to be used.
4. **Better design** — hard-to-test code is usually too coupled; tests push you to small functions with explicit inputs.
5. Python has **no compiler** to catch a typo in a rarely-run branch — only running the code finds it (type checkers help, Chapter 13).

How it works — the test pyramid



Mostly fast unit tests, fewer integration tests, a handful of E2E tests for critical paths — all run automatically in **CI** (continuous integration, e.g. GitHub Actions runs `pytest` on every push and pull request).

What makes a good test

- **Fast** (milliseconds), **isolated** (independent of other tests and order), **repeatable** (no real clock, network or unseeded randomness), **self-checking** (pass/fail, no human reading output), **focused** (one behaviour, named).
- Test **behaviour, not implementation** — what a function returns or changes, not which private helpers it calls — so refactoring doesn't break tests.

Real-world examples

CPython's suite (`python -m test`) uses `unittest`; NumPy, pandas, pip and most modern projects use pytest; Django ships a `unittest`-based runner. Their pull requests merge only when CI is green.

15.2 The module under test

Every example tests `inventory.py` (abridged — `# ...` marks omitted lines):

```python
class OutOfStock(Exception):
    """Raised when removing more units than are available."""

def slugify(name: str) -> str:
    """Lower-case, keep letters/digits, join words with '-'.

    >>> slugify("Hello, World!")
    'hello-world'
    """
    words = re.findall(r"[a-z0-9]+", name.lower())
    return "-".join(words)

def apply_discount(price: float, percent: float) -> float:
    if not 0 <= percent <= 100:
        raise ValueError(f"percent must be 0-100, got {percent}")
    return round(price * (1 - percent / 100), 2)

@dataclass
class Inventory:
    stock: dict[str, int] = field(default_factory=dict)

    def add(self, sku: str, qty: int) -> None:
        if qty <= 0:
            raise ValueError("qty must be positive")
        self.stock[sku] = self.stock.get(sku, 0) + qty

    def remove(self, sku: str, qty: int) -> None:
        have = self.stock.get(sku, 0)
        if qty > have:
            raise OutOfStock(f"{sku}: want {qty}, have {have}")
        self.stock[sku] = have - qty

    def low_stock(self, threshold: int) -> list[str]: ...   # built with TDD in §15.8
    def total_units(self) -> int: ...
    def save(self, path: Path) -> None:
        path.write_text(json.dumps(self.stock, sort_keys=True))
    @classmethod
    def load(cls, path: Path) -> "Inventory": ...

def price_in(currency: str, usd: float, client: RateClient) -> float:
    """Convert a USD price using an external rate service (mocked in tests)."""
    rate = client.get_rate(currency)
    if rate <= 0:
        raise ValueError("bad rate")
    return round(usd * rate, 2)
```

In `price_in` the rate service is **passed in** (dependency injection) rather than created inside — which makes it trivial to test with a fake (§15.5).

15.3 `unittest` basics

What it is

The standard library's test framework, modelled on Java's JUnit (the "xUnit" family). Tests are methods of a `unittest.TestCase` subclass.

How it works

1. `class TestX(unittest.TestCase)`; each method named `test*` is one test.
2. `setUp()` runs before **each** test, `tearDown()` after each; `setUpClass` / `tearDownClass` (class methods) once per class.
3. **Assertions** (checks that must hold or the test fails): `assertEqual`, `assertTrue`, `assertIn`, `assertIsNone`, `assertAlmostEqual` (7 decimal places by default), `assertRaises` (as a context manager), `assertLogs`.

4. `@unittest.skip(reason)` / `skipIf(cond, reason)`; `@unittest.expectedFailure` for known bugs.
5. Run: `python -m unittest` (auto-discovers `test*.py`), `python -m unittest -v test_inventory_unittest`, or `unittest.main()` in the file.

```
class TestInventory(unittest.TestCase):
    def setUp(self):                      # runs before EACH test method
        self.inv = Inventory()
        self.inv.add("mug", 5)

    def test_add_accumulates(self):
        self.inv.add("mug", 3)
        self.assertEqual(self.inv.stock["mug"], 8)

    def test_remove_too_many_raises(self):
        with self.assertRaises(OutOfStock):
            self.inv.remove("mug", 99)

    def test_discount(self):
        self.assertAlmostEqual(apply_discount(19.99, 10), 17.99)

    @unittest.skip("demo of skipping")
    def test_skipped(self):
        self.fail("never runs")
```

Output of `python test_inventory_unittest.py`:

```
test_add_accumulates (__main__.TestInventory.test_add_accumulates) ... ok
test_discount (__main__.TestInventory.test_discount) ... ok
test_remove_too_many_raises (__main__.TestInventory.test_remove_too_many_raises) ... ok
test_skipped (__main__.TestInventory.test_skipped) ... skipped 'demo of skipping'

----------------------------------------------------------------------
Ran 4 tests in 0.000s

OK (skipped=1)
```

Trade-offs — unittest vs pytest

	unittest	pytest
Install	built in	`pip install pytest`
Test shape	class + `self.assertX` methods	plain functions + `assert`
Setup	`setUp` / `tearDown`	fixtures (reusable, composable, scoped)
Data-driven tests	`self.subTest(...)` loop	`@pytest.mark.parametrize`
Failure messages	from the assert method	rewritten `assert` shows values and diffs
Plugins (add-on packages)	few	large ecosystem (coverage, xdist for parallel runs, asyncio, django…)
Runs the other's tests	—	yes, pytest runs `unittest` tests unchanged

When to use it

When you can't add dependencies (stdlib-only tools, CPython itself) or in an existing unittest codebase. Otherwise pytest is the default — and it runs unittest tests, so you can migrate gradually.

15.4 pytest

What it is

pytest, the most widely used Python test framework: plain functions with plain `assert`; it discovers, runs and explains failures in detail.

How it works — discovery and running

1. Collects files `test_*.py` / `*_test.py`, functions `test_*`, classes `Test*` (without `__init__`).
2. Common commands:

Command	Does
`pytest`	run everything under the current folder
`pytest -q` / `-v`	quieter / one line per test
`pytest test_x.py::test_name`	one test
`pytest -k "discount and not bad"`	tests whose names match the expression
`pytest -m "not slow"`	select by mark
`pytest -x` / `--maxfail=2`	stop after the first/second failure
`pytest --lf`	rerun only last-failed tests
`pytest -s`	show `print` output (normally captured)
`pytest --pdb`	open the debugger at a failure
`pytest --durations=5`	list the slowest tests

3. Settings: `pyproject.toml` `[tool.pytest.ini_options]` (`testpaths = ["tests"]`, `markers = ["slow: ..."]`, `addopts = "-q"`).

15.4.1 Plain `assert` and assertion rewriting

pytest rewrites `assert` in test files so a failure prints the values and a diff (§15.8 shows one). Helpers:

- `pytest.raises(Exc, match="regex")` — the block must raise `Exc` with a message matching the regex (§14.3).
- `pytest.approx(x)` — float comparison with tolerance (relative 1e-6 default): `0.1 + 0.2 == pytest.approx(0.3)`.
- `pytest.warns(Warning)` — the block must emit a warning.

15.4.2 Fixtures

A **fixture** prepares something a test needs (an object, temp file, DB connection) and optionally cleans up. A test requests it by naming it as a parameter; pytest calls it and passes the result (**dependency injection**).

```python
# conftest.py
@pytest.fixture
def inv():
    """A fresh inventory with two products for every test that asks for it."""
    inventory = Inventory()
    inventory.add("mug", 5)
    inventory.add("pen", 10)
    return inventory

# test_inventory_pytest.py
def test_total_units(inv):
    assert inv.total_units() == 15

def test_remove_reduces_stock(inv):
    inv.remove("pen", 4)
    assert inv.stock == {"mug": 5, "pen": 6}

def test_remove_too_many_raises(inv):
    with pytest.raises(OutOfStock, match="want 6, have 5"):
        inv.remove("mug", 6)
```

1. **Fresh per test by default** (`scope="function"`): `test_remove_reduces_stock` changes its `inv`; the next test still gets the original.
2. **Scopes** (how long one value is shared): `function` (default), `class`, `module`, `package`, `session` (one whole run). Wider scopes for expensive read-only resources (one DB engine per session).
3. **Teardown with `yield`** — code after `yield` runs after the test, even if it fails:

```python
@pytest.fixture
def audit_log(tmp_path):
    f = (tmp_path / "audit.txt").open("w")
    yield f          # the test runs here
    f.close()        # cleanup
```

4. Fixtures can use other fixtures (`audit_log` uses `tmp_path`).
5. `autouse=True` applies a fixture to every test in scope without naming it — sparingly.
6. **Built-in fixtures**: `tmp_path` (unique temp directory as a `Path`), `monkeypatch` (temporarily set attributes, env vars, dict items; auto-undone), `capsys` (capture stdout/stderr), `caplog` (capture log records), `request` (info about the requesting test).

15.4.3 `tmp_path` — real files, safely

```python
def test_save_and_load_roundtrip(inv, tmp_path):
    path = tmp_path / "stock.json"          # unique temp dir per test
    inv.save(path)
    assert Inventory.load(path) == inv
    assert path.read_text() == '{"mug": 5, "pen": 10}'
```

Each test gets its own directory — no collisions, nothing written into your project. Often better than mocking the file system: it tests the real code path and is still fast.

15.4.4 `parametrize` — one test, many cases

```
@pytest.mark.parametrize(
    ("price", "percent", "expected"),
    [(100, 0, 100), (100, 25, 75), (19.99, 10, 17.99), (50, 100, 0)],
)
def test_discount(price, percent, expected):
    assert apply_discount(price, percent) == pytest.approx(expected)

@pytest.mark.parametrize(("raw", "slug"), [
    ("Blue Mug", "blue-mug"),
    ("  --Hello__World-- ", "hello-world"),
    ("", ""),
], ids=["simple", "punctuation", "empty"])
def test_slugify(raw, slug):
    assert slugify(raw) == slug
```

Each tuple is a **separate test** with its own ID, so one failure doesn't hide others. Include edge cases: empty input, zero, boundaries (0 and 100), negatives, Unicode (`é`, `中`, emoji).

15.4.5 Marks

A **mark** labels a test to change how it's selected or run.

Mark	Effect
`@pytest.mark.skip(reason=...)`	never run
`@pytest.mark.skipif(sys.platform == "win32", reason=...)`	run only when the condition is false
`@pytest.mark.xfail(reason=..., strict=True)`	expected to fail; with `strict=True`, an unexpected *pass* is reported as a failure, so you notice when the bug is fixed
`@pytest.mark.parametrize(...)`	data-driven cases
`@pytest.mark.slow` (custom)	a label you select with `-m`; register it to avoid "unknown mark" warnings

```
@pytest.mark.slow
def test_slow_path():
    time.sleep(0.2)
    assert True

@pytest.mark.xfail(reason="float rounding: 2.675 is stored as 2.67499...; use Decimal",
                   strict=True)
def test_known_bug():
    assert apply_discount(2.675, 0) == 2.68
```

The xfail documents a real float issue: `2.675` is stored as slightly less, so `round(2.675, 2)` is `2.67`; the fix would be `decimal.Decimal`.

15.4.6 `conftest.py`

pytest auto-loads `conftest.py`. Its fixtures and hooks (specially named functions pytest calls, e.g. `pytest_configure`) are available to every test in that directory and below — **no import needed**; nested folders can have their own. Ours defines `inv` and registers `slow`:

```
def pytest_configure(config):
    config.addinivalue_line("markers", "slow: marks a test as slow")
```

(Real projects register markers in `pyproject.toml`.)

Output

`pytest -v test_inventory_pytest.py`:

```
test_inventory_pytest.py::test_total_units PASSED                          [  6%]
test_inventory_pytest.py::test_remove_reduces_stock PASSED                 [ 12%]
test_inventory_pytest.py::test_remove_too_many_raises PASSED               [ 18%]
test_inventory_pytest.py::test_discount[100-0-100] PASSED                  [ 25%]
...
test_inventory_pytest.py::test_discount_rejects_bad_percent[101] PASSED    [ 56%]
test_inventory_pytest.py::test_slugify[simple] PASSED                      [ 62%]
...
test_inventory_pytest.py::test_slow_path PASSED                            [ 93%]
test_inventory_pytest.py::test_known_bug XFAIL (float rounding: 2.67...) [100%]

======================== 15 passed, 1 xfailed in 0.31s =========================
```

`pytest -q -m "not slow" test_inventory_pytest.py` → `14 passed, 1 deselected, 1 xfailed`. `pytest --durations=3` (whole folder):

```
0.20s call     test_inventory_pytest.py::test_slow_path
0.12s call     test_properties.py::test_stock_never_negative
0.04s call     test_properties.py::test_slugify_is_idempotent
```

Common mistakes

- Tests depending on each other's side effects or order (use fresh fixtures).
- `==` on floats (use `pytest.approx`).
- `pytest.raises` wrapping too much — the exception may come from a setup line.
- Module-level mutable state shared between tests.
- Same-named test files in different folders without `__init__.py` → import-mismatch error.

In the interview

"I use pytest: plain asserts, fixtures for setup with `yield` for teardown, `parametrize` for edge cases, `tmp_path` and `monkeypatch` for files and environment, and marks to separate slow tests. Shared fixtures go in `conftest.py`."

- *Fixture scope?* Per function by default for isolation; session for expensive read-only resources.
- *xfail vs skip?* Skip = don't run; xfail = run, expect failure (strict flags an unexpected pass).

15.5 Test doubles and `unittest.mock`

What it is

A **test double** stands in for a real dependency (a network API, a payment gateway, the clock) during a test — like a stunt double. Standard names (Gerard Meszaros' *xUnit Test Patterns*, popularised by Martin Fowler):

Double	What it does	Example
Dummy	passed but never used	`None` for an unused logger argument
Stub	returns canned (prepared) answers	`get_rate` always returns 83.5
Spy	records how it was called, checked afterwards	a list that collects sent emails
Mock	pre-programmed with expectations about calls, which the test verifies	"`get_rate` must be called once with `'INR'`"
Fake	a working, simplified implementation	an in-memory dict instead of a database

`unittest.mock.Mock` can play stub, spy and mock at once.

Why it matters

Real dependencies make tests **slow** (network), **flaky** (pass/fail with no code change, e.g. service down), **costly** (paid APIs) or **dangerous** (real emails, real charges), and some cases (a timeout on demand) are hard to trigger.

How it works — `Mock`

1. `m = Mock()` — any attribute returns another `Mock`; calls are recorded.
2. `m.get_rate.return_value = 83.5` — **stub** the result.
3. `m.get_rate.side_effect = ...` — an **exception** (raised), an **iterable** (one value per call) or a **function** (called with the same arguments).
4. Verify: `assert_called_once_with(...)`, `assert_called_with`, `assert_not_called`, `call_count`, `call_args`, `call_args_list`.
5. `MagicMock` also supports dunder methods (`__len__`, `__iter__`, `__enter__`…); `AsyncMock` (3.8+) for `async def`.
6. **Autospec** — `create_autospec(RealClass, instance=True)` or `patch(..., autospec=True)` gives a mock with only the real object's attributes and signatures. A plain `Mock` accepts misspelled methods, so a test can pass while real code would crash.

```python
class FakeRates:                              # hand-written fake
    def __init__(self, rates): self.rates = rates
    def get_rate(self, currency): return self.rates[currency]

def test_with_fake():
    assert price_in("EUR", 10, FakeRates({"EUR": 0.9})) == 9.0

def test_with_mock_stub_and_verify():
    client = Mock()
    client.get_rate.return_value = 83.5          # stub the answer
    assert price_in("INR", 2, client) == 167.0
    client.get_rate.assert_called_once_with("INR")   # verify the interaction

def test_side_effect_exception():
    client = Mock()
    client.get_rate.side_effect = TimeoutError("rate service down")
    with pytest.raises(TimeoutError):
        price_in("EUR", 10, client)

def test_side_effect_sequence():
    client = Mock()
    client.get_rate.side_effect = [0.9, -1.0]     # one value per call
    assert price_in("EUR", 10, client) == 9.0
    with pytest.raises(ValueError, match="bad rate"):
        price_in("EUR", 10, client)
    assert client.get_rate.call_args_list == [call("EUR"), call("EUR")]

def test_autospec_catches_typos():
    client = create_autospec(FakeRates, instance=True)
    with pytest.raises(AttributeError):
        client.get_rates("EUR")                   # misspelled: plain Mock would allow it
```

How it works — `patch`

When code creates or imports its dependency itself, `patch` swaps a name **temporarily**:

```python
def test_patch_where_it_is_used(tmp_path):
    # inventory.py did `import json`; patch the name inventory.json.dumps
    with patch("inventory.json.dumps", return_value="{}") as fake_dumps:
        inventory.Inventory({"a": 1}).save(tmp_path / "x.json")
    fake_dumps.assert_called_once()
    assert (tmp_path / "x.json").read_text() == "{}"
```

1. `patch("module.name")` replaces the attribute for a `with` block, a decorated test, or between `start()` / `stop()`; the original is always restored.

2. **Patch where the name is looked up, not where it is defined.** If `shop.py` does `from requests import get`, patch `"shop.get"` — `"requests.get"` won't affect the name already bound in `shop`. (Here `inventory.json` *is* the `json` module, so the patch affects `json.dumps` everywhere during the block — keep patches short.)
3. `patch.object(obj, "attr")` for an object you hold; `patch.dict(os.environ, {...})` for dict entries.
4. pytest's `monkeypatch` covers simple cases: `monkeypatch.setenv("APP_ENV", "test")`, `monkeypatch.setattr(module, "name", value)`.

`pytest -v test_mocking.py` → all 7 tests (the six above plus `test_monkeypatch_env`) `PASSED`.

Trade-offs

Approach	Pros	Cons
Real dependency (tmp_path, SQLite in memory, local server)	tests the real thing	slower, needs setup
Fake	realistic behaviour, reusable, readable	you must write and maintain it
Mock/stub	quick, can simulate errors/timeouts	couples tests to call details; can drift from the real API
`patch`	works without changing the code	fragile if imports move; hints that dependency injection would be cleaner

When not to mock

- What you own and can run cheaply (pure functions, dataclasses, `tmp_path` files, in-memory SQLite).
- The thing you're testing.
- Every internal call — that locks in the implementation. Mock at the **boundary** (network, clock, randomness, third-party SDKs) and prefer checking outcomes over interactions.

Common mistakes

- Patching where defined instead of where used.
- Plain `Mock()` hiding typos — use `autospec`/`spec`.
- Misspelled assertions (`m.called_once_with(...)`, `m.assret_called()`) used to silently "pass" by creating child mocks. Mock now raises `AttributeError` for assertion-like typos (`assret_` etc. since 3.5; names missing the `assert_` prefix since 3.12) unless `unsafe=True`.
- Tests that only prove the mock returns what you told it to.

Real-world examples

`responses` / `respx` fake HTTP for requests / httpx; `freezegun` / `time-machine` freeze the clock; `moto` fakes AWS (Amazon Web Services) services; `fakeredis` fakes Redis (an in-memory key-value store often used as a cache).

In the interview

"I prefer dependency injection so I can pass a fake. At external boundaries I use `unittest.mock` with autospec: `return_value` to stub, `side_effect` for errors or sequences, `assert_called_once_with` only when the interaction matters. With `patch` I patch where the name is looked up."

- *Mock vs stub?* A stub supplies answers; a mock also verifies how it was called.
- *Why autospec?* The mock fails on methods or signatures the real object lacks.
- *`side_effect = [1, 2]`, three calls?* 1, then 2, then `StopIteration`.

15.6 Coverage

What it is

Code coverage measures which lines (and branches) ran during the tests. `coverage.py` is the standard tool; `pytest-cov` integrates it.

- **Line (statement) coverage** — did each line run?
- **Branch coverage** — did each `if` go both ways? A line can run while one side is never taken.

How it works

```
pytest --cov=inventory --cov-branch --cov-report=term-missing
# or without the plugin:
coverage run --branch -m pytest && coverage report -m && coverage html   # htmlcov/index.html
```

Output:

```
Name           Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------
inventory.py      48      7     10      2    84%   48, 81-86
-----------------------------------------------------------
TOTAL             48      7     10      2    84%
```

48 statements, 7 never ran; 10 branch points, 2 partially taken. Line **48** is `raise ValueError("qty must be positive")` in `Inventory.add` — no test passes a non-positive quantity, so the validation is untested. **81-86** is the `if __name__ == "__main__":` demo block (exclude with a `# pragma: no cover` comment or config).

Trade-offs / how to use the number

- Coverage shows what is **not** tested, never that tested code is **correct** — a line run without any assertion still "counts".
- Teams often enforce a CI threshold (`--cov-fail-under=85`). 100% is rarely worth it outside small critical libraries; aim high on business logic and error paths.
- In code review, read the *missing* list more than the percentage.
- **Mutation testing** (`mutmut`) goes further: it mutates code (`<` → `<=`) and checks some test fails.

15.7 doctest — examples that are tests

`doctest` finds REPL-style examples in docstrings (`>>> expr` then expected output) and checks that running them produces that output.

```
def slugify(name: str) -> str:
    """Lower-case, keep letters/digits, join words with '-'.

    >>> slugify("Hello, World!")
    'hello-world'
    """
```

- Run: `python -m doctest inventory.py -v`, `doctest.testmod()` in the module, or `pytest --doctest-modules`.
- Output is compared as **text**, so it must match the `repr` exactly.

Output of `python inventory.py` and `pytest --doctest-modules inventory.py`:

```
stock: {'mug': 3} | units: 3
doctest: TestResults(failed=0, attempted=3)
..                                                                       [100%]
2 passed in 0.07s
```

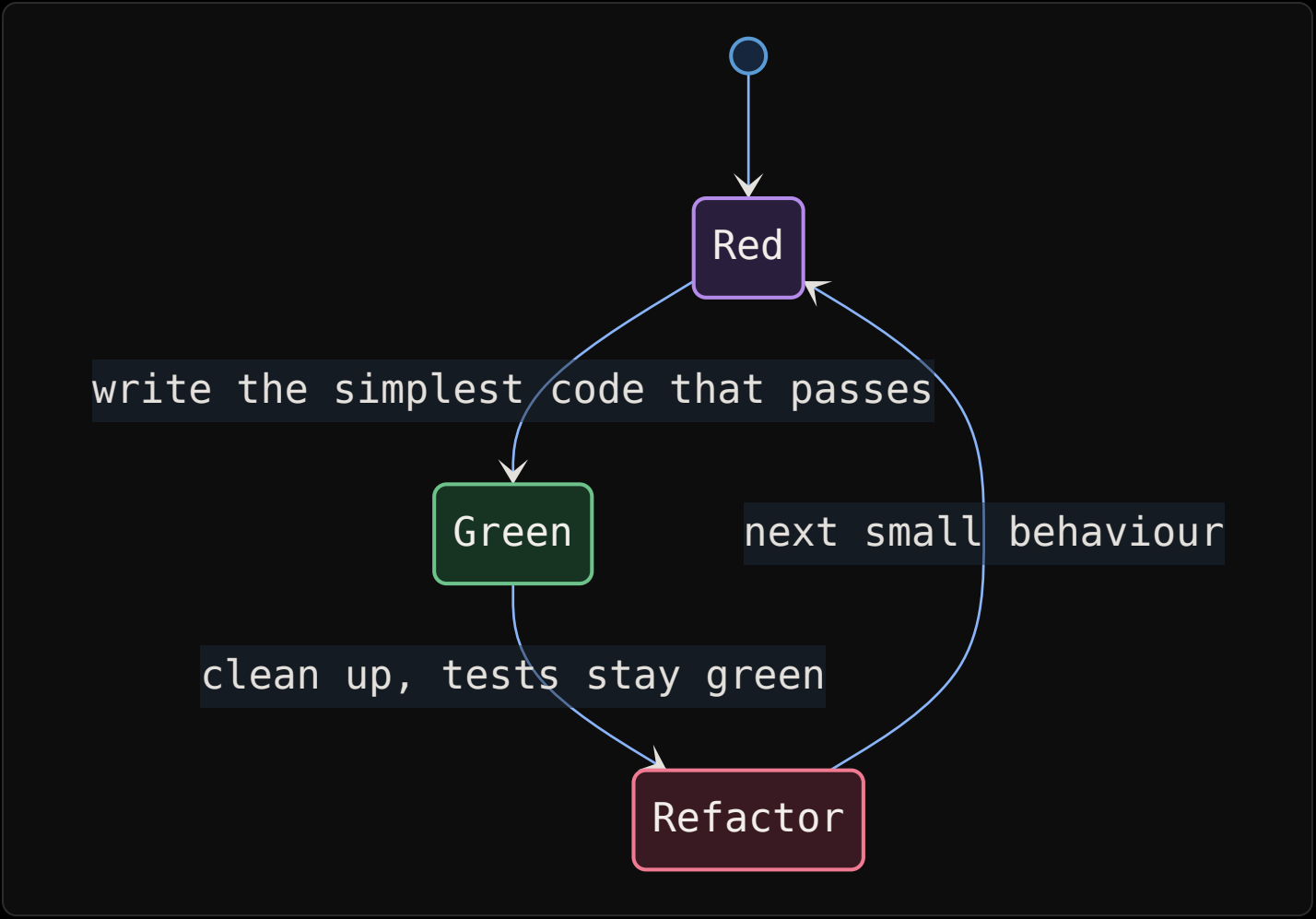
(3 examples in 2 docstrings; pytest counts each docstring as one test.)

Trade-off: great for keeping **documentation examples correct**; poor as a test suite — text comparison is brittle (dict order, float digits, memory addresses) and there are no fixtures or parametrization. Doctests for docs, pytest for tests.

15.8 The TDD loop (red, green, refactor)

What it is

Test-driven development (popularised by Kent Beck): write the test **before** the code, in tiny cycles.



1. **Red** — a small failing test for the next behaviour; see it fail *for the right reason*.
2. **Green** — the minimum code to pass.
3. **Refactor** — improve names and structure under the tests' protection.

How it works — a real cycle

Feature: `Inventory.low_stock(threshold)` → SKUs (stock-keeping units, product codes) with quantity **at or below** the threshold, sorted.

Red. Tests first (`test_tdd_low_stock.py`):

```
def test_low_stock_lists_skus_below_threshold(inv):      # inv: mug=5, pen=10
    assert inv.low_stock(threshold=6) == ["mug"]

def test_low_stock_sorted_and_inclusive(inv):
    inv.add("cap", 1)
    assert inv.low_stock(threshold=10) == ["cap", "mug", "pen"]
```

Both fail — the method doesn't exist:

```
>       assert inv.low_stock(threshold=6) == ["mug"]
               ^^^^^^^^^^^^^
E       AttributeError: 'Inventory' object has no attribute 'low_stock'
...
2 failed in 0.07s
```

Green, first try. `return [sku for sku, q in self.stock.items() if q < threshold]` passes the first test only; assertion rewriting shows why:

```
>       assert inv.low_stock(threshold=10) == ["cap", "mug", "pen"]
E       AssertionError: assert ['mug', 'cap'] == ['cap', 'mug', 'pen']
E
E         At index 0 diff: 'mug' != 'cap'
E         Right contains one more item: 'pen'
E         Use -v to get more diff
1 failed, 1 passed in 0.07s
```

Two bugs: not sorted, and `pen` (exactly 10) is missing, so the comparison must be `<=`.

Green.

```python
def low_stock(self, threshold: int) -> list[str]:
    """SKUs with quantity <= threshold, alphabetically."""
    return sorted(sku for sku, qty in self.stock.items() if qty <= threshold)
```

Whole suite: `31 passed, 1 skipped, 1 xfailed`.

Refactor. Names are already clear — next behaviour.

Trade-offs

Pros	Cons
Every line has a reason (a test)	Slower start; feels unnatural at first
Forces small, testable design	Hard when exploring an unknown problem/API (spike — a throw-away experiment — first, then test)
Instant regression safety	Can over-fit tests to one design if cycles are too big

Many teams use TDD at least for bug fixes: **first write a test that reproduces the bug**, see it fail, then fix — so it can never silently return.

15.9 Property-based testing with Hypothesis (brief)

What it is

In an **example-based** test you pick inputs. In a **property-based** test you state a rule that must hold for *all* inputs of a kind, and the library generates hundreds — including nasty ones (empty strings, huge numbers, Unicode, NaN) — to break it. **Hypothesis** is Python's main library (the idea comes from Haskell's QuickCheck). A failing input is **shrunk** to the simplest failing example and saved in `.hypothesis/` so it's retried first next time.

How it works

```python
from hypothesis import given, settings, strategies as st

@given(st.text())
def test_slugify_is_idempotent(s):            # applying it twice changes nothing
    assert slugify(slugify(s)) == slugify(s)

@given(st.text())
def test_slug_has_only_safe_chars(s):
    assert all(c.isascii() and (c.isalnum() or c == "-") for c in slugify(s))

@given(price=st.floats(0, 1e6), pct=st.floats(0, 100))
def test_discount_never_increases_price(price, pct):
    assert apply_discount(price, pct) <= round(price, 2)

@settings(max_examples=200)
@given(st.lists(st.tuples(st.sampled_from(["add", "remove"]), st.integers(1, 20))))
def test_stock_never_negative(ops):           # random sequences of operations
    inv = Inventory()
    for op, qty in ops:
        try:
            getattr(inv, op)("mug", qty)
        except OutOfStock:
            pass
        assert inv.stock.get("mug", 0) >= 0
```

All four pass. Against a naive `name.strip().lower().replace(" ", "-")`, the "safe characters" property fails and shrinks to one character:

```
E       Failing test case: test_slug_has_only_safe_chars(
E           s=':',
E       )
```

Common properties: **round-trip** (`load(save(x)) == x`, `decode(encode(x)) == x`), **idempotence** (`f(f(x)) == f(x)`), **invariants** (stock never negative, output sorted, length preserved), **oracle** (fast version equals a slow obviously-correct one). Great for parsers and serializers.

When not to use it: no general rule to state, or inputs expensive to build. Use alongside example tests, not instead.

15.10 Reading tracebacks

What it is

A **traceback** reports an uncaught exception: the **call stack** that led to it, oldest call at the top, failing line at the bottom.

How it works — read it bottom-up

1. **Last line first** — type and message: `ValueError: invalid literal for int() with base 10: 'two'`.
2. **The frame above it** (each entry is a *frame* — one function call in progress): file, line, function. Since 3.11, `~~~^^^` markers point at the exact sub-expression (PEP 657).
3. **Walk upward** to *your* code (skip library frames) — often the bug is where your code passed bad data in.
4. **Chained exceptions**: "The above exception was the direct cause of the following exception" = explicit `raise NewError(...) from original`. "During handling of the above exception, another exception occurred" = a new error *inside* an `except` block (implicit chaining).

From `debugging_demo.py` (`"two"` as a quantity):

```
Traceback (most recent call last):
  File "debugging_demo.py", line 19, in load_orders
    orders.append(parse_order(line))
                  ~~~~~~~~~~~^^^^^^
  File "debugging_demo.py", line 12, in parse_order
    return {"sku": sku, "qty": int(qty), "price": float(price)}
                               ~~~^^^^^
ValueError: invalid literal for int() with base 10: 'two'

The above exception was the direct cause of the following exception:

Traceback (most recent call last):
  File "debugging_demo.py", line 35, in <module>
    load_orders(["mug,2,4.50", "pen,two,1.20"])
    ~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "debugging_demo.py", line 21, in load_orders
    raise RuntimeError(f"bad order on line {n}: {line!r}") from e
RuntimeError: bad order on line 2: 'pen,two,1.20'
```

The final error says *which input* was bad; the cause says *what* was wrong (`int('two')`) — the payoff of `raise ... from e` with context.

Common exception → usual cause

Exception	Usual cause
`NameError`	typo, or used before assignment
`AttributeError: 'NoneType' object has no attribute ...`	a function returned `None` (forgot `return`, or an in-place method like `list.sort()`)
`TypeError: ... takes 2 positional arguments but 3 were given`	forgot `self`, or wrong call
`KeyError` / `IndexError`	missing dict key / off-by-one
`UnboundLocalError`	assigning to a name inside a function makes it local (use `nonlocal` / `global` or rename)
`ModuleNotFoundError`	wrong virtual environment (venv), or a local file shadows the module (your own `random.py`)
`RecursionError`	missing or unreachable base case

15.11 Debugging with `pdb` and `breakpoint()`

What it is

`pdb` is Python's interactive **debugger**: pause at a line, inspect variables, step, move up and down the stack. `breakpoint()` (3.7+, PEP 553) pauses there — it calls `pdb.set_trace()` by default.

How it works — ways in

Way	When
Put `breakpoint()` in the code	pause at a specific spot
`python -m pdb script.py`	start the whole script under the debugger
`pdb.pm()` after a crash (in the REPL) or `python -m pdb` (enters post-mortem — inspecting the state at the moment of the crash — on an uncaught exception)	inspect the frame where it died
`pytest --pdb`	debugger at the failing assertion
`pytest --trace`	debugger at the start of each test
IDE (integrated development environment: VS Code, PyCharm) breakpoints	same ideas, with a GUI

`PYTHONBREAKPOINT` controls `breakpoint()`: `=0` disables all (safety net in CI), `=ipdb.set_trace` switches debugger.

The commands you need

Command	Does
`l` / `ll`	list code around the current line / the whole function
`n` (next)	run the current line, stepping *over* calls
`s` (step)	step *into* a call
`r` (return)	run until the current function returns
`c` (continue)	run until the next breakpoint
`unt N` (until)	run until line N
`p expr` / `pp expr`	print / pretty-print an expression
`w` (where)	show the call stack
`u` / `d`	move up / down the stack to inspect callers
`b file:line` / `b func` / `b line, cond`	set a (conditional) breakpoint
`cl N`	clear breakpoint N
`display expr`	show `expr` each time execution stops if it changed
`interact`	a full Python REPL in the current scope
`q`	quit

A bare expression works too, but for variables named like commands (`n`, `c`, `l`…) use `p n`.

A real session (scripted, so it can be shown)

`debugging_demo.py` feeds `pdb` a command list. The function crashes on an empty list:

```python
def average_value(orders: list[dict]) -> float:
    total = 0.0
    for o in orders:
        total += o["qty"] * o["price"]
    return total / len(orders)          # bug when orders is empty
```

Commands: `b average_value`, `c`, `p orders`, `n`, `n`, `p total, len(orders)`, `c`. Transcript:

```
(Pdb) Breakpoint 1 at debugging_demo.py:26
(Pdb) > debugging_demo.py(26)average_value()
-> total = 0.0
(Pdb) []
(Pdb) > debugging_demo.py(27)average_value()
-> for o in orders:
(Pdb) > debugging_demo.py(29)average_value()
-> return total / len(orders)          # bug when orders is empty
(Pdb) (0.0, 0)
(Pdb)
then it raised: ZeroDivisionError('division by zero')
post-mortem locals: {'orders': [], 'total': 0.0}
breakpoint() skipped because PYTHONBREAKPOINT=0
```

The loop body is skipped and `len(orders)` is 0 at the return — handle the empty case (`if not orders: return 0.0` or raise a clear error). "post-mortem locals" is the same information recovered from the innermost traceback frame after the crash — what `pdb.pm()` gives interactively.

A debugging method (not just a tool)

1. **Reproduce** reliably — ideally as a failing test.
2. **Read** the traceback and code; state a hypothesis.
3. **Observe** with `breakpoint()`, logging or `p`.
4. **Narrow down**: halve the input, comment out half the code, or `git bisect` (binary-searches history for the commit that introduced the bug).
5. **Fix** the cause, not the symptom; keep the test.
6. Explaining the code line by line ("rubber-duck debugging") often reveals the bug.

Common mistakes

- Committing a `breakpoint()` — CI hangs waiting for input. Set `PYTHONBREAKPOINT=0` there and use a linter rule such as Ruff's `T100`.
- Changing several things at once.

15.12 Logging vs `print` for debugging

`print` is fine for a five-minute look. Anything that stays in the code uses **logging** (Chapter 14) — levels, timestamps, module names, configurable destinations, DEBUG enabled only where needed.

	`print`	`logging`
Levels / filtering	no	DEBUG…CRITICAL, per module
Timestamp, module, line	manual	via formatter
Destination	stdout	console, files, syslog (the OS's central log), log services
Safe to leave in code	no	yes (DEBUG off in production)
In tests	captured by pytest (`-s` shows)	`caplog` fixture lets tests assert on logs

```
log = logging.getLogger(__name__)
log.debug("pricing order=%s rate=%s", order_id, rate)    # visible only at DEBUG
```

Tips: raise one module's level temporarily (`logging.getLogger("shop.pricing").setLevel(logging.DEBUG)`); `print(f"{total=}")` prints `total=0.0`; `python -X dev` enables extra runtime checks and warnings; `faulthandler` (`python -X faulthandler`) dumps tracebacks on hard crashes (e.g. a segfault in C code) or hangs.

15.13 Common bugs checklist

Run through this when something "can't possibly be wrong":

#	Bug	Example	Fix
1	Mutable default argument	`def f(items=[])` shares one list across calls	`items=None` then `items = [] if items is None else items`
2	Late-binding closures	`[lambda: i for i in range(3)]` all return 2	`lambda i=i: i`
3	Aliasing / shallow copies	`grid = [[0]*3]*3` — one row shared three times	`[[0]*3 for _ in range(3)]`, `copy.deepcopy`
4	Modifying a list while iterating	removing items inside `for x in xs` skips items	iterate over a copy or build a new list
5	`is` vs `==`	`x is 1000` depends on object identity	`==` for values; `is` only for `None`, sentinels (unique marker objects)
6	Float equality	`0.1 + 0.2 == 0.3` is False	`math.isclose`, `pytest.approx`, `Decimal` for money
7	Integer vs float division	`7 / 2 == 3.5`, `7 // 2 == 3`, `-7 // 2 == -4`	pick the right operator
8	Methods returning `None`	`xs = xs.sort()` makes `xs` None	`xs.sort()` or `xs = sorted(xs)`
9	Swallowed exceptions	`except Exception: pass` hides the real error	catch specific exceptions, log, re-raise
10	Off-by-one	`range(1, n)` excludes `n`	test the boundaries
11	Truthiness	`if count:` is False for `0`	`if count is not None:`
12	Naive datetimes / time zones	comparing local and UTC times	aware UTC datetimes (Chapter 14)
13	Unclosed resources	files/connections left open	`with` statements
14	Wrong environment	`ModuleNotFoundError` though "it's installed"	check `sys.executable`, activate the venv, `python -m pip`
15	Module name shadowing	your `random.py` or `test.py` hides the stdlib	rename the file
16	Global/shared state between tests	tests pass alone, fail together	fresh fixtures, no module-level mutable state
17	Encoding	`UnicodeDecodeError` reading files	`open(p, encoding="utf-8")`
18	Unseeded randomness / real clock in tests	flaky tests	seed the generator, inject the clock, freeze time

---

15.14 Chapter self-check

1. Three parts of a well-shaped test? → Arrange, Act, Assert.
2. Why do most tests belong at the bottom of the pyramid? → Unit tests are fast, precise, cheap.
3. How does pytest show values in a failing `assert`? → Assertion rewriting.
4. How does a test get a fixture, and must it import from `conftest.py`? → By naming it as a parameter; no, discovered automatically.

5. Fixture cleanup? → Code after `yield`.
6. `parametrize` with four tuples produces? → Four separate tests.
7. What does `tmp_path` give you? → A unique temporary directory (`Path`) per test.
8. `xfail(strict=True)` and the test passes? → Reported as a failure (XPASS strict), so you notice the fix.
9. Stub vs mock vs fake? → Canned answers / verified interactions / working simplified implementation (e.g. an in-memory repository).
10. `shop.py` has `from requests import get` — what do you patch? → `"shop.get"`.
11. Why `create_autospec`? → Mocks reject attributes and signatures the real object lacks.
12. What does 100% line coverage prove? → Only that every line ran — not that results were checked.
13. TDD cycle? → Red, green, refactor.
14. What does Hypothesis do after finding a failing input? → Shrinks it to a minimal example and remembers it.
15. How do you read a traceback? → Bottom-up: message, failing line, then up to your code; "direct cause" = `from`, "during handling" = error inside `except`.
16. Disable every `breakpoint()` in CI? → `PYTHONBREAKPOINT=0`.
17. `xs = xs.sort()` — what is `xs`? → `None`.

Chapter TL;DR

- **Why:** tests make change safe. Many fast unit tests, fewer integration/E2E; deterministic, isolated, behaviour-focused; run in CI.
- **unittest:** `TestCase`, `setUp`, `assertEqual`/`assertRaises`; built in, more verbose.
- **pytest:** plain `assert` (rewritten failure diffs), fixtures (`yield` teardown, scopes, `conftest.py`), `parametrize`, `tmp_path`, `monkeypatch`, `capsys`, `caplog`, marks (`skip`, `skipif`, `xfail`, custom + `-m`), `-k`, `-x`, `--lf`, `--pdb`.
- **Doubles:** dummy, stub, spy, mock, fake. `Mock` + `return_value`/`side_effect`/`assert_called_once_with`; `patch` where the name is used; autospec; mock only at boundaries; prefer dependency injection.
- **Coverage:** `pytest --cov --cov-branch --cov-report=term-missing` finds untested code; it doesn't prove correctness.
- **TDD:** red, green, refactor; always reproduce a bug with a failing test first.
- **doctest** keeps doc examples true; **Hypothesis** checks properties and shrinks counter-examples.
- **Debugging:** read tracebacks bottom-up; `breakpoint()`/`pdb` (`n s c p w u d b`), `pytest --pdb`, post-mortem; reproduce → hypothesise → observe → narrow → fix → keep the test.
- **Logging over print** for anything that stays; check the **common bugs list** first (mutable defaults, aliasing, `None` returns, float equality, swallowed exceptions, wrong venv).

Chapter 16 — Concurrency and Async

What you will learn. Concurrency vs parallelism; I/O-bound work (time spent waiting on network, disk, database) vs CPU-bound work (time spent computing); Python's four tools for doing several things at once — **threads** (`threading`, locks, `queue.Queue`), **processes** (`multiprocessing`), **executors** (`concurrent.futures`) and **async/await** (`asyncio`: event loop, coroutines, tasks, `gather`, `TaskGroup`, timeouts, async iteration). What the **GIL** is, why it exists, and where **free-threaded** (no-GIL) CPython stands in 3.13 and 3.14. A "which tool?" table and the pitfalls: blocking calls in async code, race conditions, deadlocks.

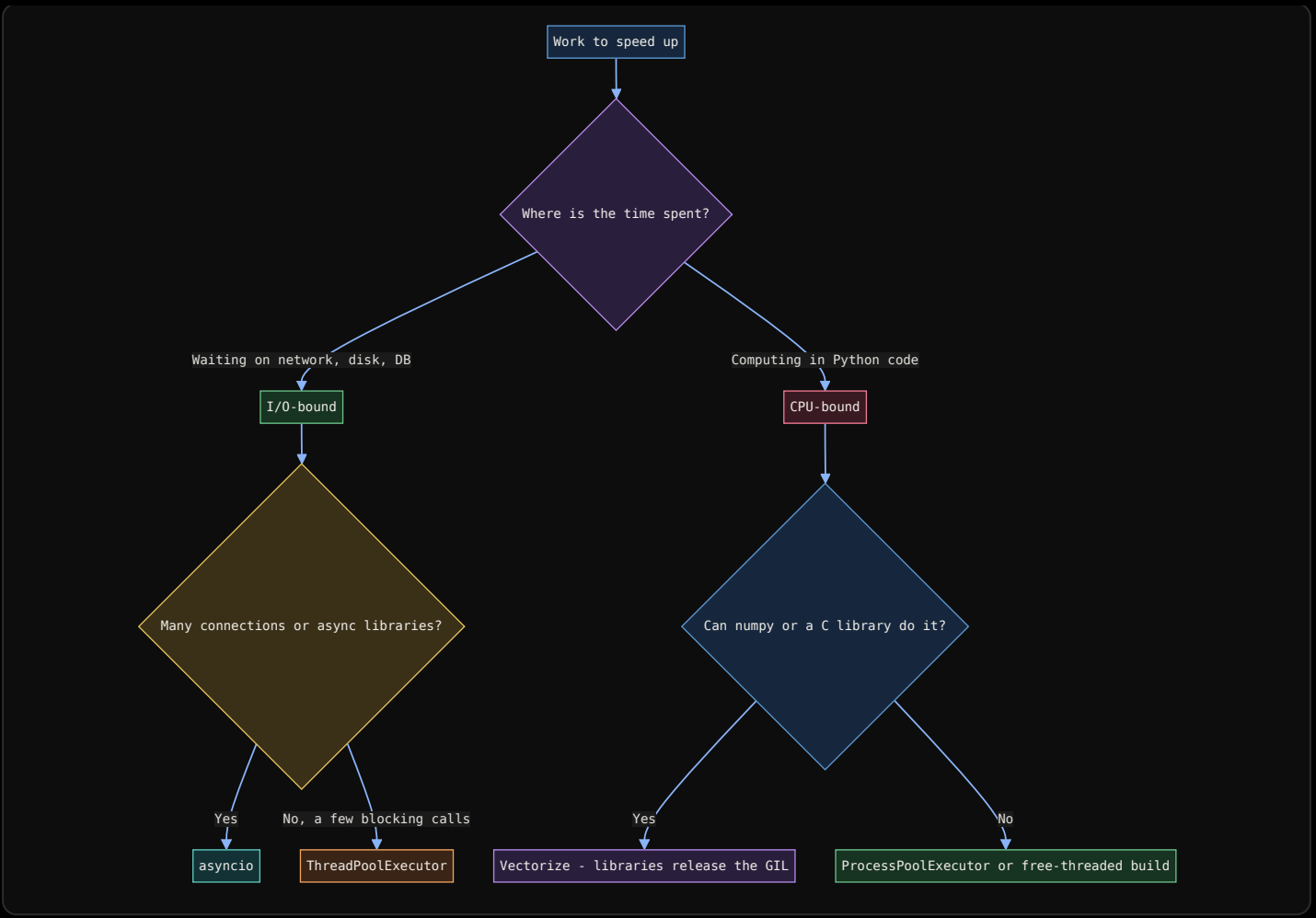
Prerequisites. Functions and closures (Ch 7), generators (Ch 11), context managers (Ch 12).

Code. `chapters/16-concurrency-and-async/code/` — `threads_basics.py`, `gil_benchmark.py` (makes the plot in `img/`), `multiprocessing_demo.py`, `futures_demo.py`, `asyncio_demo.py`, `pitfalls_demo.py`. Outputs are from Python 3.14.7 (default build, GIL on) on a 14-core Mac; timings differ on your machine, the *shape* does not.

Java side. General theory — memory models (when one thread's writes become visible to another), `synchronized`, `ReentrantLock`, semaphores, latches, atomics, concurrent collections, deadlock/livelock (threads keep reacting to each other without progress)/starvation (a thread never gets its turn) and classic LLD (low-level design) problems — is in LLD Handbook Chapter 12 — Concurrency concepts. This chapter covers what is specific to Python.

Words used everywhere in this chapter (defined once):

Term	Meaning
Process	A running program with its own memory space. Two processes do not see each other's variables.
Thread	A path of execution *inside* a process. Threads of one process share its memory.
Task (asyncio)	A coroutine (a function that can pause and resume) scheduled on an event loop (the scheduler that runs whichever task is ready next). Many tasks share one thread.
Blocking call	A call that makes the caller wait (reading a socket, `time.sleep`, a disk read).
Race condition	A bug where the result depends on the unlucky timing of two threads touching shared data.
Critical section	Code that must be run by only one thread at a time.
Lock / mutex	An object that lets only one holder into a critical section ("mutex" = *mut*ual *ex*clusion).
Deadlock	Two or more threads each waiting forever for a lock the other holds.
GIL	Global Interpreter Lock: in standard CPython only one thread runs Python bytecode at a time.



16.1 Concurrency vs parallelism

What it is

- **Concurrency** = *dealing with* many things at once: several tasks in progress, steps interleaved. Works on a single CPU core.
- **Parallelism** = *doing* many things at the same instant on different cores.

One chef boiling pasta and chopping meanwhile is concurrency; three chefs is parallelism. Concurrency is about *structure* (never idle while waiting); parallelism about *execution* (more hands). Parallelism is a kind of concurrency, not vice versa.

Why it matters

Threads on a CPU-heavy pure-Python loop in standard CPython do nothing (the GIL); threads on a program spending 95 % of its time waiting for HTTP responses can make it 10–50× faster.

How it works

	Concurrency	Parallelism
Needs multiple cores?	No	Yes
Goal	Stay busy while waiting	Finish computation sooner
Python tools	threads, asyncio	multiprocessing, free-threaded threads, C extensions that release the GIL
Typical workload	Web servers, scrapers, chat apps	Image processing, simulations, number crunching

---

16.2 I/O-bound vs CPU-bound

What it is

- **I/O-bound**: mostly *waits* (network, disk, DB, user); CPU idle. E.g. downloading 100 pages.
- **CPU-bound**: mostly *computes*; CPU at 100 %. E.g. SHA-256 of a billion strings in a Python loop.

How to tell (3 quick checks)

1. **CPU usage** (`top`, Activity Monitor): ~100 % of one core → CPU-bound; low CPU but slow → I/O-bound.
2. **Profile** (Ch 17): top functions `socket.recv`, `select`, `read` → I/O; your own loops → CPU.
3. **Wall vs CPU time**: `time.perf_counter()` vs `time.process_time()`. Wall ≫ CPU → waiting.

Measured

`gil_benchmark.py` runs 4 CPU tasks (pure-Python loop, 15 million iterations) and 4 I/O tasks (`time.sleep(0.5)`, standing in for a network wait) three ways:

```python
def cpu_task(n: int = 15_000_000) -> int:
    total = 0
    for i in range(n):                    # pure-Python loop: holds the GIL the whole time
        total += i * i
    return total

def io_task(_: int = 0) -> None:
    time.sleep(0.5)                       # sleeping releases the GIL, like waiting on a socket
```

Output:

```
Python 3.14.7, GIL enabled: True
CPU-bound: sequential 1.94s  threads 1.83s  processes 0.87s
I/O-bound: sequential 2.02s  threads 0.51s  processes 0.92s
saved gil_benchmark.png
```

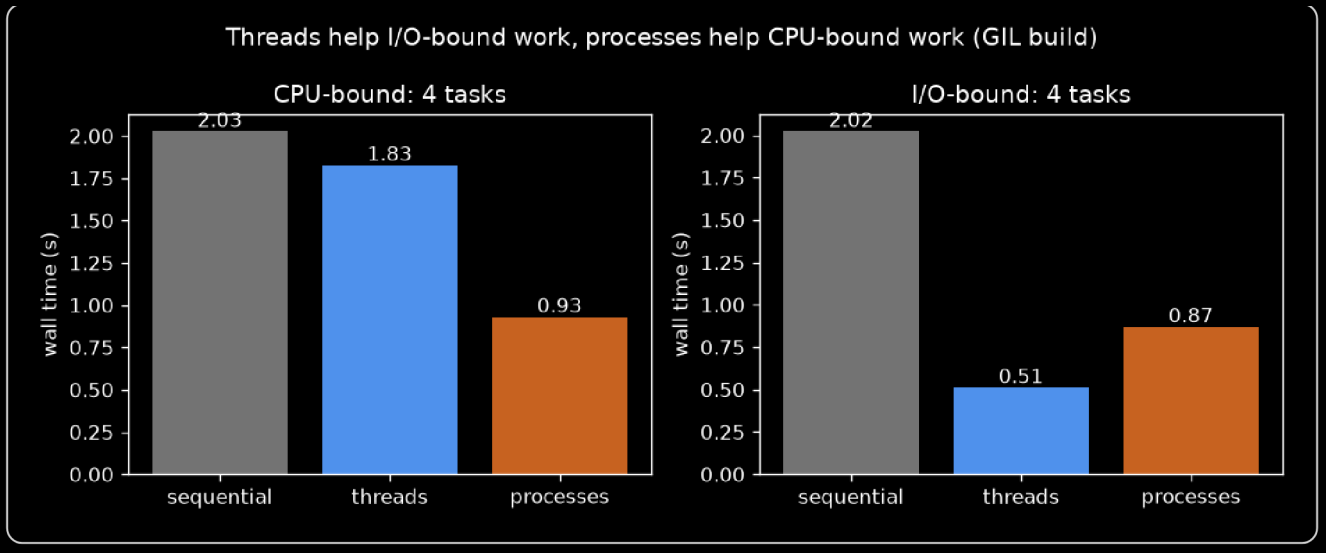


Figure — CPU-bound work only speeds up with processes; I/O-bound work speeds up with threads (processes also help but pay ~0.3–0.4 s start-up).

- **CPU + threads ≈ sequential** (1.83 vs 1.94 s is noise): one thread runs Python at a time.
- **CPU + processes ≈ 2.2× faster**: each process has its own interpreter and GIL. Not 4× because starting processes (`spawn` re-imports the module) and returning data cost time.
- **I/O + threads ≈ 4× faster**: a sleeping thread releases the GIL, so the four waits overlap.

Rule: measure first. Waiting → threads or asyncio. Computing → processes, vectorized libraries, or the free-threaded build.

16.3 Threads with `threading`

What it is

`threading` starts extra threads in your process. Threads share globals and heap objects; each has its own call stack.

How it works

1. `t = threading.Thread(target=func, args=(...), name="T1")`.
2. `t.start()` — the OS starts a thread that calls `func(*args)`.
3. `t.join()` — wait until `t` finishes (optionally `join(timeout)`).
4. A **daemon** thread (`daemon=True`) is killed abruptly when the main program exits; a normal one keeps the program alive. Daemons only for losable work (a heartbeat), never for writing files.

```
def worker(name: str, delay: float, results: dict) -> None:
    time.sleep(delay)                    # stands in for waiting on I/O
    results[name] = threading.current_thread().name

threads = [threading.Thread(target=worker, args=(f"job{i}", 0.2, results), name=f"T{i}")
           for i in range(3)]
for t in threads: t.start()              # all three begin waiting at once
for t in threads: t.join()               # wait for all to finish
```

Output (`threads_basics.py`, line 1):

```
3 x 0.2 s sleeps in threads took 0.21 s -> {'job0': 'T0', 'job1': 'T1', 'job2': 'T2'}
```

The waits overlapped. `target`'s return value is discarded; get results via a shared structure (as here), a `queue.Queue`, or — better — `ThreadPoolExecutor` (§16.8), which returns values and exceptions. Subclassing `Thread` and overriding `run()` also works, but `target` is simpler and more common.

Per-thread data. `threading.local()` gives an object whose attributes are separate per thread (a DB connection, the current request ID). In asyncio, where tasks share a thread, use `contextvars.ContextVar` — each task sees its own value (how web frameworks store the current request).

16.4 Race conditions and locks

What it is

Two threads read-modify-write the same data with interleaved steps. Even `counter += 1` is *load → add → store*: if A loads 5, then B loads 5, both store 6 — one increment lost. A **lock** (`threading.Lock`) admits one holder at a time, turning the read-modify-write into a **critical section**.

The race, demonstrated

```
def unsafe_increment(n: int) -> None:
    global counter
    for _ in range(n):
        current = counter                # read
        time.sleep(0)                    # yield: lets another thread run (widens the window)
        counter = current + 1            # write back a stale value

lock = threading.Lock()

def safe_increment(n: int) -> None:
    global counter
    for _ in range(n):
        with lock:                       # only one thread inside at a time
            current = counter
            time.sleep(0)
            counter = current + 1
```

Output (4 threads × 1,000 increments):

```
unsafe counter (expected 4000): 1000
safe counter   (expected 4000): 4000
```

The unsafe number varies (other runs: 2000, 1031). `time.sleep(0)` only makes the bug show every time; without it the bug appears rarely — worse, because it passes tests and fails in production.

How it works — the synchronization toolbox

Primitive	What it does	Typical use
`Lock`	One holder at a time. Not re-entrant: the same thread acquiring twice blocks itself.	Protect shared state
`RLock`	Re-entrant lock: the owning thread may acquire it again (must release as many times).	Recursive code that re-enters a locked method
`Semaphore(n)`	Up to *n* holders at once.	Limit concurrent DB connections to 10
`Event`	A flag threads can `wait()` on until someone calls `set()`.	"Start now" / "shut down" signal
`Condition`	Wait until a condition becomes true, woken by `notify()`.	Building your own queues
`Barrier(n)`	Blocks until *n* threads arrive, then releases all.	Phased simulations
`queue.Queue`	Thread-safe FIFO with blocking `get`/`put`.	Producer–consumer (next section)

Use locks via `with lock:` — released even on exceptions. Manual `acquire()`/`release()` only for timeouts: `if lock.acquire(timeout=1): ...`.

Common mistakes

- "The GIL makes my code thread-safe." It protects the *interpreter's* internals (a `list` is never corrupted), not *your* multi-step logic: `if key not in d: d[key] = compute()` can still run twice.
- Protecting writes but not reads of a multi-field invariant (e.g. reading `balance` mid-transfer).
- Holding a lock during slow I/O — everyone queues behind it.

In the interview

> **30-second answer:** "A race condition is when the outcome depends on thread timing, usually from an unprotected read-modify-write. Even `x += 1` is several bytecodes, so it's not atomic (indivisible). I protect shared state with a `threading.Lock` used as a context manager, keep critical sections small, or avoid sharing by passing messages through `queue.Queue`."

- *Is `list.append` thread-safe?* — One `append` on a built-in list is safe in CPython (GIL; the free-threaded build uses per-object locks for built-ins), but sequences of operations are not. Don't rely on it for logic.
- *Lock vs RLock?* — RLock lets the same thread re-acquire; Lock would deadlock on itself.
- *Which primitive limits concurrency to N?* — `Semaphore(N)`.

16.5 Producer–consumer with `queue.Queue`

What it is

Producer–consumer splits work into threads that *make* items and threads that *process* them, joined by a queue. `queue.Queue` is a thread-safe FIFO with all locking built in.

Why it matters

Threads never touch each other's data — only `put` and `get`. A **bounded** queue (`maxsize=N`) adds **backpressure**: a fast producer blocks on `put` instead of filling memory. Queues and backpressure are core system-design ideas too (System Design Fundamentals).

How it works

1. `q = queue.Queue(maxsize=5)`.
2. Producers `q.put(item)` (blocks while full).
3. Consumers `q.get()` (blocks while empty), process, `q.task_done()`.
4. To stop, each producer puts a **sentinel** ("no more work", here `None`); the consumer stops after one per producer. (3.13 added `Queue.shutdown()` for this.)

```
SENTINEL = None

def producer(q, items):
    for item in items:
        q.put(item)                    # blocks if the queue is full (backpressure)
    q.put(SENTINEL)

def consumer(q, out, n_producers):
    finished = 0
    while finished < n_producers:
        item = q.get()                 # blocks until an item is available
        if item is SENTINEL:
            finished += 1
        else:
            out.append(item * item)
        q.task_done()
```

Output:

```
consumer squared 10 items, sorted: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
```

`q.join()` (unused here) blocks until every `put` is matched by a `task_done()` — "wait for all work processed" without stopping workers.

Queue class	Order
`queue.Queue`	FIFO
`queue.LifoQueue`	Stack (last in, first out)
`queue.PriorityQueue`	Smallest item first (put `(priority, item)` tuples)
`queue.SimpleQueue`	Unbounded FIFO, no `task_done` / `join`
`multiprocessing.Queue`	Between processes (items are pickled)
`asyncio.Queue`	Between asyncio tasks (not thread-safe)

Real-world examples

Web crawlers (fetcher → parser threads), logging (`logging.handlers.QueueHandler` + `QueueListener` move slow log I/O off request threads), PyTorch's `DataLoader` (worker *processes* feeding batches through queues).

16.6 The GIL (Global Interpreter Lock)

What it is

In standard CPython the **GIL** is one lock a thread must hold to execute bytecode, so **only one thread per process runs Python code at any instant**, even on 64 cores.

Why it exists

CPython manages memory with **reference counts** (Ch 17). Updating them from several threads would need a lock per object or atomic CPU instructions everywhere — slower and much harder to get right. One big lock kept the interpreter simple, single-threaded code fast, and C extensions easy to write — a big reason numpy and the scientific stack exist.

How it works

1. A thread holds the GIL while running bytecode.
2. It **releases** it during blocking I/O (`socket.recv`, file reads, `time.sleep`) and when a C extension releases it explicitly (numpy on large arrays, `hashlib` on big buffers, `zlib`, many image and DB libraries).
3. A waiting thread asks; after the **switch interval** (`sys.getswitchinterval()`, default 0.005 s = 5 ms) the holder is asked to drop it.

Workload	Threads with the GIL
I/O-bound	Scale well — the GIL is released while waiting
CPU-bound pure Python	No speed-up; can even be slightly slower (switching overhead)
CPU-bound in numpy / C that releases the GIL	Can scale across cores

Free-threaded CPython (PEP 703) — status

- **Python 3.13 (Oct 2024):** an *optional, experimental* GIL-disabled build ships ("free-threaded"; executable `python3.13t`). PEP 703 is the design.
- **Python 3.14 (Oct 2025):** per **PEP 779**, **officially supported** (no longer experimental) but **still optional — not the default**: `python3.14` has a GIL, `python3.14t` does not. Single-threaded slowdown roughly 5–10 % (platform/compiler dependent); memory use is higher.

- **Extensions:** importing a C extension not marked free-threading-compatible **re-enables the GIL** (with a warning); force it with `PYTHON_GIL=1` or `python -X gil=1`. Many major packages (numpy among them) publish free-threaded wheels (pre-built `.whl` packages) — check before relying on it.
- **Check at runtime:**

```
import sys, sysconfig
sysconfig.get_config_var("Py_GIL_DISABLED")  # 1 on a free-threaded build, else 0/None
sys._is_gil_enabled()                        # False when running without the GIL (3.13+)
```

- **What changes for you:** CPU-bound threads can scale across cores — but races become *more* likely, so §16.4's rules still apply. Built-in `list`/`dict`/`set` use per-object locks so they aren't corrupted; multi-step logic still needs your locks.

Other ways around the GIL: **processes** (one GIL each); **subinterpreters** (independent interpreters in one process) — own GIL each since 3.12 (PEP 684), and 3.14 adds `concurrent.interpreters` and `InterpreterPoolExecutor` (PEP 734); **native code** that releases the GIL (numpy, Cython — Python-like language compiled to C, Rust via PyO3).

Common mistakes

- "Python can't do concurrency." It does I/O concurrency very well; the GIL limits *parallel bytecode*.
- "The GIL makes code thread-safe." See §16.4.
- Assuming all implementations match: Jython (JVM) and IronPython (.NET) have no GIL; PyPy (JIT compiler) has one.

In the interview

30-second answer: "The GIL is a mutex in CPython that allows only one thread to execute Python bytecode at a time. It exists because reference counting isn't thread-safe without it, and it keeps single-threaded code and C extensions simple. It's released during blocking I/O and by many C extensions, so threads are fine for I/O-bound work; for CPU-bound Python I use multiprocessing or push work into numpy. Since 3.13 there's an optional free-threaded build, officially supported in 3.14 but not the default."

- *Does the GIL make `x += 1` atomic?* — No; a switch can happen between load and store.
- *Why doesn't numpy suffer from the GIL?* — Its heavy loops run in C and release it.
- *Switch to the free-threaded build today?* — Only after checking every C-extension dependency supports it and benchmarking; single-threaded code gets a bit slower.

16.7 `multiprocessing`

What it is

Runs functions in separate **processes**, each with its own interpreter, memory and GIL — true parallelism for CPU-bound Python. The API mirrors `threading`.

How it works

1. **Start method** — how the child is created:

Method	How	Default on
`spawn`	Fresh interpreter; re-imports your main module; arguments are pickled	Windows, macOS
`fork`	Copies the parent process (fast, but unsafe if the parent has threads)	— (was Linux's default up to 3.13)
`forkserver`	Forks from a clean single-threaded server process	Linux and other POSIX (Unix-like) systems except macOS, from **3.14**

`spawn` / `forkserver` re-import your script, so **always guard start-up code with `if __name__ == "__main__":`**, or each child starts children of its own.

2. **Pickling** — arguments and results travel via `pickle` (object ↔ bytes). Lambdas, nested functions and open sockets can't be pickled; use top-level functions.
3. **No shared memory by default** — each process has its own globals (and its own pid, the OS process ID).

```
counter = 0                              # each process gets its OWN copy

def bump_global(n: int, parent_pid: int) -> None:
    global counter
    for _ in range(n):
        counter += 1
    print(f"  child: different pid from parent={os.getpid() != parent_pid}, sees counter={counter}")

def bump_shared(shared, n: int) -> None:
    for _ in range(n):
        with shared.get_lock():          # Value comes with its own lock
            shared.value += 1

if __name__ == "__main__":
    p = mp.Process(target=bump_global, args=(1000, os.getpid()))
    p.start(); p.join()
    shared = mp.Value("i", 0)            # 'i' = C int in shared memory
    # ... start 4 processes running bump_shared(shared, 1000) ...
    with mp.Pool(processes=4) as pool:
        pool.map(square, range(10))
```

Output (`multiprocessing_demo.py`):

```
start method: spawn, CPUs: 14
  child: different pid from parent=True, sees counter=1000
parent counter after child ran: 0  (not shared!)  exitcode=0
shared Value after 4 x 1000 increments: 4000
pool.map squares: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
pool.map chunksize=3: [0, 1, 4, 9, 16] ...
lambda in Pool -> PicklingError
```

Shared-state options and caveats

Tool	What it is	Caveat
Return values / `Queue` / `Pipe`	Message passing (pickled)	Preferred; cost grows with data size
`Value`, `Array`	A C number/array in shared memory	Needs `get_lock()` for read-modify-write
`multiprocessing.shared_memory.SharedMemory`	Raw shared bytes (wrap in a numpy array)	You manage layout, lifetime and `unlink()`
`Manager()`	A server process holding a dict/list that proxies talk to	Easy but slow (every access is IPC — inter-process communication)

`Pool` methods: `map` (ordered, blocks), `imap` / `imap_unordered` (lazy, streamed), `starmap` (multiple arguments), `apply_async` (one call → `AsyncResult`). `chunksize` batches items per message — big chunks cut overhead for many small tasks.

When not to use it

- Tiny tasks — pickling and start-up (tens to hundreds of ms under `spawn`) outweigh the work.
- Huge inputs copied to every worker — use shared memory or let workers load their own slice.
- I/O-bound work — threads or asyncio are lighter.

Common mistakes

- Missing `__main__` guard → `RuntimeError` or runaway process creation under `spawn`.
- Expecting a child's global change to appear in the parent.
- Lambdas/local functions → `PicklingError` / `AttributeError`.
- `fork` with threads → the child may inherit a lock held by a thread that no longer exists, and hang.

Real-world examples

Gunicorn and uWSGI (production Python web servers) run several **worker processes** of a Django/Flask app to use all cores; Celery's default worker pool is "prefork" (processes started in advance); PyTorch `DataLoader(num_workers=4)` loads batches in worker processes.

16.8 `concurrent.futures` — executors

What it is

One high-level API for "run these calls in the background and give me the results": `ThreadPoolExecutor` and `ProcessPoolExecutor` share the interface. You get a **Future** — a result that will exist later.

How it works

1. `with ThreadPoolExecutor(max_workers=8) as ex:` — leaving the block waits for all work (`shutdown(wait=True)`).
2. `fut = ex.submit(fn, *args)`; `fut.result(timeout=None)` returns the value **or re-raises the worker's exception**.
3. `ex.map(fn, iterable)` → results in **input order**, lazily.
4. `as_completed(futures)` → futures in **completion order**.
5. `wait(futures, return_when=FIRST_COMPLETED | FIRST_EXCEPTION | ALL_COMPLETED)` → `(done, pending)`.
6. `fut.cancel()` works only before the call starts.

Default pool sizes (3.13+): threads `min(32, os.process_cpu_count() + 4)`; processes `os.process_cpu_count()`.

```
with ThreadPoolExecutor(max_workers=3) as ex:
    futures = {ex.submit(fake_download, u): u for u in urls}
    for fut in as_completed(futures):
        try:
            print(f"done {futures[fut]} -> {fut.result()}")
        except ConnectionError as e:
            print(f"failed {futures[fut]} -> {e!r}")

with ThreadPoolExecutor(max_workers=3) as ex:
    it = ex.map(fake_download, urls)
    try:
        for r in it:
            print(f"map result {r}")
    except ConnectionError as e:
        print(f"map raised {e!r}")
```

Output (`futures_demo.py`; delays a=0.3 s, b=0.1 s, c=0.2 s, c fails):

```
done b.com -> ('b.com', 100)
failed c.com -> ConnectionError('c.com timed out')
done a.com -> ('a.com', 300)
map result ('a.com', 300)
map result ('b.com', 100)
map raised ConnectionError('c.com timed out')
first completed: 1 done, 1 cancelled
primes: [True, True, True, False] in 0.32s with processes
```

`as_completed` returns the fastest first; `map` keeps input order and raises on *reaching* the failed item; with one worker and three 0.2 s jobs, after the first finishes the second is already running (uncancellable), so only the third is cancelled; switching to processes for CPU work is a one-word change.

Trade-offs

	Raw `threading` / `multiprocessing`	`concurrent.futures`
Return values & exceptions	Manual (queues, shared dicts)	Built in (`Future.result()`)
Thread ↔ process switch	Rewrite	Change the class name
Fine control (daemons, custom sync)	Yes	Limited
Best for	Long-lived workers, pipelines	"Run N calls, collect results"

In the interview

30-second answer: "`concurrent.futures` gives one API over thread and process pools. `submit` returns a Future; `result()` returns the value or re-raises the worker's exception. `map` preserves order, `as_completed` yields in finish order. `ThreadPoolExecutor` for blocking I/O, `ProcessPoolExecutor` for CPU-bound pure Python."

- *Exception in a worker?* — Stored in the Future; raised on `result()` (or iterating `map`). Never checked → silently lost.
- *Timeout?* — `fut.result(timeout=5)` or `as_completed(fs, timeout=5)` → `TimeoutError`.

16.9 `asyncio` — async/await

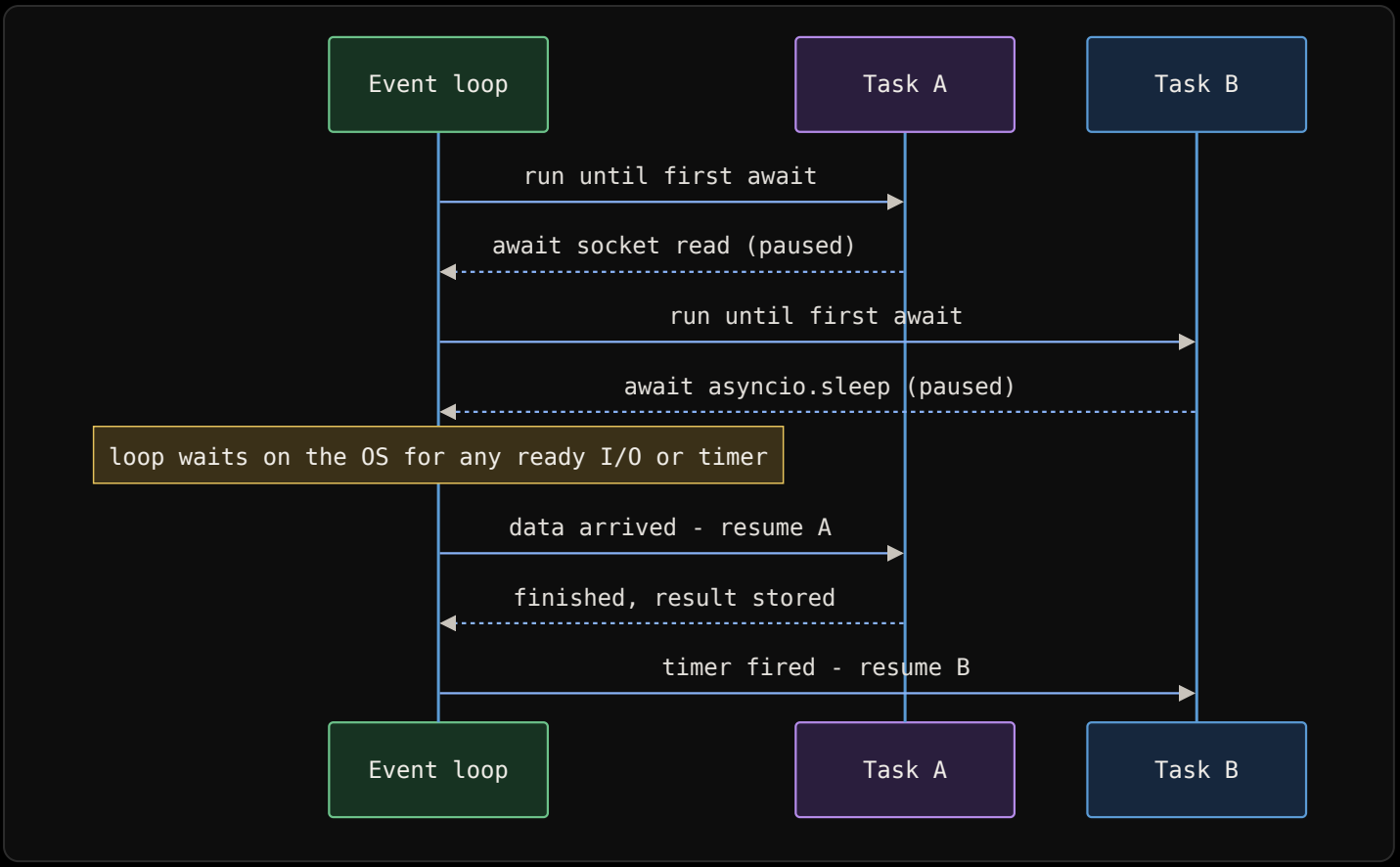
What it is

`asyncio` runs many tasks concurrently in **one thread** with an **event loop**. A **coroutine** (`async def`) pauses at each `await`, letting the loop run something else while it waits for I/O. Nothing is parallel; the speed comes from never sitting idle (one waiter serving 20 tables).

Why it matters

A thread costs stack memory and OS scheduling; 10,000 threads hurt, 10,000 asyncio tasks waiting on sockets are normal. Switches happen only at `await`, so there are far fewer race windows than with threads.

How it works



1. `asyncio.run(main())` creates a loop, runs `main()` to completion, closes it. Call once, at the top.
2. Calling `fetch()` creates a **coroutine object**; nothing runs until it's awaited or wrapped in a task.
3. `await x` pauses until `x` (coroutine, Task or Future) is done.
4. `asyncio.create_task(coro)` schedules it to run **concurrently**, returning a `Task`.
5. The loop uses OS readiness APIs (epoll on Linux, kqueue on macOS/BSD — report which of many connections have data) to resume tasks whose sockets/timers are ready.

Core API with real output (`asyncio_demo.py`)

```python
async def fetch(name: str, delay: float) -> str:
    await asyncio.sleep(delay)           # non-blocking wait
    return f"{name}:{delay}"

await fetch("a", 0.2); await fetch("b", 0.2)                    # sequential: 0.4 s
results = await asyncio.gather(fetch("a", 0.2), fetch("b", 0.2), fetch("c", 0.1))  # 0.2 s

task = asyncio.create_task(fetch("bg", 0.2))    # starts running now
await asyncio.sleep(0.1)
print(task.done())                              # False
value = await task
```

```
type before await: coroutine
awaited: x:0
[0.40s] two sequential awaits
[0.20s] gather -> ['a:0.2', 'b:0.2', 'c:0.1']  (order = argument order)
[0.10s] task done yet? False
[0.20s] task result bg:0.2
```

Key lesson: `await a(); await b()` is **sequential**; concurrency needs `gather`, `create_task` or a `TaskGroup`.

`gather` vs `TaskGroup` (3.11+)

```
res = await asyncio.gather(fetch("ok", 0.1), failing(0.05), return_exceptions=True)

try:
    async with asyncio.TaskGroup() as tg:
        slow = tg.create_task(fetch("slow", 1.0))
        tg.create_task(failing(0.1))
except* ValueError as eg:                       # except* handles an ExceptionGroup
    print(eg.exceptions, slow.cancelled())
```

An `ExceptionGroup` (3.11+) bundles several exceptions raised together (e.g. several failed tasks); `except*` handles each kind inside it.

```
gather with return_exceptions -> ['ok:0.1', ValueError('boom')]
[0.10s] TaskGroup raised (ValueError('boom'),); slow cancelled=True
```

	`asyncio.gather`	`asyncio.TaskGroup`
On first failure	Raises it, but the **other tasks keep running** unattended	**Cancels** the siblings, waits for them, raises an `ExceptionGroup`
Collect errors as values	`return_exceptions=True`	Catch with `except*`
Results	List in argument order	`task.result()` on each task
Recommended for new code	Simple fan-out (start many, collect results)	Yes — "structured concurrency": no task outlives its block

Timeouts, limits and async iteration

```
async with asyncio.timeout(0.1):                # 3.11+, can wrap several awaits
    await fetch("too-slow", 1.0)
await asyncio.wait_for(fetch("too-slow", 1.0), timeout=0.1)   # one awaitable

sem = asyncio.Semaphore(2)                      # at most 2 at a time
async def limited(i):
    async with sem:
        await asyncio.sleep(0.1)
        return i

async def ticker(n):                            # async generator
    for i in range(n):
        await asyncio.sleep(0.05)
        yield i

async with AsyncResource():                     # __aenter__ / __aexit__
    squares = [i * i async for i in ticker(4)] # async comprehension
```

```
asyncio.timeout -> TimeoutError
wait_for -> TimeoutError
[0.31s] 6 jobs, 2 at a time -> [0, 1, 2, 3, 4, 5]
[0.00s] open resource
[0.20s] close resource
async comprehension -> [0, 1, 4, 9]
```

On timeout the inner task is **cancelled** (`CancelledError` raised inside it) and the caller sees the built-in `TimeoutError` (since 3.11 `asyncio.TimeoutError` is the same class). Six jobs, limit two → 3 rounds × 0.1 s.

Async protocols: `async for` uses `__aiter__`/`__anext__`; `async with` uses `__aenter__`/`__aexit__`. Also useful: `asyncio.Queue`, `asyncio.Event`, `asyncio.Lock`, `asyncio.as_completed`.

Mixing sync and async

```
def blocking_io() -> str:
    time.sleep(0.3)                 # a sync library call (e.g. requests.get)
    return "sync result"

await asyncio.gather(asyncio.to_thread(blocking_io), asyncio.to_thread(blocking_io))
async def bad():
    return blocking_io()            # time.sleep inside a coroutine blocks EVERYTHING
await asyncio.gather(bad(), bad())
```

```
[0.31s] two blocking calls via to_thread ran together
[0.60s] two blocking calls inside coroutines ran one after another
```

One blocking call in a coroutine freezes the **entire** loop. Use async libraries (`httpx`/`aiohttp` not `requests`, `asyncpg` not sync `psycopg2`, `asyncio.sleep` not `time.sleep`) or push sync calls to a thread with `asyncio.to_thread` (3.9+) / `loop.run_in_executor`. CPU-heavy work → `ProcessPoolExecutor` via `run_in_executor`.

When to use asyncio

- **Yes:** many simultaneous network connections — web servers/APIs (FastAPI, Starlette, aiohttp), websockets (long-lived two-way connections), chat, crawlers, fan-out to many microservices (small independent services forming one application) or LLM APIs.
- **No:** CPU-bound work (one thread!), a script making three HTTP calls (threads are simpler), a codebase whose key libraries are all synchronous.

Common mistakes

- Forgetting `await` — `fetch()` alone only creates a coroutine; Python warns *"coroutine ... was never awaited"*.
- `create_task` without keeping the reference — the loop holds only a weak reference, so the task can be garbage-collected mid-run. Keep tasks in a set or use a `TaskGroup`.
- `asyncio.run()` inside a running loop (e.g. Jupyter) → `RuntimeError`; in notebooks just `await main()`.
- Catching `CancelledError` without re-raising — breaks timeouts and TaskGroup cancellation.
- Sharing `asyncio.Queue`/`Lock` across threads — not thread-safe; use `loop.call_soon_threadsafe` or `asyncio.run_coroutine_threadsafe`.

Real-world examples

FastAPI/Starlette on uvicorn (optional uvloop, a faster loop built on libuv — the C library behind Node.js); Discord bots (`discord.py`), Home Assistant, and the Anthropic and OpenAI Python SDKs' `Async...` clients.

In the interview

30-second answer: "asyncio is single-threaded cooperative concurrency — tasks voluntarily yield at `await` instead of being interrupted. The event loop resumes whichever task's I/O is ready. It scales to thousands of connections cheaply, but anything blocking stalls everything, so I use async libraries or `asyncio.to_thread`. For fan-out I prefer `TaskGroup` over `gather` because a failure cancels the siblings."

- *Is asyncio parallel?* — No, one thread.
- *`gather` vs `TaskGroup`?* — `gather` leaves siblings running on error; `TaskGroup` cancels them and raises an `ExceptionGroup`.
- *Calling sync code from async?* — `await asyncio.to_thread(fn, *args)`.
- *Cap concurrency at 10?* — `asyncio.Semaphore(10)`.

16.10 Choosing the right tool

Situation	Use	Why
A few to hundreds of blocking I/O calls, sync libraries	`ThreadPoolExecutor`	Simple, GIL released during I/O
Thousands of concurrent connections, async libraries	`asyncio`	Tasks are cheap, no thread per connection
CPU-bound pure Python	`ProcessPoolExecutor` / `multiprocessing.Pool`	Each process has its own GIL
CPU-bound numeric work	numpy/pandas (vectorize, Ch 17–18)	C loops, often multi-threaded internally
CPU-bound, want shared memory, deps support it	Free-threaded build (3.14t) + threads	Real parallel threads
Long-running pipeline stages	Threads/processes + `Queue`	Backpressure, clean shutdown
Async app needs a sync library	`asyncio.to_thread`	Keeps the loop responsive
Async app needs heavy CPU	`loop.run_in_executor(ProcessPoolExecutor(), ...)`	Moves CPU work off the loop
Scale beyond one machine	Celery, RQ, Ray, Dask, Spark	Distributed task queues/clusters across many machines

	Threads	Processes	asyncio
Parallel CPU (GIL build)	No	Yes	No
Memory per unit	~MB stack (virtual)	Whole interpreter (tens of MB)	~KB per task
Shared memory	Yes (needs locks)	No (pickle/IPC)	Yes (single thread)
Switching	Pre-emptive (the OS can interrupt at any time)	OS	Cooperative (at `await`)
Race risk	High	Low	Low
Works with sync libs	Yes	Yes	Needs `to_thread`

16.11 Common pitfalls

Blocking calls inside async code

`time.sleep`, `requests.get`, a sync DB driver, big file reads or a long CPU loop in a coroutine halts every task (§16.9). Diagnose with **debug mode** (`asyncio.run(main(), debug=True)` or `PYTHONASYNCIODEBUG=1`), which logs callbacks slower than 100 ms. Python 3.14 adds `python -m asyncio ps PID` / `pstree PID` to inspect a running process's tasks.

Deadlocks

Thread 1 holds lock A and waits for B; thread 2 holds B and waits for A — both wait forever.

```python
def transfer(first, second, name, log):
    with first:
        time.sleep(0.05)                              # make the bad interleaving likely
        if second.acquire(timeout=0.5):               # a real deadlock would wait forever
            second.release()
            log.append(f"{name} ok")
        else:
            log.append(f"{name} gave up: DEADLOCK")

run((lock_a, lock_b), (lock_b, lock_a))   # opposite order
run((lock_a, lock_b), (lock_a, lock_b))   # same order
```

Output (`pitfalls_demo.py`):

```
opposite order : ['T1 gave up: DEADLOCK', 'T2 ok']
same order     : ['T1 ok', 'T2 ok']
RLock re-entered by the same thread: fine
plain Lock re-acquire (timeout 0.1): False
```

The timeout lets the demo finish. Fixes:

1. **Global lock ordering** — always acquire in one order (e.g. by account id).
2. **Fewer locks** — one coarse lock, or no shared state (queues, immutable data).
3. **Timeouts** on `acquire` to detect and back off.
4. `RLock` when the same thread legitimately re-enters (a plain `Lock` deadlocks on itself — last line).
5. asyncio: a task awaiting something only it could produce, or a TaskGroup child waiting on its parent — same cure (clear ownership).

Other pitfalls

Pitfall	Symptom	Fix
Unprotected shared state	Wrong totals, rare	Lock, or message passing
Unhandled exception in a `Thread`	Printed to stderr, thread dies silently	Use executors and call `result()`; or set `threading.excepthook`
Too many threads	Memory, context-switch overhead (saving one thread's state, loading another's)	Bounded pool
Missing `__main__` guard	Recursive spawning / `RuntimeError`	Guard the entry point
Huge arguments to processes	Slow pickling	Shared memory, or load inside the worker
Forgotten `await` / lost task reference	Work silently never runs / vanishes	Linters (ruff), `TaskGroup`
`fork` after threads started	Random hangs in child	`spawn` / `forkserver` (defaults on macOS/Windows, and Linux from 3.14)

In the interview

- *Four conditions for deadlock?* — Mutual exclusion (one holder per resource), hold-and-wait (holding one lock while waiting for another), no pre-emption (locks can't be taken away), circular wait. Break any one (lock ordering breaks circular wait). Details: LLD Ch 12.
- *Find a hung thread?* — `faulthandler.dump_traceback_later()` or `py-spy dump --pid` show every thread's stack.

16.12 Chapter self-check

1. Concurrency vs parallelism in one line each? → Overlapping waits vs simultaneous execution on several cores.
2. Why do threads speed up downloads but not a pure-Python prime search? → I/O releases the GIL; bytecode execution doesn't.
3. Why is `counter += 1` unsafe across threads, and what does `with lock:` guarantee on an exception? → Load, add, store can interleave; the lock is released.
4. What does `q.task_done()` pair with? → `q.join()`, which waits until every item is marked done.
5. Default switch interval; how to check if the GIL is on? → 5 ms; `sys._is_gil_enabled()`.
6. Two ways to get CPU parallelism with a GIL? → Processes; C extensions that release the GIL (numpy).
7. Why must `multiprocessing` code have `if __name__ == "__main__":`? → `spawn`/`forkserver` re-import the main module in each child.
8. Linux default start method from 3.14? → `forkserver`.
9. `ex.map` vs `as_completed` ordering? → Input order vs completion order.
10. What runs when you call an `async def` function? Two `await`s in a row — concurrent? → Nothing (a coroutine object); no, sequential.
11. What does `asyncio.timeout` raise to the caller, and why is `time.sleep` in a coroutine bad? → `TimeoutError`; it blocks the whole loop.
12. Siblings when one TaskGroup task fails? → Cancelled; an `ExceptionGroup` is raised.
13. Run `requests.get` from async code without freezing the loop? → `await asyncio.to_thread(requests.get, url)`.
14. Status of free-threaded CPython in 3.14? → Officially supported (PEP 779), optional, not the default (`python3.14t`).
15. Simplest way to prevent lock-ordering deadlocks? → Always acquire locks in one global order.

Chapter TL;DR

- **First classify the work**: waiting (I/O-bound) or computing (CPU-bound). Measure.
- **Threads**: cheap concurrency for I/O; share memory → protect with `Lock`, or pass messages through `queue.Queue` (bounded, with sentinels).
- **GIL**: one thread runs Python bytecode at a time in standard CPython; released on I/O and by C extensions. Free-threaded build: experimental in 3.13, supported but optional in 3.14 (`python3.14t`, ~5–10 % single-thread cost).
- **Processes**: real parallelism; no shared globals; arguments are pickled; guard `__main__`; `spawn` on macOS/Windows, `forkserver` on Linux from 3.14.
- **Executors**: `ThreadPoolExecutor`/`ProcessPoolExecutor`, `submit` → `Future.result()`, `map` (ordered), `as_completed` (finish order). The default for "run N things".
- **asyncio**: one thread, event loop, coroutines pause at `await`. `gather`/`TaskGroup`, `asyncio.timeout`, `Semaphore`, async generators/context managers, `to_thread` for sync code. Never block the loop.
- **Pitfalls**: races, deadlocks (lock ordering), blocking calls in async, lost exceptions, missing `__main__` guard.

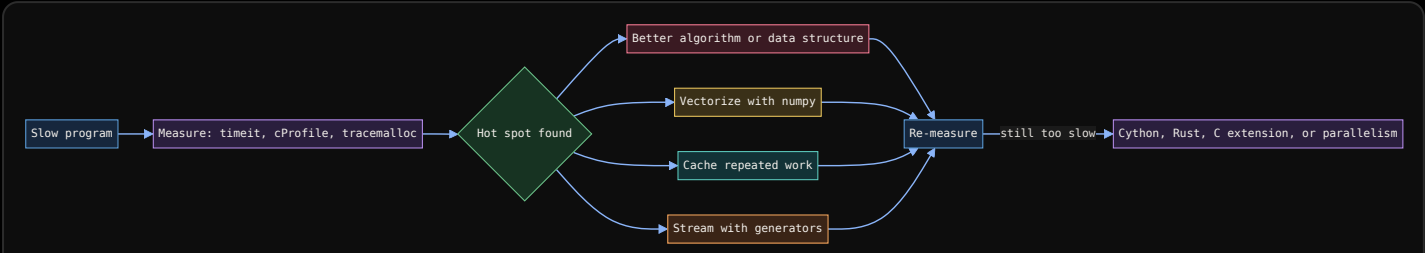
Chapter 17 — Performance, Memory and CPython Internals

What you will learn. How CPython represents every value as an object (`PyObject` : type + reference count + value); how memory is freed (reference counting plus the cyclic garbage collector); weak references; why `256 is 256` but maybe not `257 is 257` (small-int cache, string interning — one shared copy of equal values); mutability and copying recap; the **big-O cost of built-in operations** (list, dict, set, deque); how to **profile** (measure where time and memory go: `timeit` , `cProfile` , `tracemalloc` , line-level tools); and the optimisations that pay off — data structures, numpy vectorisation (one C-speed operation over a whole array instead of a Python loop), caching, generators, local lookups, `str.join` , `__slots__` — plus when to drop to C/Cython/Rust and what the "Faster CPython" releases (3.11+) changed.

Prerequisites. Ch 2 (names and objects), Ch 5–6 (containers), Ch 10 (classes), Ch 11 (generators), Ch 16 (the GIL).

OS fundamentals. What sits under every Python process — processes vs threads, CPU scheduling (which program runs on which core, when), virtual memory (each process sees a private address space the OS maps onto RAM) and page faults (the trap when touching memory not currently mapped), system calls (requests to the kernel, e.g. "read this file"), file descriptors (small integers identifying open files and sockets), blocking vs `epoll` / `kqueue` I/O (the mechanism under `asyncio` and `selectors`), and the page cache (file data kept in RAM for fast re-reads) — is explained once, language-neutrally, in the Java handbook's §16.13 Operating-system fundamentals for engineers. Python equivalents: `os.getpid()` / `multiprocessing` (processes), `threading` (threads under the GIL, Ch 16), `mmap` , `os.fsync` , `resource.getrlimit(resource.RLIMIT_NOFILE)` and `selectors` .

Code. `chapters/17-performance-memory-internals/code/` — `object_model.py` , `gc_weakref.py` , `complexity_bench.py` (plot), `profiling_demo.py` , `optimizations.py` (plot). Output from CPython 3.14.7, 64-bit macOS. Byte sizes and timings are implementation details and vary by version and machine.



17.1 The CPython object model

What it is

Everything is an object — numbers, strings, functions, classes, modules, `None` , types. Every object has:

Part	Meaning	How to see it
Identity	Which object it is; never changes during its life. In CPython it is the memory address.	`id(x)` , `x is y`
Type	What it can do; also never changes.	`type(x)`
Value	Its data. Changeable for *mutable* objects (list, dict, set), fixed for *immutable* ones (int, str, tuple, frozenset).	`==` compares values

In C, every object starts with a header struct `PyObject` holding `ob_refcnt` (the **reference count** — how many references point at it) and `ob_type` (a pointer to its type object). A variable is a **name** holding a reference to a heap object; when no name refers to it any more, it is freed.

How it works — demo

```
data = [1, 2, 3]
sys.getrefcount(data)             # 1
alias = data; holder = {"k": data}
sys.getrefcount(data)             # 3  (two more references)
del alias, holder                 # back to 1

x = 10; before = id(x); x += 1    # int is immutable -> x now names a NEW object
lst = [1]; before = id(lst); lst += [2]   # list is mutable -> same object, changed in place
```

Output (`object_model.py`):

```
refcount of new list: 1  (exact numbers are an implementation detail)
after alias + dict entry: 3
after del: 1
refcount of None: 3221225472  (immortal object, 3.12+)
a is b: True, a == c: True, a is c: False
type(a): list, type(type(a)): type, isinstance(len, object): True
int += changes id: True
list += keeps id: True
```

- `sys.getrefcount` traditionally reports one extra (its own argument), yet a fresh list shows 1: newer interpreters skip some refcount updates, so values vary by version. The docs say don't rely on it beyond 0 or 1 — watch it go **up and down**.
- `None`'s huge count: since 3.12 (PEP 683), `None`, `True`, small ints and some strings are **immortal** — refcount never changed, avoiding cache-line traffic (cores repeatedly invalidating each other's copies of the same memory) and helping the free-threaded build (Ch 16).
- `type(type(a))` is `type`: classes are objects whose type is `type`.

Object sizes (64-bit CPython 3.14)

```
getsizeof(int 0       ) =   28 bytes
getsizeof(int 1       ) =   28 bytes
getsizeof(int 2**30   ) =   32 bytes
getsizeof(int 2**100  ) =   40 bytes
getsizeof(float       ) =   24 bytes
getsizeof(str ''      ) =   41 bytes
getsizeof(str 'abc'   ) =   44 bytes
getsizeof(str 'é'     ) =   61 bytes
getsizeof(tuple ()    ) =   48 bytes
getsizeof(list []     ) =   56 bytes
getsizeof(list 3 items) =   88 bytes
getsizeof(dict {}     ) =   64 bytes
getsizeof(set()       ) =  216 bytes
getsizeof(None        ) =   16 bytes
```

A Python `int` costs ~28 bytes vs 8 in a numpy `int64` array; ints grow as needed (arbitrary precision); an ASCII string costs ~41 bytes + 1 per char, and one non-ASCII char switches the whole string to wider storage (PEP 393). `sys.getsizeof` is **shallow**: a list counts its pointer array (8 bytes per slot), not the items.

Common mistakes

- `is` for values (`if x is 1000`). `is` checks identity — only for singletons (`None`, `True`/`False`) and sentinels.
- Treating `id()` as unique forever — a dead object's address can be reused.

In the interview

> **30-second answer:** "In CPython every value is a heap object starting with a `PyObject` header — a reference count and a type pointer. Variables are names bound to objects. Identity is `id` / `is`,

type is fixed, and the value may or may not be mutable. Memory is freed when the refcount hits zero, with a cycle collector for reference cycles."

- `==` *vs* `is`*?* — Value equality (`__eq__`) vs same object.
- *Why is a list of a million ints ~36 MB but a numpy array ~8 MB?* — The list stores 8-byte pointers to separate ~28-byte ints; numpy stores raw 8-byte values contiguously.

17.2 Memory management and garbage collection

What it is

1. **Reference counting** — the main mechanism. Each new reference increments, each dropped one decrements; at 0 the object is freed **immediately**.
2. **Cyclic garbage collector (`gc`)** — a backup that finds **reference cycles** (objects referencing each other) unreachable from anything else; their counts never reach 0 on their own.

Underneath, **pymalloc** allocates small objects (≤ 512 bytes) from pools and arenas (big chunks requested from the OS once, then split) so tiny allocations are fast. Freed memory usually returns to pymalloc's pools, not the OS — why process memory can stay high after you delete data.

How it works

- **Refcounting is deterministic**: files close when the last reference goes, `__del__` runs at a predictable point (in CPython — other implementations differ, so still use `with`).
- **Cyclic GC** sorts container objects (lists, dicts, instances…) into **generations**: new objects start young, survivors are promoted; young ones are checked often, old rarely ("most objects die young"). A collection starts when allocations minus deallocations pass a threshold (`gc.get_threshold()`). Only containers are tracked; ints and strings can't form cycles.
- Version note: 3.14.0–3.14.4 shipped an *incremental* collector (old-generation work in slices to shorten pauses); 3.14.5 (May 2026) reverted to the 3.13-style generational collector after memory-pressure problems in production. The ideas hold for both.

```
class Node:
    def __init__(self, name): self.name, self.other = name, None

n = Node("solo")
weakref.finalize(n, print, "  solo freed as soon as last reference went away")
del n                                   # refcount -> 0 -> freed right here

gc.collect(); gc.disable()
a, b = Node("a"), Node("b")
a.other, b.other = b, a                 # reference cycle
weakref.finalize(a, print, "  cycle freed by gc.collect()")
del a, b                                # refcounts stay at 1 -> not freed yet
found = gc.collect()
```

Output (`gc_weakref.py`):

```
gc thresholds: (2000, 10, 10), gc enabled: True
  solo freed as soon as last reference went away
  (after del solo)
  (after del a, b: cycle still alive)
  cycle freed by gc.collect()
  gc.collect() found 3 unreachable objects
```

Weak references

A **weak reference** points to an object *without* increasing its refcount, so it doesn't keep it alive. `weakref.ref(obj)()` returns the object, or `None` once it died.

```
r = weakref.ref(obj);  del obj;  r()          # -> None
cache = weakref.WeakValueDictionary()          # entries disappear with their values
```

```
weakref alive: True
weakref after del: None
cache keys while x alive: ['x']
cache keys after del x: []
```

Uses: caches that shouldn't pin big objects, child → parent pointers in trees (avoids cycles), observer lists. `weakref.finalize(obj, fn)` is the safe way to run clean-up at death (better than `__del__`). `int`, `list`, `tuple` can't be weakly referenced; ordinary class instances can.

Trade-offs

	Reference counting	Cyclic GC
When memory is freed	Immediately	Periodically
Handles cycles	No	Yes
Cost	Small, on every reference change	Pauses proportional to tracked objects
Control	—	`gc.collect()`, `gc.disable()`, `gc.freeze()`, `gc.set_threshold()`

Common mistakes

- `__del__` for important clean-up (files, locks) — use `with` or `weakref.finalize`.
- Accidental cycles in long-lived caches, closures or tracebacks (a stored exception keeps its frames alive).
- `gc.collect()` in a loop "to be safe" — costs time, doesn't fix leaks from references you still hold.

Real-world examples

Instagram disabled, then **froze**, the cyclic GC in its Django workers: after `fork`, parent and child share memory pages until one *writes* (copy-on-write), and GC bookkeeping writes into every object, forcing huge copies; that work led to `gc.freeze()` (3.7). ORMs like SQLAlchemy keep an identity map (table of already-loaded rows) with weak references so unused rows can be freed.

In the interview

> **30-second answer:** "CPython primarily uses reference counting: at zero an object is freed immediately. Cycles never reach zero, so a generational cyclic garbage collector periodically finds unreachable groups of container objects. Weak references refer without keeping objects alive — useful for caches and back-pointers."

- *Can Python leak memory?* — Yes: objects still referenced by a global cache, an ever-growing list, or uncollectable cycles.
- *Finding a leak?* — Compare `tracemalloc` snapshots over time; `gc.get_referrers`; `objgraph`.

17.3 Small-int caching and string interning

Two optimisations that make *different-looking* values share one object:

- **Small-int cache**: CPython pre-creates ints **−5 to 256**; every result in that range reuses them.
- **String interning**: one shared copy per equal string. CPython auto-interns identifiers and many short identifier-like literals; force it with `sys.intern(s)`.

```
a, b = int("256"), int("256")           # built at run time, not constants
c, d = int("257"), int("257")
s1 = "".join(["hello", "_", "world"])     # built at run time: not interned automatically
s2 = "hello_world"
```

```
256 is 256: True   257 is 257: False   (small ints -5..256 are cached)
runtime str is literal: False, after sys.intern: True
```

(`int("...")` avoids the compiler merging equal constants in one code block, which would hide the effect.)

Why it matters: interned dict keys can match by pointer comparison (speed); millions of repeated strings (a "country" column) can be deduplicated with `sys.intern` (memory); and the interview trap — `is` on ints/strings is implementation-dependent. **Never use `is` for value comparison.**

17.4 Mutability and copying (recap)

Chapters 2 and 5 cover this in detail; the memory-level view:

```
grid = [[0, 0], [0, 0]]
alias = grid                      # same object
shallow = grid.copy()             # new outer list, SAME inner lists
deep = copy.deepcopy(grid)        # new everything
grid[0][0] = 99
bad = [[0] * 2] * 2               # two references to ONE inner list
def append_to(item, bucket=[]):  # default evaluated once, shared across calls
```

```
alias [99, 0], shallow [99, 0], deep [0, 0]
[[0]*2]*2 after bad[0][0]=1 -> [[1, 0], [1, 0]]
mutable default after two calls: [1, 2]
```

Operation	New outer object?	New inner objects?
`b = a`	No	No
`a.copy()`, `list(a)`, `a[:]`, `dict(a)`	Yes	No (shallow)
`copy.deepcopy(a)`	Yes	Yes (recursive, handles cycles)

Copying is O(n) — shallow copies copy pointers, deep copies everything (much slower). Immutables are safe to share, hashable, and thread-safe to read. Use `[[0] * cols for _ in range(rows)]` for grids and `None` as the default for mutable parameters.

17.5 Big-O of built-in operations

What it is

Big-O describes how cost grows with container size *n*: O(1) constant, O(n) proportional, O(log n) very slow growth. "Average" for dict/set assumes a good hash spread; "amortised" = occasional expensive resizes averaged over many cheap calls.

The table (CPython)

Operation	list	dict	set	deque
Index `x[i]` / key lookup `d[k]`	O(1)	O(1) avg	—	O(1) at ends, O(n) in middle
`x in c`	**O(n)**	O(1) avg	O(1) avg	O(n)
Append / add at right end	O(1) amortised	O(1) avg	O(1) avg	O(1)
Pop from right end	O(1)	`popitem` O(1)	—	O(1)
Insert / pop at left end	**O(n)** (`insert(0,x)`, `pop(0)`)	—	—	**O(1)** (`appendleft`, `popleft`)
Insert / delete in middle	O(n)	O(1) avg (`del d[k]`)	O(1) avg (`remove`)	O(n)
`len`	O(1)	O(1)	O(1)	O(1)
Iterate	O(n)	O(n)	O(n)	O(n)
Copy	O(n)	O(n)	O(n)	O(n)
Sort	O(n log n) (Timsort, Python's hybrid sort)	—	—	—
Slice `x[a:b]`	O(k) (k = slice length)	—	—	—
`min` / `max` / `sum`	O(n)	O(n)	O(n)	O(n)
Union / intersection	—	`d1 | d2` O(len d1 + len d2)	`s | t` O(len s + len t); `s & t` O(min) avg	—

Others: `heapq.heappush/heappop` O(log n), `heapify` O(n); `bisect` search O(log n) (`insort` O(n) — the list shift); `a + b` on strings O(len a + len b); `str.join` O(total length). Dict/set worst case O(n) when many keys collide (same hash slot) — rare with built-in hashes (string hashing is randomised per process to resist deliberate collisions).

Measured

```
setup = f"n={n}; L=list(range(n)); S=set(L); D=deque(L)"
per_op_us("(n-1) in L", setup, 200)               # worst case: last item
per_op_us("(n-1) in S", setup, 200_000)
per_op_us("L.append(L.pop(0))", setup, 2_000)
per_op_us("D.append(D.popleft())", setup, 200_000)
```

Output (`complexity_bench.py`, µs per operation):

```
        n            x in list            x in set    list.pop(0)+append  deque.popleft+append
    1,000                5.346               0.022                 0.098                 0.018
   10,000               54.597               0.023                 0.788                 0.017
  100,000              541.614               0.023                13.024                 0.018
1,000,000             5346.586               0.022               132.574                 0.017
```

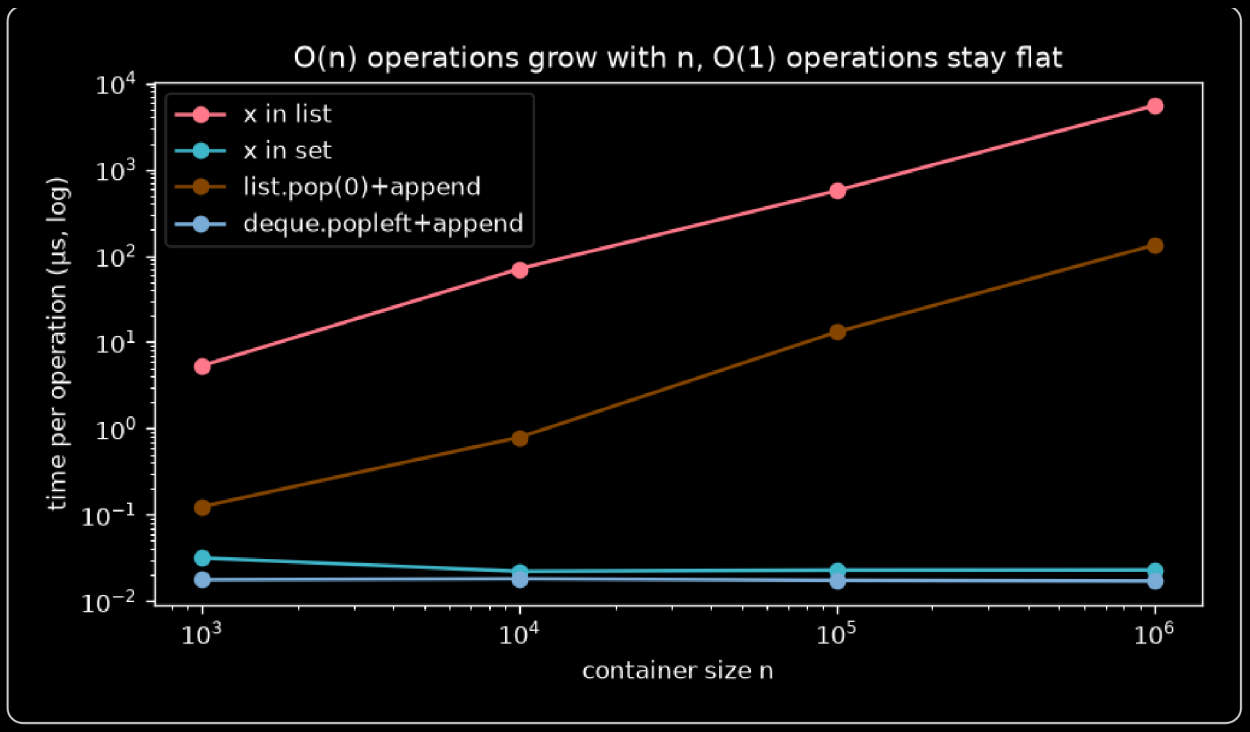


Figure — each 10× larger list makes `in` ~10× slower; set and deque don't care about n. At a million items, set lookup is ~250,000× faster.

Common mistakes

- `if x in some_list` in a loop → O(n²) (10× data, 100× slower). Build a `set` once.
- List as a queue (`pop(0)`) → `collections.deque`.
- Repeated `sorted()` in a loop for a running minimum → `heapq`.
- Building a set (O(n)) for a *single* lookup — the list is fine.

In the interview

30-second answer: "Lists are dynamic arrays (they grow by reallocating a bigger block and copying): O(1) index and append, O(n) insert/pop at the front and O(n) membership. Dicts and sets are hash tables with O(1) average lookup, insert and delete. Deque gives O(1) at both ends. The classic fix for slow code is turning repeated list membership into a set lookup."

Key term — hash table: items stored in an array of slots. A **hash function** turns each key into a number that picks the slot, so a lookup is one hash plus one probe instead of a scan — O(1) on average. When two keys pick the same slot (a *collision*) the table probes for another; with many collisions lookups degrade toward O(n). `dict` and `set` are hash tables, which is why keys must be hashable.

- *Why is `list.append` "amortised" O(1)?* — The array over-allocates; occasional resizes copy everything, but averaged per append it's O(1).
- *When is dict lookup O(n)?* — Pathological hash collisions.

17.6 Profiling — measure before you optimise

Profiling measures where time or memory goes. Guessing is usually wrong; the hot spot is often one innocent-looking line.

Tool	Answers	How
`timeit`	"How fast is this small snippet?"	`python -m timeit "sum(range(1000))"` or `timeit.repeat(...)`; in Jupyter `%timeit`
`cProfile` + `pstats`	"Which *functions* take the time?" (deterministic: every call counted)	`python -m cProfile -s cumtime script.py`
`line_profiler` (3rd-party)	"Which *lines* inside a function?"	decorate with `@profile`, run `kernprof -l -v script.py`; Jupyter `%lprun`
`py-spy`, `scalene`, `pyinstrument` (3rd-party)	Sampling profilers (peek at the program many times a second instead of recording every call): low overhead, attach to running processes; scalene also profiles memory and CPU vs native time	`py-spy top --pid 1234`
`tracemalloc`	"Which lines allocate memory?"	snapshots and `statistics("lineno")`
`time.perf_counter()`	Quick wall-clock timing of a block	subtract two readings

Python 3.15 (release candidates since August 2026; final due October 2026) adds a statistical sampling profiler to the stdlib, `profiling.sampling` (PEP 799); on 3.14 and earlier use the tools above.

How it works — a real hunt

```
def is_vip(customer_id: int, vip_ids: list[int]) -> bool:
    return customer_id in vip_ids             # O(len(vip_ids)) per call!

def build_report(n: int = 20_000) -> dict:
    records = load_records(n)
    vip_ids = list(range(0, n, 7))            # a list, not a set
    ...
prof = cProfile.Profile(); prof.enable(); build_report(); prof.disable()
pstats.Stats(prof).strip_dirs().sort_stats("cumulative").print_stats(4)
```

Output (`profiling_demo.py`):

```
timeit sum(range(1000)): 3.79 µs per call
         22861 function calls in 0.287 seconds
   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.003    0.003    0.287    0.287 profiling_demo.py:19(build_report)
    20000    0.281    0.000    0.281    0.000 profiling_demo.py:15(is_vip)
        1    0.003    0.003    0.003    0.003 profiling_demo.py:10(load_records)
     2858    0.000    0.000    0.000    0.000 {method 'get' of 'dict' objects}
list lookup: 278 ms  set lookup: 2.8 ms  speed-up x101  same result: True
records: 50,000, current 11.2 MB, peak 11.2 MB
top allocation site: line 11, 9.6 MB
```

Columns: **ncalls** = calls; **tottime** = time in the function itself; **cumtime** = including callees; **percall** = averages. `is_vip` has 98 % of the time → make `vip_ids` a `set` → ~100× faster. `tracemalloc` flags line 11 (the list of dicts) as the memory hog.

Steps that work every time:

1. Reproduce with realistic input size.
2. Profile the whole run (`cProfile`, or `py-spy` on a live server).
3. Sort by `cumtime` to find the expensive branch, then `tottime` for the leaf doing the work.
4. Fix the **algorithm/data structure** first; micro-optimise last.
5. Re-measure with `timeit`; check the result is unchanged.

Common mistakes

- A single timing run (noise) — `timeit.repeat`, take the minimum.
- Profiling tiny inputs where start-up dominates.
- Forgetting `cProfile`'s per-call overhead makes many tiny functions look worse; sampling profilers avoid this.

17.7 Optimisation techniques

Measured on 3.14

```
def concat_plus():              s = ""; for i in range(N): s += str(i)
def concat_join():              return "".join(str(i) for i in range(N))
def global_lookup(): ... total += math.sqrt(i)          # attribute lookup each time
def local_lookup():  sqrt = math.sqrt; ... total += sqrt(i)
def numpy_sum_squares():        a = np.arange(N, dtype=np.int64); return int((a * a).sum())
@lru_cache(maxsize=None)
def fib_cached(n): return n if n < 2 else fib_cached(n - 1) + fib_cached(n - 2)
```

Output (`optimizations.py`, N = 100,000):

```
string += vs join             6.33 ms ->    5.11 ms   x  1.2
for+append vs listcomp        2.45 ms ->    2.15 ms   x  1.1
math.sqrt vs local sqrt       3.27 ms ->    2.83 ms   x  1.2
python sum vs numpy           2.70 ms ->    0.04 ms   x 61.1
fib(25) vs lru_cache          5.85 ms ->   0.002 ms   x 2419   CacheInfo(hits=23, misses=26, maxsize=None,
currsize=26)
sum over 1M: list peak 40.4 MB vs generator peak 0.4 KB
100k objects: regular class 12.8 MB vs __slots__ 8.8 MB
__slots__ blocks new attributes: 'PointSlots' object has no attribute 'z' and no __dict__ for setting new
attributes
```

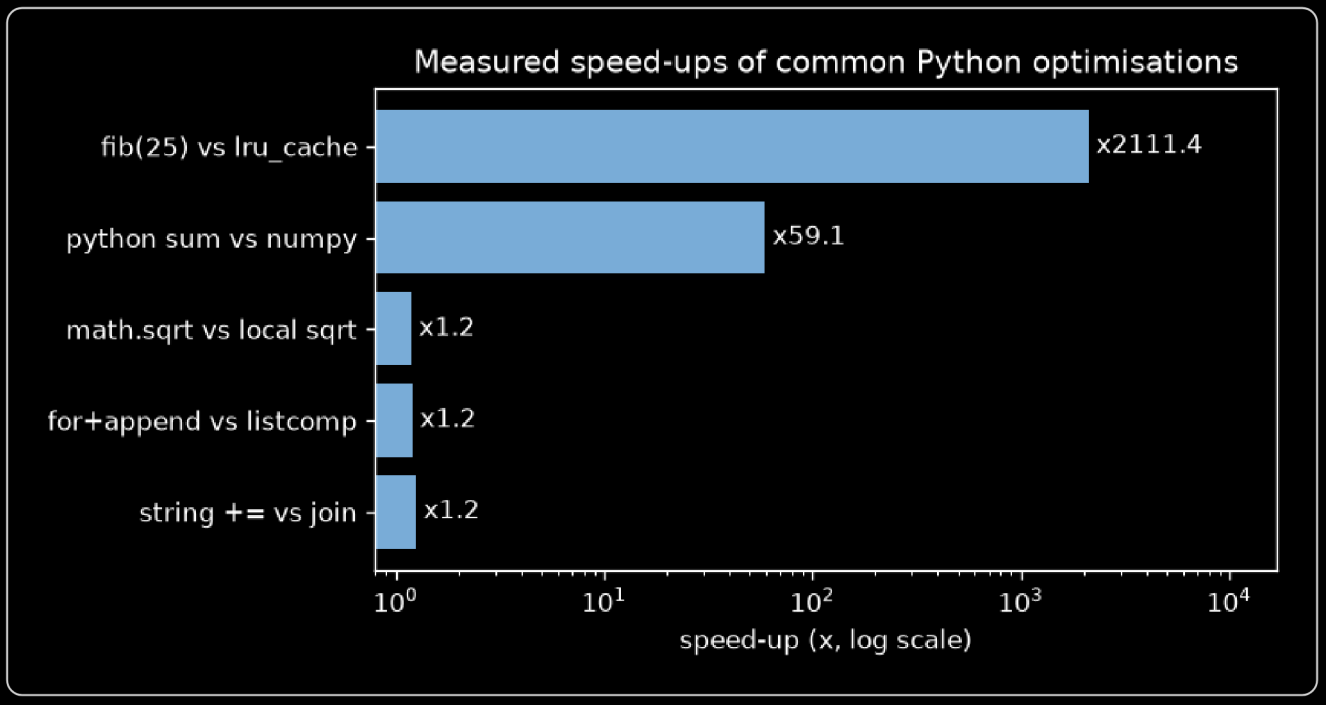


Figure — algorithmic and vectorisation changes win by orders of magnitude; micro-tweaks give 10–20 %.

The techniques, ranked by typical pay-off

#	Technique	Why it helps	Typical gain
1	**Better algorithm / data structure** (set for membership, dict for lookup, deque for queues, heapq for top-k)	Changes the big-O	10×–100,000×
2	**Caching** (`functools.lru_cache` / `cache` , memoisation, dicts)	Don't recompute the same answer	Up to exponential → linear
3	**Vectorise with numpy/pandas**	One C loop over raw numbers instead of a Python loop over objects	10×–100×
4	**Generators / streaming**	Constant memory, earlier results	Memory: GB → KB
5	**Built-ins and comprehensions** (`sum` , `max` , `sorted` , `"".join` , list comprehensions)	Loops run in C / specialised bytecode	1.1×–3×
6	**Local variable lookups** (alias `math.sqrt` before a hot loop)	Local access is an array index; globals/attributes are dict lookups	~1.1× on 3.11+
7	`__slots__` on classes with millions of instances	No per-instance `__dict__`	Less memory per object (31 % less total here)
8	**Parallelism** (Ch 16)	Use more cores	≤ number of cores

Details worth knowing:

- `str.join` : repeated `s += piece` can be O(n²), since strings are immutable and each `+=` may copy. CPython optimises in place when the string has one reference — hence the small gap above — but that's an implementation detail (not on PyPy; breaks once another reference exists). `"".join(parts)` is always linear. For bytes: `bytearray` or `io.BytesIO` .
- `lru_cache` : hashable arguments; `maxsize=None` (or `@functools.cache`) is unbounded — leak risk with many distinct inputs. Caches per function, not per instance; on methods it keeps `self` alive (use `functools.cached_property` for per-instance values).
- **Local lookups**: the 3.11+ specialising interpreter already caches global and attribute lookups, so the gain is small — only in measured hot loops.
- **Avoid needless work in loops**: hoist invariant computations, compute `len()` once, avoid exceptions for normal control flow on very hot paths (though `try` itself is zero-cost in 3.11+ when nothing is raised).

`__slots__`

Instances normally keep attributes in a per-instance dict; `__slots__ = ("x", "y")` replaces it with fixed slots:

```
class PointSlots:
    __slots__ = ("x", "y")              # fixed attributes, no per-instance __dict__
    def __init__(self, x, y):
        self.x, self.y = x, y
```

100,000 points: 12.8 MB vs 8.8 MB (the total includes the list and the ints, so the per-object saving is bigger than the ratio). Slightly faster attribute access too. Costs: no new attributes, no `__dict__` (some tools expect it), no weak references unless `"__weakref__"` is listed, and every class in the inheritance chain needs its own `__slots__` . `@dataclass(slots=True)` (3.10+) generates them. Since 3.11–3.13 ordinary instances store attributes more compactly ("inline values"), so the gap is smaller than old blog posts claim — measure.

When to drop to C, Cython or Rust

When a profiled hot loop can't be expressed in numpy/pandas and is still too slow:

Option	What it is	Good for
numba	JIT-compiles (compiles while the program runs) numeric Python functions (`@njit`) to machine code	Loops over numpy arrays
Cython	Python-like language with C types, compiled to a C extension	Gradual speed-up of existing Python; wrapping C libraries
Rust + PyO3/maturin	Write the extension in Rust	Safe, fast new libraries (pydantic-core, polars, ruff, tokenizers are written this way)
C/C++ + pybind11 / C API	Classic native extension	Existing C/C++ code
PyPy	Alternative Python with a tracing JIT (compiles the loops it sees running hot)	Pure-Python long-running programs (check C-extension support)
mypyc	Compiles type-annotated Python to C extensions	Typed codebases (mypy and the black formatter use it)

Exhaust algorithmic fixes and vectorisation first; native code adds a build step and a second language.

Python version performance (Faster CPython)

Version	What changed (from the official "What's New" notes)
3.11 (2022)	Specialising adaptive interpreter (PEP 659): bytecode rewrites itself for the types it sees. Cheaper frames, inlined Python-to-Python calls, "zero-cost" exceptions. Official figure: **10–60 % faster than 3.10, ~1.25× on average** on pyperformance (the standard benchmark suite).
3.12 (2023)	Inlined comprehensions (PEP 709, up to 2× faster for comprehension-heavy code), immortal objects (PEP 683), per-interpreter GIL (PEP 684).
3.13 (2024)	Experimental JIT compiler (PEP 744, off by default), experimental free-threaded build (PEP 703), improved interactive REPL.
3.14 (2025)	Free-threaded build officially supported (PEP 779); optional tail-calling interpreter (new internal dispatch; ~3–5 % faster when built with recent Clang); experimental JIT in the Windows and macOS installers, still off by default.

Upgrading from 3.10 to 3.11+ is often the cheapest speed-up you'll ever get.

When not to optimise

Code that runs once, I/O-bound code (1 ms CPU saved next to a 200 ms network call), anything a profiler doesn't show hot. Readability usually wins.

Common mistakes

- Micro-optimising before fixing an O(n²) algorithm.
- "Vectorising" by calling numpy element-by-element in a Python loop (slower than plain Python!).
- Unbounded caches growing forever in long-running services.
- Premature `__slots__` on classes with 50 instances.

Real-world examples

pandas and scikit-learn are fast because inner loops are C/Cython; polars and pydantic v2 moved their cores to Rust; Django uses `lru_cache` / `cached_property` for settings and URL resolution; mypyc was built by the mypy team (originally at Dropbox) to compile mypy itself, making it several times faster.

In the interview

> **30-second answer:** "First I profile — cProfile or py-spy — to find the hot spot. The biggest wins are algorithmic: the right data structure, caching repeated work, vectorising numeric loops with numpy. Then smaller things: built-ins, comprehensions, `join` instead of `+=`, generators for memory, `__slots__` for many small objects. If that's not enough, I move the hot loop to numba, Cython or Rust, or parallelise."

- *Why is numpy faster than a Python loop?* — Contiguous typed memory, one C loop, no per-element object creation or dispatch, often SIMD (single instruction, multiple data — one CPU instruction processing several numbers).
- *Downside of `lru_cache`?* — Memory growth and stale results; arguments must be hashable.
- *How much faster was 3.11 than 3.10?* — ~25 % on average (10–60 %).

17.8 Chapter self-check

1. Three properties of every object, and what starts every `PyObject`? → Identity, type, value; refcount and type pointer.
2. Does `x += 1` on an int change the object? → No, it rebinds `x` to a new int.
3. Is `sys.getsizeof([big objects])` the total memory? → No, it's shallow.
4. When is an object freed in CPython? → Refcount reaches zero, or the cyclic GC finds it unreachable in a cycle.
5. Why do GC generations exist? → Most objects die young; scan young often, old rarely.
6. Does a weak reference keep an object alive? What does `WeakValueDictionary` do when a value dies? → No; removes the entry.
7. Which ints are cached? → −5 to 256.
8. Cost of `x in list` vs `x in set`? → O(n) vs O(1) average.
9. Best structure for a FIFO queue? → `collections.deque`.
10. `tottime` vs `cumtime`? → Own time vs including callees.
11. Why does `sum(i*i for i in range(10**6))` use almost no memory? → The generator streams values.
12. Why `''.join` over `+=`? → Guaranteed linear; `+=` can be quadratic.
13. Downside of `__slots__`? → No new attributes, no `__dict__`, no weakrefs unless listed.
14. Two ways to speed up a numeric loop numpy can't express? → numba, Cython (or Rust/PyO3).
15. First thing to try before rewriting in Rust? → Profile and fix the algorithm / vectorise.

Chapter TL;DR

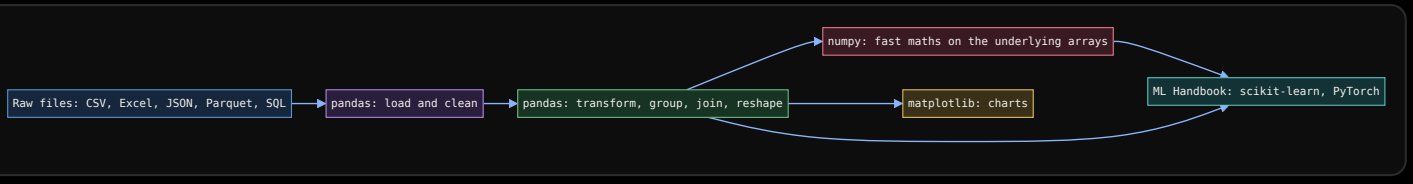
- **Objects**: identity (`id`, `is`), type, value; `PyObject` = refcount + type pointer; names are references; immortal objects since 3.12.
- **Memory**: refcounting frees immediately; generational cyclic GC handles cycles; `weakref` refers without owning; `gc.freeze()` helps pre-fork servers.
- **Caching quirks**: small ints −5…256 and interned strings are shared — always compare with `==`.
- **Copying**: assignment shares, `.copy()` is shallow, `deepcopy` is deep; immutables are safe to share.
- **Big-O**: list O(1) end / O(n) front and `in`; dict/set O(1) average; deque O(1) both ends; heapq O(log n).
- **Profile**: `timeit`, `cProfile` / `pstats`, `line_profiler`, `py-spy` / `scalene`, `tracemalloc`.
- **Optimise**: data structure → caching → numpy → generators → built-ins/join/locals → `__slots__` → numba/Cython/Rust → parallelism. Newer CPython (3.11+) is substantially faster for free.
- **OS layer** (processes, threads, paging, syscalls, fds, epoll, page cache): see Java handbook §16.13.

Chapter 18 — The Data Stack: numpy, pandas, matplotlib

What you will learn. The three libraries almost every Python data, science and ML (machine learning — programs that learn patterns from data rather than hand-written rules) project stands on. **numpy**: fast n-dimensional arrays, dtypes, indexing and views, broadcasting, vectorisation, aggregation along axes, basic linear algebra (vectors and matrices) and the modern random API. **pandas**: `Series` and `DataFrame`, file I/O, `loc` / `iloc`, filtering, `groupby` / `agg` / `transform`, `merge` / `concat`, `pivot_table` / `melt`, missing data, datetimes, why `apply` is slow, performance tips — and what changed in **pandas 3.0** (string dtype, Copy-on-Write). **matplotlib**: Figure and Axes, the four workhorse charts, subplots, labels, saving images. Ends with a small end-to-end analysis: messy CSV → clean → analyse → chart.

Where this leads. The ML Handbook, Chapter 3 — The Python ML stack, takes these tools into ML (feature matrices — numeric inputs, one row per example; scikit-learn pipelines — chained preprocessing + model steps; train/test splits — holding back data to test the model on unseen examples). This chapter stays with general data work, no scikit-learn.

Code. `chapters/18-data-stack/code/` — `numpy_basics.py`, `pandas_basics.py`, `matplotlib_demo.py`, `end_to_end.py`, `plot_style.py` (shared chart style). Plots go to `img/`. Versions: Python 3.14.7, numpy 2.5, pandas 3.0.6, matplotlib 3.11.



Library	Core object	Think of it as
numpy	`ndarray`	A grid of numbers of one type, stored in one block of memory
pandas	`DataFrame` / `Series`	A spreadsheet table with labelled rows and columns; columns can have different types
matplotlib	`Figure` / `Axes`	A canvas with one or more plots drawn on it

---

18.1 numpy arrays

What it is

numpy (Numerical Python) provides the `ndarray`: an n-dimensional array whose elements share one **dtype** (`int64`, `float64`, `bool` …) and sit in one contiguous block of memory. **shape** is the tuple of sizes per dimension (`(3, 4)` = 3 rows, 4 columns); `ndim` is the number of dimensions. A list is a shelf of pointers to objects stored elsewhere; an array is an egg carton — uniform slots in one piece, processed all at once.

Why it matters

- **Speed**: compiled C loops over raw numbers — typically 10–100× faster than Python loops (35× measured below).
- **Memory**: 8 bytes per `float64` vs ~24–32 bytes per Python float plus an 8-byte pointer.
- **Ecosystem**: pandas columns, scikit-learn inputs, OpenCV/Pillow images and PyTorch tensors (which convert to and from numpy) all build on it.

How it works — creating and inspecting

```
a = np.array([1, 2, 3])                       # from a list
np.zeros((2, 3)); np.ones(3, dtype=np.int32)  # filled arrays
np.arange(0, 10, 3); np.linspace(0, 1, 5)     # ranges: step-based vs count-based
m = np.arange(12).reshape(3, 4)               # 3 rows x 4 cols
np.array([1, 2.5]).dtype                      # mixed -> upcast to float64
np.array([1.7, -1.7]).astype(int)             # truncates toward zero
```

Output (`numpy_basics.py`):

```
[1 2 3] int64 (3,) 1
[[0. 0. 0.]
 [0. 0. 0.]]
[1 1 1]
[0 3 6 9]
[0.   0.25 0.5  0.75 1.  ]
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
m.shape=(3, 4) m.size=12 m.dtype=int64 m.nbytes=96 itemsize=8
mixed ints and floats -> float64; astype -> [ 1 -1]
```

`reshape` needs the same total size (12 = 3 × 4); `-1` lets numpy compute one dimension (`reshape(-1, 2)`). "Upcast" = numpy picks a wider type that holds every value. Other creators: `np.full`, `np.eye` (identity matrix), `np.empty` (uninitialised — fast, contains garbage), `np.zeros_like(m)`.

Indexing, slicing, views and masks

```
m[1, 2]; m[0]; m[:, 1]; m[-1, ::2]            # element, row, column, strided
view = m[:2, :2]; view[0, 0] = 100            # slicing returns a VIEW -> m changes too
mask = m % 2 == 0; m[mask]                    # boolean mask selects matching elements
m[[2, 0]]                                     # fancy (list) indexing returns a COPY
np.where(m > 5, m, -1)                        # vectorised if/else
```

```
m[1, 2]=6  m[0]=[0 1 2 3]  m[:, 1]=[1 5 9]  m[-1, ::2]=[ 8 10]
after view[0,0]=100, m[0,0]=100  shares memory: True
even values: [ 0  2  4  6  8 10]
fancy rows [2, 0]:
[[ 8  9 10 11]
 [ 0  1  2  3]]
np.where(m > 5, m, -1):
[[-1 -1 -1 -1]
 [-1 -1  6  7]
 [ 8  9 10 11]]
```

Indexing kind	Example	Returns
Basic (ints, slices)	`m[1:, ::2]`	**View** (shares memory)
Boolean mask	`m[m > 5]`	Copy (1-D)
Fancy (int lists/arrays)	`m[[2, 0]]`	Copy

Unlike lists, a numpy slice is a **view** — a new array object over the *same* memory, so changing it changes the original ("strided" = every k-th element). Use `.copy()` for independence. Combine masks with `&`, `|`, `~` and parentheses: `m[(m > 2) & (m < 9)]` (`and`/`or` raise on arrays).

Vectorised maths and aggregation along an axis

```
x*2=[2. 4. 6. 8.]  x**2=[ 1.  4.  9. 16.]  np.sqrt(x)=[1.    1.414 1.732 2.   ]  x>2=[False False  True  True]
m.sum()=66  m.sum(axis=0)=[12 15 18 21]  m.sum(axis=1)=[ 6 22 38]
m.mean()=5.50  m.max(axis=1)=[ 3  7 11]  m.argmax()=11  cumsum=[ 1  3  6 10]
```

Arithmetic and **ufuncs** (universal functions: `np.sqrt`, `np.exp`, `np.log`, `np.abs` …) work element-wise. For reductions (many values → one, like `sum`, `max`), `axis=0` collapses the rows — one result **per column**; `axis=1` — one result **per row**. `argmax` gives the position of the maximum (in the flattened 1-D array unless `axis` is given).

Vectorisation, measured

```
[v * 2 + 1 for v in big]      # Python: one bytecode loop, one int object per element
arr * 2 + 1                   # numpy: two C loops over raw int64s
```

```
1M elements: list comprehension 25.8 ms, numpy 0.75 ms, x35
```

Vectorisation = whole-array operations instead of a Python loop. If you're writing `for i in range(len(arr))` over an array, look for the array operation (`np.where`, masks, `cumsum`, `np.diff`, `np.clip`, broadcasting).

Everyday array functions: **join** — `np.concatenate([a, b])` (existing axis), `np.stack` / `np.vstack` / `np.hstack` (new or row/column axis); **sort** — `np.sort(a)` (copy), `a.sort()` (in place), `np.argsort(a)` (index order, e.g. to sort one array by another); **deduplicate/count** — `np.unique(a, return_counts=True)`; **reshape views** — `a.T`, `a.ravel()`, `a[:, None]`.

Common mistakes

- Expecting a slice to be a copy (it's a view) — or the reverse with fancy indexing.
- Integer overflow: fixed-size dtypes wrap (`np.int8(127) + np.int8(1)` → −128 — numpy warns for scalars, arrays wrap silently); Python ints never overflow.
- `==` on floats — use `np.isclose` / `np.allclose`.
- `np.append` in a loop (copies each time, O(n²)) — collect in a list, convert once.
- Mixing up `axis=0` and `axis=1`.

18.2 Broadcasting

What it is

Broadcasting combines arrays of different shapes without copying: the smaller one is (virtually) stretched to match.

How it works — the rule

Compare shapes **from the right-most dimension leftwards**:

1. Pad the shorter shape on the left with 1s.
2. Two dimensions are compatible if **equal** or **one is 1**.
3. A size-1 dimension stretches; any other mismatch → `ValueError`.

Worked example: `(3, 1) + (4,)` → `(3, 1)` vs `(1, 4)` → `(3, 4)`.

```
col = np.array([[10], [20], [30]])                # shape (3, 1)
row = np.array([1, 2, 3, 4])                      # shape (4,)
col + row                                         # shape (3, 4)
z = (data - data.mean(axis=0)) / data.std(axis=0)   # standardise each column: (3,2) op (2,)
np.ones((3, 2)) + np.ones(3)                      # (3,2) vs (3,) -> last dims 2 vs 3 -> error
```

```
(3,1) + (4,) -> (3, 4)
[[11 12 13 14]
 [21 22 23 24]
 [31 32 33 34]]
column-standardised:
[[-1.225 -1.225]
 [ 0.     0.   ]
 [ 1.225  1.225]]
(3,2) + (3,) -> ValueError: operands could not be broadcast together with shapes (3,2) (3,)
```

To subtract a **per-row** value, make it a column: `data - data.mean(axis=1, keepdims=True)` (or `[:, None]` / `np.newaxis`).

Real-world: standardising features (column means/stds), normalising images (`img / 255.0`, or a per-channel `(3,)` mean from an `(H, W, 3)` image), pairwise distances (`a[:, None, :] - b[None, :, :]`).

Self-check: `(5, 1, 3) + (4, 1)` → `(5, 4, 3)`. `(3, 2) + (3,)` fails because the right-most dims 2 and 3 differ and neither is 1.

18.3 Linear algebra and random numbers

Linear algebra basics

```
A = np.array([[3.0, 1.0], [1.0, 2.0]]); b = np.array([9.0, 8.0])
A @ A                       # matrix multiplication (same as np.matmul); A * A is element-wise!
np.linalg.solve(A, b)       # solve A x = b
np.linalg.det(A); np.linalg.inv(A); np.linalg.eigh(A); np.linalg.norm(b); A.T
```

```
A @ A =
[[10.  5.]
 [ 5.  5.]]
solve A x = b -> x = [2. 3.], check A @ x = [9. 8.]
det = 5.000, inv =
[[ 0.4 -0.2]
 [-0.2  0.6]]
eigenvalues = [1.382 3.618], norm of b = 12.042, A.T =
[[3. 1.]
 [1. 2.]]
```

Glossary: **determinant** — one number summarising a square matrix (zero → no inverse); **inverse** — the matrix that undoes `A`; **eigenvalues** — factors by which `A` stretches its special directions (eigenvectors); **norm** — a vector's length. Checks: 3·2 + 1·3 = 9, 1·2 + 2·3 = 8 ✓; det = 3·2 − 1·1 = 5 ✓; norm = √(9² + 8²) = √145 ≈ 12.042 ✓.

Need	Use	Note
Matrix product	`A @ B`	`*` is element-wise
Solve `A x = b`	`np.linalg.solve`	Faster and more accurate than `inv(A) @ b`
Least squares	`np.linalg.lstsq`	Over-determined systems (more equations than unknowns)
Eigen-decomposition	`np.linalg.eigh` (symmetric), `eig` (general)	With numpy 2.5 here, `eig` returned a complex dtype even for this real-valued case; `eigh` returns real values sorted ascending
SVD (singular value decomposition — a matrix split into rotation and scaling parts)	`np.linalg.svd`	Basis of PCA (principal component analysis — compressing data to its most informative directions)
Dot product / length	`a @ b`, `np.linalg.norm(a)`	

The maths (vectors, matrices, eigenvalues) is in the ML Handbook's maths chapter.

Random numbers

```
rng = np.random.default_rng(seed=42)        # modern Generator API
rng.integers(0, 10, size=5); rng.normal(0, 1, size=3); rng.choice(["a", "b", "c"], size=4)
rng.shuffle(arr5)                           # in place
```

```
integers: [0 7 6 4 4]
normal:   [ 0.941 -1.951 -1.302]
choice:   ['c' 'c' 'c' 'c']
shuffle:  [1 0 2 4 3]
same seed, same numbers: [0 7 6 4 4]
```

Use `np.random.default_rng(seed)` (same seed → same sequence) and pass the `rng` around. The old global functions (`np.random.seed`, `np.random.rand`) share hidden global state, making reproducibility across modules fragile. `rng.integers(low, high)` excludes `high`. Also: `random`, `uniform`, `permutation`, `binomial`, `poisson`. (Four 'c's is just chance with this seed.)

In the interview

30-second answer: "A numpy array is a typed, contiguous block of memory with a shape, so operations run as C loops — vectorised code is often tens of times faster than Python loops. Slices are views, fancy indexing copies. Broadcasting lines up shapes from the right, and a dimension of 1 stretches. I use `@` for matrix multiply, `linalg.solve` rather than inverting, and `default_rng(seed)` for reproducible randomness."

- *Why is numpy fast?* — Contiguous typed memory, compiled loops, no per-element Python objects, SIMD, and BLAS/LAPACK (decades-old, heavily optimised linear-algebra libraries).
- *`a * b` vs `a @ b`?* — Element-wise vs matrix product.
- *Reproducible results?* — A seeded `Generator` passed explicitly.

18.4 pandas: Series and DataFrame

What it is

- **Series** — a 1-D labelled array: values plus an **index** (row labels).
- **DataFrame** — a table: Series (columns) sharing one index, each with its own dtype.

```
s = pd.Series([10, 20, 30], index=["a", "b", "c"], name="score")
s["b"]; s.iloc[0]; s[s > 15]; s.mean()
```

```
a    10
b    20
c    30
Name: score, dtype: int64
s['b']=20  s.iloc[0]=10  s[s > 15].tolist()=[20, 30]  s.mean()=20.0
```

Operations between Series **align on the index**: `s1 + s2` adds matching labels; labels in only one become `NaN` (missing). This alignment is pandas' biggest difference from numpy.

Reading data and first look

```
df = pd.read_csv(io.StringIO(CSV), parse_dates=["date"])   # usually pd.read_csv("orders.csv", ...)
df.head(3); df.dtypes; df.shape; df.describe()
```

```
   order_id       date customer   city   product   qty  price
0         1 2026-01-03      ana   Pune       pen  10.0    1.5
1         2 2026-01-03      ben  Delhi  notebook   2.0    4.0
2         3 2026-01-10      ana   Pune  notebook   NaN    4.0
order_id             int64
date        datetime64[us]
customer               str
city                   str
product                str
qty                float64
price              float64
shape=(8, 7)
       order_id                 date        qty      price
count       8.0                    8   7.000000   7.000000
mean        4.5  2026-02-03 00:00:00   6.142857   5.928571
min         1.0  2026-01-03 00:00:00   1.000000   1.500000
max         8.0  2026-03-05 00:00:00  20.000000  25.000000
```

- `qty` is `float64` because it has a missing value: `NaN` (Not a Number, numpy's missing marker) is a float. The nullable dtype `"Int64"` (capital I) keeps integers with missing values (`<NA>`).
- Text columns are **`str`** — new in pandas 3.0 (earlier: generic `object`). It holds only strings (and missing values), backed by PyArrow (Python library for Apache Arrow, a fast standard columnar in-memory format) when installed, else Python objects.
- Dates parse at **microsecond** resolution (`datetime64[us]`) in pandas 3.0 (earlier: nanoseconds).
- First-look checklist: `head()`, `info()` (dtypes, non-null counts, memory), `describe()` (count, mean, std, min, quartiles — the 25/50/75 % points — max), `value_counts()` on categorical columns, `isna().sum()`.

Input and output

Format	Read	Write	Notes
CSV	`pd.read_csv(path, usecols=, dtype=, parse_dates=, chunksize=)`	`df.to_csv(path, index=False)`	Universal, slow, loses dtypes
Excel	`pd.read_excel`	`df.to_excel`	Needs `openpyxl`
JSON	`pd.read_json(lines=True)`	`df.to_json(orient="records")`	APIs, logs
Parquet	`pd.read_parquet`	`df.to_parquet`	Columnar (each column stored together, so reading a few is cheap), compressed, keeps dtypes, fast; needs `pyarrow`. Preferred for anything big.
SQL	`pd.read_sql(query, conn)`	`df.to_sql(name, conn)`	Via SQLAlchemy or `sqlite3`
Clipboard / HTML	`read_clipboard`, `read_html`		Quick grabs

`chunksize=100_000` makes `read_csv` return an iterator of DataFrames — process larger-than-memory files piecewise.

18.5 Selecting data: `loc`, `iloc` and filters

What it is

Accessor	Selects by	Slice end	Example
`df["col"]` / `df[["a", "b"]]`	Column name(s)	—	one Series / a DataFrame
`df.loc[rows, cols]`	**Labels** (index values, column names) or boolean masks	**Inclusive**	`df.loc[1:2, ["customer", "qty"]]` → rows labelled 1 **and** 2
`df.iloc[rows, cols]`	**Integer positions**	Exclusive (like Python)	`df.iloc[0, 2]` → first row, third column
`df.at[r, c]` / `df.iat[i, j]`	One cell, fast	—	

```
df.loc[0, "customer"]; df.iloc[0, 2]; df.loc[1:2, ["customer", "qty"]]
df[(df["city"] == "Pune") & (df["product"] == "pen")]       # boolean mask, parentheses required
df.query("qty >= 5 and price < 2")                          # same idea as a string expression
df[df["product"].isin(["bag", "notebook"])]
```

```
ana | ana | {'customer': ['ben', 'ana'], 'qty': [2.0, nan]}
{'order_id': [1, 8], 'qty': [10.0, 20.0]}
[1, 4, 8]
[2, 3, 5, 6, 7]
```

Other filters: `.between(a, b)`, `.str.contains("x")`, `.str.startswith`, `.isna()` / `.notna()`, `~mask` for NOT.

Sorting and ranking: `df.sort_values("revenue", ascending=False)` (or a list of columns), `df.sort_index()`, `df.nlargest(3, "revenue")`, `df["city"].value_counts()` (most common first; `normalize=True` for shares).

Setting values and Copy-on-Write

Set with **one** `.loc` call:

```
df.loc[df["revenue"] >= 20, "tier"] = "big"      # correct
# df["tier"][df["revenue"] >= 20] = "big"       # chained assignment: never modifies df in pandas 3
```

In **pandas 3.0, Copy-on-Write (CoW)** is always on: every indexing result *behaves as a copy*, so changing it never changes the original, and chained assignment (`df[a][b] = v`) does nothing to `df` (pandas warns). `SettingWithCopyWarning` is gone. Memory is still shared until one side is modified, so it stays fast.

```
sub = df[df["city"] == "Pune"]
sub["qty"] = 0                                   # modifies only `sub`, never df
```

```
df Pune qty still: [10, 0, 2, 20]
```

(The 0 is the order whose missing quantity was filled with 0 earlier.)

Common mistakes

- `and` / `or` instead of `&` / `|` → `ValueError: The truth value of a Series is ambiguous`.
- Missing parentheses around each condition (`&` binds tighter than `==`).
- Assuming `loc` slices exclude the end label.
- Relying on chained assignment (silently ineffective in pandas 3).

18.6 Cleaning: missing data and new columns

Missing data

Missing values are `NaN` (floats), `NaT` (datetimes) or `pd.NA` (nullable dtypes); `None` in input becomes one of these.

Task	Method
Find	`df.isna()`, `df.isna().sum()`, `df.info()`
Drop	`df.dropna()` (any NaN in row), `dropna(subset=["qty"])`, `dropna(thresh=5)`
Fill with constant	`df["qty"].fillna(0)`
Fill with statistic	`fillna(df["price"].median())` or per group with `transform` (below)
Fill from neighbours	`ffill()` (copy the previous value down), `bfill()` (backward), `interpolate()` (estimate in-between values; time series)
Replace sentinel values (placeholder codes like -1 = "unknown")	`df.replace(-1, np.nan)`

```
df["qty"] = df["qty"].fillna(0).astype(int)
df["price"] = df["price"].fillna(df.groupby("product")["price"].transform("median"))
```

```
{'qty': 1, 'price': 1}
after fill: nulls=0, order 6 (missing price, bag) filled with bag median=25.0
```

The missing bag price got the median price of *bags*, not all products — group-aware filling is usually more sensible. Drop vs fill is a modelling decision: drop when few rows are affected at random; fill (maybe with a "was_missing" flag column) when missingness is common or informative. Aggregations skip `NaN` by default, and `NaN != NaN`, so test with `isna()`.

New columns, vectorised

```
df = df.assign(revenue=df["qty"] * df["price"], month=df["date"].dt.to_period("M"))
df.loc[df["revenue"] >= 20, "tier"] = "big"
df["tier"] = df["tier"].fillna("small")
```

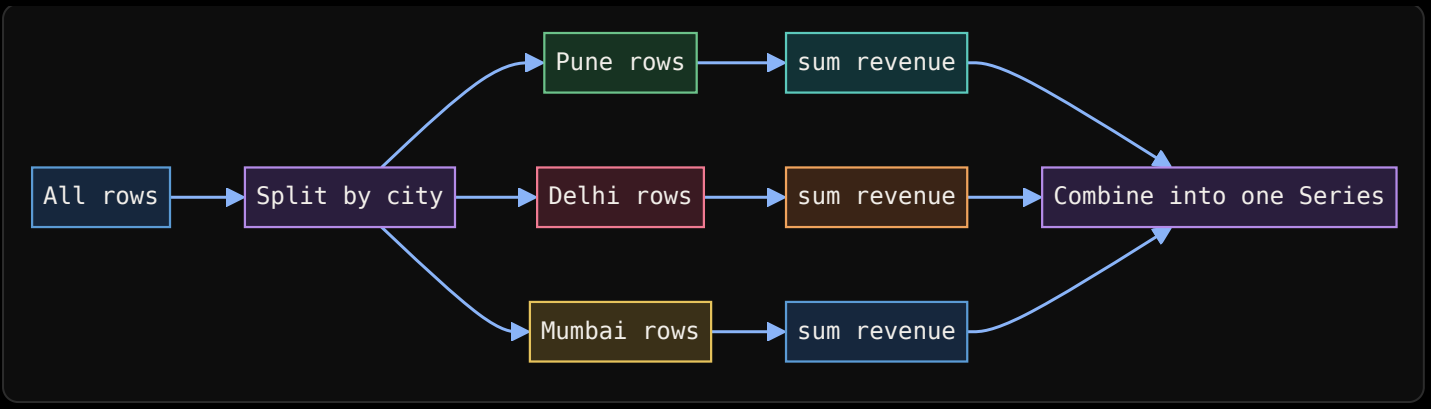
```
   order_id  revenue    month   tier
0         1     15.0  2026-01  small
1         2      8.0  2026-01  small
2         3      0.0  2026-01  small
3         4      7.5  2026-02  small
```

`assign` returns a new DataFrame, so it chains. Several conditions: `np.select([cond1, cond2], [val1, val2], default=...)`; bins: `pd.cut` (fixed edges) or `pd.qcut` (quantile edges — equal rows per bin). Other tools: `astype`, `pd.to_numeric(errors="coerce")` (bad strings → NaN), `.str.strip().str.lower()`, `rename(columns=...)`, `drop_duplicates()`, `drop(columns=...)`. Never loop row by row.

18.7 `groupby`: split–apply–combine

What it is

`groupby` implements **split–apply–combine**: split rows into groups by key(s), apply a function per group, combine the results — SQL's `GROUP BY`.



How it works

```
df.groupby("city")["revenue"].sum().sort_values(ascending=False)
df.groupby("customer").agg(orders=("order_id", "count"),          # named aggregation
                           revenue=("revenue", "sum"),
                           avg_qty=("qty", "mean"))
df["share_of_city"] = df["revenue"] / df.groupby("city")["revenue"].transform("sum")
```

```
city
Pune      95.0
Delhi     33.0
Mumbai    19.5
Name: revenue, dtype: float64
          orders  revenue  avg_qty
customer
ana            3     45.0     10.0
ben            2     33.0      1.5
cara           2     19.5      4.0
dev            1     50.0      2.0
   order_id   city  share_of_city
0         1   Pune           0.16
1         2  Delhi           0.24
2         3   Pune           0.00
```

Method	Output shape	Use
`agg` / `sum` / `mean` / `size` / `count`	One row **per group**	Summaries
`transform`	Same length as the input (group result broadcast back to each row)	Group-relative features: share of total, deviation from group mean, group-wise fill
`filter`	Subset of original rows	Keep groups meeting a condition (`lambda g: len(g) > 1`)
`apply`	Anything	Flexible but slowest — last resort

`size()` counts rows including NaN; `count()` counts non-null values. Several keys (`["city", "product"]`) → a MultiIndex (multi-level labels); `.reset_index()` flattens it, or pass `as_index=False`. Categorical keys: `observed=True` skips combinations that never occur.

Real-world: revenue per region per month, average session length per user cohort (users sharing a start point, e.g. sign-up month), top product per store, feature engineering such as "user's average order value".

18.8 Combining tables: `merge`, `join`, `concat`

- `merge` joins two tables on key columns (e.g. customer ID), like SQL `JOIN`.
- `join` — shortcut for merging on the index.

- `concat` stacks tables vertically (more rows) or side by side (`axis=1`).

```
customers = pd.DataFrame({"customer": ["ana", "ben", "cara", "eve"],
                          "segment": ["retail", "business", "retail", "business"]})
df.merge(customers, on="customer", how="inner")      # only matching keys
df.merge(customers, on="customer", how="left")       # keep all orders
customers.merge(df[["customer"]].drop_duplicates(), on="customer", how="outer", indicator=True)
pd.concat([df, april], ignore_index=True)            # stack rows; missing columns -> NaN
```

```
inner rows=7, left rows=8, dev's segment=nan
  customer   segment      _merge
0      ana    retail        both
1      ben  business        both
2     cara    retail        both
3      dev       NaN  right_only
4      eve  business   left_only
concat rows: 9
```

how=	Keeps	SQL
`inner` (default)	Keys in both	`INNER JOIN`
`left`	All left rows	`LEFT JOIN`
`right`	All right rows	`RIGHT JOIN`
`outer`	All keys from both	`FULL OUTER JOIN`
`cross`	Every pair	`CROSS JOIN`

`indicator=True` adds `_merge` showing each row's origin — a quick data-quality check ("which orders have no customer?"). `validate="many_to_one"` raises if the right table has duplicate keys, which would otherwise **multiply** rows silently. Different key names: `left_on=` / `right_on=`; overlapping columns get `suffixes=("_x", "_y")`.

Common mistakes: row explosion from duplicate keys (compare `len` before/after, or `validate=`); keys with different dtypes (`int` vs `str`) → no matches; `concat` in a loop (quadratic) — collect in a list, concat once.

18.9 Reshaping: `pivot_table` and `melt`

- **Wide** format: one row per entity, one column per variable (a spreadsheet report).
- **Long** ("tidy") format: one row per observation — `(city, product, revenue)`; plotting and grouping libraries prefer it.
- `pivot_table`: long → wide (aggregating duplicates, like an Excel pivot); `melt`: wide → long (unpivot).

```
pivot = df.pivot_table(index="city", columns="product", values="revenue",
                       aggfunc="sum", fill_value=0)
long = pivot.reset_index().melt(id_vars="city", var_name="product", value_name="revenue")
```

```
product   bag  notebook   pen
city
Delhi    25.0       8.0   0.0
Mumbai    0.0      12.0   7.5
Pune     50.0       0.0  45.0
     city product  revenue
0   Delhi     bag     25.0
1  Mumbai     bag      0.0
2    Pune     bag     50.0
```

Related: `pivot` (no aggregation — fails on duplicate pairs), `stack`/`unstack` (move a level between index and columns), `pd.crosstab` (frequency tables), `margins=True` for totals.

18.10 Dates and times

- Parse: `pd.to_datetime(col, format="%Y-%m-%d", errors="coerce")` or `parse_dates=` in `read_csv`; an explicit `format` is faster and avoids day/month ambiguity.
- `.dt` accessor: `.dt.year`, `.dt.month`, `.dt.day_name()`, `.dt.hour`, `.dt.to_period("M")`, `.dt.floor("h")`.
- Arithmetic: `date + pd.Timedelta(days=7)`, `df["end"] - df["start"]` → timedelta, `pd.date_range("2026-01-01", periods=3, freq="D")`.
- Time series: set a `DatetimeIndex`, then `resample("ME")` (month end), `"W"`, `"D"`, `"h"` for calendar buckets, and `rolling(7).mean()` for moving averages.
- Time zones: `tz_localize("Asia/Kolkata")` then `tz_convert("UTC")`; pandas 3 uses stdlib `zoneinfo`.

```
df["date"].dt.day_name().head(5); df["date"].dt.month.unique()
df.set_index("date")["revenue"].resample("ME").sum()
```

```
['Saturday', 'Saturday', 'Saturday', 'Sunday', 'Saturday'] [1, 2, 3]
date
2026-01-31    23.0
2026-02-28    82.5
2026-03-31    42.0
Freq: ME, Name: revenue, dtype: float64
```

Version note: pandas 3.0 removed the old frequency aliases — month end `"ME"` (not `"M"`), quarter end `"QE"`, year end `"YE"`, hour `"h"`. Old tutorials using `resample("M")` now fail.

18.11 `apply` vs vectorised, and performance tips

Measured

```
big.apply(lambda r: r["qty"] * r["price"], axis=1)   # a Python function call per row
big["qty"] * big["price"]                            # one C loop per column
```

```
100k rows: apply(axis=1) 210 ms, vectorised 0.08 ms, x2523
```

`apply(axis=1)` builds a Series per row and calls your Python function — a loop with extra overhead. `iterrows()` is even slower; if you must loop, `itertuples()` is much faster.

Performance checklist

Tip	Why
Vectorise: arithmetic on columns, `np.where` / `np.select`, `.str` and `.dt` methods	C loops instead of Python calls
Right dtypes: `category` for low-cardinality (few distinct values) text, smaller numeric types (`float32`, `int32`) where precision allows	Less memory, faster group-bys
Read less: `usecols=`, `dtype=`, `nrows=`, `chunksize=`	Don't load what you don't need
Store as Parquet	Faster reads, keeps dtypes, column pruning (skipping unrequested columns)
Use built-in aggregations (`"sum"`, `"mean"`) rather than `lambda` in `agg`	Built-ins run in optimised (Cython) code
Avoid growing DataFrames in a loop	Collect rows/frames in a list, build once
`merge` on the right keys, with `validate`	Avoid accidental row explosions
Check `df.memory_usage(deep=True)`	Find heavy columns
When data outgrows pandas	Polars (fast Rust DataFrame library), DuckDB (embedded analytics database), Dask or Spark (spread work over many cores or machines)

```
cities = pd.Series(rng.choice(["Pune", "Delhi", "Mumbai"], 100_000))
cities.memory_usage(deep=True); cities.astype("category").memory_usage(deep=True)
```

```
dtype str: 5.40 MB, category: 0.10 MB
```

A **categorical** stores each distinct value once plus a small integer code per row — 54× less memory here. (With PyArrow, the `str` dtype is also far more compact than Python string objects.)

In the interview

30-second answer: "pandas is fast when you stay vectorised: column arithmetic, `np.where`, `.str` / `.dt` accessors and built-in group aggregations run in compiled code. `apply(axis=1)` and `iterrows` are Python loops. For memory I use categoricals and smaller dtypes, read only needed columns and store Parquet. In pandas 3, Copy-on-Write means every indexing result behaves as a copy, so I always assign with a single `.loc`."

- *`loc` vs `iloc`?* — Labels (inclusive end) vs positions (exclusive end).
- *`transform` vs `agg`?* — One value per original row vs one per group.
- *Orders with no matching customer?* — Left merge with `indicator=True`, filter `left_only`.
- *Why does `qty` load as float?* — It contains `NaN`, which is a float.

18.12 matplotlib: Figure, Axes and the four workhorse charts

What it is

matplotlib is Python's foundational plotting library (pandas' `.plot()` and seaborn, a higher-level statistical charting library, build on it).

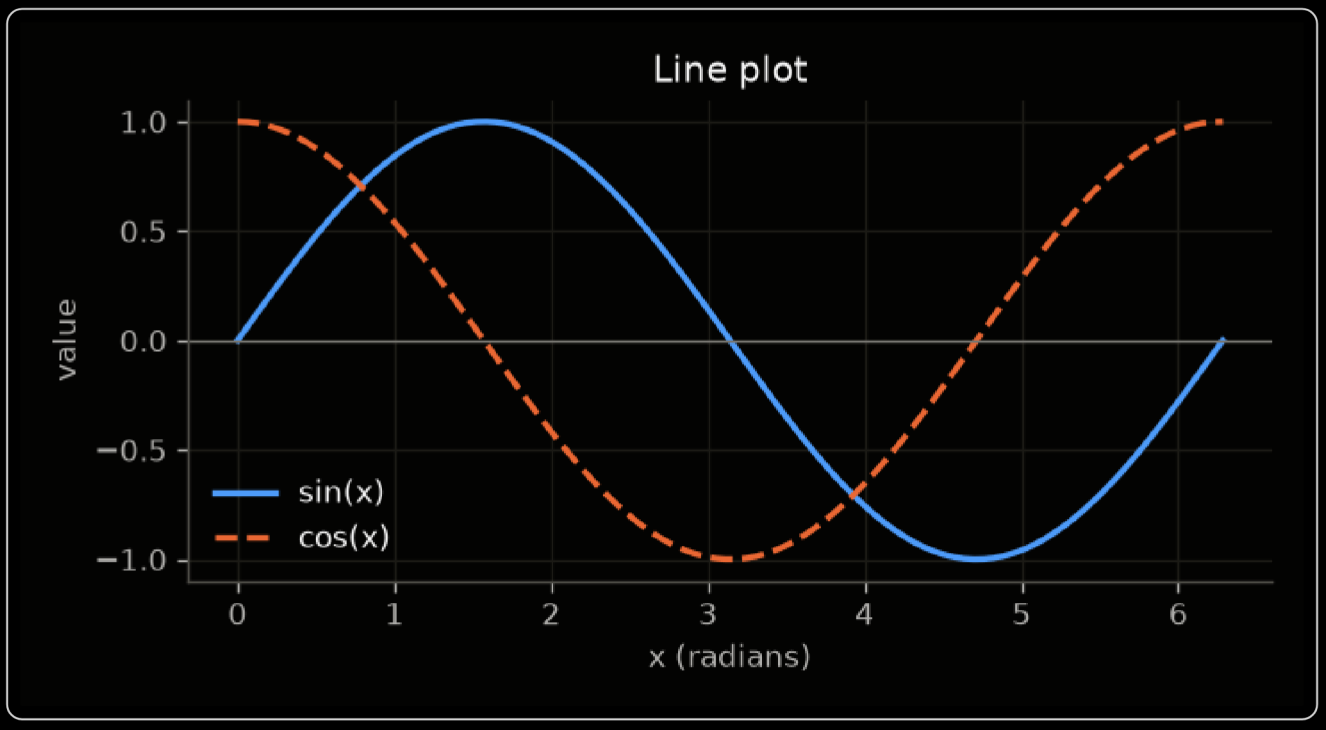
- A **Figure** is the whole canvas/image.
- An **Axes** is one plot in it — its own x/y axis, title, labels, legend. A figure can hold many.

- Two styles: the **pyplot state machine** (`plt.plot(...)` acts on a remembered "current" axes — quick in notebooks) and the **object-oriented (OO) API** (`fig, ax = plt.subplots(); ax.plot(...)` — explicit). Use the OO API for scripts and multiple plots.

How it works

```
import matplotlib
matplotlib.use("Agg")                        # render to files only (servers, scripts)
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(6, 3.2))      # Figure = canvas, Axes = one plot
ax.plot(x, np.sin(x), label="sin(x)")
ax.plot(x, np.cos(x), label="cos(x)", linestyle="--")
ax.set(title="Line plot", xlabel="x (radians)", ylabel="value")
ax.legend(frameon=False)
fig.tight_layout()                            # stop labels overlapping
fig.savefig(IMG / "line_plot.png", dpi=120)   # format from the extension: png, svg, pdf
plt.close(fig)                                # free memory in scripts/loops
```




```
fig, axes = plt.subplots(2, 2, figsize=(9, 6))              # axes is a 2x2 numpy array
axes[0, 0].plot(months, sales, marker="o")                  # line: change over time
axes[0, 1].bar(["pen", "notebook", "bag"], [95, 20, 75])    # bar: compare categories
axes[1, 0].scatter(h, w, s=14, alpha=0.6)                   # scatter: relationship
axes[1, 1].hist(rng.normal(50, 10, 1_000), bins=30)         # histogram: distribution
fig.suptitle("Four basic chart types")
```

Output (`matplotlib_demo.py`):

```
corr(height, weight) = 0.80
saved line_plot.png, four_charts.png
```

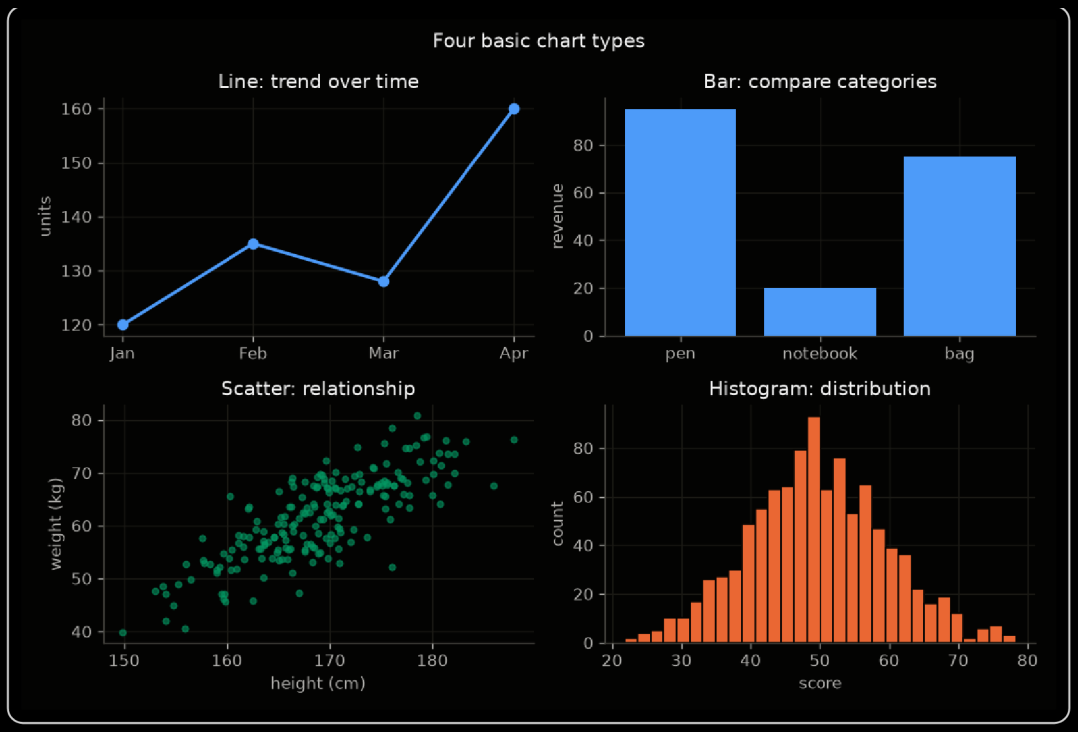


Chart	Answers	Call	Tip
Line	How does it change over an ordered x (time)?	`ax.plot(x, y)`	Don't use for unordered categories
Bar	How do categories compare?	`ax.bar` / `ax.barh`	Start the y-axis at 0; horizontal bars for long labels
Scatter	Are two numbers related?	`ax.scatter(x, y)`	`alpha` (transparency) against overplotting (points so dense you can't see density)
Histogram	How are values distributed?	`ax.hist(values, bins=30)`	Try several `bins` values
Box plot	Spread and outliers (values far from the rest) by group	`ax.boxplot`	

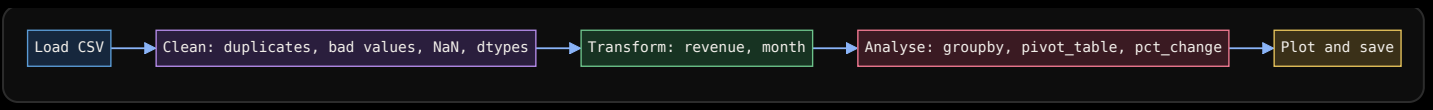
Labelling essentials: `ax.set_title`, `ax.set_xlabel`/`set_ylabel` (with units), `ax.legend()` for two or more series, `ax.tick_params(axis="x", rotation=30)` for crowded dates, `ax.set_xlim`/`set_ylim`, `ax.axhline` for reference lines. `savefig(..., bbox_inches="tight")` trims margins; `dpi=150–300` (dots per inch) for print. From pandas: `df.plot(kind="bar", ax=ax)`.

Common mistakes

- Mixing pyplot and OO calls, so labels land on the wrong subplot.
- No `plt.close(fig)` in loops → memory grows; matplotlib warns about too many open figures.
- `plt.show()` in a headless script (no screen, e.g. a server) instead of `savefig` with the `Agg` backend (renders straight to files).
- Two y-axes (`twinx`) for unrelated measures — usually misleading; use two panels.
- Rainbow colour maps for ordered data — use a single-hue sequential map (e.g. `viridis`).

18.13 Mini end-to-end analysis

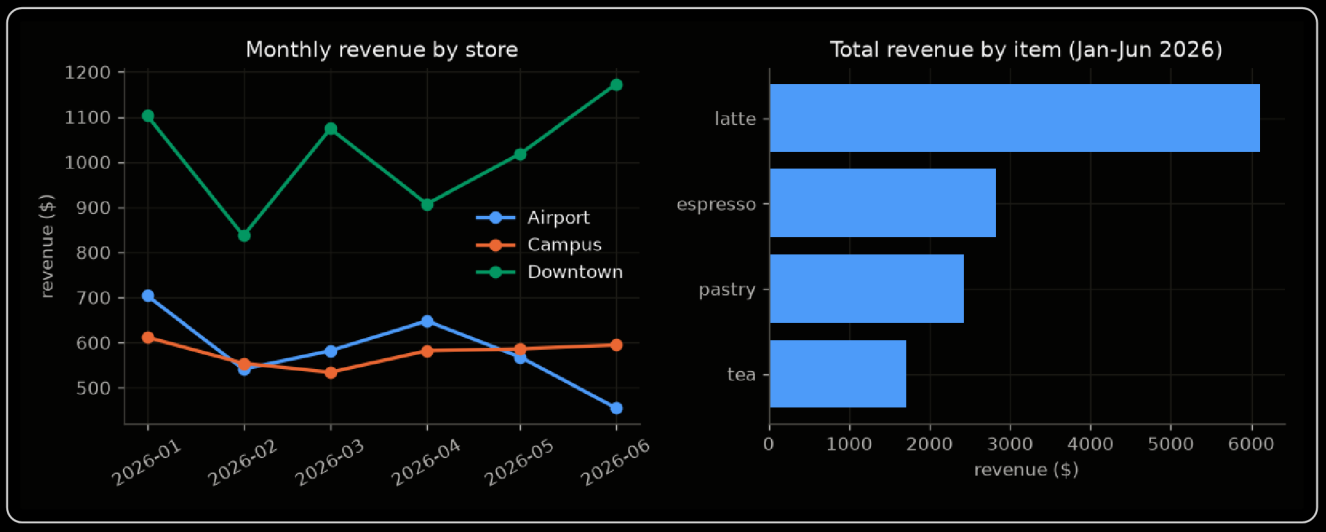
A coffee chain with three stores asks: which store earns most, how did revenue move monthly, which item matters most? The script generates a realistic messy CSV (6 months, 2,000 orders, 3 % missing quantities, a negative price, 10 duplicated rows), then runs the standard pipeline.




```
# 1. Load
raw = pd.read_csv(io.StringIO(make_raw_csv()), parse_dates=["date"])
# 2. Clean
df = (raw.drop_duplicates()
         .query("unit_price > 0")
         .assign(qty=lambda d: d["qty"].fillna(d["qty"].median()).astype(int),
                 store=lambda d: d["store"].astype("category"),
                 item=lambda d: d["item"].astype("category")))
# 3. Transform
df["revenue"] = df["qty"] * df["unit_price"]
df["month"] = df["date"].dt.to_period("M")
# 4. Analyse
by_store = df.groupby("store", observed=True).agg(orders=("qty", "size"),
                                                  revenue=("revenue", "sum"),
                                                  avg_ticket=("revenue", "mean")).round(2)
monthly = df.pivot_table(index="month", columns="store", values="revenue",
                         aggfunc="sum", observed=True).round(0)
growth = monthly.sum(axis=1).pct_change().mul(100).round(1)
# 5. Plot  (line per store + horizontal bar per item, saved to img/coffee_analysis.png)
```

Output (`end_to_end.py`):

```
raw: 2010 rows, missing qty=65, duplicates=10, negative prices=2
clean: 1999 rows, nulls=0
          orders  revenue  avg_ticket
store
Downtown    1010   6112.0        6.05
Airport      412   3497.2        8.49
Campus       577   3461.5        6.00
store    Airport  Campus  Downtown
month
2026-01    704.0   612.0    1103.0
2026-02    541.0   554.0     837.0
2026-03    582.0   534.0    1074.0
2026-04    648.0   582.0     907.0
2026-05    568.0   586.0    1018.0
2026-06    455.0   595.0    1173.0
best item per store: {'Airport': 'latte', 'Campus': 'latte', 'Downtown': 'latte'}
month-over-month total revenue change %: {'2026-02': -20.1, '2026-03': 13.4, '2026-04': -2.4, '2026-05': 1.6,
'2026-06': 2.3}
saved coffee_analysis.png
```



Reading the results:

1. **Data quality** — 10 duplicates removed (the duplicated block contained the bad row: 2 negative prices before dedup, 1 after); 1 negative price dropped; 65 missing quantities filled with the median.

Always report what cleaning did.

2. **Stores** — Downtown has the most orders and revenue; Airport the fewest orders but the highest average ticket (8.49 vs ~6), matching the 40 % airport markup built into the data. (`pct_change` = percentage change from one row to the next.)
3. **Trend** — February dropped 20 % (28 days, and the synthetic data is uniform per day — normalise by days or compare like periods); Airport declines towards June while Downtown grows.
4. **Items** — lattes lead everywhere: highest drink price and most frequent item.

This is most analytics work and the first half of any ML project; ML Handbook Chapter 3 continues from a clean DataFrame to features, models and evaluation.

18.14 Chapter self-check

1. Why is `arr * 2` faster than a list comprehension? → One compiled loop over contiguous typed memory, no per-element Python objects.
2. Is `m[:2, :2]` a copy? → No, a view.
3. Result shape of `(4, 1) * (3,)`? → `(4, 3)`. How to subtract each row's mean? → `x - x.mean(axis=1, keepdims=True)`.
4. Better than `inv(A) @ b`? → `np.linalg.solve(A, b)`.
5. Reproducible random numbers? → `rng = np.random.default_rng(seed)`.
6. `df.loc[1:3]` vs `df.iloc[1:3]` with a default index? → Labels 1–3 (3 rows) vs positions 1–2 (2 rows).
7. What changed about chained assignment in pandas 3? → It never updates the original (Copy-on-Write).
8. Each row's share of its group total? → `col / df.groupby(key)[col].transform("sum")`.
9. What does `how="left"` keep, and how to detect keys missing from a lookup table? → Every left row; `indicator=True`, look for `left_only`.
10. Long → wide? Wide → long? → `pivot_table`; `melt`.
11. Month-end alias in pandas 3? Faster than `apply` for `qty * price`? → `"ME"`; `df["qty"] * df["price"]`.
12. Figure vs Axes? → The canvas vs one plot on it.

Chapter TL;DR

- **numpy**: typed contiguous arrays; shape/dtype; slices are views; masks and fancy indexing copy; vectorise instead of looping (35× here); `axis=0` per column, `axis=1` per row; broadcasting aligns shapes from the right (equal or 1); `@`, `solve`, `eigh`; `default_rng(seed)`.
- **pandas**: Series/DataFrame with label alignment; `read_csv` / `read_parquet`; `loc` (labels, inclusive) vs `iloc` (positions); masks with `&` / `|`; fill missing values thoughtfully; `groupby` + `agg` / `transform`; `merge` with `how`, `indicator`, `validate`; `pivot_table` / `melt`; `.dt` and `resample("ME")`.
- **pandas 3.0**: `str` dtype for text, Copy-on-Write always on (assign with one `.loc`), microsecond datetimes, new frequency aliases.
- **Performance**: vectorise (2,500× over `apply` here), categoricals (54× less memory here), Parquet, read less.
- **matplotlib**: OO API (`fig, ax = plt.subplots()`), line/bar/scatter/hist, label everything, `savefig` + `close`.
- **Workflow**: load → clean → transform → analyse → plot → explain, then hand the clean table to ML.

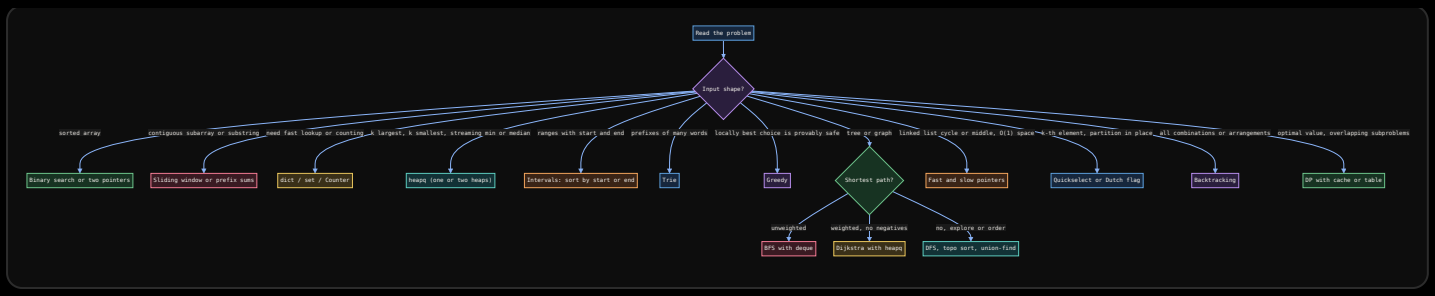
Chapter 19 — Python for Coding Interviews

Coding interviews (the "LeetCode-style" rounds — LeetCode being the best-known practice site, whose online "judges" run your code against hidden tests) test two things: whether you know the classic **data-structure and algorithm (DSA) patterns**, and whether you express them cleanly. Python is the most popular interview language because it is short: a heap (§19.5), a hash map (`dict`, §19.3), a queue (§19.4) and binary search (§19.6) are each one import away. But the shortness hides costs — `list.pop(0)` looks innocent and is O(n), `[[0]*n]*m` looks like a grid and is a bug.

For each pattern: the idea, the Python tools, a tiny runnable example with `assert` tests, its complexity, and the usual mistakes. Assumes Chapters 5–7 (lists, dicts, functions) and 11 (iterators). Scripts are in `code/`, run with Python 3.14 (all 3.12-compatible); output shown is real.

What you'll learn

#	Topic	Script
19.1	Big-O refresher and the complexity cheat table for built-ins	`01_builtins_complexity.py`
19.2	Arrays & strings: two pointers, sliding window, prefix sums	`02_arrays_strings.py`
19.3	Hashing: `dict`, `set`, `Counter`, `defaultdict`	`03_hashing.py`
19.4	Stacks and queues: `list`, `deque`, monotonic stack/deque	`04_stacks_queues.py`
19.5	Heaps: `heapq`, the max-heap trick, top-k, merge k, two heaps (running median, maximise capital)	`05_heaps.py`
19.6	Sorting with keys; binary search by hand and with `bisect`; weighted random pick	`06_sorting_bisect.py`
19.7	Linked lists and trees as Python classes; serialize/deserialize a tree	`07_linked_lists_trees.py`
19.8	Graphs: adjacency lists, BFS/DFS, grids, Dijkstra, topological sort, recursion limit	`08_graphs.py`
19.9	Union-Find (disjoint sets), Kruskal	`09_union_find.py`
19.10	Dynamic programming: `@cache` (top-down) and tabulation (bottom-up)	`10_dynamic_programming.py`
19.11	Backtracking	`11_backtracking.py`
19.12	Bit tricks and math (`gcd`, `pow(b, e, m)`, `isqrt`, `comb`, sieve)	`12_bits_math.py`
19.13	Tries: prefix search, autocomplete, wildcard search, word search on a board	`13_tries.py`
19.14	Greedy: jump game, gas station, candy, interval scheduling	`14_greedy.py`
19.15	Intervals: merge, insert, intersect, meeting rooms (heap and sweep line)	`15_intervals.py`
19.16	Fast & slow pointers: cycle start, middle, palindrome list, happy number, find duplicate	`16_fast_slow_pointers.py`
19.17	Sort & search: merge sort on a list, quicksort, quickselect, Dutch national flag	`17_sort_search.py`
19.18	Math & geometry: spiral matrix, rotate, reverse a 32-bit int, collinear points, Josephus	`18_math_geometry.py`
19.19	Common pitfalls	`19_pitfalls.py`
19.20	How to talk through a solution	—



19.1 Big-O and the built-ins cheat table

What it is

Big-O notation describes how time (or memory) grows with input size n, ignoring constant factors: O(1) doesn't grow, O(n) grows proportionally, O(n log n) is good sorting, O(n²) is "every pair". An **amortised** O(1) operation is occasionally expensive but constant on average: `list.append` sometimes reallocates and copies, rarely enough that the average stays O(1).

Why it matters

Interviewers always ask "what's the complexity?", and Python hides loops inside innocent calls: `x in my_list` scans, `my_list.insert(0, x)` shifts every element, `s = s + ch` copies the string.

How it works — the cheat table

CPython's `list` is a dynamic array of pointers (reallocated bigger when full); `dict` and `set` are hash tables (§19.3); `collections.deque` is a doubly-linked list of fixed-size blocks; `heapq` treats a plain list as a binary heap (§19.5). Costs follow (average case; hash tables degrade to O(n) under pathological collisions — many keys landing in the same slot):

Structure	Operation	Cost	Note
`list`	`a[i]`, `a[i] = x`, `len(a)`	O(1)	array indexing
	`a.append(x)`, `a.pop()`	O(1) amortised	end of the array
	`a.insert(i, x)`, `a.pop(i)`, `del a[i]`	O(n)	shifts the tail; `pop(0)` is the trap
	`x in a`, `a.index(x)`, `a.count(x)`, `min/max/sum`	O(n)	linear scan
	`a[i:j]`, `a.copy()`, `a + b`	O(k)	slicing **copies** k elements
	`a.sort()`, `sorted(a)`	O(n log n)	Timsort, stable (equal items keep their order); O(n) if already sorted
	`a.reverse()`, `a[::-1]`	O(n)	
`deque`	`append`, `appendleft`, `pop`, `popleft`	O(1)	the right queue type
	`dq[i]` in the middle	O(n)	O(1) at either end
`dict`	`d[k]`, `d[k] = v`, `del d[k]`, `k in d`, `d.get`	O(1) avg	keys must be hashable
	iterate over keys/values/items	O(n)	insertion-ordered (3.7+)
`set`	`add`, `remove`, `discard`, `x in s`	O(1) avg	
	`s | t`, `s - t`	O(len s + len t) / O(len s)	
	`s & t`	O(min(len s, len t))	
`heapq`	`heappush`, `heappop`, `heappushpop`	O(log n)	min-heap on a list
	`h[0]` (peek)	O(1)	
	`heapify(list)`	O(n)	faster than n pushes
	`nlargest(k, it)`, `nsmallest(k, it)`	O(n log k)	
`bisect`	`bisect_left/right`	O(log n)	list must be sorted
	`insort`	O(n)	search is log n, the insert shifts
`str`	`s[i]`, `len(s)`	O(1)	immutable
	`s + t`, slicing, `s.replace`, `s.split`	O(n)	always builds a new string
	`"".join(parts)`	O(total length)	the right way to build strings
`Counter`	`Counter(iterable)`	O(n)	`most_common(k)` O(n log k)

Python `int`s have **arbitrary precision** (no overflow), so `(lo + hi) // 2` is safe — though arithmetic on huge numbers isn't O(1); treat interview-sized numbers as O(1).

`01_builtins_complexity.py` measures the two most important rows (machine-dependent numbers, same shape; plotted in §17.5):

```
     n |   x in list |  x in set |  pop(0) all | popleft all
  1000 |      5.3 us |   0.03 us |     0.07 ms |     0.02 ms
  5000 |     26.3 us |   0.04 us |     1.06 ms |     0.13 ms
 20000 |    108.3 us |   0.03 us |    24.50 ms |     0.51 ms
 50000 |    321.3 us |   0.18 us |   174.89 ms |     1.47 ms
```

`x in list` grows linearly while `x in set` stays flat; draining a list with `pop(0)` is quadratic, a deque linear.

Common mistakes

- `x in list` inside a loop (O(n²) total) when a `set` makes it O(n); a list as a queue with `pop(0)`.
- Forgetting slicing copies: `helper(nums[1:])` in recursion adds O(n) per call — pass indices.
- Building a string with `+=` in a loop (CPython sometimes optimises it in place; don't rely on it) — collect pieces, `"".join` once.

In the interview

> **30-second answer:** "Lists are dynamic arrays — O(1) index and append, O(n) insert/delete at the front and membership. Dicts and sets are hash tables with O(1) average lookup. For a queue I use `deque`, for a priority queue `heapq`, and sorting is O(n log n) Timsort."

- *Why is `append` amortised O(1)?* → The array over-allocates; when full it grows by a proportion and copies, so total copying over n appends is O(n).
- *Worst case for a dict lookup?* → O(n) if many keys collide; O(1) in practice.
- *Space of `sorted(a)`?* → O(n) for the new list (Timsort also uses up to n/2 temporary space).

19.2 Arrays and strings: two pointers, sliding window, prefix sums

These turn many "check every pair/subarray" O(n²) problems into O(n). (A subarray is a contiguous run of elements; a substring the same for strings.)

What it is

- **Two pointers** — two indices move from both ends inward (sorted pair sum, palindrome check) or both forward at different speeds (a slow "write" and fast "read" pointer for in-place filtering).
- **Sliding window** — `left` and `right` bound a contiguous window: `right` grows it, `left` shrinks it when a rule breaks. Each index only moves forward → O(n) total.
- **Prefix sums** — precompute `prefix[k] = sum of the first k elements`; any range `nums[i:j]` sums to `prefix[j] - prefix[i]` in O(1).

How it works

Two pointers on a sorted array (pair sum = target): start at both ends. Too small → only moving `lo` up can increase it; too big → only moving `hi` down can decrease it. Each step discards one element forever → O(n).

Variable sliding window (longest substring without repeats): expand `right`, remembering each character's last index; if the new character was seen inside the window, jump `left` past its earlier copy. Answer = largest `right - left + 1`.

Prefix sums + hash map (count subarrays summing to k, negatives allowed): subarray `(i, j]` sums to k exactly when `prefix[i] = prefix[j] - k`. Walk once, count prefix values seen, add `seen[running - k]` each step. For `[1, 1, 1]`, k = 2: running sums 1, 2, 3; at 2 look up 0 (seen once, the empty prefix) → 1; at 3 look up 1 → 1; total 2.

```python
def two_sum_sorted(nums, target):
    lo, hi = 0, len(nums) - 1
    while lo < hi:
        s = nums[lo] + nums[hi]
        if s == target:
            return lo, hi
        if s < target:
            lo += 1          # need a bigger sum -> move the small end up
        else:
            hi -= 1          # need a smaller sum -> move the big end down
    return None

def longest_unique_substring(s):
    last_seen = {}
    left = best = 0
    for right, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1      # shrink: jump past the earlier copy
        last_seen[ch] = right
        best = max(best, right - left + 1)
    return best

def min_window_at_least(nums, target):   # positive numbers only
    left = total = 0
    best = float("inf")
    for right, x in enumerate(nums):
        total += x
        while total >= target:            # shrink while still valid
            best = min(best, right - left + 1)
            total -= nums[left]
            left += 1
    return 0 if best == float("inf") else int(best)

class RangeSum:
    def __init__(self, nums):
        self.prefix = [0, *accumulate(nums)]   # prefix[k] = sum(nums[:k])
    def query(self, i, j):                     # half-open [i, j)
        return self.prefix[j] - self.prefix[i]

def count_subarrays_sum_k(nums, k):
    seen = defaultdict(int)
    seen[0] = 1           # the empty prefix
    running = count = 0
    for x in nums:
        running += x
        count += seen[running - k]
        seen[running] += 1
    return count

assert two_sum_sorted([1, 3, 4, 6, 9], 10) == (0, 4)
assert longest_unique_substring("pwwkew") == 3
assert min_window_at_least([2, 3, 1, 2, 4, 3], 7) == 2
assert RangeSum([3, -1, 4, 1, 5]).query(1, 4) == 4
assert count_subarrays_sum_k([1, 1, 1], 2) == 2
```

("Half-open `[i, j)`" = i included, j not, like slices.) The script also has `is_palindrome` (two pointers from both ends, skipping non-alphanumerics) and `remove_duplicates_sorted` (slow/fast pointers, in place). Output:

```
prefix: [0, 3, 2, 6, 7, 12]
two_sum_sorted([1,3,4,6,9], 10) -> (0, 4)
longest_unique_substring('pwwkew') -> 3
min_window_at_least([2,3,1,2,4,3], 7) -> 2
count_subarrays_sum_k([3,4,7,-2,2,1,4,2], 7) -> 6
all asserts passed
```

Python helpers worth knowing

Need	Python
index + value	`for i, x in enumerate(nums)`
adjacent pairs	`for a, b in itertools.pairwise(nums)` (3.10+) or `zip(nums, nums[1:])`
running totals	`itertools.accumulate(nums)`; `accumulate(nums, max)` for running max
reverse	`nums[::-1]` (copy) or `nums.reverse()` (in place)
letter → 0..25	`ord(ch) - ord("a")`; back with `chr(i + ord("a"))`
character class	`ch.isalnum()`, `ch.isdigit()`, `ch.isalpha()`, `ch.lower()`
strings are immutable	convert to `list(s)`, edit, then `"".join(chars)`

Trade-offs

Pattern	Needs	Time	Space
Two pointers (pair sum)	sorted input	O(n) (+ O(n log n) to sort)	O(1)
Hash map (pair sum, 19.3)	nothing	O(n)	O(n)
Sliding window	a condition that stays monotonic (moves one way only) as the window grows/shrinks (e.g. positive numbers)	O(n)	O(1)–O(alphabet)
Prefix sums	static array	O(n) build, O(1) query	O(n)

When not to use it

A shrinking window assumes "adding grows it, removing shrinks it". **Negative numbers** break that — use prefix sums + hash map (`count_subarrays_sum_k`). If the array changes between queries, prefix sums must be rebuilt; use a Fenwick/segment tree (updates and range sums in O(log n)).

Common mistakes

- Off-by-one window length (it's `right − left + 1`); forgetting `seen[0] = 1` (misses subarrays starting at index 0).
- Moving `left` backwards in the unique-substring problem — the `last_seen[ch] >= left` check prevents it.

In the interview

> **30-second answer:** "Contiguous subarrays or substrings → sliding window: expand right, shrink left while invalid, O(n). Pairs in a sorted array → two pointers from both ends. Range sums or 'subarray sums to k' → prefix sums with a hash map of seen prefixes."

- *Why is the window O(n) with a nested `while`?* → Each index enters and leaves at most once; the inner loop's total iterations are ≤ n.
- *Two-sum on unsorted data?* → Hash map, O(n) time/space; sorting + two pointers is O(n log n) and loses indices unless you sort `(value, index)` pairs.
- *`prefix` for `[2, 4, 1]`, and `sum(nums[1:3])` from it?* → `[0, 2, 6, 7]`; `prefix[3] − prefix[1] = 5`.

19.3 Hashing: `dict`, `set`, `Counter`, `defaultdict`

What it is

A **hash table** stores items in buckets chosen by a **hash** of the key: O(1) average insert and lookup. Python's flavours: `dict` (key → value), `set` (keys only), `collections.Counter` (dict subclass that counts; missing keys read as 0) and `collections.defaultdict(factory)` (creates `list()`, `int()`, `set()`… on first access to a missing key). Only **hashable** objects can be keys — `int`, `str`, tuples of hashables, `frozenset`; convert a list to a `tuple` first.

Why it matters

Many easy/medium problems are "trade O(n) memory for O(1) lookup": two-sum, anagrams (same letters, "listen"/"silent"), duplicates, frequency counts, "seen this before?".

How it works

```
def two_sum(nums, target):
    index_of = {}
    for i, x in enumerate(nums):
        if target - x in index_of:        # the complement was seen earlier
            return index_of[target - x], i
        index_of[x] = i
    return None

def is_anagram(a, b):
    return Counter(a) == Counter(b)

def group_anagrams(words):
    groups = defaultdict(list)
    for w in words:
        groups[tuple(sorted(w))].append(w)   # key must be hashable -> tuple
    return list(groups.values())

def longest_consecutive(nums):
    values = set(nums)
    best = 0
    for x in values:
        if x - 1 not in values:           # x starts a run -> count it once
            length = 1
            while x + length in values:
                length += 1
            best = max(best, length)
    return best

assert two_sum([2, 7, 11, 15], 9) == (0, 1)
assert longest_consecutive([100, 4, 200, 1, 3, 2]) == 4

c = Counter("mississippi")
print("Counter:", c)
print("most_common(2):", c.most_common(2))
print("missing key:", c["z"])
print("c - Counter('sip'):", c - Counter("sip"))
```

Output:

```
Counter: Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
most_common(2): [('i', 4), ('s', 4)]
missing key: 0
c - Counter('sip'): Counter({'i': 3, 's': 3, 'm': 1, 'p': 1})
group_anagrams: [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
all asserts passed
```

`longest_consecutive` looks nested but is O(n): the inner `while` only runs from the start of each run, so each number is visited by it at most once.

Handy API

Want	Code
read with default	`d.get(k, 0)`
count / group	`Counter`, `defaultdict(int)`, `defaultdict(list)` + `.append`
insert if missing, return value	`d.setdefault(k, [])`
top k counts	`Counter(xs).most_common(k)`
counter arithmetic	`c1 + c2`, `c1 - c2` (drops ≤ 0), `c1 & c2` (min), `c1 | c2` (max)
fixed alphabet count	`[0] * 26` indexed by `ord(ch) - 97` — also hashable as `tuple(...)`
set algebra	`a & b`, `a | b`, `a - b`, `a ^ b`

Trade-offs

Hashing costs O(n) memory and loses order-based tricks: on sorted input, two pointers use O(1) space. For keys from a tiny fixed range (26 letters), an array is faster and simpler.

Common mistakes

- A list as a key (`TypeError: unhashable type: 'list'`) — use `tuple(lst)`.
- Reading a `defaultdict` with `d[k]` just to check it **inserts** the key. Use `k in d`.
- Mutating a dict while iterating (`RuntimeError: dictionary changed size during iteration`) — iterate over `list(d)`.

In the interview

> **30-second answer:** "I use a dict to remember something about values already seen — index, count, group — turning an O(n²) pair search into O(n). Keys must be hashable, so composite keys are tuples. `Counter` and `defaultdict` remove boilerplate."

- *Dict key from a list of counts?* → `tuple(counts)`.
- *`Counter.most_common()` with no argument?* → All items by count, O(n log n); with k it uses a heap, O(n log k).
- *Can a `set` be a dict key?* → No; a `frozenset` can.

19.4 Stacks and queues: `list` and `deque`

What it is

- **Stack** — last-in, first-out (LIFO): a `list` with `append` (push) and `pop()`, both O(1) at the end.
- **Queue** — first-in, first-out (FIFO): `collections.deque` with `append` and `popleft`, both O(1). (`queue.Queue` is a slower thread-safe queue for concurrency, Chapter 16 — not for interviews.)
- **Monotonic stack** — keeps elements in increasing (or decreasing) order by popping whatever breaks the order before pushing — "next greater/smaller element" in O(n). A **monotonic deque** also drops elements leaving a window from the front — sliding-window maximum in O(n).

How it works

Valid parentheses: push openers; a closer must match the top; the stack must end empty.

Next greater element: walk left to right holding indices still waiting for a bigger value. A new `x` *is* the answer for every smaller value on top — pop them and record `x`. Each index is pushed and popped once → O(n).

```python
def valid_parentheses(s):
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif not stack or stack.pop() != pairs[ch]:
            return False
    return not stack

def next_greater(nums):
    result = [-1] * len(nums)
    stack = []                          # indices, values decreasing
    for i, x in enumerate(nums):
        while stack and nums[stack[-1]] < x:
            result[stack.pop()] = x
        stack.append(i)
    return result

def sliding_window_max(nums, k):
    dq = deque()                        # indices, values decreasing
    out = []
    for i, x in enumerate(nums):
        if dq and dq[0] <= i - k:
            dq.popleft()                # front fell out of the window
        while dq and nums[dq[-1]] <= x:
            dq.pop()                    # smaller values can never be the max
        dq.append(i)
        if i >= k - 1:
            out.append(nums[dq[0]])
    return out

assert valid_parentheses("([]{})") and not valid_parentheses("(]")
assert next_greater([2, 1, 2, 4, 3]) == [4, 2, 4, -1, -1]
assert sliding_window_max([1, 3, -1, -3, 5, 3, 6, 7], 3) == [3, 3, 5, 5, 6, 7]
```

The script also has `daily_temperatures` (same idea, storing distances), a `MinStack` (entries are `(value, min_so_far)` so `get_min` is O(1)), and `eval_rpn` for Reverse Polish Notation (operators after operands, `3 4 +`, evaluated with a stack). Its division uses `int(a / b)`, which truncates toward zero as required, while `a // b` floors (`−7 // 2 = −4`). Deque tour output:

```
deque: [0, 1, 2, 3, 4]
popleft -> 0 | pop -> 4 | now deque([1, 2, 3])
after rotate(1): deque([3, 1, 2])
deque(maxlen=3) after 0..5: deque([3, 4, 5], maxlen=3)
sliding_window_max: [3, 3, 5, 5, 6, 7]
all asserts passed
```

`deque(maxlen=k)` silently drops from the opposite end when full — a ready-made "last k items" buffer.

Common mistakes

- `stack.pop()` / `stack[-1]` on an empty list raises `IndexError` — check `if stack` first (the `not stack or ...` short-circuit above never pops an empty stack).
- Storing values instead of indices in a monotonic stack when you later need distances or window positions.

In the interview

> **30-second answer:** "Stack = list with append/pop. Queue = deque with append/popleft, never list.pop(0). For 'next greater/smaller' or 'span' I use a monotonic stack of indices — O(n), each index pushed and popped once."

- *Queue with two stacks?* → Push to `inbox`; to dequeue, if `outbox` is empty move everything from `inbox` (reversing it), then pop `outbox`. Amortised O(1).

- *Why does `sliding_window_max` pop smaller values from the back?* → A smaller value left of a bigger one can never be a future window's max.
- *`deque.append` / `rotate(1)`?* → Adds on the right (`appendleft` on the left); moves the last element to the front.

19.5 Heaps with `heapq`

What it is

A **binary heap** is a complete binary tree (every level full except possibly the last, filled from the left) stored in an array where every parent ≤ its children (**min-heap**), so the smallest item is at index 0. `heapq` provides functions over a plain `list` — no heap class. A heap is the standard **priority queue** (serve the most urgent item next).

How it works

- Children of i are at 2i+1 and 2i+2. `heappush` appends and "sifts up" (swaps with the parent while smaller); `heappop` moves the last item to the root and "sifts down" (swaps with the smaller child while bigger) — both O(log n).
- **Max-heap trick:** push `-x`, negate on the way out. (3.14 adds public `heapq.heappush_max` / `heappop_max` / `heapify_max`, but interview platforms often run older versions, so negation is the portable answer.)
- **Tuples:** `(priority, item)` compares priority first; on ties Python compares the next field — add a unique counter `(priority, count, item)` when items aren't comparable.
- **Top-k:** for the k largest of n, keep a **min-heap of size k**, popping the smallest whenever it exceeds k; the root is then the k-th largest. O(n log k) time, O(k) space.

```python
def kth_largest(nums, k):
    heap = []
    for x in nums:
        heapq.heappush(heap, x)
        if len(heap) > k:
            heapq.heappop(heap)       # drop the smallest
    return heap[0]

def top_k_frequent(words, k):
    counts = Counter(words)
    return heapq.nsmallest(k, counts, key=lambda w: (-counts[w], w))  # ties alphabetical

def merge_k_sorted(lists):
    heap = [(lst[0], i, 0) for i, lst in enumerate(lists) if lst]
    heapq.heapify(heap)
    out = []
    while heap:
        val, i, j = heapq.heappop(heap)       # i breaks ties
        out.append(val)
        if j + 1 < len(lists[i]):
            heapq.heappush(heap, (lists[i][j + 1], i, j + 1))
    return out

class RunningMedian:
    """low = max-heap (negated) of the smaller half, high = min-heap of the larger half."""
    def __init__(self):
        self.low, self.high = [], []
    def add(self, x):
        heapq.heappush(self.low, -x)
        heapq.heappush(self.high, -heapq.heappop(self.low))
        if len(self.high) > len(self.low):
            heapq.heappush(self.low, -heapq.heappop(self.high))
    def median(self):
        if len(self.low) > len(self.high):
            return float(-self.low[0])
        return (-self.low[0] + self.high[0]) / 2
```

Output of the tour in `__main__`:

```
heapified: [1, 2, 8, 3, 5] smallest: 1
pop order: [1, 2, 3, 5, 8]
max via negation: 8
priority queue first: (1, 'fix prod bug')
nlargest(2): [9, 7]
running medians: [5.0, 10.0, 5.0, 4.0]
max capital (k=2): 4
all asserts passed
```

`heapified` is **not** sorted — only `h[0]` is guaranteed smallest. Popping repeatedly yields sorted order (heapsort).

The two-heaps pattern

Two heaps facing each other split data into a "smaller" and a "larger" group with the boundary available fast.

- **Running (streaming) median** — `RunningMedian` above: `low` (max-heap) holds the smaller half, `high` (min-heap) the larger. Invariants: every value in `low` ≤ every value in `high`; `len(low)` equals `len(high)` or one more. Median = `low`'s top (odd count) or the average of both tops (even). `add` passes each new value through `low` into `high`, then moves one back if `high` grew bigger. Trace for 5, 15, 1, 3:

add	`low` (max-heap)	`high` (min-heap)	median
5	{5}	{}	5
15	{5}	{15}	(5 + 15) / 2 = 10
1	{1, 5}	{15}	5
3	{1, 3}	{5, 15}	(3 + 5) / 2 = 4

- **Maximise capital** (`max_capital`): at most k projects, each needing `cost` capital and returning `profit`. One min-heap holds **locked** projects by cost; a max-heap holds **affordable** ones by profit. Each round, move every now-affordable project across, then take the most profitable — a greedy choice (19.14) that heaps make O(n log n).
- **Sliding-window median** (hard follow-up): same two heaps, but values also leave; heaps can't delete from the middle cheaply, so use **lazy deletion** — note "remove x later" in a `Counter` and discard x when it reaches a top.

Trade-offs

Need	Best tool	Cost
k largest once, small k	`heapq.nlargest(k, xs)`	O(n log k)
k close to n	`sorted(xs)[-k:]`	O(n log n)
k-th element, average fast	quickselect (partition-based selection, 19.17)	O(n) average, $O(n^2)$ worst
repeated "give me the min" with inserts	heap	O(log n) each
delete arbitrary item	heap + "lazy deletion" (mark, skip when popped)	amortised

Common mistakes

- Assuming the heap list is sorted; negating on push but not on pop in a max-heap.
- Pushing `(priority, dict)` — on a tie Python compares dicts → `TypeError`. Add a tie-breaker counter.
- `heappush` onto a list of unordered items without `heapify` first.

In the interview

> **30-second answer:** "`heapq` is a min-heap on a list: push/pop O(log n), peek `h[0]` O(1), `heapify` O(n). For a max-heap I push negatives. For top-k I keep a size-k min-heap, O(n log k). Dijkstra, merge-k-lists and running median are all heap problems."

- *Why a min-heap for k largest?* → The root is the weakest of the current top k — the one to evict.
- *Running median complexity?* → O(log n) per insert, O(1) per median.
- *Why not a sorted list with `insort`?* → Search is O(log n) but the insert shifts, O(n) per add.

19.6 Sorting with keys; binary search and `bisect`

What it is

- `sorted(iterable, key=..., reverse=...)` returns a new list; `list.sort(...)` sorts in place and returns `None`. Both are **Timsort** — O(n log n) worst, O(n) on sorted input, **stable** (equal keys keep their order).
- A **key function** maps each item to what's compared (`key=len`, `key=str.lower`). Tuple keys give multi-level sorting; negate a numeric field for "descending on this field only". `functools.cmp_to_key` adapts a pairwise comparator (`cmp(a, b)` → negative/zero/positive) for problems where order depends on the *pair* (e.g. "largest number").
- **Binary search** halves a sorted range each step: O(log n). `bisect_left(a, x)` = first index with `a[i] >= x`; `bisect_right(a, x)` = first with `a[i] > x`; `insort` inserts in order. Since 3.10 they accept `key=`.

How it works

Binary search on the answer — interviewers' favourite: when a yes/no test `ok(v)` is *monotonic* (false…false, true…true), binary-search the smallest true v. E.g. minimum eating speed to finish banana piles in h hours — if speed s works, any faster one does.

```
people = [("ana", 31), ("bo", 25), ("cy", 31), ("di", 25)]
print(sorted(people, key=lambda p: (p[1], p[0])))      # age, then name
print(sorted(people, key=lambda p: (-p[1], p[0])))     # age desc, name asc

def merge_intervals(intervals):
    out = []
    for start, end in sorted(intervals):
        if out and start <= out[-1][1]:
            out[-1][1] = max(out[-1][1], end)
        else:
            out.append([start, end])
    return out

def lower_bound(nums, target):        # same as bisect_left
    lo, hi = 0, len(nums)              # half-open [lo, hi)
    while lo < hi:
        mid = (lo + hi) // 2
        if nums[mid] < target:
            lo = mid + 1
        else:
            hi = mid
    return lo

def min_eating_speed(piles, hours):
    def ok(speed):
        return sum(ceil(p / speed) for p in piles) <= hours
    lo, hi = 1, max(piles)
    while lo < hi:
        mid = (lo + hi) // 2
        if ok(mid):
            hi = mid          # mid works -> try smaller
        else:
            lo = mid + 1
    return lo

a = [1, 3, 3, 3, 7, 9]
assert lower_bound(a, 3) == bisect.bisect_left(a, 3) == 1
assert bisect.bisect_right(a, 3) == 4
assert min_eating_speed([3, 6, 7, 11], 8) == 4
```

Output:

```
by age then name: [('bo', 25), ('di', 25), ('ana', 31), ('cy', 31)]
age desc, name asc: [('ana', 31), ('cy', 31), ('bo', 25), ('di', 25)]
itemgetter(1) (stable): [('bo', 25), ('di', 25), ('ana', 31), ('cy', 31)]
case-insensitive: ['Apple', 'banana', 'cherry']
by length desc: ['banana', 'cherry', 'Apple']
count of 3s via bisect: 3
insort 5: [1, 3, 3, 3, 5, 7, 9]
first age>=31 at index 2
weighted pick shares (expect ~0.17, 0.50, 0.33): [0.17, 0.5, 0.33]
all asserts passed
```

The script also has `largest_number` (`cmp_to_key`: put `a` before `b` if `a+b > b+a` as strings) and `search_rotated` (binary search in a rotated sorted array: one half is always sorted; check whether the target lies in it).

Weighted random selection

Problem: given weights `w = [1, 3, 2]`, return index i with probability `w[i] / sum(w)` (1/6, 3/6, 2/6) — used for load balancing across servers of different sizes (System Design Fundamentals) and sampling.

Idea: lay the weights end to end with **prefix sums**: `[1, 4, 6]` means index 0 owns [0, 1), index 1 owns [1, 4), index 2 owns [4, 6). Draw an integer r in [0, 6) and find the first prefix **greater than** r — exactly `bisect_right`. Build O(n), each pick O(log n).

```python
class WeightedPicker:
    def __init__(self, weights, seed=None):
        self.prefix = list(accumulate(weights))     # [1, 3, 2] -> [1, 4, 6]
        self.rng = random.Random(seed)
    def pick(self):
        r = self.rng.randrange(self.prefix[-1])     # integer in [0, total)
        return bisect.bisect_right(self.prefix, r)
```

60,000 seeded picks gave shares `[0.17, 0.5, 0.33]` (above). `random.choices(range(n), weights=w)` does the same internally — mention it, but interviewers usually want the hand-written version. `bisect_left` here is an off-by-one: r = 1 would map to index 0.

The two binary-search templates

Template	Loop	Update	Returns
Exact match	`while lo <= hi` on `[lo, hi]`	`lo = mid + 1` / `hi = mid - 1`	index or −1
Boundary (first true)	`while lo < hi` on `[lo, hi)`	`lo = mid + 1` / `hi = mid`	first position where condition holds

Pick one and use it every time; most binary-search bugs come from mixing them (infinite loops when `lo = mid`).

Common mistakes

- `nums = nums.sort()` sets `nums` to `None`; `reverse=True` reverses the whole key, not one field.
- `bisect` on an unsorted list (no error, wrong answers); thinking `insort` is O(log n) — the insertion is O(n).

In the interview

> **30-second answer:** "Python sorts with Timsort, stable, O(n log n). I pass a `key` — tuples for multi-level ordering and negation for descending fields. For binary search I use `bisect_left/right` or a half-open `lo < hi` template, and for 'minimum value that satisfies X' I binary-search the answer space when X is monotonic."

- *Why does stability matter?* → Sort by the secondary key, then the primary; equal primaries keep the secondary order.
- *TreeMap/sorted set in Python?* → Not built in: sorted list + `bisect` (O(n) inserts), a heap, or third-party `sortedcontainers.SortedList` (preinstalled on some platforms such as LeetCode).
- *`bisect_left([1,2,2,3], 2)` and `bisect_right`?* → 1 and 3.
- *Strings by length, ties alphabetical?* → `sorted(ws, key=lambda w: (len(w), w))`.

19.7 Linked lists and trees in Python

What it is

No built-in linked list or tree for interviews — you define tiny node classes. A **singly linked list** node has `val` and `next`; a **binary tree** node `val`, `left`, `right`. A **binary search tree (BST)** keeps every left-subtree value < node < every right-subtree value. `@dataclass` (Chapter 10) writes `__init__`; `from __future__ import annotations` lets a class mention itself in hints.

How it works — linked lists

- **Dummy head:** start with a fake node so "empty list" is never a special case when building or merging.

- **Reversal:** walk with `prev`, pointing each node back. Tuple assignment evaluates the right side first, so `head.next, prev, head = prev, head, head.next` is a correct one-liner.
- **Fast/slow pointers (Floyd, after computer scientist Robert Floyd):** `slow` moves 1, `fast` 2. When `fast` ends, `slow` is at the middle; with a cycle they meet. Compare nodes with `is`.

```python
@dataclass
class ListNode:
    val: int
    next: ListNode | None = None

def reverse(head):
    prev = None
    while head:
        head.next, prev, head = prev, head, head.next
    return prev

def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:
            return True
    return False

def merge_two(a, b):
    dummy = tail = ListNode(0)
    while a and b:
        if a.val <= b.val:
            tail.next, a = a, a.next
        else:
            tail.next, b = b, b.next
        tail = tail.next
    tail.next = a or b
    return dummy.next
```

How it works — trees

- **DFS traversals:** pre-order (node, left, right), in-order (left, node, right — sorted order for a BST), post-order (left, right, node). Recursive = 3 lines; iterative = explicit stack.
- **BFS / level order:** a `deque`; process `len(q)` nodes per outer loop to separate levels.
- **Validate a BST:** pass down the allowed `(lo, hi)` range — checking only parent vs child is a classic bug.
- **Lowest common ancestor (LCA — the deepest node with both targets below it):** recurse; if both sides return something, the current node is the LCA.

```python
@dataclass
class TreeNode:
    val: int
    left: TreeNode | None = None
    right: TreeNode | None = None

def inorder_iterative(root):
    out, stack, node = [], [], root
    while stack or node:
        while node:                 # go left as far as possible
            stack.append(node)
            node = node.left
        node = stack.pop()
        out.append(node.val)
        node = node.right
    return out

def level_order(root):
    if not root:
        return []
    levels, q = [], deque([root])
    while q:
        levels.append([])
        for _ in range(len(q)):
            node = q.popleft()
            levels[-1].append(node.val)
            q.extend(c for c in (node.left, node.right) if c)
    return levels

def max_depth(root):
    return 0 if not root else 1 + max(max_depth(root.left), max_depth(root.right))

def is_valid_bst(root, lo=float("-inf"), hi=float("inf")):
    if not root:
        return True
    if not lo < root.val < hi:
        return False
    return is_valid_bst(root.left, lo, root.val) and is_valid_bst(root.right, root.val, hi)

def lca(root, p, q):
    if root is None or root.val in (p, q):
        return root
    left, right = lca(root.left, p, q), lca(root.right, p, q)
    return root if left and right else left or right
```

`build_tree([3, 9, 20, None, None, 15, 7])` in the script turns LeetCode's level-order list format into nodes for local testing. Output:

```
reversed list: [5, 4, 3, 2, 1]
inorder: [9, 3, 15, 20, 7] preorder: [3, 9, 20, 15, 7]
levels: [[3], [9, 20], [15, 7]] depth: 3
serialized: 3,9,#,#,20,15,#,#,7,#,#
all asserts passed
```

Serialize and deserialize a binary tree

Serialize = tree → string (to store or send); **deserialize** = rebuild the identical tree. A plain pre-order list is ambiguous (`[1, 2]`: is 2 a left or right child?); writing a `#` for every missing child fixes it — reading pre-order, each value's next tokens are *exactly* its left subtree then its right subtree.

```python
def serialize(root):
    out, stack = [], [root]
    while stack:                        # iterative pre-order
        node = stack.pop()
        if node is None:
            out.append("#"); continue
        out.append(str(node.val))
        stack.append(node.right)        # right first, so left is popped first
        stack.append(node.left)
    return ",".join(out)

def deserialize(data):
    tokens = iter(data.split(","))      # a shared iterator = "read the next token"
    def build():
        tok = next(tokens)
        if tok == "#":
            return None
        node = TreeNode(int(tok))
        node.left = build()
        node.right = build()
        return node
    return build()
```

Both O(n) time and space; n nodes give 2n + 1 tokens (n values + n + 1 `#`). The script checks the round trip `deserialize(serialize(t)) == t`. Recursive `build` has the usual depth limit — for very deep trees use the level-order format `build_tree` reads.

Complexity

Traversals are O(n) time. Space: O(h) for DFS (h = height: log n if balanced, n for a degenerate "linked-list" tree), O(w) for BFS (w = widest level, up to n/2).

Common mistakes

- Mutable default `def f(node, path=[])` in tree recursion (19.19).
- `if node.val:` when the value can be `0` — test `if node:` / `is None`.
- Recursing on a 10^5-deep degenerate tree → `RecursionError`; go iterative.

In the interview

30-second answer: "I define small `ListNode` / `TreeNode` classes. Lists: dummy head, fast/slow pointers, in-place reversal with three pointers. Trees: recursive DFS for clarity, an explicit stack if depth can be large, BFS with a deque for level order. Space is O(height)."

- *Where does a cycle start?* → After slow and fast meet, move one to the head; step both by 1; they meet at the start.
- *Inorder successor in a BST?* → With a right child, its leftmost node; else the nearest ancestor reached via a left edge.

19.8 Graphs: adjacency lists, BFS, DFS, Dijkstra, topological sort

What it is

A **graph** is **nodes** (vertices) joined by **edges**, possibly **directed** (one-way) and **weighted** (with a cost). Standard Python form: an **adjacency list**, `defaultdict(list)` of node → neighbours (or `(neighbour, weight)` pairs). A **grid** is an implicit graph of 4 (or 8) adjacent cells.

- **BFS (breadth-first search)** — rings of increasing distance with a queue → shortest paths in **unweighted** graphs.
- **DFS (depth-first search)** — as deep as possible before backtracking (recursion or stack) → connectivity, cycles, topological order, flood fill.

- **Dijkstra's algorithm** — shortest paths with **non-negative** weights via a min-heap of `(distance, node)`.
- **Topological sort** — orders a **DAG** (directed acyclic graph) so every edge goes earlier → later: course prerequisites, build order.

How it works

BFS: start in a deque and in `visited`; pop the front, push unvisited neighbours, **marking them visited when enqueued** (marking on pop lets a node be enqueued many times).

Dijkstra (lazy version):

1. `dist = {src: 0}`, heap = `[(0, src)]`.
2. Pop the smallest `(d, u)`; if `d` exceeds the best known `dist[u]`, it's stale — skip.
3. For each edge `u → v` with weight w: if `d + w < dist[v]`, update and push `(d + w, v)`.
4. O((V + E) log V).

Topological sort, Kahn's algorithm: count in-degrees (edges pointing *into* each node); start with all in-degree-0 nodes; pop one, append it, decrement its neighbours, enqueue any reaching 0. Fewer than V nodes in the order → a cycle.

Topological sort, DFS: three colours — white (unvisited), grey (on the current path), black (finished). Meeting a grey node = a back edge = cycle. Append nodes as they finish (post-order), reverse at the end.

```
def bfs_shortest_path(g, start, goal):
    parent = {start: None}          # doubles as the visited set
    q = deque([start])
    while q:
        node = q.popleft()
        if node == goal:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            return path[::-1]
        for nxt in g.get(node, []):
            if nxt not in parent:   # mark when ENQUEUED
                parent[nxt] = node
                q.append(nxt)
    return None

def dijkstra(g, src):
    dist = {src: 0}
    heap = [(0, src)]
    while heap:
        d, u = heapq.heappop(heap)
        if d > dist.get(u, float("inf")):
            continue                    # stale entry
        for v, w in g.get(u, []):
            nd = d + w
            if nd < dist.get(v, float("inf")):
                dist[v] = nd
                heapq.heappush(heap, (nd, v))
    return dist

def topo_sort_kahn(n, prereqs):         # (a, b): b before a
    g = defaultdict(list)
    indeg = [0] * n
    for a, b in prereqs:
        g[b].append(a)
        indeg[a] += 1
    q = deque(i for i in range(n) if indeg[i] == 0)
    order = []
    while q:
        u = q.popleft()
        order.append(u)
        for v in g[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                q.append(v)
    return order if len(order) == n else None   # None -> cycle
```

The script also has `dfs_recursive`, `dfs_iterative` (pushes neighbours in reverse so the order matches), `num_islands` (grid flood fill with a stack and four direction offsets), and `topo_sort_dfs`. Output:

```
adjacency: {'A': ['B', 'C'], 'B': ['A', 'D'], 'C': ['A', 'D'], 'D': ['B', 'C', 'E'], 'E': ['D']}
dfs recursive: ['A', 'B', 'D', 'C', 'E'] | iterative: ['A', 'B', 'D', 'C', 'E']
dijkstra from A: {'A': 0, 'B': 3, 'C': 1, 'D': 4}
course order (Kahn): [0, 1, 2, 3] | DFS: [0, 2, 1, 3]
recursion limit: 1000
depth 5000 -> RecursionError
after setrecursionlimit(10_000): depth 5000
all asserts passed
```

Both topological orders are valid — a DAG usually has several.

The recursion limit

CPython won't nest calls deeper than `sys.getrecursionlimit()` (1000 by default) — `RecursionError`. Python has **no tail-call optimisation** (reusing the frame when a function's last action is calling itself), so tail-recursive code counts too. Options:

1. Prefer the **iterative** version (explicit stack/deque) for possibly deep graphs/trees — safest.

2. `sys.setrecursionlimit(10**6)` — accepted on many judges, but very deep recursion can still crash the interpreter by exhausting the real C stack; a workaround, not a fix.

Grid template

```
for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
    nr, nc = r + dr, c + dc
    if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == "1" and (nr, nc) not in seen:
        ...
```

The chained comparison `0 <= nr < rows` keeps bounds checks readable.

Trade-offs

Problem	Algorithm	Complexity
Shortest path, unweighted	BFS	O(V + E)
Shortest path, weights 0/1	0-1 BFS (deque, `appendleft` for 0-edges)	O(V + E)
Shortest path, non-negative weights	Dijkstra	O((V + E) log V)
Negative weights	Bellman–Ford (relaxes every edge V−1 times)	O(V·E)
All pairs, small V	Floyd–Warshall (DP over intermediate nodes)	O(V^3)
Order with dependencies	Topological sort	O(V + E)
Connected components (groups of mutually reachable nodes), dynamic unions	Union-Find (19.9)	~O(α(n)) per op (effectively constant, see 19.9)

Common mistakes

- Marking visited on pop in BFS (duplicates in the queue, sometimes TLE — "time limit exceeded"); a list instead of a set for `visited`.
- Dijkstra with negative edges (wrong answers).
- Forgetting isolated nodes absent from the edge list (loop over `range(n)`, not `g`).

In the interview

30-second answer: "I store graphs as `defaultdict(list)` adjacency lists. Unweighted shortest path → BFS with a deque, marking visited on enqueue. Weighted non-negative → Dijkstra with heapq, skipping stale entries. Dependencies → Kahn's topological sort, which also detects cycles. For deep graphs, iterative DFS because of the recursion limit."

- *Why not Dijkstra with negative edges?* → It finalises a node when popped, assuming no later path is shorter; a negative edge breaks that.
- *Cycle in an undirected graph?* → DFS ignoring the edge back to the parent, or union-find (an edge joining two nodes already in one set).

19.9 Union-Find (disjoint set union)

What it is

Union-Find (**DSU**, disjoint set union) tracks which items share a group while groups merge: `find(x)` → the group's representative ("root"); `union(a, b)` → merge two groups. Like asking whether two employees work for the same firm after mergers — follow "who owns whom" to the top parent.

How it works

1. `parent[i] = i` initially (everyone is their own root).
2. `find` follows parents to the root; **path compression** then points every node on the path straight at the root.
3. `union` finds both roots; if different, attaches the **smaller** tree under the larger (**union by size**) and decrements the component count. Returns `False` if already joined — detecting a cycle-closing edge.
4. With both optimisations each operation is O(α(n)) amortised — α is the inverse Ackermann function, ≤ 4 for any realistic n, "effectively constant".

```python
class DSU:
    def __init__(self, n):
        self.parent = list(range(n))
        self.size = [1] * n
        self.components = n

    def find(self, x):
        root = x
        while self.parent[root] != root:
            root = self.parent[root]
        while self.parent[x] != root:          # path compression (iterative)
            self.parent[x], x = root, self.parent[x]
        return root

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False
        if self.size[ra] < self.size[rb]:
            ra, rb = rb, ra
        self.parent[rb] = ra
        self.size[ra] += self.size[rb]
        self.components -= 1
        return True

def kruskal_mst(n, edges):              # edges (weight, u, v)
    dsu, total = DSU(n), 0
    for w, u, v in sorted(edges):
        if dsu.union(u, v):
            total += w
    return total

assert kruskal_mst(4, [(1, 0, 1), (4, 0, 2), (3, 1, 2), (2, 2, 3), (5, 1, 3)]) == 6
```

Output: `parent: [0, 0, 0, 3, 3, 5] components: 3` then `all asserts passed`. The script also solves "number of connected components" and "redundant connection" (the first edge whose `union` returns `False`). The iterative `find` avoids the recursion limit; in `self.parent[x], x = root, self.parent[x]` the right side (old parent) is evaluated before `x` is rebound.

When not to use it: it can't **split** groups or give paths — use BFS/DFS. For a one-off component count on a static graph, BFS/DFS is equally good.

In the interview

30-second answer: "Union-Find tracks groups under merging. `find` with path compression and `union` by size give near-constant amortised time. I use it for connected components, cycle detection in undirected graphs, Kruskal's MST (minimum spanning tree — the cheapest edge set connecting every node), and 'accounts merge'-type problems."

- *Why union by size?* → Keeps trees shallow (height O(log n) even without compression).
- *Kruskal's complexity?* → O(E log E) for the sort; the DSU part is near-linear.

19.10 Dynamic programming: `@cache` and tabulation

What it is

Dynamic programming (DP) combines answers to smaller **overlapping subproblems**, computing each once; it applies when there is **optimal substructure** (the best answer is built from best sub-answers).

- **Top-down (memoisation):** write the natural recursion, cache it with `functools.cache` (3.9+) or `lru_cache(maxsize=None)`.
- **Bottom-up (tabulation):** fill an array from the smallest subproblems with loops; no recursion, often only the last row kept.

How it works — the recipe

1. **State:** parameters describing a subproblem (`i`, `(i, j)` prefixes of two strings, `(i, capacity)` …).
2. **Transition:** how a state's answer comes from smaller states.
3. **Base cases.**
4. **Order** (bottom-up) or **memoise** (top-down).
5. **Answer location** (`dp[n]`, `dp[m][n]`, `max(dp)` …).

Coin change, coins `[1, 2, 5]`, amount 11: `dp[a] = 1 + min(dp[a − c])` over coins c ≤ a, `dp[0] = 0`. `dp[5] = 1`, `dp[10] = 2`, `dp[11] = 1 + min(dp[10], dp[9], dp[6]) = 3`.

```python
from functools import cache

@cache
def climb(n):                    # 1 or 2 steps at a time
    return 1 if n <= 1 else climb(n - 1) + climb(n - 2)

def coin_change(coins, amount):
    INF = amount + 1
    dp = [0] + [INF] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c <= a:
                dp[a] = min(dp[a], dp[a - c] + 1)
    return dp[amount] if dp[amount] != INF else -1

def lcs(a, b):
    m, n = len(a), len(b)
    dp = [[0] * (n + 1) for _ in range(m + 1)]    # NOT [[0]*(n+1)]*(m+1)
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1] + 1
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]

def edit_distance(a, b):
    @cache
    def go(i, j):
        if i == len(a):
            return len(b) - j
        if j == len(b):
            return len(a) - i
        if a[i] == b[j]:
            return go(i + 1, j + 1)
        return 1 + min(go(i + 1, j), go(i, j + 1), go(i + 1, j + 1))
    return go(0, 0)

def knapsack_01(weights, values, cap):
    dp = [0] * (cap + 1)
    for w, v in zip(weights, values):
        for c in range(cap, w - 1, -1):    # backwards: each item used once
            dp[c] = max(dp[c], dp[c - w] + v)
    return dp[cap]
```

Output:

```
climb.cache_info(): CacheInfo(hits=8, misses=11, maxsize=None, currsize=11)
coin_change([1,2,5], 11) = 3
edit_distance('horse','ros') = 3
knapsack = 9
all asserts passed
```

`cache_info()` shows 11 real computations for `climb(10)`; the naive recursion makes 177 calls, growing exponentially with n.

The script also has `house_robber` (O(1)-space DP with two variables), `unique_paths` (one-row grid DP) and `lis_length` — O(n log n) longest increasing subsequence keeping `tails[k]` = smallest tail of an increasing run of length k+1, placing each number with `bisect_left`.

Common DP families

Family	State	Examples
1-D sequence	`dp[i]`	climbing stairs, house robber, decode ways, LIS (longest increasing subsequence)
Unbounded choice	`dp[amount]`	coin change, combination sum count
0/1 knapsack (each item at most once, maximise value within a weight limit)	`dp[capacity]` iterated backwards	subset sum, partition equal subset
Two strings	`dp[i][j]`	LCS (longest common subsequence), edit distance (fewest single-letter edits between strings), regex/wildcard matching
Grid	`dp[r][c]`	unique paths, minimum path sum
Intervals	`dp[i][j]` over lengths	burst balloons, palindrome partitioning

Top-down vs bottom-up

	`@cache` recursion	Tabulation
Writing it	closest to the recurrence (state in terms of smaller states); quick	must choose iteration order
Computes	only reachable states	all states
Limits	recursion limit; function-call overhead	none
Memory tricks	hard	easy to keep one row (O(n) instead of O(n·m))

Common mistakes

- `@cache` on a function taking a **list** → `TypeError: unhashable type`. Pass tuples or indices.
- A module-level `@cache` reused across test cases with different global data returns stale answers — define the helper **inside** the solving function (as `edit_distance` does) or `.cache_clear()`.
- 2-D tables via `[[0] * n] * m` (all rows are one list — 19.19).
- 0/1 knapsack capacity loop **forwards** (becomes unbounded knapsack).

In the interview

> **30-second answer:** "I first write the state and recurrence, then memoise with `functools.cache` — top-down. If recursion depth or memory matters I convert to a bottom-up table and often compress it to one row. Complexity is states × work per state."

- *Edit distance complexity?* → O(m·n) time; O(m·n) space top-down, O(min(m, n)) with rolling rows.
- *`lru_cache` vs `cache`?* → `cache` is `lru_cache(maxsize=None)`: unbounded, cheaper since it never tracks recency; `lru_cache(maxsize=k)` evicts least-recently-used entries.
- *How many states did `@cache` store?* → `f.cache_info().currsize`.

19.11 Backtracking

What it is

Backtracking builds candidates step by step and abandons ("prunes") a partial candidate once it can't lead to a valid answer — DFS over a tree of choices. Template: **choose** → **explore** → **un-choose**.

How it works

1. Keep a mutable `path` (partial solution) and maybe `used` arrays/sets for constraints.
2. At each level loop over choices, skipping invalid ones (prune).
3. Append, recurse, pop to restore state.
4. On a complete path, record **a copy** (`path[:]`).

```
def subsets(nums):
    out, path = [], []
    def go(start):
        out.append(path[:])             # copy! path keeps changing
        for i in range(start, len(nums)):
            path.append(nums[i])        # choose
            go(i + 1)                   # explore
            path.pop()                  # un-choose
    go(0)
    return out

def combination_sum(cands, target):     # reuse allowed
    cands = sorted(cands)
    out, path = [], []
    def go(start, remaining):
        if remaining == 0:
            out.append(path[:])
            return
        for i in range(start, len(cands)):
            if cands[i] > remaining:
                break                   # prune: later ones are bigger
            path.append(cands[i])
            go(i, remaining - cands[i])  # i, not i+1: reuse allowed
            path.pop()
    go(0, target)
    return out

def n_queens(n):
    cols, diag, anti = set(), set(), set()
    def go(r):
        if r == n:
            return 1
        count = 0
        for c in range(n):
            if c in cols or r - c in diag or r + c in anti:
                continue
            cols.add(c); diag.add(r - c); anti.add(r + c)
            count += go(r + 1)
            cols.remove(c); diag.remove(r - c); anti.remove(r + c)
        return count
    return go(0)
```

Output:

```
subsets: [[], [1], [1, 2], [1, 2, 3], [1, 3], [2], [2, 3], [3]]
combination_sum([2,3,6,7], 7): [[2, 2, 3], [7]]
8-queens solutions: 92
all asserts passed
```

The tests check `subsets`/`permute` against `itertools.combinations`/`permutations`; `word_search` marks grid cells visited by overwriting them with `"#"` and restoring afterwards.

`itertools` **shortcuts** (Chapter 11): `permutations(xs, r)`, `combinations(xs, r)`, `combinations_with_replacement`, `product(xs, repeat=k)`. Interviewers usually want the hand-written version when pruning matters, but mentioning these shows fluency.

Complexity: exponential — subsets $O(2^n \cdot n)$, permutations $O(n! \cdot n)$. Pruning shrinks real work, not the worst case.

Common mistakes

- Appending `path` instead of `path[:]` — every stored result is the same (final, usually empty) list.
- Forgetting to undo the choice (sets/`used` flags drift).
- Duplicate inputs (e.g. subsets II): sort, skip `nums[i] == nums[i-1]` when `i > start`.

In the interview

> **30-second answer:** "Backtracking is DFS over choices: choose, recurse, undo. I store a copy of the path at the leaves and prune early — sort candidates so I can break, and use sets for O(1) constraint checks, like columns and diagonals in N-Queens."

- *Why `r - c` and `r + c` for diagonals?* → Cells on one "" diagonal share r − c; on one "/" diagonal r + c.
- *Subsets of a 4-element set?* → $2^4 = 16$.

19.12 Bit tricks and math

What it is

Python `int`s are arbitrary-precision and act like infinite **two's-complement** numbers (−x = flip all bits, add 1) for bitwise operators: `&`, `|`, `^` (xor — 1 when exactly one bit is 1), `~` (`~x == -x - 1`), `<<`, `>>` (arithmetic shift — keeps the sign, floors negatives). `math` and built-in `pow` cover interview number theory (divisibility, primes, remainders).

Bit cheat table (x = 180 = 0b10110100, from the real output)

Trick	Expression	Result
test bit i	`x >> i & 1`	bit 2 → 1
set bit i	`x | (1 << i)`	set bit 0 → `0b10110101`
clear bit i	`x & ~(1 << i)`	
toggle bit i	`x ^ (1 << i)`	
lowest set bit	`x & -x`	4
clear lowest set bit	`x & (x - 1)`	`0b10110000`
power of two?	`x > 0 and x & (x - 1) == 0`	
count 1-bits	`x.bit_count()` (3.10+) or `bin(x).count("1")`	4
bits needed	`x.bit_length()`	8
format	`f"{x:08b}"`, `bin(x)`, `int("1011", 2)`	

XOR identities power "single number": `a ^ a = 0`, `a ^ 0 = a`, order-independent. A **bitmask** (an int whose bits are on/off flags) running `0` … $2^n - 1$ enumerates all subsets (bit i set ⇒ item i chosen) — the basis of bitmask DP.

Math toolkit

Need	Python
gcd / lcm	`math.gcd(a, b)`, `math.lcm(a, b)` (3.9+, both accept many args)
modular power	`pow(b, e, m)` — fast square-and-multiply, O(log e)
modular inverse (the x with a·x ≡ 1 mod m; m prime or gcd = 1)	`pow(a, -1, m)` (3.8+)
integer square root	`math.isqrt(n)` — exact, no float error
combinations	`math.comb(n, k)`, `math.perm(n, k)`, `math.factorial(n)`
floor/ceil division	`a // b`, `-(-a // b)` or `math.ceil(a / b)` (float — careful with huge ints)
quotient and remainder	`divmod(a, b)`
infinity	`float("inf")`, `math.inf`; `sys.maxsize` is just a big int
big modulus	`MOD = 10**9 + 7`

Output of the script (excerpt):

```
~5 = -6  (two's complement: -(5+1))
-7 >> 1 = -4  (arithmetic shift floors)
gcd(84, 36) = 12 | lcm(4, 6) = 12
pow(2, 100, MOD) = 976371285
comb(5,2) = 10 | perm(5,2) = 20 | factorial(5) = 120
isqrt(10**20 + 1) = 10000000000
primes <= 30: [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]
2**100 = 1267650600228229401496703205376 (arbitrary precision)
divmod(-7, 2) = (-4, 1) | math.fmod(-7, 2) = -1.0
```

The script also hand-writes `fast_pow` (square-and-multiply: square the base each step, multiply it in when the current exponent bit is 1) and a Sieve of Eratosthenes (list primes ≤ n by crossing out multiples from i², O(n log log n)).

Common mistakes

- Expecting 32-bit overflow: "return the 32-bit result" needs masking (`x & 0xFFFFFFFF`, converting back if the top bit is set).
- `~x` is not "flip bits of an unsigned number"; it's `-x - 1`.
- `math.sqrt` on big integers (float precision) — use `math.isqrt`.
- `(a ** b) % m` builds a giant number first — use `pow(a, b, m)`.

In the interview

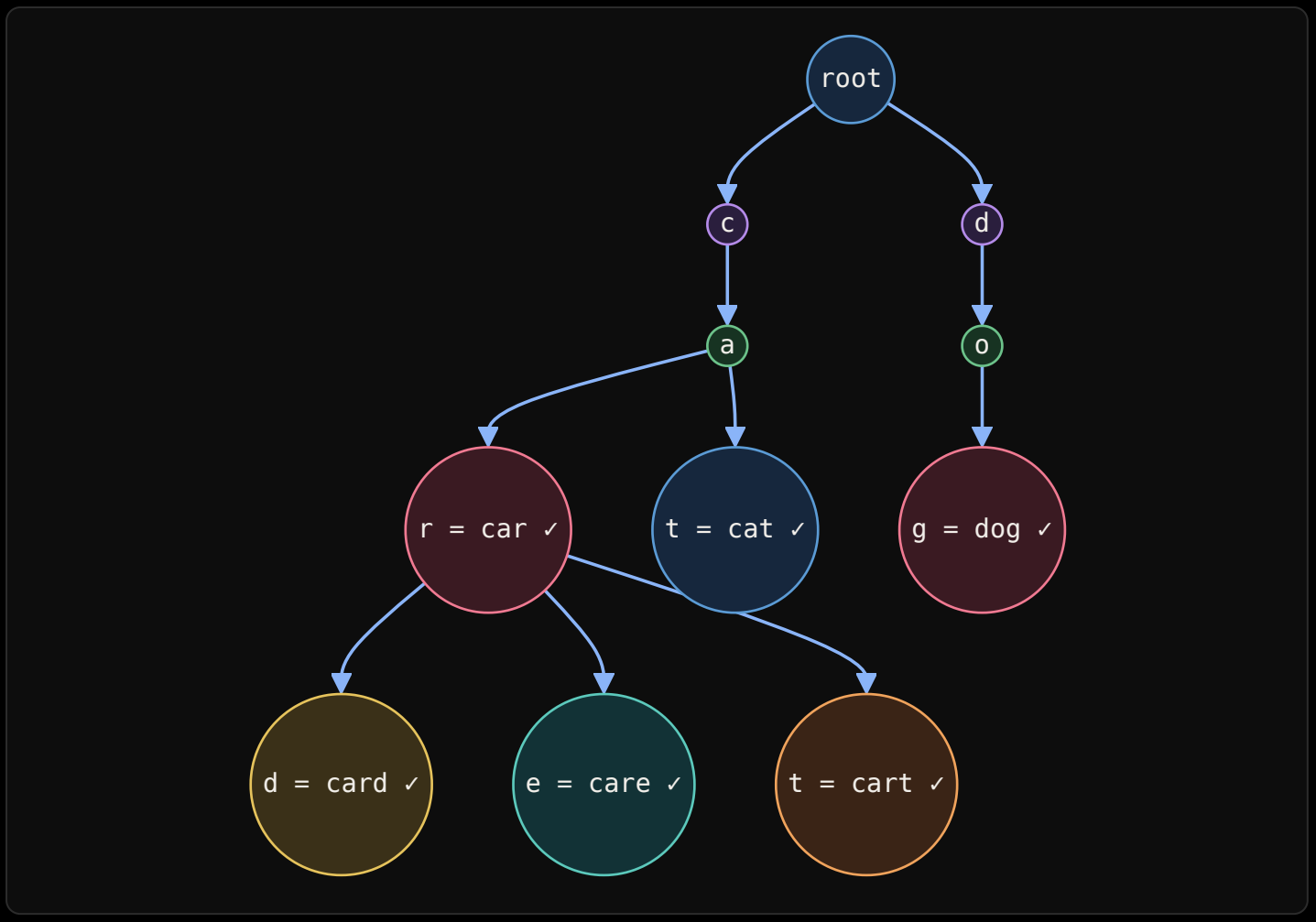
30-second answer: "Python ints never overflow, so I use `pow(b, e, m)` for modular exponentiation, `math.gcd`, `math.isqrt` and `math.comb` instead of hand-rolling. For bits: `x & (x-1)` drops the lowest set bit, `x & -x` isolates it, XOR cancels pairs."

- *Power of four?* → Power of two with the bit at an even position: `n > 0 and n & (n-1) == 0 and n & 0x55555555 != 0` (32-bit), or `n.bit_length() % 2 == 1` for any size. (`&` binds tighter than `==` in Python, unlike C.)
- *`6 & 5`, `6 | 5`, `6 ^ 5`?* → 4, 7, 3.
- *Inverse of 3 mod 7?* → `pow(3, -1, 7) = 5` (3·5 = 15 ≡ 1).

19.13 Tries (prefix trees)

What it is

A **trie** ("try", from re*trie*val) stores strings letter by letter. Each node is a **prefix**, each edge adds one letter, and a flag marks "a word ends here". Words starting with "car" share c → a → r, so **"which words start with this prefix?"** is one walk down the prefix — no scan of the word list.



The problem it solves

With a `set` of words, "is `car` a word?" is O(L) (L = word length), but "does any word start with `ca`?" checks every word — O(n·L). A trie makes both O(L) and lets a search **stop early** once a path is no prefix of anything — what makes board word search fast.

How it works

A node is a `dict` of children plus the word ending there; `setdefault(ch, TrieNode())` walks or creates in one step.

```python
class TrieNode:
    __slots__ = ("children", "word")        # many small nodes -> save memory
    def __init__(self):
        self.children = {}                   # letter -> TrieNode
        self.word = None                     # full word if one ends here

class Trie:
    def __init__(self):
        self.root = TrieNode()
    def insert(self, word):
        node = self.root
        for ch in word:
            node = node.children.setdefault(ch, TrieNode())
        node.word = word
    def _walk(self, s):
        node = self.root
        for ch in s:
            node = node.children.get(ch)
            if node is None:
                return None
        return node
    def search(self, word):
        node = self._walk(word)
        return node is not None and node.word is not None
    def starts_with(self, prefix):
        return self._walk(prefix) is not None
```

Problem 1 — autocomplete. Walk to the prefix node, DFS below it in alphabetical order, collect the first `limit` words (`words_with_prefix`).

Problem 2 — wildcard search. `.` matches any one letter: follow one child for a letter, try **every** child for `.` (`any(...)` stops at the first success):

```python
def search_wildcard(self, pattern):
    def dfs(node, i):
        if i == len(pattern):
            return node.word is not None
        if pattern[i] == ".":
            return any(dfs(child, i + 1) for child in node.children.values())
        child = node.children.get(pattern[i])
        return child is not None and dfs(child, i + 1)
    return dfs(self.root, 0)
```

Problem 3 — word search on a board. Find every listed word traceable through horizontally/vertically adjacent cells without reusing a cell. Instead of one grid search per word, put all words in one trie and run **one backtracking DFS from each cell that walks the trie simultaneously**; a path leaving the trie is abandoned at once.

```python
def find_words(board, words):
    trie = Trie()
    for w in words:
        trie.insert(w)
    rows, cols, found = len(board), len(board[0]), []

    def dfs(r, c, parent):
        ch = board[r][c]
        node = parent.children.get(ch)
        if node is None:
            return                          # no word continues this way
        if node.word is not None:
            found.append(node.word)
            node.word = None                # report each word once
        board[r][c] = "#"                   # mark visited in place
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols and board[nr][nc] != "#":
                dfs(nr, nc, node)
        board[r][c] = ch                    # undo (backtracking, 19.11)
        if not node.children and node.word is None:
            del parent.children[ch]         # prune a finished branch

    for r in range(rows):
        for c in range(cols):
            dfs(r, c, trie.root)
    return sorted(found)
```

Output of `13_tries.py`:

```
autocomplete 'car': ['car', 'card', 'care']
wildcard 'ca.e': True | '.o': False
words on board: ['eat', 'oat', 'oath']
all asserts passed
```

Complexity

Operation	Time	Notes
insert / search / starts_with	O(L)	L = length of the word or prefix
wildcard search	O(L) with no dots; up to O(26^d · L) with d dots	every dot branches
build for n words	O(total letters)	space: up to one node per letter
word search on an r × c board, max word length L	O(r·c·4·3^(L−1)) worst case	pruning makes it far faster in practice

When not to use it: only exact lookups → a `set` (simpler, less memory); one prefix query on a static list → `sorted` + `bisect`.

Common mistakes

- "Path exists" ≠ "word exists" — `search("ca")` must be `False` when only "car" was inserted; that's the end-of-word field's job.
- Board search: not restoring the cell, or reporting a word twice when two paths spell it (clear `node.word` after the first find).
- `defaultdict(TrieNode)`-style auto-creation in `search` — every miss grows the trie.

In the interview

> **30-second answer:** "A trie stores words letter by letter; each node is a prefix, so insert, search and prefix checks are O(L). In Python a node is a dict of children plus an end-of-word marker. For a wildcard I branch into every child; for board word search I put all words in one trie and backtrack from every cell while walking it, pruning as soon as the path isn't a prefix."

- *Trie vs a hash set of all prefixes?* → The set costs O(total L^2) memory and can't enumerate words under a prefix; a trie shares prefixes.
- *Top 3 suggestions by popularity?* → Store each node's top-3 words (updated on insert) → O(L) lookup.
- *Too much memory?* → A **radix tree** (compress single-child chains into one edge), or 26-slot arrays instead of dicts for lowercase-only input.

19.14 Greedy

What it is

A **greedy algorithm** builds the answer step by step, always taking the choice that looks best **now** and never revisiting it. It's correct only if the local best never blocks the global best — typically shown by an **exchange argument**: "take any optimal answer; swapping its first choice for the greedy one keeps it optimal."

The problem it solves

Optimisation problems that seem to need DP (19.10) over every combination collapse to one pass — O(n), or O(n log n) after sorting, O(1) extra space — when a greedy rule is provably safe.

How it works

No single template: look for "keep only the best so far" or "sort, then take whatever still fits". Four classics:

Problem 1 — jump game. `nums[i]` = max jump from i. *Reachable?* Track `farthest`; standing beyond it means stuck. *Fewest jumps?* BFS levels (19.8) without a queue: indices reachable in j jumps form a range; at the range's end you must jump again, and the new range ends at the farthest point seen.

```python
def can_jump(nums):
    farthest = 0
    for i, n in enumerate(nums):
        if i > farthest:
            return False
        farthest = max(farthest, i + n)
    return True

def min_jumps(nums):
    jumps = end = farthest = 0
    for i in range(len(nums) - 1):
        farthest = max(farthest, i + nums[i])
        if i == end:                # finished this level -> jump again
            jumps += 1
            end = farthest
    return jumps
```

Problem 2 — gas station. Stations on a circle; station i gives `gas[i]`, reaching the next costs `cost[i]`. Find the start that completes the loop (−1 if none). If total gas < total cost, none works. If the tank goes negative at i after starting at s, **no** station between s and i works either: starting there means arriving with an empty tank, whereas the run from s arrived with ≥ 0 and still failed. So jump the start to i + 1.

```python
def gas_station(gas, cost):
    if sum(gas) < sum(cost):
        return -1
    start = tank = 0
    for i, (g, c) in enumerate(zip(gas, cost)):
        tank += g - c
        if tank < 0:
            start, tank = i + 1, 0
    return start
```

Problem 3 — candy. Children in a line with ratings; each gets ≥ 1 candy and a higher-rated child gets more than that neighbour. Minimise the total: a left-to-right pass satisfies left neighbours, a right-to-left pass right neighbours, `max` keeps both.

```python
def candy(ratings):
    n = len(ratings)
    c = [1] * n
    for i in range(1, n):
        if ratings[i] > ratings[i - 1]:
            c[i] = c[i - 1] + 1
    for i in range(n - 2, -1, -1):
        if ratings[i] > ratings[i + 1]:
            c[i] = max(c[i], c[i + 1] + 1)
    return sum(c)
```

Problem 4 — interval scheduling (`max_non_overlapping`): to fit the most non-overlapping talks in one room, sort by **end** time and take every talk starting after the last taken. The talk ending first leaves the most room — the exchange argument. Sorting by start or by length both fail on small counter-examples.

Output of `14_greedy.py`:

```
can_jump [3,2,1,0,4]: False | min_jumps [2,3,1,1,4]: 2
gas station start: 3
candy [1,3,4,5,2]: 11
max non-overlapping talks: 4
all asserts passed
```

Trade-offs

	Greedy	DP
Speed	usually O(n) or O(n log n)	states × work per state
Correctness	needs a proof; a plausible rule is often wrong	follows from the recurrence
Example of greedy failing	coins {1, 3, 4}, amount 6: greedy 4+1+1 = 3 coins; best 3+3 = 2	DP gets 2

Common mistakes

- A greedy rule never checked against a small counter-example (the coin case). Can't justify it in one sentence → consider DP.
- Candy: one pass only, or `c[i] = c[i+1] + 1` in the second pass, breaking the first (use `max`).
- Min jumps: looping to the last index inclusive counts one extra jump.

In the interview

30-second answer: "Greedy takes the locally best choice and never revisits it, usually one pass after a sort. I only use it when I can justify it — typically an exchange argument, like 'the interval that ends first never hurts' — otherwise DP. Jump game tracks the farthest reachable index; gas station resets the start when the tank goes negative; candy is two passes."

- *How do you know greedy is right?* → Exchange argument, or "greedy stays ahead": after every step its partial answer is at least as good as any other.
- *Why is min jumps O(n), not BFS O(n²)?* → Each level is a contiguous range; track only its end and the farthest reach.

19.15 Intervals

What it is

An **interval** is `[start, end]` — a meeting, booking, ID range. Problems ask about overlap, merging, or how many are active at once. `[a, b]` and `[c, d]` **overlap** when `max(a, c) <= min(b, d)` (`<` if touching ends don't count — ask).

The problem it solves

All pairs is O(n²). **Sorting** puts overlapping intervals next to each other, so most problems become one O(n) pass after an O(n log n) sort. The key matters: **start** for merging, **end** for "keep as many as possible" (19.14).

How it works

Problem 1 — merge. Sort by start; each interval extends the last kept one or starts a new one (snippet in 19.6).

Problem 2 — insert into a sorted, non-overlapping list. No re-sort: copy intervals ending before the new one, absorb all that overlap it, copy the rest. O(n).

```
def insert(intervals, new):
    out, i, n = [], 0, len(intervals)
    s, e = new
    while i < n and intervals[i][1] < s:          # entirely before
        out.append(intervals[i]); i += 1
    while i < n and intervals[i][0] <= e:         # overlapping -> absorb
        s, e = min(s, intervals[i][0]), max(e, intervals[i][1]); i += 1
    out.append([s, e])
    out.extend(intervals[i:])                     # entirely after
    return out
```

Problem 3 — intersection of two sorted lists. Two pointers (19.2): the current pair's overlap is `[max(starts), min(ends)]` if non-empty; advance whichever **ends first** — it can't overlap anything later in the other list.

Problem 4 — minimum meeting rooms (peak simultaneous meetings):

```
def min_rooms_heap(meetings):          # heap of end times of meetings in progress
    ends = []
    for start, end in sorted(meetings):
        if ends and ends[0] <= start:  # the earliest-ending room is free again
            heapq.heapreplace(ends, end)
        else:
            heapq.heappush(ends, end)
    return len(ends)

def min_rooms_sweep(meetings):         # sweep line: +1 at a start, -1 at an end
    events = sorted([(s, 1) for s, _ in meetings] + [(e, -1) for _, e in meetings])
    return max(accumulate(d for _, d in events), default=0)
```

Sweep line: each interval becomes two events walked in time order with a running count. Tuple sorting puts `(t, -1)` before `(t, 1)`, so a meeting ending at 10 frees its room for one starting at 10.

Output of `15_intervals.py`:

```
merged: [[1, 6], [8, 10], [15, 18]]
insert [4,8]: [[1, 2], [3, 10], [12, 16]]
intersection: [[1, 2], [5, 5], [8, 10], [15, 23], [24, 24], [25, 25]]
rooms needed: 2
all asserts passed
```

The script also has `erase_overlaps` (fewest deletions so the rest don't overlap = n − the greedy maximum from 19.14).

Trade-offs

Question	Sort by	Technique	Cost
merge / union	start	extend last kept	O(n log n)
insert one	already sorted	three phases	O(n)
overlap of two lists	already sorted	two pointers	O(m + n)
max active at once	start (heap) or event time (sweep)	min-heap of ends / running sum	O(n log n)
keep the most / delete the fewest	**end**	greedy	O(n log n)

Common mistakes

- Not asking whether `[1, 4]` and `[4, 5]` overlap — `<=` vs `<` changes the answer.
- Merge: `out[-1][1] = end` instead of `max(out[-1][1], end)` — breaks on a nested interval (`[1, 10]`, `[2, 3]`).
- Sweep line: starts sorting before ends at the same time over-counts rooms.

In the interview

> **30-second answer:** "I sort the intervals — by start to merge or count overlaps, by end to keep as many as possible — then make one pass. Two intervals overlap when max of starts ≤ min of ends. For meeting rooms I use a min-heap of end times or a sweep line of +1/−1 events; both O(n log n)."

- *Which room does each meeting get?* → Heap of `(end, room_id)`; reuse the popped room's id.
- *Intervals arrive as a stream?* → Keep them sorted by start (`sortedcontainers.SortedList` or `bisect`) and merge with neighbours on insert.

19.16 Fast & slow pointers

What it is

Two pointers walk the **same** sequence at different speeds — `slow` one step, `fast` two: **Floyd's cycle-finding algorithm** ("tortoise and hare"). Works on anything with a "next" — a linked list, or an implicit sequence x → f(x) — in O(1) extra memory. 19.7 used it for the middle and a yes/no cycle check; here is the full set.

The problem it solves

Loop detection or finding a midpoint otherwise needs a visited `set` (O(n) memory) or a counting pass. Fast/slow does both in one pass with two variables.

How it works

- **Middle:** when `fast` ends, `slow` has done half the steps (even length → second middle).

- **Cycle start:** inside a loop `fast` gains a step per move, so they must meet. Then reset one pointer to the head and step both by one: they meet at the entry. (Why: with a = head-to-entry distance, meeting point b steps into a loop of length c, `fast` walked twice as far as `slow`, giving a = k·c − b for some whole k — so a steps from the head and a steps from the meeting point land on the entry.)

```python
def cycle_start(head):
    slow = fast = head
    while fast and fast.next:
        slow, fast = slow.next, fast.next.next
        if slow is fast:                    # `is`: same node, not equal values
            slow = head
            while slow is not fast:
                slow, fast = slow.next, fast.next
            return slow
    return None
```

Problem 1 — palindrome linked list in O(1) space. Find the middle, reverse the second half in place (19.7), compare halves, reverse it back so the caller's list is unchanged.

Problem 2 — happy number. Replace n by the sum of the squares of its digits (19 → 82 → 68 → 100 → 1); **happy** if it reaches 1. The sequence either hits 1 or loops, so treat `f(n)` as "next" and run Floyd:

```python
def is_happy(n):
    def f(x):
        return sum(int(d) ** 2 for d in str(x))
    slow, fast = n, f(n)
    while fast != 1 and slow != fast:
        slow, fast = f(slow), f(f(fast))
    return fast == 1
```

Problem 3 — find the duplicate number. n + 1 values in 1..n, one repeated, read-only, O(1) space. Treat index i → `nums[i]` as "next": two indices point at the same value, which is therefore a cycle's entry — `cycle_start` on an array (`find_duplicate`).

Output of `16_fast_slow_pointers.py`:

```
cycle starts at value: 2
palindrome [1,2,3,2,1]: True
happy numbers <= 30: [1, 7, 10, 13, 19, 23, 28]
all asserts passed
```

Complexity: all O(n) time, O(1) extra space (n = list length, or distinct values before the sequence repeats).

Common mistakes

- `while fast.next and fast` checks `fast.next` before knowing `fast` isn't `None`. Always `while fast and fast.next`.
- `==` on `@dataclass` nodes: the generated `__eq__` compares values, so distinct nodes with equal fields are "equal" (and on a cyclic list the comparison may never finish). Use `is`, or `@dataclass(eq=False)` as the script does.
- Leaving the list half-reversed after a palindrome check — "did you modify the input?"

In the interview

> **30-second answer:** "Slow moves one, fast moves two. When fast ends, slow is at the middle; if they meet there's a cycle, and resetting one to the head and stepping both by one finds the entry. O(n) time, O(1) space, and it works on implicit sequences — happy number, or find-the-duplicate with index → value as the next pointer."

- *Cycle length?* → After meeting, hold one still and count steps until the other returns.

- *Why not a visited set?* → Also O(n) time and simpler, but O(n) memory; offer it first, then optimise.

19.17 Sort & search: quicksort, quickselect, Dutch flag, merge sort on a list

What it is

You'll normally call `sorted` (19.6), but the ideas inside sorting algorithms solve other problems:

- **Partition** — rearrange around a **pivot** so smaller items come first, larger after. Core of **quicksort** (partition, sort both sides) and **quickselect** (partition, continue into **one** side to find the k-th element).
- **Three-way partition** (Dutch national flag) — sort an array of three distinct values in one pass.
- **Merge sort** — split, sort halves, merge. On a linked list it needs no extra arrays — the list sorting algorithm.

The problem it solves

k-th largest: sorting O(n log n); a heap (19.5) O(n log k); quickselect **O(n) average**, since each partition discards one side. Sorting 0s/1s/2s by counting takes two passes; the Dutch flag one.

How it works

Lomuto partition with a random pivot (making the O(n²) worst case — always picking the smallest — extremely unlikely):

```
def partition(nums, lo, hi, rng):
    p = rng.randint(lo, hi)
    nums[p], nums[hi] = nums[hi], nums[p]    # move the pivot to the end
    pivot, store = nums[hi], lo
    for i in range(lo, hi):
        if nums[i] < pivot:                  # grow the "smaller" region [lo, store)
            nums[i], nums[store] = nums[store], nums[i]
            store += 1
    nums[store], nums[hi] = nums[hi], nums[store]
    return store                             # pivot is now in its final sorted place

def kth_largest(nums, k, seed=0):
    nums, rng = nums[:], random.Random(seed)
    target = len(nums) - k                   # k-th largest = index n-k when sorted
    lo, hi = 0, len(nums) - 1
    while True:
        p = partition(nums, lo, hi, rng)
        if p == target:
            return nums[p]
        if p < target:
            lo = p + 1
        else:
            hi = p - 1
```

`quicksort` in the script uses the same `partition` with an explicit stack of `(lo, hi)` ranges instead of recursion.

Dutch national flag — three regions grow while "unknown" shrinks. Invariant: `[0, low)` 0s, `[low, mid)` 1s, `(high, end]` 2s, `[mid, high]` unseen.

```python
def sort_colors(nums):
    low, mid, high = 0, 0, len(nums) - 1
    while mid <= high:
        if nums[mid] == 0:
            nums[low], nums[mid] = nums[mid], nums[low]
            low += 1; mid += 1
        elif nums[mid] == 1:
            mid += 1
        else:                                # a 2 goes to the back; the value swapped
            nums[mid], nums[high] = nums[high], nums[mid]   # in is unseen, so mid stays
            high -= 1
```

Merge sort on a linked list — find the middle with fast/slow (19.16; start `fast` at `head.next` so `slow` stops at the end of the left half), cut, sort halves recursively, merge with a dummy head (19.7). `sort_list` in the script.

Output of `17_sort_search.py` (also checks `quicksort` and `kth_largest` against `sorted` on 200 random lists):

```
merge-sorted list: [1, 2, 2, 3, 4, 5]
quicksort: [1, 2, 4, 7, 8, 9] | 2nd largest of [3,2,1,5,6,4]: 5
sort colors: [0, 0, 1, 1, 2, 2]
all asserts passed
```

Trade-offs

Algorithm	Time (avg / worst)	Extra space	Stable?	Use when
`sorted` (Timsort)	O(n log n) / O(n log n)	O(n)	yes	always, in real code
quicksort	O(n log n) / O(n²)	O(log n) stack	no	asked to implement in place
merge sort	O(n log n) / O(n log n)	O(n) array, O(log n) list	yes	linked lists; external sorting (data too big for memory, sorted in chunks on disk)
quickselect	O(n) / O(n²)	O(1)	—	one k-th element, input may be reordered
heap top-k	O(n log k)	O(k)	—	streaming data, or k ≪ n
Dutch flag	O(n)	O(1)	no	exactly three key values

Common mistakes

- Always pivoting on the first/last element — O(n²) on sorted input. Randomise.
- Many equal values: Lomuto puts all equals on one side and degrades; a three-way partition fixes it.
- Dutch flag: advancing `mid` after swapping with `high`.
- Quickselect silently mutating the caller's list — copy, or ask.

In the interview

30-second answer: "Partition puts a pivot in its final place, smaller left, larger right. Quicksort recurses on both sides — O(n log n) average; quickselect follows only the side containing index n−k — O(n) average; random pivots avoid the worst case. Three distinct values: Dutch flag, one pass with low/mid/high. Linked lists: merge sort, splitting with fast/slow pointers."

- *Guaranteed O(n) selection?* → Median of medians (guarantees a reasonably central pivot), O(n) worst case but slower in practice; mention, don't code.
- *Quickselect vs heap for top-k?* → Quickselect is O(n) average but needs all data in memory and reorders it; a size-k heap streams in O(k) memory.

19.18 Math & geometry

What it is

Problems where the trick is a small mathematical fact rather than a data structure: walking a matrix in a pattern, integer limits, comparing slopes exactly, replacing a simulation with a formula. Toolkit in 19.12.

How it works

Problem 1 — spiral order. Keep four boundaries — `top`, `bottom`, `left`, `right` — and peel one ring at a time (top row, right column, bottom row, left column), shrinking each boundary after its side. The two `if` checks stop a single remaining row or column being read twice.

```python
def spiral_order(matrix):
    out = []
    if not matrix:
        return out
    top, bottom, left, right = 0, len(matrix) - 1, 0, len(matrix[0]) - 1
    while top <= bottom and left <= right:
        out += [matrix[top][c] for c in range(left, right + 1)];  top += 1
        out += [matrix[r][right] for r in range(top, bottom + 1)]; right -= 1
        if top <= bottom:
            out += [matrix[bottom][c] for c in range(right, left - 1, -1)]; bottom -= 1
        if left <= right:
            out += [matrix[r][left] for r in range(bottom, top - 1, -1)]; left += 1
    return out
```

`rotate` turns an n × n matrix 90° clockwise in place: **transpose** (swap `m[i][j]` and `m[j][i]`), then reverse each row.

Problem 2 — reverse an integer with 32-bit overflow. Return 0 if the reversed value leaves $[-2^{31}, 2^{31} - 1]$. Python ints never overflow, so check yourself — interviewers often want the check a 32-bit language needs, **before** multiplying: `res > (INT_MAX - digit) // 10`.

```python
INT_MIN, INT_MAX = -2**31, 2**31 - 1

def reverse_int(x):
    sign = -1 if x < 0 else 1
    x, res = abs(x), 0
    while x:
        x, digit = divmod(x, 10)
        if res > (INT_MAX - digit) // 10:
            return 0
        res = res * 10 + digit
    res *= sign
    return res if INT_MIN <= res <= INT_MAX else 0
```

(`abs(x)` first avoids a trap: `-123 % 10` is 7 in Python, not −3 — `%` follows the divisor's sign, 19.19.)

Problem 3 — most points on one line. For each anchor, group the other points by slope through it; the biggest group plus the anchor and its exact duplicates is the best line through that anchor. Store slopes as **reduced fractions** `(dx // g, dy // g)` with `g = gcd(dx, dy)` and one fixed sign — floats can round two slopes together, and vertical lines divide by zero.

Problem 4 — Josephus. n people in a circle, every k-th leaves; who survives? Simulating with `list.pop` is $O(n^2)$. The recurrence $J(1) = 0$, $J(m) = (J(m-1) + k) \bmod m$ (0-based) is $O(n)$: after the first removal the circle is a size $m - 1$ problem shifted by k.

```python
def josephus(n, k):
    survivor = 0
    for m in range(2, n + 1):
        survivor = (survivor + k) % m
    return survivor
```

Output of `18_math_geometry.py` (checks `josephus` against simulation for all n < 30, k < 8):

```
spiral: [1, 2, 3, 4, 8, 12, 11, 10, 9, 5, 6, 7]
rotated: [[7, 4, 1], [8, 5, 2], [9, 6, 3]]
reverse 1534236469 -> 0 (overflow)
max collinear: 4
Josephus n=7, k=3 survivor index: 3
all asserts passed
```

Complexity

Problem	Time	Space
spiral / rotate	O(r·c)	O(1) extra (rotate in place)
reverse integer	O(number of digits)	O(1)
max points on a line	O(n²) (the gcd adds a log factor)	O(n) per anchor
Josephus	O(n)	O(1)

Common mistakes

- Spiral without the two `if` guards — a middle row/column printed twice on non-square matrices.
- Slopes as floats, or unnormalised signs — `(1, -2)` and `(-1, 2)` are the same direction.
- Forgetting the anchor's duplicates in "max points on a line".
- Relying on big ints when told "assume a 32-bit environment".

In the interview

> **30-second answer:** "Matrix walks: explicit boundaries that shrink; rotation is transpose plus reverse rows. Python ints don't overflow, so for 32-bit problems I check the range explicitly before multiplying, as C or Java would. Geometry: slopes as reduced integer fractions via `gcd`, never floats. A slow simulation → look for a recurrence, like Josephus's `(J + k) % m`."

- *Rotate counter-clockwise?* → Transpose, then reverse each column (or reverse rows first, then transpose).
- *Why does Josephus's recurrence work?* → After removing person k−1, renumber the remaining m−1 from the next one; that circle's survivor, shifted back by k (mod m), survives the larger one.

19.19 Common pitfalls

Each is demonstrated (bug, then fix) in `19_pitfalls.py`. Chapter 20 covers general gotchas in depth; these bite in timed coding.

```
1. [1, 2] [1, 2] <- shared list
   fixed: [1] [2]
2. [[0]*3]*2 -> [[9, 0, 0], [9, 0, 0]] | comprehension -> [[9, 0, 0], [0, 0, 0]]
3. shallow sees change: 99 | deep does not: 1
4. stored references: [[1, 2], [1, 2]] | use path[:] -> [[1], [1, 2]]
5. default limit: 1000
   depth(3000) -> RecursionError; use an explicit stack or raise the limit
6. 7 // 2 = 3 | -7 // 2 = -4 | int(-7 / 2) = -3 | -7 % 3 = 2
   7 / 2 = 3.5 (always float) | round(2.5) = 2 | round(3.5) = 4
7. 0.1 + 0.2 == 0.3: False | isclose: True
8. a == b: True | a is b: False | use `is` only for None/sentinels
9. remove while iterating -> [1, 2, 3] | comprehension -> [1, 3]
10. ''.join(parts) = 01234 (O(n); repeated += can be O(n^2))
11. xs.sort() returns None - xs is now [1, 2, 3] | sorted() returns a new list
12. list still callable: ['a', 'b'] [1, 2]
```

#	Pitfall	Why	Fix
1	Mutable default `def f(x, acc=[])`	defaults are evaluated **once** when `def` runs; every call shares the list	`acc=None`, then `if acc is None: acc = []`
2	`[[0] * n] * m`	the outer `*` repeats a **reference** to one row	`[[0] * n for _ in range(m)]`
3	Shallow copy of nested data	`a[:]`, `list(a)`, `a.copy()`, `copy.copy` copy only the outer list	`copy.deepcopy`, or `[row[:] for row in grid]` for 2-D
4	`res.append(path)` in backtracking	stores the same list object	`res.append(path[:])`
5	Deep recursion	limit 1000, no tail-call optimisation	iterative stack/deque; `sys.setrecursionlimit` as last resort
6	Integer division	`//` **floors** (toward −∞); `int(a / b)` truncates toward 0; `%` takes the sign of the divisor; `/` always returns float; `round` uses banker's rounding (ties to even)	choose deliberately; for "truncate toward zero" use `int(a / b)` (small ints) or sign-aware `//`
7	Float equality	binary floats can't represent 0.1 exactly	`math.isclose`, or stay in integers
8	`is` vs `==`	`is` compares identity	`==` for values; `is` only for `None` and sentinels
9	Mutating a list while iterating	the iterator's index skips elements	build a new list, or iterate over a copy
10	String `+=` in loops	may copy each time → O(n²)	collect parts, `"".join`
11	`nums = nums.sort()`	in-place methods return `None`	`nums.sort()` alone or `nums = sorted(nums)`
12	Shadowing built-ins (`list`, `dict`, `sum`, `max`, `input`, `id`)	later calls hit your variable	pick other names (`nums`, `total`, `list_`)

Also: `float("inf")` compares fine with ints but `int(float("inf"))` raises `OverflowError`; `@cache` on a helper with global state gives stale answers across test cases (19.10).

19.20 How to talk through a solution

What it is

Interviewers score **communication and process** as well as code; a predictable structure keeps you calm and shows your thinking.

How it works — the seven steps

1. **Clarify** (1–2 min): input sizes, ranges (negatives? duplicates? empty?), output format, may you modify the input? Size hints the target: $n \le 20$ → exponential is fine; $n \le 10^3$ → $O(n^2)$; $n \le 10^5$–10^6 → $O(n \log n)$ or $O(n)$.
2. **Examples:** one normal and one edge case by hand.
3. **Brute force:** state it and its complexity in one sentence — proof you understand, and a fallback.
4. **Optimise:** name the pattern ("sorted → two pointers", "k-th → heap"), explain *why* it's faster, agree before coding.
5. **Code:** clear names, small helpers, idioms (`enumerate`, `defaultdict`, unpacking); narrate briefly.
6. **Test:** trace your example, then edge cases — empty, single element, all equal, negatives, maximum sizes.
7. **Analyse:** time and space, and what changes for follow-ups (streaming, memory limits).



Phrases that help

- "Before I optimise, the brute force is $O(n^2)$ because we check every pair."
- "Since the array is sorted, I can discard half each step — binary search, $O(\log n)$."
- "I'll use a `deque` here rather than a list because `popleft` is $O(1)$."
- "Space is $O(n)$ for the hash map; if memory mattered, I could sort and use two pointers at $O(1)$ extra space but $O(n \log n)$ time."

Python-specific tips

- `Counter`, `heapq`, `bisect`, `itertools` are expected — but know each call's complexity and be ready to hand-write it ("without `Counter`?").
- Type hints in the signature (`def solve(nums: list[int]) -> int:`) read well; don't code-golf (squeeze into fewest characters) — a clear four-line loop beats a one-liner you can't explain.
- Know the platform's Python version; newer helpers (`itertools.pairwise`/`math.lcm` need 3.9–3.10, `itertools.batched` 3.12, `heapq.heappush_max` 3.14) may be missing.

Chapter TL;DR

Pattern	Reach for it when…	Python tool
Costs	always state them	list = array (O(1) end, O(n) front/search); dict/set O(1) avg; slices and `+` copy
Two pointers / window / prefix sums	sorted pairs; contiguous + monotonic rule; range sums or "sum = k"	indices, `accumulate`, dict of prefixes
Hashing	seen-before, counts, groups	`set`, `dict`, `Counter`, `defaultdict` (hashable keys)
Stack / queue / monotonic	brackets, "next greater", window max	`list`, `deque`
Heap	top-k, merge-k, two heaps (running median, capital), Dijkstra	`heapq` (negate for max, tie-breakers)
Sort / binary search	ordering, "first position", monotonic feasibility, weighted random	`key=` tuples, `bisect_left/right`, prefix sums
Lists / trees / graphs	pointers, traversals, shortest paths, dependencies	node classes, `deque` BFS, `heapq` Dijkstra, Kahn, DSU
DP / backtracking	overlapping subproblems; enumerate with pruning	`@cache` or tables; choose → explore → un-choose
Bits / math	masks, parity, number theory	`x & (x-1)`, `pow(b, e, m)`, `gcd`, `isqrt`, `comb`
Tries	prefix search, autocomplete, wildcard, many words on a grid	dict-of-children nodes + end-of-word field
Greedy	a locally best choice is provably safe (exchange argument)	one pass, often after `sorted(key=...)`
Intervals	merge, insert, overlap, rooms needed	sort by start (merge) or end (keep most); heap of ends; sweep line
Fast & slow pointers	cycle and its entry, middle, O(1)-space list checks, implicit sequences	`slow`, `fast`; compare with `is`
Sort & search	k-th element, partition in place, three values, sort a linked list	quickselect, Dutch flag, merge sort
Math & geometry	matrix walks, 32-bit limits, collinear points, circle elimination	boundaries, explicit range checks, `gcd` slopes, recurrences

Avoid the pitfalls of 19.19 (mutable defaults, `[[0]*n]*m`, shallow copies, the 1000-frame recursion limit, `//` flooring), and narrate: clarify → brute force → optimise → code → test → analyse.

Chapter 20 — Pythonic Idioms, Gotchas and Interview Q&A

Pythonic code uses the language as its designers intended: readable, short without being clever, leaning on built-in protocols (the special methods Python's syntax calls — iteration, context managers, unpacking) instead of re-implementing them. Interviewers notice at once — `for i in range(len(xs))` or `if x == None` marks someone who learned Python from another language. The flip side is a handful of **gotchas**: behaviours that surprise even experienced developers (mutable default arguments, late-binding closures, shallow copies).

This chapter pulls the book together for review: idioms, then gotchas (each with demo and fix), then about 45 classic interview questions with short answers pointing to the chapter that teaches each topic. "Bank Qn" points to the same question in the Chapter 22 question bank, which adds many more. Scripts are in `code/`, run with Python 3.14 (all 3.12-compatible); outputs are real.

What you'll learn

#	Topic	Script
20.1	Pythonic idioms: EAFP, unpacking, `enumerate` / `zip`, comprehensions, truthiness, f-strings, context managers and `with open`, `pathlib`, and smaller habits	`01_idioms.py`
20.2	13 gotchas, each demonstrated and fixed	`02_gotchas.py`
20.3	Interview Q&A: language basics, data structures, functions, OOP, internals and memory, concurrency, typing/tooling	`03_interview_qa.py`

> The Zen of Python (`import this`, PEP 20; PEPs — *Python Enhancement Proposals* — are the numbered documents defining the language and its conventions) sums up the taste: *explicit is better than implicit, simple is better than complex, readability counts, errors should never pass silently, there should be one obvious way to do it.* PEP 8 is the style guide (4-space indents, `snake_case` functions, `CapWords` classes, lines ≤ 79–99 chars); `ruff` (linter) and `black` (formatter) enforce it automatically.

20.1 Pythonic idioms

What it is

An **idiom** is the community's conventional way to express a common task. Python idioms usually replace manual bookkeeping (indexes, flags, cleanup calls) with a language feature.

Why it matters

Fewer lines → fewer bugs (no off-by-one on an index you didn't need); others read it faster; and many idioms are faster because the work runs inside C built-ins (`sum`, `join`, comprehensions).

The idioms at a glance

Instead of…	Write…	Section
`if key in d: v = d[key] else: v = default`	`v = d.get(key, default)` or `try/except KeyError`	EAFP
`tmp = a; a = b; b = tmp`	`a, b = b, a`	Unpacking
`for i in range(len(xs)): x = xs[i]`	`for i, x in enumerate(xs)`	enumerate
`for i in range(len(a)): a[i], b[i]`	`for x, y in zip(a, b)`	zip
`out = []` + loop + `append`	`[f(x) for x in xs if cond(x)]`	Comprehensions
`if len(xs) == 0` / `if flag == True`	`if not xs` / `if flag`	Truthiness
`"Hi " + name + ", you are " + str(age)`	`f"Hi {name}, you are {age}"`	f-strings
`f = open(p); ...; f.close()`	`with open(p) as f:`	Context managers
`os.path.join(d, "x.txt")`	`Path(d) / "x.txt"`	pathlib
`s = ""` + `s += piece` in a loop	`"".join(pieces)`	Strings
`x == None`	`x is None`	Identity
`found = False` flag + loop	`for ... else:` or `any(...)`	Loops

EAFP vs LBYL

LBYL ("look before you leap") checks first: `if "port" in config: ...`. **EAFP** ("easier to ask forgiveness than permission") tries and handles failure: `try: port = int(config["port"]) except (KeyError, ValueError): port = 80`. Python favours EAFP because:

1. It avoids **race conditions** — between "check the file exists" and "open it", another process can delete it; opening inside `try` has no gap.
2. One `try` covers several failure modes (missing key *and* bad number).
3. The happy path (normal, no-error flow) stays unindented.

Use LBYL when failure is common and cheap to check (exceptions would be slow and noisy) or the check has no side effects to race with. `contextlib.suppress(FileNotFoundError)` is a tidy EAFP shortcut for "ignore this error". Never a bare `except:` (gotcha 11).

Unpacking

Any iterable unpacks into names; `*name` collects "the rest" into a list; `**` unpacks mappings.

```
a, b = b, a                                # swap, no temp variable
first, *middle, last = [10, 20, 30, 40, 50]
(x, y), z = (1, 2), 3                      # nested
merged = {**point, "z": 5}                 # dict unpacking (or point | {"z": 5}, 3.9+)
both = [*range(3), *"ab"]
_, ext = "report.final.pdf".rsplit(".", 1)  # _ = "don't care"
```

```
swap: 2 1 | star: 10 [20, 30, 40] 50 | nested: 1 2 3
dict merge: {'x': 3, 'y': 4, 'z': 5} | point | {'z': 5} -> {'x': 3, 'y': 4, 'z': 5} | list: [0, 1, 2, 'a', 'b']
rsplit + _: pdf
```

Also in `for name, score in pairs:` and calls (`f(*args, **kwargs)`, Chapter 7).

`enumerate` and `zip`

`enumerate(xs, start=1)` yields `(index, item)`; `zip(a, b)` yields same-position tuples and **stops at the shortest** input — `strict=True` (3.10+) raises `ValueError` on unequal lengths, a cheap guard against silent data loss. `zip(*rows)` "unzips" (transposes).

```python
for rank, name in enumerate(names, start=1):
    print(f"  {rank}. {name}")
dict(zip(names, scores))
list(zip(names, [1, 2], strict=True))   # ValueError
nums, letters = zip(*[(1, "a"), (2, "b")])
```

```
  1. ann
  2. bob
  3. cy
zip: [('ann', 91), ('bob', 78), ('cy', 85)] | dict(zip): {'ann': 91, 'bob': 78, 'cy': 85}
zip(strict=True): zip() argument 2 is shorter than argument 1
unzip: (1, 2) ('a', 'b')
```

Comprehensions and generator expressions

A **comprehension** builds a list/dict/set in one expression: `[expr for x in it if cond]`, `{k: v for ...}`, `{x for ...}`. A **generator expression** `(expr for x in it)` is lazy — pass it straight to `sum`, `any`, `max`, `"".join` to avoid building a list.

```python
[n * n for n in range(6)]                          # [0, 1, 4, 9, 16, 25]
[n for n in range(10) if n % 2 == 0]               # filter at the end
["even" if n % 2 == 0 else "odd" for n in range(4)] # if-else goes BEFORE for
{w: len(w) for w in ["hi", "hello"]}
sum(n * n for n in range(1_000_000))               # no million-item list
[x for row in [[1, 2], [3]] for x in row]          # loops read left-to-right
```

One `for` and one condition is ideal, two `for`s the maximum; past a line or two, write a loop. Don't use a comprehension for side effects (`[print(x) for x in xs]` builds a useless list).

Truthiness

Falsy: `None`, `False`, numeric zeros (`0`, `0.0`, `0j`), empty containers (`""`, `[]`, `()`, `{}`, `set()`, `range(0)`); everything else is truthy unless its class defines `__bool__`/`__len__` otherwise. So `if items:`, `if not name:`. Trap: `if count:` is also false for `0` — when `0`/`""` is valid and you mean "was it provided?", write `if count is not None:`. `and`/`or` return one of their **operands**, not a bool: `name or "anonymous"` defaults any falsy name.

f-strings

f-strings (3.6+) evaluate `{}` expressions and apply the **format mini-language** (width, alignment, number style) after a colon:

```python
print(f"{name:<8}|{price:>12,.2f}|{ratio:.1%}|{42:05d}|{255:#x}|{255:b}")
print(f"{price=}")          # 3.8+: prints the expression and its value
```

```
Widget  |    1,234.57|45.7%|00042|0xff|11111111
price=1234.5678
```

`<`/`>`/`^` align, the number is the width, `,` thousands separators, `.2f` decimals, `%` ×100, `x`/`b` hex/binary; `!r` uses `repr()`. For logging pass arguments lazily (`log.info("user %s", uid)`) so formatting happens only if emitted. Full detail: Chapter 4.

Context managers and `with open`

A **context manager**'s `__enter__` runs at the start of a `with` block and `__exit__` at the end — **even on an exception** (Chapter 12) — so cleanup of files, locks, transactions, temp dirs can't be forgotten.

```python
with open(path, "w", encoding="utf-8") as f:   # closed even on error
    f.write("line one\nline two\n")
with open(path, encoding="utf-8") as f:
    for line_no, line in enumerate(f, 1):      # stream: one line in memory at a time
        print(line_no, line.rstrip())

@contextmanager
def timer(label):
    start = time.perf_counter()
    try:
        yield                                   # the with-block body runs here
    finally:
        print(f"{label}: done")
```

Always pass `encoding="utf-8"` for text: the default comes from the platform's locale (e.g. often cp1252 on Windows) — "works on my machine" bugs. One `with` can hold several managers: `with open(a) as src, open(b, "w") as dst:`.

`pathlib`

`pathlib.Path` (Chapter 9): paths as objects, `/` joins, methods replace scattered `os.path` functions.

```python
base = Path(tmp)
notes = base / "notes" / "day1.txt"
notes.parent.mkdir(parents=True, exist_ok=True)
(base / "b.csv").write_text("a,b\n1,2\n", encoding="utf-8")
notes.name, notes.stem, notes.suffix     # 'day1.txt', 'day1', '.txt'
[p.name for p in base.glob("*.csv")]     # ['b.csv']
[p.relative_to(base).as_posix() for p in base.rglob("*.txt")]
```

```
name/stem/suffix: day1.txt day1 .txt
glob *.csv: ['b.csv']
rglob *.txt: ['notes/day1.txt']
exists: True | read_text: 'line one'
```

Smaller habits (all in `01_idioms.py`)

- **Chained comparisons:** `0 < x < 10`.
- **Conditional expression:** `"big" if x > 3 else "small"`.
- **`any` / `all` with generators** instead of flag loops; `max(xs, key=len, default=None)` for empty-safe maxima.
- **`for … else`:** `else` runs only if the loop didn't `break` — the "search failed" branch.
- **Walrus `:=`** (3.8+) assigns inside an expression: `if (m := len(s)) > 3:` — handy in `while` loops reading chunks.
- **`defaultdict` / `Counter`** instead of "if key not in d: d[key] = …".
- **`"sep".join(strings)`** to build strings.
- **`dataclass`** instead of hand-written `__init__` / `__repr__` / `__eq__` (Chapter 10).
- **`if __name__ == "__main__":`** so importing doesn't run script code (Chapter 8).

Common mistakes

- Comprehensions nested three deep — readable beats short.
- `except Exception: pass` — EAFP means *handling* a specific failure, not hiding all failures.
- Truthiness tests where `0` / `""` are valid.

In the interview

30-second answer: "Pythonic code uses the language's protocols: iterate directly with `enumerate` / `zip`, build collections with comprehensions, use truthiness, `with` for any resource, f-strings for formatting, `pathlib` for paths, and EAFP — try the operation and catch the specific exception, which also avoids check-then-use races."

- *EAFP or LBYL before opening a file?* → EAFP: `try: open(...) except FileNotFoundError:` — the file could vanish between check and open.
- *List comprehension or generator expression?* → Generator when consumed once (e.g. `sum`), especially for large data; list when you index, reuse or `len()` it.
- *`zip([1, 2, 3], "ab")`?* → `[(1, 'a'), (2, 'b')]` — stops at the shortest.

20.2 Top gotchas — demo and fix

Each block is printed by `02_gotchas.py`.

1. Mutable default arguments

```
def add_item_bad(item, bucket=[]):
    bucket.append(item)
    return bucket
```

```
bad : ['a', 'b'] ['a', 'b'] | defaults: (['a', 'b'],)
good: ['a'] ['b']
```

Why: defaults are evaluated **once**, when `def` runs, and stored on the function (`__defaults__`); every call omitting `bucket` shares one list. **Fix:** default to `None`, create inside: `if bucket is None: bucket = []` (same for `{}`, `set()`, any mutable). Linters flag it (ruff `B006`). Bank Q71, Q236.

2. Late-binding closures

```
fns = [lambda: i for i in range(3)]
[f() for f in fns]                          # [2, 2, 2]
fns = [lambda i=i: i for i in range(3)]     # [0, 1, 2]
```

Why: a closure references the *variable* `i`, looked up when **called** — after the loop, when it's 2. **Fix:** bind the current value with a default (`i=i`), `functools.partial(f, i)`, or a factory `def make(i): return lambda: i`. Bank Q81, Q237.

3. `is` vs `==`

```
a == b: True | a is b: False
x is None: True (the right way to test for None)
```

Why: `==` calls `__eq__` and compares **values**; `is` compares **identity** (`id(a) == id(b)`). **Fix:** `==` for values; `is` only for singletons — `None`, rarely `True`/`False`, and your own sentinels (unique "nothing was passed" markers, e.g. `_MISSING = object()`).

4. Integer caching

```
256 is 256: True | 257 is 257: False | 257 == 257: True
```

Why: CPython pre-creates −5…256 and reuses them; bigger runtime ints are separate objects. (Equal literals in one compiled block may be shared, so `a = 257; b = 257; a is b` can print `True` — an implementation detail either way.) Some strings are **interned** too (§17.3). **Fix:** never `is` for numbers or strings. Bank Q44, Q188.

5. Modifying a list while iterating

```
bad : [1, 2, 3]
good: [1, 3] | or iterate a copy: for n in nums[:]
dict: dictionary changed size during iteration
dict good: {'b': 2}
```

Why: the list iterator walks by index; removing shifts the rest left, so the next item is skipped (the second `2` survived). Dicts and sets detect size changes and raise `RuntimeError`. **Fix:** build a new collection, iterate a copy (`nums[:]`, `list(d)`), or collect keys and delete afterwards. Bank Q245.

6. Shallow copy

```
original after shallow edit: {'name': 'svc', 'ports': [80, 443]} | deep copy: {'name': 'svc', 'ports': [80]}
alias is config: True
```

Why: assignment copies nothing; a **shallow copy** (`d.copy()`, `list(x)`, `x[:]`, `copy.copy`) is a new outer container sharing inner objects; a **deep copy** (`copy.deepcopy`) copies recursively (memo dict handles cycles). **Fix:** `deepcopy` for nested data you'll change independently — or design with immutables (tuples, frozen dataclasses). For a 2-D list, `[row[:] for row in grid]`.

7. Shared mutable values: `dict.fromkeys` and `defaultdict` reads

```
fromkeys bad : {'a': [1], 'b': [1]}
fromkeys good: {'a': [1], 'b': []}
defaultdict after a read: {'missing': []} | use `'missing' in dd` or dd.get()
```

Why: `dict.fromkeys(keys, [])` stores the **same** list under every key (like `[[]] * n`); a `defaultdict` inserts a default on *any* `d[key]`, even a read, so `if dd["x"]:` grows the dict. **Fix:** `{k: [] for k in keys}`; test with `in` or `dd.get(key)` (doesn't insert). Bank Q55.

8. Class attributes vs instance attributes

```
bad : sam.tricks = ['roll']
good: sam.tricks = []
assignment shadows: wolf | canis familiaris | canis familiaris
```

Why: class-body attributes are shared by all instances; mutating one through an instance (`r.tricks.append`) changes it for everyone, while *assigning* (`r.species = "wolf"`) creates an instance attribute shadowing the class one for `r` only. **Fix:** per-instance mutable state in `__init__` (`self.tricks = []`); class attributes for constants and genuinely shared data. `dataclass` rejects `tricks: list = []` and requires `field(default_factory=list)`. Bank Q91, Q246.

9. Circular imports

```
bad : ImportError: cannot import name 'helper_a' from 'mod_a'
good: ba
```

Why: importing runs a module top to bottom. `mod_a` starts, hits `from mod_b import helper_b`, so `mod_b` runs — and its `from mod_a import helper_a` finds `mod_a` only *partially initialised*, without that name yet. (Message wording varies by version.) **Fixes, best first:**

1. Move the shared code into a third module both import (usually the real fix — a cycle signals muddled responsibilities).
2. `import mod_a` and use `mod_a.helper_a` at call time (the "good" version).
3. Import inside the function that needs it.
4. Type-hint-only imports: `from typing import TYPE_CHECKING` + `if TYPE_CHECKING: import ...`.

Bank Q169.

10. Floating point

```
0.1 + 0.2 = 0.30000000000000004 | == 0.3: False
math.isclose: True
Decimal('0.1') + Decimal('0.2') = 0.3 | Decimal(0.1) = 0.1000000000000000555...
Fraction(1, 10) + Fraction(2, 10) = 3/10
round(2.675, 2) = 2.67 | round(0.5), round(1.5) = 0 2
float('nan') == float('nan'): False
```

Why: `float` is IEEE-754 binary64 (the standard 64-bit binary format every modern CPU uses); most decimal fractions (0.1, 2.675) aren't exactly representable, so the nearest value is stored. `round` is **round-half-to-even** ("banker's rounding"), and `2.675` is really `2.67499999…`. `NaN` is unequal to everything, itself included. **Fix:** `math.isclose` (or an explicit tolerance); `decimal.Decimal` **from strings** for money; `fractions.Fraction` for exact rationals (one integer over another, like 1/3); integer cents for currency; `math.isnan` for NaN. Bank Q27, Q239.

11. Bare `except:`

```
bad : bare except swallowed KeyboardInterrupt
good: `except Exception` let KeyboardInterrupt through
```

Why: bare `except:` catches `BaseException`, including `KeyboardInterrupt` (Ctrl+C) and `SystemExit` (`sys.exit()`) — the program can't be stopped and real bugs vanish. **Fix:** catch the narrowest exception you can handle (`except KeyError:`); at the top level `except Exception:` and **log** (`logger.exception(...)`); re-raise with `raise` if you can't handle it. Bank Q152.

12. Shadowing built-ins

```
bad : 'list' object is not callable
good: ['a', 'b', 'c']
```

Why: lookup is Local → Enclosing → Global → **Built-in** (LEGB, Chapter 7), so a variable named `list`, `dict`, `str`, `id`, `input`, `sum`, `max`, `type`, `file` or `len` hides the built-in for that scope. **Fix:** descriptive names (`items`, `user_id`) or a trailing underscore (`type_`); ruff `A001` catches it. Don't name *files* after stdlib modules (`random.py`, `json.py`) either — they shadow the real module.

13. Bonus: return values that surprise

- `list.sort()`, `list.append()`, `random.shuffle()`, `dict.update()` modify in place and return `None` — `xs = xs.sort()` loses your list.
- `and` / `or` return an operand: `"" or "default"` → `'default'`, `0 and 1` → `0`.

Bank Q253.

Gotcha summary

Gotcha	Rule to remember
Mutable default	Defaults are evaluated once → use `None`
Late binding	Closures see variables, not values → `i=i`
`is` vs `==`	`is` only for `None`/sentinels
Int caching / interning	Implementation detail — never rely on it
Mutating while iterating	Build a new collection or iterate a copy
Shallow copy	Copies the container, shares the contents
`fromkeys` / `defaultdict`	Shared default object; reads insert
Class vs instance attrs	Mutable state goes in `__init__`
Circular imports	Restructure, or import the module not the name
Floats	`isclose`, `Decimal("…")`, `Fraction`, ints of cents
Bare `except`	Catch specific exceptions; log the rest
Shadowing built-ins	Don't name things `list`, `id`, `sum` …

In the interview

> **30-second answer:** "The classic ones: mutable default arguments are evaluated once, so use `None`; closures bind late, so capture loop variables with a default; `is` is identity, not equality; copying is shallow unless you `deepcopy`; don't mutate a collection while iterating it; floats are binary, so use `isclose` or `Decimal`; and never a bare `except`."

- *Why evaluate defaults once?* → They're part of the function object, created when `def` runs; re-evaluating per call would be slower and change the meaning of defaults referring to outer names.
- `a = [[0] * 2] * 2; a[0][0] = 1` → `a`*?* → `[[1, 0], [1, 0]]`.
- *What does bare* `except:` *catch that* `except Exception:` *doesn't?* → `KeyboardInterrupt`, `SystemExit`, `GeneratorExit` — `BaseException` subclasses outside `Exception`.

20.3 Python interview Q&A

Short answers to say in 20–40 seconds. "→ Ch N" is where the topic is taught in full; "Bank Qn" is the same question in Chapter 22. Demos come from `03_interview_qa.py`.

A. Language basics

Q1. Is Python compiled or interpreted? Both. CPython **compiles** source to **bytecode** (instructions for Python's own virtual machine, cached as `.pyc` in `__pycache__`), then its VM **interprets** it. No ahead-of-time compilation to machine code by default (3.13+ ships an experimental JIT — just-in-time compiler — off by default). `dis` shows the bytecode:

```
== bytecode: CPython compiles to bytecode, then interprets it
167           RESUME                   0
              LOAD_FAST_BORROW         0 (x)
              LOAD_SMALL_INT           1
              BINARY_OP                0 (+)
              RETURN_VALUE
```

(Opcodes change between versions.) Other implementations: **PyPy** (tracing JIT, often much faster for pure-Python loops), MicroPython (microcontrollers), historically Jython/IronPython. → Ch 1, 17 · Bank Q2

Q2. Python is dynamically and strongly typed — what does that mean? *Dynamic*: types belong to objects, not variables; names can be rebound to any type, checked at runtime. *Strong*: no silent conversion between unrelated types — `"1" + 1` raises `TypeError` (JavaScript gives `"11"`). → Ch 2 · Bank Q6

Q3. Mutable vs immutable types? Immutable: `int`, `float`, `bool`, `str`, `tuple`, `frozenset`, `bytes`. Mutable: `list`, `dict`, `set`, `bytearray`, most user classes. "Changing" an immutable (`s += "x"`) creates a new object and rebinds. Only hashable values can be dict keys/set members — a tuple *containing* a list is immutable itself, but its list can change, so it isn't hashable:

```
== mutable vs immutable, list vs tuple
   tuple holding a list: (1, [2, 3, 4])
   hash(tuple with list): unhashable type: 'list'
   sizes: list 88 bytes | tuple 72 bytes
```

→ Ch 2, 5 · Bank Q21

Q4. How are arguments passed — by value or by reference? By object reference ("call by sharing"): the parameter is a new name bound to the *same object*. Mutating it is visible to the caller; rebinding the parameter is not.

```
== pass by object reference
   after mutate: ['orig', 'changed']
```

(`lst.append` was visible; `lst = ["rebound"]` was not.) → Ch 7 · Bank Q70

Q5. `is` vs `==`? `==` compares values via `__eq__`; `is` compares identity. `is` only for `None` and sentinels (gotchas 3–4). → Ch 2 · Bank Q23

Q6. What does `if __name__ == "__main__":` do? `__name__` is `"__main__"` when the file runs directly, the import name when imported. The guard makes a file both an importable module and a script — and stops multiprocessing children (which re-import the main module under `spawn`) re-running your program. → Ch 8 · Bank Q166

Q7. Module vs package; what is `__init__.py`? A **module** is one `.py` file; a **package** a directory of modules. A *regular* package has `__init__.py` (run on first import, often exposing the public API); without it you get a *namespace package* (can span several directories). Imports are cached in `sys.modules`, so a module body runs once per process. → Ch 8 · Bank Q165

Q8. What is a virtual environment and why use one? An isolated directory with its own interpreter link and `site-packages` (where third-party packages install), made with `python -m venv .venv`. Each project pins its own dependency versions without conflicts or touching the system Python. Tools: `pip` + `requirements.txt`, or `uv` / `poetry` with `pyproject.toml` and lock files (exact versions of every package, for reproducible installs). → Ch 8 · Bank Q171

Q9. What's new that interviewers like to hear about? `match` (structural pattern matching — branching on data's *shape*, 3.10), `X | Y` unions (3.10), exception groups and `except*` (3.11), "Faster CPython" speed-ups (3.11+), type-parameter syntax `def f[T](x: T)` (3.12), the optional free-threaded (no-GIL) build (experimental 3.13, officially supported 3.14), and template strings (t-strings, 3.14 — like f-strings but producing a template object code can inspect and process safely before rendering). → Ch 1, 3, 13, 16 · Bank Q17

B. Data structures

Q10. List vs tuple?

	list	**tuple**
Mutable	yes	no
Hashable (dict key)	no	yes, if its items are
Typical meaning	many items of one kind (a collection)	a fixed record of positions (`(lat, lon)`)
Memory / speed	over-allocates to allow growth	smaller and slightly faster to create
Tuples also signal "this won't change". → Ch 5 · Bank Q45		

Q11. How is a list implemented? A dynamic array of pointers. Index O(1); `append` amortised O(1) (over-allocates, occasionally grows and copies); insert/delete at the front O(n). → Ch 5, 19 · Bank Q46

Q12. How does a dict work internally? A **hash table**: `hash(key)` picks a slot; collisions are resolved by **open addressing** (probing other slots in a pseudo-random sequence). Since CPython 3.6 the layout is *compact* — a small index table into a dense entry array — saving memory and preserving **insertion order**, guaranteed since 3.7. It resizes at about two-thirds full, keeping O(1) average (O(n) worst with many collisions). Keys must be hashable; equal objects must have equal hashes. → Ch 6 · Bank Q51

Q13. What makes an object hashable? What if I define `__eq__`? A `__hash__` returning an int that never changes over its lifetime, consistent with `__eq__` (equal ⇒ same hash). Defining `__eq__` without `__hash__` sets `__hash__ = None` → unhashable. `@dataclass(frozen=True)` generates both. → Ch 6, 10 · Bank Q52, Q102

Q14. Set vs list for membership? Set: O(1) average `in`, no duplicates, unordered. List: O(n) scan, keeps order and duplicates. Convert to a set for repeated membership tests. → Ch 6, 19 · Bank Q58

Q15. Shallow vs deep copy? Shallow copies the container, sharing elements; deep copies recursively.

```
== copy vs deepcopy
   shallow: {'tags': ['a', 'b']} | deep: {'tags': ['a']}
```

Customise with `__copy__` / `__deepcopy__`. → Ch 5, gotcha 6 · Bank Q60

Q16. `array` vs `list` vs NumPy array? `list` holds pointers to arbitrary objects; `array.array` stores raw C numbers of one type compactly; a **NumPy** `ndarray` is a typed contiguous block *with* C-implemented vectorised operations — orders of magnitude faster for numeric work. → Ch 5, 18 · Bank Q66

Q17. What's in `collections`? `Counter` (counting), `defaultdict` (auto-defaults), `deque` (O(1) both ends), `namedtuple` (tuple with named fields), `OrderedDict` (order-aware equality, `move_to_end` — handy for LRU caches), `ChainMap` (layered lookups). → Ch 6 · Bank Q61

C. Functions

Q18. What are `*args` and `**kwargs`? In a definition `*args` collects extra positional arguments into a **tuple**, `**kwargs` extra keyword arguments into a **dict** (names are convention). At a call, `*` / `**` **unpack** an iterable/mapping. Signature order: positional-only (before `/`, can't be passed by name), normal, `*args`, keyword-only (must be `name=value`), `**kwargs`.

```
== *args / **kwargs
   show(1, 2, a=3) -> ((1, 2), {'a': 3}) | show(*[1, 2], **{'b': 4}) -> ((1, 2), {'b': 4})
```

Typical use: wrappers/decorators forwarding arguments unchanged. → Ch 7 · Bank Q69

Q19. What is a closure? A function that remembers variables from its enclosing scope after that function returned. They live in *cells* (`f.__closure__`); `nonlocal` rebinds them. Closures bind late (gotcha 2). → Ch 7 · Bank Q80

Q20. Explain LEGB and `global` / `nonlocal`. Lookup order: **L**ocal, **E**nclosing functions, **G**lobal (module), **B**uilt-ins. Assigning anywhere in a function makes the name local for the whole function (hence `UnboundLocalError` if read first). `global x` rebinds the module variable; `nonlocal x` the nearest enclosing function's. → Ch 7 · Bank Q77

Q21. What is a decorator? Write one with arguments. A callable taking a function and returning a replacement; `@d` above `def f` means `f = d(f)`. With arguments, one more layer: `@retry(3)` calls `retry(3)`, which returns the real decorator. Always `functools.wraps`, keeping `__name__`, `__doc__`, `__wrapped__`.

```python
def retry(times):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except ValueError:
                    print(f"   attempt {attempt} failed")
            raise RuntimeError("gave up")
        return wrapper
    return decorator
```

```
== decorators
   attempt 1 failed
   attempt 2 failed
    ok | name kept by wraps: flaky | Fails twice, then works.
```

Real uses: `@app.get("/")` (FastAPI/Flask routing), `@functools.cache`, `@pytest.fixture`, `@login_required` (Django). → Ch 12 · Bank Q135, Q138, Q274

Q22. Iterator vs iterable vs generator? An **iterable** has `__iter__` returning an iterator (lists, dicts, files). An **iterator** has `__next__` (next item or `StopIteration`) and `__iter__` returning itself; single-pass. A **generator** is an iterator from a `yield` function (or generator expression), its state saved between `next` calls. Every generator is an iterator, not vice versa.

```
== generators vs iterators
   iter(gen) is gen: True | values: [3, 2, 1] | second pass: []
   list 85176 bytes vs generator 208 bytes
```

The second pass is empty — iterators exhaust; the generator's size doesn't depend on how many items it yields. → Ch 11 · Bank Q121

Q23. `yield` vs `return`; what is `yield from`? `return` ends the function with one value; `yield` produces a value and **pauses** until the next `next()`. `yield from sub` delegates to a sub-iterator (re-yielding its values, passing `send()`/`throw()` through). `send(value)` pushes values in — the historical basis of coroutines (`async`). → Ch 11 · Bank Q123

Q24. What is a lambda and when shouldn't you use it? An anonymous single-expression function, `lambda x: x * 2` — good for short `key=` arguments. Don't assign one to a name (`def` gives a proper traceback name) or cram logic into it. → Ch 7 · Bank Q74

Q25. `map`/`filter` vs comprehensions? Both lazy in Python 3 (`map`/`filter` return iterators). Comprehensions usually read better, especially when a `lambda` would be needed; `map(str, xs)` with an existing function is fine. → Ch 11 · Bank Q76

D. Object-oriented programming

Q26. `__init__` vs `__new__`? `__new__(cls, …)` is a static method that **creates** and returns the instance; `__init__(self, …)` **initialises** it and returns `None`. Override `__new__` only for immutable subclasses (`int`, `str`, `tuple`), singletons/instance caching, or metaclass work (Q35). `__init__` still runs on every call:

```
== __new__ vs __init__
  __new__: creating the object
  __init__: setting value=1
  __init__: setting value=2
  same object: True | value: 2
```

→ Ch 10 · Bank Q90

Q27. `@staticmethod` vs `@classmethod` vs instance method?

Kind	First parameter	Can access	Typical use
Instance method	`self`	instance and class	normal behaviour
`@classmethod`	`cls`	the class (and subclasses correctly)	alternative constructors (`dict.fromkeys`, `datetime.fromtimestamp`, `from_fahrenheit`)
`@staticmethod`	none	nothing implicit	a helper that logically belongs to the class

```
== classmethod / staticmethod / property / __slots__
   Temperature(100.0) | is_freezing(-3): True
```

→ Ch 10 · Bank Q92

Q28. What does `@property` do? Makes a method a computed attribute with optional setter/deleter — start with a plain attribute, add validation later without changing callers (`t.celsius = -300` → `ValueError: below absolute zero`). No need for Java-style getters/setters. → Ch 10 · Bank Q93

Q29. What is the MRO and how does `super()` work? The **Method Resolution Order** — the order classes are searched for an attribute — comes from **C3 linearisation** (merging all parent orders consistently): children before parents, base order left-to-right preserved. `super()` means "the next class in the MRO of the *instance's* class", not "my parent", so cooperative multiple inheritance (each parent calls `super()`; "diamonds" share a grandparent) calls each class once:

```
== MRO / super()
  D.__mro__: ['D', 'B', 'C', 'A', 'object'] | D().hello(): D>B>C>A
```

`B`'s `super()` went to `C`, not `A`, because the instance is a `D`. → Ch 10 · Bank Q97

Q30. `__str__` vs `__repr__`? `__repr__` is for developers — unambiguous, ideally code that recreates the object (`Temperature(100.0)`); used by the REPL, debuggers and inside containers. `__str__` is for users

(`print`, `str()`, f-strings), falling back to `__repr__`. Define `__repr__` first. → Ch 10 · Bank Q99

Q31. What are dunder (magic) methods? `__x__` methods Python calls for syntax and built-ins — the **data model**: `len(x)` → `__len__`, `a + b` → `__add__`, `x[i]` → `__getitem__`, `for` → `__iter__`, `with` → `__enter__` / `__exit__`, `==` → `__eq__`, `bool(x)` → `__bool__`, calling → `__call__`. They make classes behave like built-ins. → Ch 10 · Bank Q100

Q32. What does `__slots__` do? Declares fixed attribute names stored in a compact array instead of a per-instance `__dict__` — less memory per object (millions of small objects) and slightly faster access, but no new attributes:

```
  __slots__ blocks new attributes: 'Temperature' object has no attribute 'colour' and no __dict__ for setting new attributes
```

`@dataclass(slots=True)` (3.10+) generates it. → Ch 10, 17 · Bank Q111

Q33. Abstract base classes vs duck typing vs `Protocol`? Duck typing: if it has the method, it works — no inheritance. An **ABC** (`abc.ABC` + `@abstractmethod`) enforces at instantiation that subclasses implement methods (nominal: must inherit). A **`typing.Protocol`** describes required methods structurally, checked by static type checkers without inheritance — "duck typing mypy can verify" (`Square` satisfies `HasArea` without inheriting it). → Ch 10, 13 · Bank Q107, Q108

Q34. What is a dataclass? `@dataclass` generates `__init__`, `__repr__`, `__eq__` from annotated fields; options add ordering (`order=True`), immutability + hashing (`frozen=True`), `slots=True`, and `field(default_factory=list)` for mutable defaults. Plain data records; Pydantic when you need runtime validation/parsing. → Ch 10 · Bank Q109

Q35. What are descriptors and metaclasses (briefly)? A **descriptor** is a class attribute with `__get__` / `__set__` / `__delete__` that routes instance attribute access; `property`, methods (binding `self` via `__get__`), `classmethod`, `staticmethod` are descriptors. A **metaclass** is the class of a class (default `type`), controlling class *creation*; ORMs (Django models), `abc.ABCMeta` and `enum.EnumMeta` / `EnumType` use them. A class decorator or `__init_subclass__` is usually simpler. → Ch 12 · Bank Q118, Q119

E. Errors, resources, memory and internals

Q36. Explain `try / except / else / finally` and `raise … from`. `except` handles matching exceptions; `else` runs only if `try` raised nothing (put code that shouldn't be protected there); `finally` always runs (cleanup). `raise NewError(...) from err` chains, storing the cause in `__cause__` so the traceback shows both. Custom exceptions subclass `Exception`. → Ch 9 · Bank Q151

Q37. How does a context manager work? `with obj as x:` calls `obj.__enter__()` (return value bound to `x`), runs the block, then always `obj.__exit__(exc_type, exc, tb)`; a true return suppresses the exception. `contextlib.contextmanager` builds one from a generator: before `yield` is enter, after it (in `finally`) is exit. → Ch 12 · Bank Q143

Q38. How does Python manage memory?

1. **Reference counting**: freed **immediately** at count zero (deterministic cleanup).
2. A generational **cyclic garbage collector** (`gc`) periodically frees groups of objects that only reference each other.
3. Small objects come from **pymalloc**, CPython's pooling allocator, avoiding many small `malloc` (C's general allocator) calls.

```
== memory: reference counting + cycle collector
   refcount (includes getrefcount's own arg): 2
   freed immediately by refcounting: True
   cycle alive before gc.collect(): True
   cycle freed after gc.collect(): True
```

"Leaks" are usually lingering references: growing global caches/lists, unbounded `lru_cache`s, closures holding big objects. Diagnose with `tracemalloc`. → Ch 17 · Bank Q182, Q185

Q39. What is pickling? Is it safe? `pickle` **serialises** objects to bytes (`dumps` / `dump`) and back (`loads` / `load`); `multiprocessing` uses it between processes, and scikit-learn models are saved with it

(joblib builds on it). Functions and classes pickle *by reference* (module + name), so lambdas and local functions can't:

```
== pickling
   round trip: {'model': 'v1', 'weights': [0.1, 0.2]} | 61 bytes
   pickle a lambda: PicklingError
```

Never unpickle untrusted data — loading can execute arbitrary code. For exchange use JSON (or Parquet / `safetensors` — a safe model-weights format — for arrays). → Ch 9 · Bank Q162

Q40. How do you find and fix slow Python code? Measure first: `cProfile` / `pstats` (which functions), `timeit` (micro-benchmarks), `tracemalloc` (memory). Then: better algorithms/data structures (sets for membership, deque for queues), built-ins or NumPy vectorisation, caching (`functools.cache`), fewer attribute lookups in hot loops — and only then PyPy, Cython/Rust extensions, or processes. → Ch 17 · Bank Q192

F. Concurrency

Q41. What is the GIL? The **Global Interpreter Lock** is a mutex in CPython letting only one thread execute bytecode at a time, keeping reference counting and interpreter internals simple and fast single-threaded. Consequences:

- **CPU-bound** pure Python does **not** speed up with threads — use `multiprocessing` / `ProcessPoolExecutor` (one GIL each) or C extensions that release the GIL (NumPy, for many operations).
- **I/O-bound** code *does* benefit — the GIL is released while waiting.
- It doesn't make your code thread-safe: `counter += 1` is several bytecodes and can race — use `threading.Lock`.

```
== GIL: threads vs processes on CPU-bound work
   GIL enabled: True
   CPU-bound x4  serial 0.95s | 4 threads 1.02s | 4 processes 0.56s
```

(Machine-dependent; the pattern matters.) Processes helped, less than 4× due to start-up and data transfer. PEP 703's **free-threaded** build (`python3.13t` / `3.14t`) removes the GIL; it's optional, the default build keeps it. → Ch 16 · Bank Q197

Q42. Threads vs processes vs asyncio — which when?

Workload	Choose	Why
CPU-bound (math, parsing, image processing)	`multiprocessing` / `ProcessPoolExecutor`	true parallelism, one GIL per process
I/O-bound, moderate concurrency, blocking libraries	`threading` / `ThreadPoolExecutor`	GIL released during I/O; simple
I/O-bound, thousands of connections, async libraries	`asyncio`	one thread, very cheap tasks
Numeric arrays	NumPy/vectorisation	C loops, often release the GIL
→ Ch 16 · Bank Q201		

Q43. How does `asyncio` work? A single-threaded **event loop** runs **coroutines** (`async def`). At each `await` on something not ready, the coroutine yields to the loop, which runs another ready task — **cooperative multitasking** (tasks hand over control voluntarily). `asyncio.gather` runs many concurrently:

```
== asyncio: many waits overlap on one thread
   10 x 0.2s waits concurrently -> 0.20s, ['req0', 'req1', 'req2']...
```

Ten 0.2 s waits took 0.2 s total. Pitfall: a blocking call (`time.sleep`, `requests.get`, heavy CPU) in a coroutine freezes the loop — use async libraries (`httpx`, `aiohttp`) or `await asyncio.to_thread(func)`. → Ch 16 · Bank Q211

G. Typing and tooling

Q44. Are type hints enforced? No — stored in `__annotations__`, ignored at runtime:

```
== typing is not enforced at runtime; Protocol = structural typing
   double('ab') runs anyway: abab
   total_area: 13
```

Static checkers (**mypy**, **pyright**) and IDEs use them before running; **Pydantic** and **FastAPI** read them to validate at runtime. Vocabulary: `list[int]`, `dict[str, float]`, `X | None` (= `Optional[X]`), `Callable[[int], str]`, `TypeVar`/generics (`def first[T](xs: list[T]) -> T`, 3.12), `Protocol`, `TypedDict`, `Literal`, `Any`. → Ch 13 · Bank Q174

Q45. How do you test Python code? `pytest`, the de-facto standard: plain `assert`, fixtures for setup, `@pytest.mark.parametrize` for many cases; `unittest` is the stdlib xUnit style (class-based, like JUnit). `unittest.mock` (`patch`, `Mock`) replaces external dependencies, `coverage` measures what ran, `hypothesis` does property-based tests (state a rule; the library generates inputs to break it). → Ch 15 · Bank Q226

Q46. What is monkey patching? Replacing a module or class attribute at runtime (`module.func = fake`). Fine in tests — `unittest.mock.patch("pkg.mod.name")` does it safely and restores the original — but dangerous in production: global and invisible. Patch where the name is **looked up**, not where defined. → Ch 12, 15 · Bank Q147, Q229

Rapid-fire round

Question	One-line answer
`range` in Python 3?	A lazy, immutable sequence; O(1) `len` and `in` for ints.
Are strings mutable?	No; build with `join`, or `io.StringIO`.
What does `pass` do?	Nothing — a placeholder where a statement is required.
`append` vs `extend`?	`append(x)` adds one item; `extend(it)` adds each item of an iterable.
`sorted` vs `.sort()`?	New list vs in-place (returns `None`); both stable Timsort.
`del` vs `remove` vs `pop`?	By index/slice/name; by value (first match); by index returning the item.
What is `None`?	The single instance of `NoneType`, meaning "no value"; test with `is None`.
Why `self`?	Methods receive the instance explicitly; `obj.m()` is `Class.m(obj)`.
What is `__call__`?	Makes instances callable like functions.
What is a namedtuple?	A tuple subclass with named fields: lightweight immutable records.
`@functools.cache` vs `lru_cache`?	Unbounded vs size-limited memoisation.
`copy.copy` on a tuple of ints?	Returns the same tuple (immutable — no need to copy).
What is `__pycache__`?	Cached bytecode (`.pyc`) to skip recompiling unchanged modules.
What is `*` alone in a signature?	Everything after it is keyword-only.
What does `/` in a signature mean?	Everything before it is positional-only.
`match` statement?	Structural pattern matching (3.10): destructures sequences, mappings, classes.
What does `enumerate` return?	A lazy iterator of `(index, item)` tuples.

How to answer well

1. **Lead with a one-sentence definition**, then the mechanism, then a use case or trade-off.
2. **Show you know the edges**: "…except the free-threaded build", "…O(1) on average, O(n) worst case".
3. **Offer a tiny example** ("`@app.get` in FastAPI is a decorator").

4. **Say "I'd check the docs"** rather than invent a detail — accuracy beats bluffing.

Self-check

1. "The first item and the rest" in one line? → `first, *rest = items`.
2. Why pass `encoding="utf-8"` to `open`? When does a loop's `else` run? → The default text encoding is platform-dependent; when the loop ends without `break`.
3. `def f(x, seen=set())` — what's wrong? → `seen` is shared between calls.
4. Compare two floats for "equality"? → `math.isclose(a, b)` (with `abs_tol` near zero).
5. Why doesn't threading speed up a CPU-bound pure-Python loop in standard CPython? → The GIL lets one thread run bytecode at a time.
6. What does `super()` resolve to in a diamond? → The next class in the instance's MRO.
7. When is `__new__` worth overriding? → Immutable subclasses, singletons/caching, metaclass-level control.
8. The two parts of CPython's memory reclamation? → Reference counting + cyclic GC.
9. Is `pickle.loads` on a user upload safe? → No — it can run arbitrary code.

Chapter TL;DR

- **Idioms:** EAFP with specific exceptions; unpacking (`a, b = b, a`, `first, *rest`); `enumerate` / `zip(strict=True)`; comprehensions and generator expressions; truthiness (`if not xs`, but `is not None` when 0 is valid); f-strings with format specs and `{x=}`; `with` for every resource; `open(..., encoding="utf-8")`; `pathlib.Path`; `"".join`; `for … else`; `:=`.
- **Gotchas:** mutable defaults (use `None`), late-binding closures (`i=i`), `is` vs `==`, int caching/interning (don't rely), mutating while iterating (new list or copy), shallow vs deep copy, `dict.fromkeys` / `defaultdict` surprises, class vs instance attributes, circular imports (restructure / import the module), float precision (`isclose`, `Decimal("…")`), bare `except` (catch specific), shadowing built-ins.
- **Interview core:** bytecode + interpreter; dynamic + strong typing; call by object reference; list = dynamic array, dict = ordered hash table; hashable = immutable + `__hash__` / `__eq__`; decorators (`f = d(f)`), closures, generators; `__new__` creates, `__init__` initialises; static vs class methods; MRO/C3 and `super()`; dunder methods; `__slots__`; ABC vs Protocol; refcounting + cycle GC; pickle is unsafe for untrusted data; GIL → processes for CPU, threads/asyncio for I/O; type hints checked by tools, not at runtime.

Chapter 21 — Cheat Sheets and Glossary

This last chapter compresses Chapters 1–20 into pages you can scan in a minute: one-page **cheat sheets** by topic, a **"which tool for which job"** table, and an alphabetical **glossary** of 560+ terms, each pointing to the section where it is explained properly. Nothing here is new — if a line doesn't make sense, follow its section reference (written like **§6.2** = Chapter 6, section 2) back to the full explanation.

> **New to a word?** Every term, acronym and tool name used in this book — from *API*, *CLI* and *PATH* to *GIL*, *DAG* and *Copy-on-Write* — has a one-line plain-English definition in the glossary (21.22). Words about networks and large systems (HTTP, latency, caching, queues) are explained in more depth in System Design Fundamentals.

Every snippet with a visible result was checked by `code/cheatsheet_checks.py` (Python 3.14; all 10 checks pass; everything shown is valid on 3.12 unless a version is noted). Each sheet's heading names the chapters it compresses.

Chapter links: 1 · 2 · 3 · 4 · 5 · 6 · 7 · 8 · 9 · 10 · 11 · 12 · 13 · 14 · 15 · 16 · 17 · 18 · 19 · 20

21.1 Syntax and control flow (§1.9–1.11, Ch 3)

Concept	Syntax / rule
Blocks	A line ending in `:` opens a block; its body is indented **4 spaces**; no braces (§1.9)
Comments / docstrings	`# comment` (discarded); `"""Docstring."""` as the first statement → `__doc__` (§1.10)
Line continuation	Implicit inside `()`, `[]`, `{}`; avoid backslashes (§1.9)
Names	Case-sensitive; `snake_case` variables/functions, `CapWords` classes, `UPPER_CASE` constants (§1.11)
Input / output	`input(prompt)` always returns `str`; `print(*values, sep=" ", end="\n")` (§1.7, §1.9)
Script entry point	`if __name__ == "__main__": main()` (§8.5)
`if`	`if c1: … elif c2: … else: …` — first truthy branch wins (§3.2)
Conditional expression	`x = a if cond else b` (§3.3)
`match` (3.10+)	`match cmd:` / `case "quit": …` / `case ["go", direction]: …` / `case {"id": id_}:` / `case Point(x=0):` / `case _:` — no fall-through; bare names **capture** (§3.4)
`for`	`for x in iterable:` — any iterable; `range(start, stop, step)` stop excluded (§3.6)
`while`	`while cond:`; `while True: … break` for loop-and-a-half (§3.7)
Loop control	`break` (leave innermost loop), `continue` (next iteration), `pass` (do nothing) (§3.8)
Loop `else`	Runs only if the loop finished **without** `break` (§3.9)
Walrus	`while (line := f.readline()):` — assign inside an expression, 3.8+ (§2.8)

```
for i, name in enumerate(names, start=1):      # index + value
for a, b in zip(xs, ys, strict=True):          # parallel, error on length mismatch
for k, v in d.items():                          # dict pairs
for x in reversed(seq): ...                     # backwards
for x in sorted(xs, key=len): ...               # sorted view, original unchanged
```

Zen and style: PEP 20 (`import this`) — readability counts, explicit beats implicit, errors shouldn't pass silently. PEP 8 is the style guide; let a formatter (Black/Ruff) apply it (§1.11).

21.2 Types, numbers and operators (Ch 2)

Type	Example	Mutable?	Notes
`int`	`42`, `0b101`, `0xff`, `1_000_000`	no	unbounded precision (§2.5)
`float`	`3.14`, `1e-3`, `float("inf")`	no	IEEE-754 double, ~15–17 significant digits (§2.5)
`complex`	`2 + 3j`	no	`.real`, `.imag`
`bool`	`True`, `False`	no	subclass of `int` (`True + 1 == 2`) (§2.6)
`str`	`"hi"`, `'hi'`, `"""multi"""`, `r"\d"`, `f"{x}"`	no	Unicode text (Ch 4)
`bytes` / `bytearray`	`b"\x00"`	no / yes	raw binary (§4.9)
`list`	`[1, "a"]`	yes	dynamic array (Ch 5)
`tuple`	`(1, 2)`, `1, 2`, `(1,)`	no	the comma makes the tuple (§5.9)
`range`	`range(0, 10, 2)`	no	lazy sequence
`dict`	`{"k": 1}`	yes	insertion-ordered (Ch 6)
`set` / `frozenset`	`{1, 2}`, `set()`	yes / no	`{}` is an empty **`dict`** (§6.6)
`NoneType`	`None`	no	the single "no value" object; test with `is None`

Falsy values: `None`, `False`, `0`, `0.0`, `0j`, `Decimal(0)`, `Fraction(0)`, `""`, `[]`, `()`, `{}`, `set()`, `range(0)`. Everything else is truthy (§2.6).

Conversions (§2.7): `int("42")`, `int("ff", 16)`, `int(3.9) → 3` (truncates), `float("1.5")`, `str(3)`, `bool([]) → False`, `list("ab") → ['a', 'b']`, `ord("A") → 65`, `chr(65) → 'A'`. Bad input raises `ValueError`; wrong type raises `TypeError`.

Operators (§2.8):

Kind	Operators	Watch out
Arithmetic	`+ - * / // % **`	`/` always float; `//` floors (`-7 // 2 == -4`); `%` takes the divisor's sign (`-7 % 3 == 2`); `**` is right-associative and binds tighter than unary minus (`-2 ** 2 == -4`)
Comparison	`== != < <= > >=`, chains `0 < x < 10`	compare floats with `math.isclose`
Identity / membership	`is`, `is not`, `in`, `not in`	`is` only for `None`/singletons
Logical	`and`, `or`, `not`	short-circuit; return an **operand** (`"" or "x" → 'x'`)
Bitwise	`& | ^ ~ << >>`	`~x == -x - 1`
Assignment	`=`, `+=` …, `:=`	`+=` mutates lists in place, rebinds immutables
Built-in maths	`abs`, `round` (half-to-even: `round(2.5) == 2`), `divmod`, `pow(b, e, m)`, `min`, `max`, `sum`	

Money and exactness: `Decimal("0.1")` (from a string!) for money, `Fraction(1, 3)` for exact ratios, `math.isclose` for float comparison (§2.5, §14.8).

21.3 Data structures and big-O (§5.3, §6.2, §6.12, §17.5, §19.1)

Structure	Use for	Fast (avg)	Slow
`list`	ordered, growable sequence	`a[i]`, `a[i]=x`, `append`, `pop()` → O(1)	`insert(0,x)`, `pop(0)`, `x in a`, `remove`, `index` → O(n)
`tuple`	fixed record, dict key	`t[i]` O(1)	no mutation
`dict`	key → value	`d[k]`, `d[k]=v`, `del d[k]`, `k in d` → O(1)	worst case O(n) with collisions
`set`	uniqueness, membership	`add`, `discard`, `x in s` → O(1)	no indexing
`collections.deque`	queue / both-ends stack / last N	`append`, `appendleft`, `pop`, `popleft` → O(1)	middle index O(n)
`heapq` on a list	priority queue, top-k	push/pop O(log n), peek `h[0]` O(1), `heapify` O(n)	search O(n)
`bisect` on a sorted list	ordered search	search O(log n)	`insort` O(n)
`array.array`	compact numbers	O(1) index	Python-level maths not faster
`numpy.ndarray`	numeric arrays	vectorised maths in C	appending is O(n) (copies)

Operation	Cost	Operation	Cost
`sorted(xs)` / `xs.sort()`	O(n log n), O(n) if already sorted, **stable**	`a[i:j]` slice	O(j − i) — copies
`len(x)` (built-ins)	O(1)	`"".join(parts)`	O(total length)
`s1 | s2`	O(len s1 + len s2)	`s1 & s2`	O(min(len s1, len s2))
`min` / `max` / `sum`	O(n)	`Counter(xs)`	O(n)
`heapq.nlargest(k, xs)`	O(n log k)	`list(dict)`	O(n)

Big-O: O(1) constant, O(log n) halving, O(n) linear, O(n log n) good sorting, O(n²) nested loops; "avg" = typical, a rare worst case can be slower (glossary: *Big-O*, *Amortised O(1)*).

Rules: membership tests in a loop → use a `set`; queues → `deque`; repeated "smallest next" → `heapq`; never build strings with `+=` in a loop → `"".join` (§4.8, §17.5).

21.4 String methods and formatting (Ch 4)

Strings are **immutable**: every method returns a new string (§4.2). Index from 0; negative indexes count from the end; slices `s[start:stop:step]` exclude `stop` and never raise (§4.4).

Group	Methods (result for `s = " Hello, World "` where shown)
Case	`lower()`, `upper()`, `title()`, `capitalize()`, `swapcase()`, `casefold()` (aggressive lower for comparisons)
Strip	`strip()` → `'Hello, World'`, `lstrip()`, `rstrip()`, `strip("xy")` removes those *characters*; `removeprefix("ab")`, `removesuffix(".py")` remove an exact prefix/suffix (3.9+)
Split / join	`split()` (any whitespace), `split(",")`, `split(",", 1)`, `rsplit`, `splitlines()`, `partition(":")` → 3-tuple; `", ".join(items)`
Search	`x in s` (preferred test), `find(sub)` → index or **`-1`**, `index(sub)` → index or **`ValueError`**, `rfind`, `count(sub)`, `startswith(("a", "b"))`, `endswith`
Replace	`replace(old, new, count)`, `translate(str.maketrans("abc", "xyz"))`, `expandtabs()`
Checks	`isdigit()`, `isdecimal()`, `isnumeric()`, `isalpha()`, `isalnum()`, `isspace()`, `isupper()`, `islower()`, `isidentifier()`
Align	`center(20, "*")`, `ljust(10)`, `rjust(10)`, `zfill(5)` (`"42".zfill(5)` → `'00042'`)
Encode	`s.encode("utf-8")` → `bytes`; `b.decode("utf-8")` → `str` (§4.9)

Escapes: `\n \t \\ \' \" \xhh \uXXXX \N{name}`; raw strings `r"C:\path\n"` keep backslashes (use for regex) (§4.5).

f-string format spec — `{value:[[fill]align][sign][#][0][width][,][.precision][type]}` (§4.6):

Spec	Result	Spec	Result
`f"{3.14159:.2f}"`	`3.14`	`f"{1234567:,}"`	`1,234,567`
`f"{0.256:.1%}"`	`25.6%`	`f"{42:05d}"`	`00042`
`f"{'hi':>6}"`	`hi`	`f"{'hi':*^6}"`	`**hi**`
`f"{255:#x}"` / `f"{255:b}"`	`0xff` / `11111111`	`f"{1234.5:e}"`	`1.234500e+03`
`f"{x=}"` (3.8+)	`x=…`	`f"{obj!r}"`	uses `repr()`

Other formatting: `"{} {}".format(a, b)` for stored templates, `"%s" % x` in `logging` calls, `string.Template` for user-supplied templates, t-strings `t"…"` (3.14) produce a `Template` object for safe processing (§4.6).

Regex (§4.10, §14.3): `re.search` (anywhere), `re.match` (at start), `re.fullmatch` (whole string), `re.findall`, `re.finditer`, `re.sub(pattern, repl, s)`, `re.split`, `re.compile`; groups `(...)`, named `(?P<name>...)`; always raw strings; beware catastrophic backtracking on nested quantifiers.

21.5 List, tuple, dict and set methods (Ch 5, 6)

List (§5.4, §5.5):

Method	Does	Returns
`append(x)`	add one item at the end	`None`
`extend(iterable)`	add each item	`None`
`insert(i, x)`	insert before index i	`None`
`remove(x)`	delete first `x` (`ValueError` if absent)	`None`
`pop(i=-1)`	remove and return item i	the item
`clear()`	empty the list	`None`
`index(x[, start[, end]])`	first position (`ValueError` if absent)	int
`count(x)`	occurrences	int
`sort(key=None, reverse=False)`	sort in place (stable)	`None`
`reverse()`	reverse in place	`None`
`copy()`	shallow copy (= `a[:]`)	new list

Slicing: `a[1:3]`, `a[::-1]`, `a[::2]`; slice assignment `a[1:3] = [9, 9, 9]` can resize; `del a[::2]`. Copies: `=` aliases, `a.copy()` / `a[:]` / `list(a)` shallow, `copy.deepcopy(a)` deep; grids with `[[0] * n for _ in range(m)]`, never `[[0] * n] * m` (§5.8).

Tuple (§5.9–5.13): `count(x)`, `index(x)`; packing `t = 1, 2`; unpacking `a, *rest = t`; `namedtuple("P", "x y")` / `typing.NamedTuple` for named fields (`_asdict()`, `_replace()`).

Dict (§6.1, §6.4):

Method	Does
`d[k]` / `d[k] = v` / `del d[k]`	read (`KeyError` if missing) / insert-or-update / delete
`get(k, default=None)`	read without error
`setdefault(k, default)`	return `d[k]`, inserting `default` first if missing
`update(other)` / `d |= other` / `d1 | d2`	merge (right side wins); `|` returns a new dict (3.9+)
`pop(k[, default])`	remove and return
`popitem()`	remove and return the **last** inserted pair
`keys()`, `values()`, `items()`	live **views**; `keys()`/`items()` support set operations
`dict.fromkeys(keys, value)`	new dict — the *same* `value` object for every key
`copy()`, `clear()`	shallow copy, empty

Iterating a dict yields keys, in insertion order (guaranteed since 3.7). Don't add/remove keys while iterating — loop over `list(d)`.

Set (§6.6–6.8):

Operator	Method	Meaning
`a | b`	`union`	in either
`a & b`	`intersection`	in both
`a - b`	`difference`	in a, not b
`a ^ b`	`symmetric_difference`	in exactly one
`a <= b`, `a < b`	`issubset`	subset, proper subset
`a >= b`	`issuperset`	superset
—	`isdisjoint(b)`	no common items

Mutators: `add(x)`, `update(it)`, `remove(x)` (raises if absent), `discard(x)` (doesn't), `pop()` (arbitrary item), `clear()`, and in-place `|=`, `&=`, `-=`, `^=`. Methods accept any iterable; operators need sets. `frozenset` is the immutable, hashable version.

21.6 Comprehensions, iterators and generators (§5.6, §6.5, Ch 11)

```
[x * x for x in xs]                        # list
[x for x in xs if x > 0]                   # filter: `if` at the END
["pos" if x > 0 else "neg" for x in xs]    # choose: if-else BEFORE `for`
[y for row in grid for y in row]           # nested: same order as the loops
{k: v for k, v in pairs}                   # dict
{w.lower() for w in words}                 # set
(x * x for x in xs)                        # generator expression — lazy
sum(x * x for x in xs)                     # parentheses can be dropped as sole argument
```

Concept	Rule
Iterable	has `__iter__` → returns an iterator (list, str, dict, file, range) (§11.1)
Iterator	has `__next__` (raises `StopIteration` at the end) and `__iter__` returning itself; **single-pass**
`for` loop	`it = iter(x)` then `next(it)` until `StopIteration`
Generator function	contains `yield`; calling it returns a generator (an iterator) without running the body (§11.2)
`yield from sub`	delegate to a sub-iterator (§11.3)
`send`, `throw`, `close`	two-way generators (§11.3)
Lazy vs eager	generators/`map`/`filter`/`zip`/`range` produce on demand → constant memory, early exit (§11.4)
Built-in consumers	`list`, `tuple`, `set`, `dict`, `sum`, `min`, `max`, `any`, `all`, `sorted`, `"".join`, `next(it, default)`

```
def read_records(path):          # a pipeline stage: constant memory
    with open(path, encoding="utf-8") as f:
        for line in f:
            if line.strip():
                yield line.rstrip("\n").split(",")
```

21.7 Functions and arguments (Ch 7)

```
def f(pos_only, /, normal, *args, kw_only, **kwargs) -> None:
    """Docstring."""
```

Part	Meaning
before `/`	positional-only (§7.2)
normal	positional or keyword
`*args`	extra positionals → **tuple**
after `*` or `*args`	keyword-only
`**kwargs`	extra keywords → **dict**
defaults `x=1`	evaluated **once** at `def` time → use `None` for mutable defaults
call-site unpacking	`f(*seq, **mapping)`
return several	`return a, b` returns a tuple; no `return` → `None`

Topic	One-liner
Scope	**LEGB**: Local → Enclosing → Global → Built-in; assignment makes a name local for the whole function; `global x`, `nonlocal x` to rebind outer names (§7.3)
Closure	inner function remembering outer variables; binds **late** (§7.4)
First-class	functions are objects: pass, return, store; `key=` functions (§7.5)
`lambda`	`lambda x: x * 2` — one expression; don't assign to a name
`map` / `filter`	lazy iterators; comprehensions usually clearer (§7.5, §11.7)
Decorator	`@d` over `def f` ≡ `f = d(f)`; use `functools.wraps` (§7.6, §12.1)
Recursion	base case + progress; limit ~1000 frames (`sys.getrecursionlimit()`); no tail-call optimisation (§7.7)
Memoisation	`@functools.cache` / `@lru_cache(maxsize=128)`; arguments must be hashable (§7.6)
Docs and hints	docstring → `help(f)`; annotations are not enforced (§7.8)

21.8 OOP and dunder methods (Ch 10, §12.4–12.6)

```python
from dataclasses import dataclass, field

class Account:
    bank = "PyBank"                        # class attribute (shared)
    def __init__(self, owner: str, balance: float = 0):
        self.owner = owner                 # instance attribute
        self._balance = balance            # _ = internal by convention
    @property
    def balance(self) -> float:            # computed/validated attribute
        return self._balance
    @classmethod
    def empty(cls, owner):                 # alternative constructor
        return cls(owner)
    @staticmethod
    def valid_amount(x) -> bool:           # helper, no self/cls
        return x > 0
    def __repr__(self):
        return f"Account({self.owner!r}, {self._balance})"

@dataclass(frozen=True, slots=True)
class Point:
    x: float
    y: float = 0.0
    tags: tuple[str, ...] = ()             # mutable defaults need field(default_factory=list)
```

Syntax / built-in	Dunder	Syntax / built-in	Dunder
`Cls(...)` create / init	`__new__` / `__init__`	`repr(x)` / `str(x)`	`__repr__` / `__str__`
`x == y`, `x < y`	`__eq__`, `__lt__` …	`hash(x)`	`__hash__`
`x + y`, `y + x` fallback, `x += y`	`__add__`, `__radd__`, `__iadd__`	`bool(x)`	`__bool__` (else `__len__`)
`len(x)`	`__len__`	`x[k]`, `x[k] = v`, `del x[k]`	`__getitem__`, `__setitem__`, `__delitem__`
`for a in x`	`__iter__` / `__next__`	`a in x`	`__contains__`
`x(...)`	`__call__`	`with x:`	`__enter__` / `__exit__`
`x.attr` missing	`__getattr__`	every `x.attr`	`__getattribute__`
descriptor protocol	`__get__`, `__set__`, `__delete__`	subclass hook	`__init_subclass__`
`f"{x:spec}"`	`__format__`	pattern matching positions	`__match_args__`

Rule	Section
Always write `__repr__`; `__str__` falls back to it	§10.2
Defining `__eq__` without `__hash__` makes instances unhashable	§10.2
Return `NotImplemented` (not `False`) for unsupported operand types	§10.3
`super()` = next class in the instance's **MRO** (C3 linearisation); `Cls.__mro__`	§10.4, §10.5
Mixins add small capabilities via multiple inheritance	§10.6
Duck typing by default; ABCs enforce methods at instantiation; `Protocol` for structural static typing	§10.7
Prefer composition (has-a) unless it's a genuine is-a	§10.8
`_x` internal by convention; `__x` is name-mangled to `_Cls__x`, not private	§10.9
`@dataclass` options: `frozen`, `order`, `slots`, `kw_only`, `field(default_factory=…)`	§10.10
`enum.Enum`, `IntEnum`, `StrEnum`, `Flag`, `auto()`	§10.11
`__slots__` removes the per-instance `__dict__` → less memory, no new attributes	§10.12
`functools.total_ordering` fills in comparisons from `__eq__` + one of `__lt__` …	§10.2, §12.2

21.9 Exceptions (Ch 9)

```
try:
    value = int(text)
except ValueError as e:               # specific first; the first matching except wins
    log.warning("bad input %r: %s", text, e)
    value = 0
except (TypeError, KeyError):
    raise                             # re-raise unchanged
else:
    print("parsed OK")                # only if no exception
finally:
    cleanup()                         # always

class AppError(Exception): ...        # custom hierarchy
class NotFound(AppError): ...
raise NotFound("user 42") from err    # chain: keeps the cause (__cause__)
e.add_note("while loading config")    # 3.11+
```

Hierarchy (partial)	Raised when
`BaseException` → `SystemExit`, `KeyboardInterrupt`, `GeneratorExit`, `Exception`	root; don't catch it
`Exception` → `ArithmeticError` → `ZeroDivisionError`, `OverflowError`	maths
`LookupError` → `IndexError`, `KeyError`	bad index / missing key
`ValueError` (→ `UnicodeError`)	right type, bad value
`TypeError`	wrong type
`AttributeError`, `NameError` (→ `UnboundLocalError`)	missing attribute / name
`OSError` → `FileNotFoundError`, `PermissionError`, `TimeoutError`, `ConnectionError`	OS / I/O
`ImportError` → `ModuleNotFoundError`	import failed
`RuntimeError` → `RecursionError`, `NotImplementedError`	generic runtime problems
`StopIteration`	iterator exhausted
`ExceptionGroup` + `except*` (3.11+)	several errors at once, e.g. from `asyncio.TaskGroup` (§9.6)

Rules: keep `try` blocks small; never bare `except:` (catches Ctrl+C); prefer EAFP (§9.5); `assert` is for internal invariants only — it is removed by `python -O` (§9.7); read tracebacks bottom-up (§9.1, §15.10).

21.10 Files, paths and data formats (§9.8–9.11, §14.6)

```
from pathlib import Path
p = Path("data") / "sales.csv"
p.parent.mkdir(parents=True, exist_ok=True)
text = p.read_text(encoding="utf-8");  p.write_text("a,b\n", encoding="utf-8")
with p.open(encoding="utf-8") as f:       # stream big files line by line
    for line in f: ...
with open("img.png", "rb") as f: data = f.read()        # binary: bytes, no encoding
for q in Path(".").rglob("*.py"): print(q.name, q.stem, q.suffix)
```

Mode	Meaning	Mode	Meaning
`"r"`	read text (default)	`"rb"` / `"wb"`	binary read / write
`"w"`	write, **truncates**	`"a"`	append
`"x"`	create, fail if it exists	`"r+"`	read and write

Task	Tool
Paths, globbing, read/write small files	`pathlib.Path` (§9.10)
Copy/move/delete trees, archives	`shutil.copytree`, `shutil.move`, `shutil.rmtree`, `shutil.make_archive`
Scratch files/dirs	`tempfile.TemporaryDirectory()`, `NamedTemporaryFile`
Atomic write	write to a temp file in the same folder, then `os.replace(tmp, final)`
CSV	`csv.reader/writer`, `csv.DictReader/DictWriter`; open with `newline=""`; values come back as `str` (§9.11)
JSON	`json.load/dump` (files), `json.loads/dumps` (strings), `indent=2`, `default=str` for unknown types
Python objects	`pickle` — Python-only, **never load untrusted data**
Tabular analytics	pandas `read_csv` / `read_parquet` (§18.4)
Embedded SQL	`sqlite3` with `?` placeholders (§14.11)

Always pass `encoding="utf-8"` for text files (§9.9).

21.11 `itertools`, `functools`, `collections` (§6.9, §7.6, §11.5, §12.2)

`itertools` (all lazy) — results for small inputs:

Function	Example → result
`count(start, step)`	`count(10, 5)` → 10, 15, 20, … (infinite)
`cycle(it)` / `repeat(x, n)`	`cycle("ab")` → a, b, a, …; `repeat(0, 3)` → 0, 0, 0
`chain(a, b)` / `chain.from_iterable(xss)`	`chain([1, 2], [3])` → 1, 2, 3
`islice(it, stop)` / `islice(it, start, stop, step)`	`islice(count(), 3)` → 0, 1, 2
`takewhile` / `dropwhile`	`takewhile(lambda x: x < 3, [1, 2, 5, 1])` → 1, 2
`groupby(it, key)`	groups **consecutive** equal keys — sort first
`accumulate(it[, func])`	`accumulate([1, 2, 3])` → 1, 3, 6
`pairwise(it)` (3.10+)	`pairwise("abc")` → ('a','b'), ('b','c')
`batched(it, n)` (3.12+)	`batched("abcde", 2)` → ('a','b'), ('c','d'), ('e',)
`zip_longest(a, b, fillvalue=None)`	pads the shorter input
`product(a, b)` / `product(a, repeat=2)`	Cartesian product
`permutations(it, r)`	`permutations("abc", 2)` → 6 ordered pairs
`combinations(it, r)`	`combinations("abc", 2)` → ab, ac, bc
`combinations_with_replacement`	allows repeats
`starmap(f, pairs)`	`f(*pair)` for each
`tee(it, n)`	n independent iterators from one

`functools`:

Tool	Use
`@cache` / `@lru_cache(maxsize=…)`	memoise pure functions; `.cache_info()`, `.cache_clear()`
`@cached_property`	compute an attribute once per instance
`@wraps(func)`	keep name/docstring in decorators
`partial(f, *args, **kw)`	pre-fill arguments
`reduce(f, it, initial)`	fold to one value (prefer `sum`, `max`, `math.prod` when they fit)
`@total_ordering`	derive comparison methods
`cmp_to_key(cmp)`	use an old-style comparator as `key=`
`@singledispatch`	overload a function by the first argument's type

collections:

Type	One-liner
`Counter(it)`	counts; `most_common(k)`; missing key → 0; `+ - & |`
`defaultdict(list)`	auto-creates missing values (a *read* inserts)
`deque(maxlen=n)`	O(1) at both ends; `rotate`; bounded buffer
`namedtuple("P", "x y")`	tuple with field names
`OrderedDict`	`move_to_end`, `popitem(last=False)`; order-sensitive `==`
`ChainMap(a, b)`	look up keys in several mappings in order

21.12 Decorators, context managers, metaprogramming (Ch 12)

```python
import functools, time
from contextlib import contextmanager

def timed(func):                                  # plain decorator
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        t0 = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            print(f"{func.__name__}: {time.perf_counter() - t0:.3f}s")
    return wrapper

def retry(times):                                 # decorator factory (3 levels)
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(times):
                try:
                    return func(*args, **kwargs)
                except OSError:
                    if attempt == times - 1:
                        raise
        return wrapper
    return decorator

@contextmanager
def opened(path):                                 # generator-based context manager
    f = open(path, encoding="utf-8")
    try:
        yield f                                   # value bound by `as`
    finally:
        f.close()                                 # always runs
```

Fact	Section
Stacked decorators are applied bottom-up, and the wrappers run top-down	§12.1
Decorators run at **definition** time; wrappers run at **call** time	§12.1
`__exit__` returning a truthy value suppresses the exception	§12.3
`contextlib` : `suppress` , `closing` , `nullcontext` , `redirect_stdout` , `chdir` (3.11+), `ExitStack` (dynamic number of managers, closes LIFO)	§12.3
Async versions: `async with` , `@asynccontextmanager`	§12.3
Descriptors (`__get__` / `__set__`) power `property` , methods, `classmethod` , `staticmethod` ; data descriptors beat the instance `__dict__`	§12.4
`__getattr__` runs only when normal lookup fails; `__getattribute__` runs for every access	§12.5
Metaclass = class of a class (`type` by default); prefer class decorators or `__init_subclass__`	§12.6
Monkey patching: fine in tests with automatic restore (`mock.patch` , `monkeypatch`); avoid in production	§12.7
Introspection: `type` , `isinstance` , `dir` , `vars` , `getattr` / `setattr` / `hasattr` , `inspect` ; never `eval` / `exec` untrusted input	§12.8

21.13 Type hints (Ch 13)

```python
from typing import Protocol, TypedDict, Literal, Final, NewType, Callable, Any

def greet(name: str, times: int = 1) -> str: ...
scores: dict[str, list[float]] = {}
maybe: int | None = None                      # = Optional[int]
def first[T](xs: list[T]) -> T: ...           # 3.12 generic syntax
type Matrix = list[list[float]]               # 3.12 type alias statement
Handler = Callable[[str, int], bool]
Mode = Literal["r", "w"]
MAX: Final = 100
UserId = NewType("UserId", int)

class SupportsClose(Protocol):                # structural: no inheritance needed
    def close(self) -> None: ...

class Movie(TypedDict):
    title: str
    year: int
```

Concept	Meaning	Section
Not enforced	the interpreter ignores hints; mypy/pyright check them statically	§13.1
Gradual typing	add hints where they pay off; unannotated code is `Any`	§13.1
Accept abstract, return concrete	parameters `Iterable` / `Sequence` / `Mapping` (from `collections.abc`), return `list` / `dict`	§13.3
Invariance	`list[bool]` is not a `list[int]`; `Sequence` is covariant	§13.3
Narrowing	`if x is not None:` / `isinstance` refine a union	§13.4
`Any` vs `object`	`Any` switches checking off; `object` allows only what every object supports	§13.5
`TypeVar` / bounds / constraints	generic functions and classes	§13.6
`ParamSpec`	type decorators that preserve signatures	§13.7
`Protocol`, `TypedDict`, `Literal`, `Final`, `NewType`	structural interface, typed dict shape, fixed values, constant, distinct zero-cost type	§13.8–13.10
Tools	`mypy --strict`, `pyright`, `reveal_type(x)`, `# type: ignore[code]`, stub packages `types-*`	§13.13
Runtime validation	pydantic / FastAPI at the boundary	§13.14

21.14 Standard-library quick picks (Ch 14)

Need	Module / call	Section
Dates and times	`datetime.now(UTC)`, `ZoneInfo("Europe/Paris")`, `isoformat()`, `fromisoformat()`, `timedelta` — store aware UTC	§14.1
Durations / timeouts	`time.perf_counter()` / `time.monotonic()`; `time.time()` = epoch seconds	§14.2
Regex	`re` (see 21.4)	§14.3
Logging	`log = logging.getLogger(__name__)`; configure once (`basicConfig` / `dictConfig`); `log.info("x=%s", x)`; `log.exception(...)` in `except`	§14.4
CLI arguments	`argparse.ArgumentParser`, `add_argument`, subcommands	§14.5
Environment, argv, exit	`os.environ.get("KEY", default)`, `sys.argv`, `sys.exit(code)`	§14.6
Run commands	`subprocess.run(["git", "status"], check=True, capture_output=True, text=True, timeout=30)` — a list, not a shell string	§14.6
Random	`random.Random(42)` for reproducible simulations; `secrets.token_urlsafe()` for tokens/passwords	§14.7
Maths / stats	`math.isclose`, `math.gcd`, `math.isqrt`, `math.comb`; `statistics.mean/median/stdev/quantiles`	§14.8
Exact numbers	`decimal.Decimal("0.10")`, `fractions.Fraction(1, 3)`	§14.8
Copy / pretty print	`copy.deepcopy`, `pprint.pp`	§14.9
IDs / hashes	`uuid.uuid4()`; `hashlib.sha256(b).hexdigest()` (use a KDF such as `hashlib.scrypt` for passwords)	§14.9
Embedded DB	`sqlite3.connect`, `with con:` transactions, `?` placeholders	§14.11
URLs / HTTP	`urllib.parse`; `urllib.request` for quick scripts; `requests` / `httpx` for real work	§14.12

21.15 Modules, packages and environments (Ch 8)

Task	Command / rule	Section
Import styles	`import json`, `from pathlib import Path`, `import numpy as np`; avoid `import *`	§8.1
How import works	check `sys.modules` → search `sys.path` → create module, register it, run it once → bind the name	§8.2
Package	folder with `__init__.py` (regular) or without (namespace); `__all__` controls `import *`	§8.3
Relative import	`from .utils import x` — only inside packages; run with `python -m pkg.mod`	§8.4
Script vs module	`if __name__ == "__main__":`	§8.5
Circular import fix	restructure; `import module` instead of `from module import name`; import inside the function	§8.6
Virtual env	`python -m venv .venv` → `source .venv/bin/activate` (Windows: `.venv\Scripts\activate`) → `python -m pip install pkg`	§8.7
Freeze / reproduce	`pip freeze > requirements.txt`, `pip install -r requirements.txt`	§8.8
Modern projects	`pyproject.toml` (metadata, dependencies, tool config) + lock file; `uv add pkg`, `uv run`, `uv sync`; Poetry similar	§8.8
Layout and publishing	`src/` layout; build sdist + wheel (`python -m build` / `uv build`); TestPyPI → PyPI (Trusted Publishing)	§8.9

21.16 Testing and debugging (Ch 15)

```python
# test_prices.py — run with: pytest -q
import pytest
from prices import apply_discount

@pytest.fixture
def cart():
    return {"apple": 1.0, "pear": 2.0}

@pytest.mark.parametrize("pct, expected", [(0, 3.0), (50, 1.5)])
def test_discount(cart, pct, expected):
    assert apply_discount(cart, pct) == pytest.approx(expected)

def test_rejects_negative(cart):
    with pytest.raises(ValueError, match="negative"):
        apply_discount(cart, -5)
```

Tool / idea	Key facts	Section
Test types	unit (one piece, fast), integration (pieces together), end-to-end (whole system); Arrange–Act–Assert	§15.1
`unittest`	`class T(unittest.TestCase)`, `setUp`, `assertEqual`, `assertRaises`; `python -m unittest`	§15.3
pytest	plain `assert`; fixtures (`yield` for teardown, `scope=`), `conftest.py`, `parametrize`, `tmp_path`, `monkeypatch`, `capsys`, `caplog`; marks `skip` / `skipif` / `xfail`	§15.4
pytest CLI	`-q`, `-x` (stop at first failure), `-k expr`, `-m mark`, `--lf` (last failed), `--pdb`	§15.4
Test doubles	dummy, stub, spy, mock, fake; `Mock(return_value=…, side_effect=…)`, `assert_called_once_with`; `patch("where.it.is.used")`; `autospec=True`	§15.5
Coverage	`pytest --cov=pkg --cov-branch --cov-report=term-missing`; coverage ≠ correctness	§15.6
doctest	examples in docstrings run as tests	§15.7
TDD	red → green → refactor	§15.8
Hypothesis	property-based tests: `@given(st.lists(st.integers()))`; shrinks failures	§15.9
Tracebacks	read the last line first, then walk up the frames	§15.10
Debugger	`breakpoint()` → pdb: `n` next, `s` step, `c` continue, `p expr`, `w` where, `u` / `d` up/down, `b` breakpoint, `q` quit	§15.11
Logging vs print	logging for anything that stays in the code	§15.12

21.17 Concurrency: which tool? (Ch 16)

Situation	Use	Why
Tens–hundreds of blocking I/O calls (HTTP, DB, files), sync libraries	`ThreadPoolExecutor` / `threading`	GIL is released while waiting
Thousands of concurrent connections, async libraries	`asyncio`	one thread, cheap tasks
CPU-bound pure Python	`ProcessPoolExecutor` / `multiprocessing`	one GIL per process → real parallelism
CPU-bound numeric work	numpy/pandas vectorisation	C loops
CPU-bound threads with shared memory, compatible deps	free-threaded build (`python3.14t`)	no GIL (optional build)
Pipeline stages	threads/processes + `queue.Queue` (bounded)	backpressure, sentinels for shutdown
Blocking call inside async code	`await asyncio.to_thread(func, ...)`	keeps the event loop responsive
Beyond one machine	Celery, RQ, Dask, Ray, Spark	distributed

	Threads	Processes	asyncio
Parallel CPU (standard build)	no	yes	no
Shared memory	yes (needs locks)	no (pickled messages)	yes (one thread)
Switching	pre-emptive	OS	cooperative at `await`
Cost per unit	~MB stack	whole interpreter	~KB

```python
from concurrent.futures import ThreadPoolExecutor, as_completed
with ThreadPoolExecutor(max_workers=8) as ex:
    futures = [ex.submit(fetch, url) for url in urls]
    for fut in as_completed(futures):          # completion order; ex.map keeps input order
        print(fut.result())                    # re-raises the worker's exception

import asyncio
async def main():
    async with asyncio.TaskGroup() as tg:      # 3.11+: structured concurrency
        tasks = [tg.create_task(fetch_async(u)) for u in urls]
    return [t.result() for t in tasks]
asyncio.run(main())
```

Synchronisation (§16.4): `Lock` (mutual exclusion; `with lock:`), `RLock` (re-entrant), `Semaphore(n)` (at most n holders), `Event`, `Condition`. Pitfalls (§16.11): races on shared state, deadlocks (take locks in a global order, use timeouts), blocking calls in coroutines, lost exceptions in fire-and-forget tasks, missing `if __name__ == "__main__":` with processes.

21.18 Performance and internals (Ch 17)

Fact	Section
Everything is an object with identity (`id`), type and value; CPython objects carry a reference count and a type pointer	§17.1
Memory: reference counting frees objects immediately; a generational cyclic GC collects reference cycles; `weakref` refers without owning	§17.2
Small ints −5…256 and some strings are cached/interned → never compare values with `is`	§17.3
Assignment shares; `.copy()` is shallow; `deepcopy` is deep	§17.4

Measure first (§17.6):

Question	Tool
How fast is this snippet?	`python -m timeit "..."`, `timeit.timeit(fn, number=…)`, `%timeit` in Jupyter
Which functions are slow?	`python -m cProfile -s cumtime script.py`, `pstats` (ncalls, tottime, cumtime)
Which lines?	`line_profiler` (`kernprof -l -v`)
Live / low-overhead	`py-spy top --pid N`, `scalene`, `pyinstrument`
Which lines allocate memory?	`tracemalloc` snapshots

Optimise in this order (§17.7): better algorithm/data structure (set for membership, dict for lookup, deque for queues) → cache repeated work (`functools.cache`) → vectorise with numpy/pandas → stream with generators → built-ins, comprehensions, `str.join`, local variables in hot loops → `__slots__` for many small objects → compile hot spots (numba, Cython, Rust/PyO3, mypyc) or use PyPy → parallelise (processes). Newer CPython releases (3.11+) are faster for free.

21.19 numpy, pandas, matplotlib essentials (Ch 18)

numpy (§18.1–18.3):

```
import numpy as np
a = np.array([[1, 2, 3], [4, 5, 6]])       # shape (2, 3), dtype int64
a.shape, a.ndim, a.dtype, a.size
np.zeros((2, 3)); np.ones(3); np.arange(0, 10, 2); np.linspace(0, 1, 5); np.eye(3)
a[0, 1]; a[:, 1]; a[a > 2]                  # index, column slice (view), boolean mask (copy)
a.sum(axis=0)                               # per column -> [5, 7, 9]
a.mean(axis=1)                              # per row    -> [2., 5.]
a * 10; a + np.array([10, 20, 30])          # vectorised + broadcasting
a @ a.T                                     # matrix product
a.reshape(3, 2); a.T; np.concatenate([a, a]); np.vstack; np.hstack
rng = np.random.default_rng(42); rng.normal(size=3); rng.integers(0, 10, 5)
np.linalg.solve(A, b); np.linalg.eigh(S)
```

Rules: slices are **views** (changing them changes the original; `.copy()` to detach); masks and fancy indexing return copies; broadcasting compares shapes from the right — dimensions must be equal or one of them 1; `axis=0` collapses rows (result per column), `axis=1` collapses columns (result per row); vectorise instead of Python loops.

pandas (§18.4–18.11, pandas 3.x):

Task	Code
Load / save	`pd.read_csv(path, parse_dates=["date"])`, `pd.read_parquet`, `df.to_parquet`
First look	`df.head()`, `df.info()`, `df.describe()`, `df["col"].value_counts()`, `df.isna().sum()`
Select	`df["col"]`, `df[["a", "b"]]`, `df.loc[row_labels, cols]` (labels, end **inclusive**), `df.iloc[0:5, 1]` (positions, end exclusive)
Filter	`df[(df.qty > 5) & (df.city == "Oslo")]` (use `&`, `|`, `~` with parentheses), `df.query("qty > 5")`
New columns	`df["rev"] = df.qty * df.price`, `df.assign(rev=lambda d: d.qty * d.price)`, `np.where`, `np.select`, `pd.cut`
Missing data	`isna`, `dropna`, `fillna`, `pd.to_numeric(errors="coerce")`
Group	`df.groupby("city")["rev"].agg(["sum", "mean"])`, named aggregation `agg(total=("rev", "sum"))`, `transform` keeps the original shape
Combine	`pd.merge(a, b, on="id", how="left", validate="m:1", indicator=True)`, `pd.concat([a, b])`
Reshape	`pivot_table(index=, columns=, values=, aggfunc=)`, `melt` (wide → long)
Dates	`pd.to_datetime(col, format=...)`, `.dt.month`, `set_index("date").resample("ME").sum()` (`"ME"` month end in pandas ≥ 2.2/3.0), `rolling(7).mean()`
Sort / top-N	`sort_values("rev", ascending=False)`, `nlargest(3, "rev")`
Speed / memory	vectorise instead of `apply`; `astype("category")` for repeated strings; Parquet over CSV

pandas 3.0 notes (§18.4–18.5): text columns use a dedicated `str` dtype; **Copy-on-Write** is always on → assign with a single `df.loc[mask, "col"] = value`, never chained indexing.

matplotlib (§18.12):

```
import matplotlib
matplotlib.use("Agg")                       # headless scripts
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(6, 4))      # object-oriented API
ax.plot(x, y, label="sales"); ax.bar(cats, vals); ax.scatter(x, y); ax.hist(data, bins=20)
ax.set(title="Sales", xlabel="Month", ylabel="EUR"); ax.legend()
fig.tight_layout(); fig.savefig("img/sales.png", dpi=120); plt.close(fig)
```

21.20 Interview patterns and gotchas (Ch 19, 20)

Pattern → Python tool (Ch 19):

Signal in the problem	Pattern	Python
sorted array, pair/triple sums	two pointers	indices `lo`, `hi` (§19.2)
contiguous subarray/substring	sliding window / prefix sums	`dict` of last seen, `accumulate`, `defaultdict(int)` (§19.2)
"seen before?", counts, groups	hashing	`set`, `Counter`, `defaultdict(list)` (§19.3)
matching brackets, "next greater"	stack / monotonic stack	`list.append/pop` (§19.4)
FIFO, level order, window max	queue / monotonic deque	`collections.deque` (§19.4)
k largest / smallest, merge k, running median	heap / two heaps	`heapq` (negate for max-heap) (§19.5)
sorted search, minimum feasible value, weighted random pick	binary search	`bisect_left/right`, `lo < hi` template, prefix sums (§19.6)
linked list, tree, serialize a tree	node classes, DFS/BFS	`@dataclass` nodes, pre-order with `#` markers (§19.7)
shortest path unweighted / weighted / dependencies	BFS / Dijkstra / topological sort	`deque`, `heapq`, in-degree counts (§19.8)
dynamic connectivity, cycles, MST	union-find	`parent` list + path compression (§19.9)
optimal value with overlapping subproblems	DP	`@cache` or a table (§19.10)
all combinations / permutations / placements	backtracking	choose → recurse → undo; `path[:]` (§19.11)
bits, parity, powers	bit tricks, number theory	`x & (x-1)`, `pow(b, e, m)`, `math.gcd`, `math.isqrt` (§19.12)
prefix lookups, autocomplete, wildcard, words on a grid	trie	`dict` of children + end-of-word field (§19.13)
locally best choice is provably safe	greedy	one pass after `sorted(key=...)`; justify by exchange argument (§19.14)
merge / insert / overlap ranges, rooms needed	intervals	sort by start (merge) or end (keep most); `heapq` of ends; sweep line (§19.15)
cycle entry, middle, O(1)-space list checks	fast & slow pointers	`slow`, `fast`, compare with `is` (§19.16)
k-th element, three values, sort a linked list	quickselect / Dutch flag / merge sort	random-pivot partition; `low` / `mid` / `high` (§19.17)
spiral/rotate matrix, 32-bit limits, collinear points	math & geometry	boundaries; explicit range check; `gcd`-reduced slopes (§19.18)

Top gotchas (§19.19, §20.2): mutable default args (`None` sentinel) · late-binding closures (`lambda i=i: i`) · `is` vs `==` · int caching/interning · mutating a list/dict while iterating · shallow copies and `[[0]*n]*m` · `dict.fromkeys(k, [])` shares one list; a `defaultdict` read inserts · mutable class attributes shared by instances · circular imports · float precision (`isclose`, `Decimal("…")`) · bare `except:` · shadowing built-ins (`list = …`) · `xs.sort()` returns `None` · `//` floors toward −∞ · recursion limit ~1000.

21.21 Which tool for which job

Job	Reach for	Not	Section
Ordered collection you append to	`list`	`tuple` (immutable)	§5.2
Fixed record / multiple return values / dict key	`tuple`, `NamedTuple`, frozen `dataclass`	`list`	§5.9, §5.13, §10.10
Look up by key	`dict`	list of pairs + search	§6.1
Membership / de-duplication	`set` (`dict.fromkeys` to keep order)	`x in list` in a loop	§6.6
Counting	`Counter`	manual dict with `if k in d`	§6.9
Grouping	`defaultdict(list)`	`setdefault` everywhere	§6.9
Queue / sliding buffer	`deque` (`maxlen`)	`list.pop(0)`	§6.9
Priority queue / top-k	`heapq`	repeated `sort`	§6.10
Sorted inserts and range queries	`bisect` (or `sortedcontainers`)	re-sort after each insert	§6.11
Numeric arrays / maths	NumPy	Python loops over lists	§5.14, §18.1
Tables, group-by, joins	pandas	nested dicts	§18.4
Simple data class	`@dataclass`	hand-written `__init__` / `__repr__` / `__eq__`	§10.10
Validate untrusted input (JSON, API)	pydantic	trusting type hints	§13.14
Named constants / fixed choices	`enum.Enum` / `StrEnum`	bare strings	§10.11
Interface without inheritance	`typing.Protocol`	ABC (when you don't control the classes)	§10.7, §13.8
Reuse behaviour	composition, small mixins	deep inheritance trees	§10.6, §10.8
Cross-cutting behaviour (timing, retry, auth)	decorator	copy-pasted code	§12.1
Setup/teardown that must happen	`with` + context manager	manual `close()`	§9.8, §12.3
Huge or infinite sequences	generators, `itertools`	building full lists	§11.2, §11.5
Paths	`pathlib`	string concatenation, `os.path` sprawl	§9.10
Config / data exchange	JSON (or TOML for config)	pickle	§9.11
Persist Python objects you trust	`pickle` / `joblib`	pickle from strangers	§9.11
Small local database	`sqlite3`	ad-hoc files	§14.11
Diagnostics in running code	`logging`	`print`	§14.4, §15.12
CLI arguments	`argparse` (or click/typer)	parsing `sys.argv` by hand	§14.5
External commands	`subprocess.run([...], check=True)`	`os.system`, `shell=True` with user input	§14.6

Job	Reach for	Not	Section
Tokens / passwords	`secrets`	`random`	§14.7
Money	`Decimal("…")` or integer cents	`float`	§2.5, §14.8
Timestamps	aware `datetime` in UTC + `zoneinfo`	naive local times	§14.1
Durations	`time.perf_counter()`	`time.time()`	§14.2
Tests	pytest (+ fixtures, parametrize)	manual prints	§15.4
Replace a dependency in a test	`unittest.mock.patch` / `monkeypatch`	editing production code	§15.5
Concurrency (I/O waits, CPU-heavy work)	see the table in 21.17	—	§16.10
Find the slow part	`cProfile`, `py-spy`	guessing	§17.6
Memoise a pure function	`functools.cache`	hand-rolled global dict	§7.6
Dependencies per project	`venv` + `pyproject.toml` (+ `uv`)	global `pip install`	§8.7, §8.8
Type-check a codebase	mypy / pyright in CI	runtime `isinstance` everywhere	§13.13

21.22 Glossary (A–Z)

Each entry: **term** — plain meaning — section where it is explained. Links go to the chapter README.

Term	Meaning	Where
ABC (abstract base class) / abstract method	A class (from `abc`) whose `@abstractmethod`-marked methods a subclass must implement before it can be instantiated	10.7
Absolute import	Import by full path from the project/package root (`from pkg.sub import x`)	8.4
Accumulate	`itertools.accumulate`: running totals (or running results of any two-argument function)	11.5
Adjacency list	Graph stored as "node → list of its neighbours", usually a `dict` of lists	19.8
Agg backend	matplotlib's non-interactive renderer that only writes image files — for scripts and servers without a screen	18.12
Aliasing	Two names bound to the same object, so a change through one is seen through the other	5.8
Amortised O(1)	An operation that is occasionally expensive but constant on average over many calls (e.g. `list.append`)	5.3
Annotation / type hint	Note describing the expected type of a parameter, return value or variable, for tools and readers; stored in `__annotations__`, not enforced	13.2
`Any`	Type-hint escape hatch that disables checking for a value	13.5
API	Application programming interface: the set of functions, classes or HTTP endpoints one piece of software offers to others	1.3
`apply` (pandas)	Runs a Python function per row/column/group — flexible but slow; prefer vectorised operations	18.11
argparse	Standard-library module for command-line options, arguments and subcommands	14.5
Argument	The actual value passed at a call (vs the parameter name in the definition)	7.1
`*args`	Parameter collecting extra positional arguments into a tuple	7.2
Arrange–Act–Assert	Test structure: set up, perform the action, check the result	15.1
`array.array`	Compact, typed array of C numbers from the standard library	5.1
ASCII	Old 7-bit character set (128 characters: English letters, digits, punctuation); the first 128 code points of Unicode	4.9
`assert`	Statement checking an internal invariant; removed when Python runs with `-O`	9.7
Associativity	Which side groups first when the same operator repeats: most are left-to-right, `**` is right-to-left	2.8
AST (abstract syntax tree)	Tree form of parsed source code that CPython compiles to bytecode	1.8
`async def` / `await`	`async def` defines a coroutine function; `await` pauses it until another awaitable finishes, letting the event loop run other work	16.9
asyncio	Standard library for single-threaded concurrency with an event loop and `async` / `await`	16.9
Atomic operation	Happens completely or not at all, with no half-finished state visible (e.g. `os.replace` of a file)	9.10
Attribute	A name attached to an object (`obj.name`): data or method	10.1
Augmented assignment	`x += 1` and friends; mutates in place for mutable objects that support it	2.8
Autospec	Mock option that makes a mock follow the real object's signature	15.5
Aware datetime	A `datetime` that carries a time zone (`tzinfo`); comparable across zones	14.1

Term	Meaning	Where
Axis (NumPy/pandas)	A dimension of an array: `axis=0` runs down rows (result per column), `axis=1` across columns (result per row)	18.1
Backpressure	Slowing producers when consumers can't keep up, e.g. with a bounded queue	16.5
Backtracking	Search that builds a candidate step by step and undoes steps that can't succeed	19.11
Banker's rounding	Round-half-to-even, used by `round()` (`round(2.5) == 2`)	2.5
Base class / subclass	The class another inherits from (parent, superclass) / the class that inherits (child)	10.4
`BaseException`	Root of all exceptions, including `SystemExit` and `KeyboardInterrupt`; catch `Exception` instead	9.1
BFS (breadth-first search)	Graph/tree traversal level by level with a queue; shortest paths in unweighted graphs	19.8
Big-O	Notation for how cost grows with input size, ignoring constants	17.5
Binary heap / `heapq` / min-heap	Complete binary tree stored in an array where each parent is ≤ its children, so the smallest item is at index 0; `heapq` implements it on a list (O(log n) push/pop)	6.10
Binary mode	Opening a file with `"b"` so reads/writes are raw `bytes` with no encoding or newline translation	9.9
Binary search	Finding a position in sorted data by halving the range each step, O(log n)	19.6
Binary search on the answer	Binary-searching a range of candidate answers using a yes/no "is this value feasible?" check	19.6
Binary tree	Tree in which every node has at most two children (left and right)	19.7
bisect	Standard-library binary search and sorted insertion on lists	6.11
Bit / byte	A bit is one binary digit (0 or 1); a byte is 8 bits, holding a value 0–255	4.9
Bitwise operator	`& | ^ ~ << >>`: work on the individual bits of integers	2.8
Black	Opinionated code formatter that rewrites Python files into one consistent style	1.11
Block	Group of statements under a line ending in `:`, marked by indentation	1.9
Blocking call	A call that makes the current thread wait (for I/O, a lock, `time.sleep`) until it finishes	16.9
Boolean	A true/false value; Python's `bool` type with `True` and `False`	2.6
Boolean mask	Array/Series of `True`/`False` used to select matching elements (`a[a > 2]`, `df[df.qty > 5]`)	18.1
Bound method	A function retrieved through an instance, with `self` already filled in	10.1
Branch coverage	Coverage metric that checks both outcomes of each decision were executed	15.6
`breakpoint()`	Built-in that drops into the debugger (pdb by default) at that line	15.11
Broadcasting	NumPy rule for combining arrays of different shapes by stretching size-1 dimensions	18.2
Brute force	Solving a problem by trying every possibility; a correct but usually slow first answer	19.20
BST (binary search tree)	Binary tree where every left subtree holds smaller keys and every right subtree larger ones	19.7

Term	Meaning	Where
Buffering	Collecting data in memory and reading/writing it in larger chunks; `flush()` forces buffered output out	9.8
Build backend	Tool named in `pyproject.toml` (hatchling, setuptools, …) that turns a project into an sdist and a wheel	8.8
Built-in function	Function always available without an import (`print`, `len`, `sorted`, `open`)	1.7
Bytecode	The low-level instructions CPython compiles source into and then interprets	1.8
`bytes`	Immutable sequence of raw bytes (0–255); text must be encoded to become bytes	4.9
C extension	Module written in C (or C++/Rust) and compiled, then imported like a normal Python module	17.7
`cached_property`	Decorator computing an attribute once per instance and storing it	12.4
Call site	The place in code where a function is called (as opposed to where it is defined)	7.2
Call stack	The stack of active function frames; its depth is limited for recursion	7.7
`Callable`	Type hint for function values, e.g. `Callable[[int], str]`	13.7
Callable (object)	Anything you can call with `()`: functions, classes, methods, objects with `__call__`	12.1
Callback	Function passed to other code so that code can call it later (e.g. `key=`, an event handler)	7.4
Cancellation	Asking a running asyncio task to stop: `task.cancel()` raises `CancelledError` inside it at its next `await`	16.9
Capacity	Number of slots a dynamic array has allocated, which can exceed its current length	5.3
Capture pattern	A bare name in a `case` pattern that binds the matched value	3.4
CapWords / snake_case	Naming styles: `CapWords` for classes, `snake_case` for variables and functions, `UPPER_CASE` for constants	1.11
Cartesian product	Every combination taking one item from each input (`itertools.product`)	11.5
Catastrophic backtracking	Regex with nested repetition (e.g. `(a+)+`) that takes exponential time on some inputs	4.10
Categorical (dtype)	pandas type storing repeated strings as small integer codes — saves memory	18.11
Chained comparison	`a < b < c` meaning `a < b and b < c`, evaluating `b` once	2.8
ChainMap	`collections` mapping that searches several dicts in order	6.9
CI (continuous integration)	Service that automatically runs tests, linters and type checks on every push (e.g. GitHub Actions)	15.1
Circular import	Two modules importing each other so one sees the other half-initialised	8.6
Class	Blueprint that bundles data (attributes) and behaviour (methods)	10.1
Class attribute	Attribute defined in the class body, shared by all instances	10.1
Class decorator	A decorator applied to a class instead of a function	12.1
`classmethod`	Method receiving the class (`cls`) — used for alternative constructors	10.1
CLI (command-line interface)	A program you run by typing a command and options in a terminal	14.5

Term	Meaning	Where
click / typer	Third-party libraries for building command-line interfaces with decorators or type hints	14.13
Closure	Inner function that remembers variables from its enclosing function	7.4
Code object	Compiled bytecode plus constants and names for one function/module	1.8
Code point	The number Unicode assigns to a character (`ord("A") == 65`)	4.9
Collision (hash)	Two different keys mapping to the same hash-table slot; the table must probe for another slot	6.2
Combinations / permutations	Ways to choose r items ignoring order / respecting order (`itertools.combinations` , `permutations`)	11.5
Compact dict	CPython's dict layout since 3.6: a small index table pointing into a dense, insertion-ordered entries array	6.2
Compiler	Program that translates source code into another form (bytecode or machine code) before it runs	1.8
Comprehension	Expression building a list/dict/set from an iterable: `[f(x) for x in xs if c]`	5.6
Composition	Building objects that *have* other objects instead of inheriting from them	10.8
Concat (pandas)	`pd.concat` stacks DataFrames on top of each other (or side by side) without matching keys	18.8
Concurrency	Making progress on several tasks by interleaving them (not necessarily at the same instant)	16.1
`concurrent.futures`	High-level executors (`ThreadPoolExecutor` , `ProcessPoolExecutor`) returning futures	16.8
Condition (threading)	Synchronisation object letting threads wait until another thread signals that some state has changed	16.4
Conditional expression	One-line `a if cond else b` that evaluates to one of two values	3.3
`conftest.py`	pytest file whose fixtures are shared by tests in its directory	15.4
Constructor	The call that creates an object (`Cls(...)`), running `__new__` then `__init__`	10.1
Container	Object that holds other objects (`list` , `dict` , `set` , `tuple` …); supports `in`	2.4
Context manager	Object with `__enter__` / `__exit__` used by `with` to guarantee setup and cleanup	9.8
`contextmanager`	`contextlib` decorator turning a one- `yield` generator into a context manager	12.3
Contiguous	Stored side by side in one unbroken block of memory (or a range with no gaps)	5.1
Cooperative multiple inheritance	Every class in the hierarchy calls `super()` so each `__init__` /method in the MRO runs exactly once	10.5
Cooperative multitasking	Tasks voluntarily give up control (at `await`), unlike threads, which the OS pre-empts at any moment	16.9
Copy-on-Write	pandas 3 rule: derived objects behave as copies; modify with one `.loc` assignment	18.5
Core (CPU)	One independent processing unit of a processor; several cores can run code truly in parallel	16.1

Term	Meaning	Where
Coroutine	The object returned by calling an `async def` function (a *coroutine function*); the event loop runs it, pausing at each `await`	16.9
Counter	`collections` dict subclass for counting hashable items	6.9
Covariance	A generic type that follows its element's subtyping: a `Sequence[bool]` is accepted where `Sequence[int]` is expected	13.3
Coverage	Measurement of which lines/branches the tests executed	15.6
cProfile	Built-in deterministic profiler reporting time per function	17.6
CPU time	Time the processor actually spent running your code (excludes waiting); compare with wall time	16.2
CPU-bound	Work limited by computation speed rather than waiting	16.2
CPython	The reference Python implementation, written in C	1.8
Critical section	Code touching shared state that only one thread may run at a time	16.4
Cross-cutting concern	Behaviour needed in many unrelated places (logging, timing, retries, auth) — a natural fit for decorators	12.1
CSV	Comma-separated text tables; `csv` module; all values read back as strings	9.11
Cython	Python-like language compiled to C to speed up hot code or wrap C libraries	17.7
Daemon thread	Background thread that is killed abruptly when the main program exits	16.3
DAG (directed acyclic graph)	Directed graph with no cycles — e.g. task dependencies; can be topologically sorted	19.8
Data descriptor	Descriptor defining `__set__` or `__delete__`; it takes priority over the instance `__dict__` (e.g. `property`)	12.4
Data model	Python's rules for how objects plug into syntax and built-ins via dunder methods	10.2
dataclass	`@dataclass` generates `__init__`, `__repr__`, `__eq__` from annotated fields	10.10
DataFrame	pandas 2-D labelled table of columns (each column a Series)	18.4
Deadlock	Threads waiting for each other's locks forever	16.11
Debugger	Tool that pauses a running program so you can step line by line and inspect variables (pdb, IDE debuggers)	15.11
`Decimal`	Exact decimal arithmetic type for money; construct from strings	14.8
Decorator	Callable that takes a function (or class) and returns a replacement; `@d` means `f = d(f)`	12.1
Decorator factory	Function taking arguments that returns a decorator (`@retry(times=3)`), giving three nested levels	12.1
Deep copy	Recursive copy of an object and everything it contains (`copy.deepcopy`)	5.8
`defaultdict`	Dict that creates a default value for a missing key on access	6.9
Deferred import	Import placed inside a function so it runs only when needed — breaks circular imports or speeds start-up	8.6
Dependency	External package your project needs; also any object a piece of code relies on (replaced by doubles in tests)	8.8
Dependency injection	Passing collaborators (clock, DB, HTTP client) in as arguments so tests can supply fakes	15.5

Term	Meaning	Where
deque	Double-ended queue with O(1) appends/pops at both ends	6.9
Descriptor	Object with `__get__` / `__set__` / `__delete__` that controls attribute access on a class	12.4
Deserialize / serialize	Turning stored bytes/text back into objects / turning objects into bytes or text (JSON, pickle)	9.11
DFS (depth-first search)	Traversal that goes as deep as possible before backtracking (recursion or stack)	19.8
dict	Mutable mapping from hashable keys to values; hash table, insertion-ordered	6.1
Dict view	Live, set-like object from `keys()` / `items()` / `values()`	6.4
Dijkstra's algorithm	Shortest paths with non-negative weights using a min-heap	19.8
Django / Flask / FastAPI	Popular Python web frameworks: full-featured / minimal / async and type-hint driven	1.3
Docstring	String literal as the first statement of a module/class/function, shown by `help()`	1.10
doctest	Runs examples written in docstrings as tests	15.7
DST (daylight saving time)	Seasonal clock shift that makes some local times repeat or not exist — why you store UTC	14.1
dtype	NumPy/pandas element type (`int64`, `float64`, `bool`, `category`…)	18.1
Duck typing	Using an object by what it can do, not by its class	10.7
Dummy	Test double passed only to fill a parameter, never actually used	15.5
Dummy head	Placeholder node placed before a linked list's first node so inserts/deletes at the front need no special case	19.7
Dunder method	"Double underscore" special method like `__len__` that Python calls for syntax	10.2
Dutch national flag	One-pass, three-way partition of an array into "less / equal / greater" regions using low/mid/high pointers	19.17
Dynamic array	Array that over-allocates and grows by copying — how `list` works	5.3
Dynamic programming (DP)	Solving overlapping subproblems once and combining their answers	19.10
Dynamic typing	Types belong to objects; names can be rebound to any type	2.3
EAFP	"Easier to ask forgiveness than permission": try, then handle the exception	9.5
Eager evaluation	Computing every result immediately (a list comprehension) — the opposite of lazy	11.4
Editable install	Installing a project so source edits take effect without reinstalling (`pip install -e .`)	8.8
Ellipsis (`...`)	Built-in singleton object, often used as a placeholder body in stubs; also NumPy slicing shorthand	3.8
Encoding / encode / decode	Rule turning text into bytes (UTF-8 is the standard); `str.encode()` text → bytes, `bytes.decode()` bytes → text	4.9
End-to-end (E2E) test	Test of the whole system as a user would exercise it, from input to final output	15.1
Enum	Class of named constant members (`enum.Enum`, `StrEnum`, `auto()`)	10.11
`enumerate`	Built-in yielding `(index, item)` pairs	3.6

Term	Meaning	Where
Environment variable	Named setting provided by the operating system to a process (`os.environ["HOME"]`)	14.6
Epoch	Reference time 1970-01-01 UTC that `time.time()` counts seconds from	14.2
Escape sequence	Backslash code in a string literal standing for a special character (`\n`, `\t`, `é`)	4.5
`eval` / `exec`	Built-ins that run a string as Python code — dangerous on untrusted input	12.8
Event (threading)	Flag threads can wait on until another thread sets it	16.4
Event loop	The asyncio scheduler that runs ready coroutines and waits for I/O	16.9
Exception	Object representing an error, raised to unwind the stack until handled	9.1
Exception chaining	Linking a new exception to its cause with `raise … from …`	9.4
ExceptionGroup	Exception bundling several errors, handled with `except*` (3.11+)	9.6
Exchange argument	Greedy-correctness proof: swapping any optimal solution's choice for the greedy one never makes it worse	19.14
Executor	Pool that runs callables on threads or processes and returns futures	16.8
Exit code	Integer a process returns when it ends: 0 means success, non-zero means failure (`sys.exit(code)`)	14.6
ExitStack	`contextlib` tool managing a dynamic number of context managers	12.3
Expression / statement	An expression produces a value (`x + 1`); a statement performs an action (`if`, `for`, `x = 1`)	1.9
f-string	String literal `f"…{expr:spec}…"` that evaluates and formats expressions	4.6
Fake	Test double with a working lightweight implementation (e.g. in-memory DB)	15.5
Fall-through	In other languages' `switch`, running into the next case; Python's `match` never does this	3.4
Falsy / truthy	Value treated as false in a condition (`None`, `False`, zeros, empty containers) / as true (anything else)	2.6
Fancy indexing	NumPy indexing with an array of positions (`a[[0, 2]]`); returns a copy	18.1
Fast and slow pointers	Two pointers moving at speeds 1 and 2 to find a list's middle or a cycle's entry in O(1) space (Floyd)	19.16
Figure / Axes	matplotlib's whole image / a single plot area within it	18.12
File object	Object returned by `open()` with `read`, `write`, iteration and `close`	9.8
File-like object	Any object with file methods (`read` / `write`), e.g. `io.StringIO`, usable where a file is expected	9.9
`Final`	Type hint marking a name that must not be reassigned	13.10
Finder / loader	Parts of the import system: a finder locates a module, a loader executes it to create the module object	8.2
First-class function	Functions are objects: they can be stored, passed and returned	7.5
Fixture	pytest function providing setup (and teardown) injected into tests by name	15.4
Flaky test	Test that sometimes passes and sometimes fails with no code change (timing, randomness, network)	15.5
float	IEEE-754 double-precision number; approximate for most decimals	2.5
Floor division	`//`: divide and round toward negative infinity	2.5

Term	Meaning	Where
for-else	`else` block that runs when a loop ends without `break`	3.9
Format spec	Mini-language after `:` in f-strings (`>10`, `,.2f`, `%`, `x`)	4.6
Forward reference	Type hint naming a class not yet defined, written as a string (`"Node"`) or resolved lazily	13.2
`Fraction`	Exact rational number type (`fractions.Fraction(1, 3)`)	14.8
Frame	Per-call workspace holding a function's local variables	7.1
Free variable	Variable used in a function but defined in an enclosing function — what a closure captures	7.4
Free-threaded build	Optional CPython build without the GIL (experimental in 3.13, supported in 3.14)	16.6
frozenset	Immutable, hashable set	6.8
Functional programming	Style built from pure functions, immutable data and functions passed as values (`map`, `filter`, `reduce`)	11.7
functools	Module of higher-order tools: `cache`, `lru_cache`, `partial`, `reduce`, `wraps`	7.6
Future	Placeholder for a result that will be available later (`.result()`)	16.8
Garbage collector (cyclic)	CPython component that finds and frees reference cycles	17.2
Generations (GC)	The cyclic garbage collector groups objects by age and scans young ones most often	17.2
Generator / generator object	Iterator returned by calling a function containing `yield`; its body runs only as values are requested, pausing between them	11.2
Generator expression	Lazy comprehension in parentheses: `(x * x for x in xs)`	11.2
Generic type / generic class	Type parameterised by other types (`list[int]`, `dict[str, float]`); a generic class takes a type variable (`class Box[T]`) so `Box[int]` and `Box[str]` are checked separately	13.6
GIL (Global Interpreter Lock)	CPython lock letting one thread run Python bytecode at a time	16.6
Git	Version-control system that records every change to a project's files	1.4
`global`	Statement letting a function rebind a module-level name	7.3
Gotcha	Behaviour that surprises people and causes bugs (mutable defaults, late binding, …)	20.2
GraalPy / MicroPython	Alternative Python implementations: on the Java VM / on tiny microcontrollers	1.8
Gradual typing	Adding type hints incrementally; unannotated code is treated as `Any`	13.1
Graph (nodes, edges; directed / weighted)	Set of nodes (vertices) connected by edges — maps, networks, dependencies; directed edges have a direction, weighted edges carry a cost	19.8
Greedy algorithm	Builds an answer by always taking the locally best choice; correct only when an exchange argument proves it	19.14
groupby (itertools)	Groups *consecutive* items with equal keys — sort first	11.5
groupby (pandas)	Split–apply–combine: aggregate or transform rows per group	18.7
Guard (match)	`if` condition attached to a `case` pattern	3.4
Guard clause	Early `return` / `continue` for special cases to keep code flat	3.2

Term	Meaning	Where
Has-a / is-a	Composition (a car *has an* engine) vs inheritance (a car *is a* vehicle)	10.8
Hash / hash function	Hash function: maps any data to a fixed-size value; `hash()` gives the integer that places a value in a hash table; cryptographic ones (SHA-256) are hard to reverse or collide	6.2
Hash randomisation	str/bytes hashes are salted with a random per-process seed (`PYTHONHASHSEED`), so set order can change between runs	6.2
Hash table	Array of slots indexed by hash — the structure behind dict and set	6.2
Hashable	Has a hash that never changes and consistent `__eq__`; can be a dict key or set item	6.2
hashlib	Standard-library cryptographic hash functions (SHA-256…)	14.9
Heapify	Turning a list into a heap in place in O(n) (`heapq.heapify`)	6.10
Hexadecimal / octal / binary	Base-16 (`0xff`), base-8 (`0o17`) and base-2 (`0b101`) ways of writing integers	2.5
Higher-order function	Function that takes or returns functions (`map`, `sorted(key=…)`, decorators)	7.5
HTTP	Protocol browsers and APIs use to send requests and responses over the web	14.12
httpx / requests	Third-party HTTP client libraries; `httpx` also supports async	14.12
Hypothesis	Library for property-based testing that generates and shrinks inputs	15.9
I/O	Input/output: reading or writing files, network, terminal — usually slow compared with the CPU	16.2
I/O-bound	Work that spends most of its time waiting on network, disk or other I/O	16.2
IDE	Integrated development environment (VS Code, PyCharm)	1.6
Idempotent	Doing it twice has the same effect as doing it once: `f(f(x)) == f(x)`	15.9
Identifier	A name for a variable, function, class, etc.	1.9
Identity	Which object a name refers to; compared with `is`, shown by `id()`	2.2
Idiom	The usual, natural way to express something in a language	20.1
IEEE-754	Standard for floating-point numbers; Python's `float` is its 64-bit "double"	2.5
`iloc` / `loc`	pandas selection by integer position / by label (label slices include the end)	18.5
Immortal object	Object (like `None` or small ints) whose refcount never changes, since 3.12	17.1
Immutable	Cannot change after creation (`int`, `str`, `tuple`, `frozenset`)	2.2
Import system	Machinery that finds, loads, caches (`sys.modules`) and binds modules	8.2
In place	Changing the existing object rather than building a new one (`list.sort()` vs `sorted()`)	5.5
In-degree	Number of edges pointing into a node; Kahn's topological sort starts from nodes with in-degree 0	19.8
Index (pandas)	The row labels of a Series/DataFrame; operations align rows by these labels	18.4
Index (position)	An item's position in a sequence, counting from 0; negative indexes count from the end	4.4
Inheritance	A subclass reuses and extends a base class ("is-a")	10.4

Term	Meaning	Where
Instance	An object created from a class	10.1
Instance attribute	Attribute stored on one object (usually set in `__init__`)	10.1
Instance method	Ordinary method that receives the instance as `self`	10.1
Integer	Whole number; Python's `int` has unlimited size	2.5
Integration test	Test of several components working together	15.1
Interning (strings)	Reusing one object for equal strings (identifier-like literals, `sys.intern`)	17.3
Interpreter	Program that reads and runs code directly; `python` is CPython's interpreter	1.8
Interval	A range with a start and end (meeting times, number ranges)	19.15
Introspection	Examining objects at runtime (`type`, `dir`, `getattr`, `inspect`)	12.8
Invariance	Generic container types don't accept subtypes: `list[bool]` is not `list[int]`	13.3
Invariant	A condition that must always hold at a certain point (e.g. "the heap's smallest item is at index 0")	9.7
Iterable	Object that can return an iterator (`__iter__`): anything you can loop over	11.1
Iterator	Object producing items one at a time with `__next__`; single-pass	11.1
itertools	Standard module of lazy iterator building blocks	11.5
JIT (just-in-time) compiler	Compiles hot code to machine code while the program runs (PyPy, numba; experimental in CPython 3.13+)	17.7
Josephus problem	Classic puzzle: people in a circle, every k-th is removed — who survives?	19.18
JSON	Text data format; `json` module; safe and portable, limited types	9.11
Jupyter	Browser-based notebooks mixing code, output, charts and text; popular for data work	1.6
Kahn's algorithm	Topological sort that repeatedly removes nodes with in-degree 0 using a queue	19.8
KDF (key-derivation function)	Deliberately slow, salted hash for storing passwords (`hashlib.scrypt`, `pbkdf2_hmac`)	14.9
Key function	Function passed as `key=` that maps each item to the value used for sorting/min/max	5.7
Keyword	Reserved word of the language (`if`, `def`, `class`…)	1.9
Keyword argument	Argument passed by name (`f(x=1)`)	7.2
Keyword-only parameter	Parameter after `*` or `*args` that must be passed by name	7.2
`**kwargs`	Parameter collecting extra keyword arguments into a dict	7.2
`lambda`	Anonymous one-expression function	7.5
Late binding	Closures look up variables when called, not when created	7.4
Lazy deletion	Leaving stale heap entries in place and skipping them when they reach the top	19.5
Lazy evaluation	Computing values only when requested (generators, `range`, `map`)	11.4
LBYL	"Look before you leap": check conditions before acting	9.5

Term	Meaning	Where
LEGB	Name lookup order: Local, Enclosing, Global, Built-in	7.3
Library / framework	Library: code you call; framework: code that calls yours (Django, pytest)	1.3
LIFO / FIFO	Last-in-first-out (stack) / first-in-first-out (queue) order	6.9
Linked list	Nodes each pointing to the next; built with small classes in Python	19.7
Linter	Tool that flags bugs and style problems without running code (Ruff, Flake8)	1.6
list	Mutable, ordered sequence of any objects (dynamic array)	5.2
`Literal`	Type hint restricting a value to specific constants	13.10
Literal (value)	A value written directly in code: `42`, `"hi"`, `[1, 2]`, `{"a": 1}`	2.4
Load factor	How full a hash table is; CPython resizes a dict at about 2/3 full to keep lookups fast	6.2
Lock / mutex	"Mutual exclusion" synchronisation primitive: only one holder (thread) at a time may enter a critical section	16.4
Lock file	File pinning exact versions of every dependency for reproducible installs	8.8
Log level	Severity of a log message: DEBUG < INFO < WARNING < ERROR < CRITICAL	14.4
Logger / Handler / Formatter	logging's named source / output destination / message layout	14.4
LogRecord	Object holding one log event (message, level, time, source) that handlers format and output	14.4
Lomuto partition	Simple quicksort partition that sweeps once, moving items smaller than the pivot to the left	19.17
Loop control	`break` (leave the loop), `continue` (next iteration) and `pass` (do nothing)	3.8
Lowest common ancestor (LCA)	The deepest node that is an ancestor of two given nodes in a tree	19.7
`lru_cache`	Memoising decorator with a size limit, evicting least-recently-used entries	7.6
`map` / `filter` / `reduce`	Apply a function to each item / keep items passing a test / fold items into one value	11.7
Mapping	Object that maps keys to values (`dict`, `ChainMap`, `MappingProxyType`)	2.4
`match` statement	Structural pattern matching on a subject's shape and values (3.10+)	3.4
Matrix	Two-dimensional grid of numbers (rows × columns); a 2-D NumPy array or a list of lists	18.3
Max-heap trick	`heapq` is a min-heap only, so push negated values to get the largest item first	19.5
Memoization	Caching a function's results by its arguments	7.7
Memory address	Location of an object in memory; CPython's `id(x)` returns it	17.1
merge (pandas)	SQL-style join of two DataFrames on key columns	18.8
Merge sort	Divide-and-conquer sort: split in half, sort each, merge; O(n log n) and stable	19.17
Metaclass	The class of a class (default `type`); controls class creation	12.6
Method	Function defined in a class and called on its instances	10.1
Mixin	Small class adding one capability through multiple inheritance	10.6

Term	Meaning	Where
Mock	Test double that records calls and returns configured values	15.5
Module	A `.py` file whose names can be imported	8.1
Mojibake	Garbled text produced by decoding bytes with the wrong encoding	4.9
Monkey patching	Replacing attributes of modules/classes at runtime	12.7
Monotonic clock	Clock that never goes backwards (`time.monotonic`, `perf_counter`) — correct for timeouts and durations	14.2
Monotonic stack / deque	Stack (or deque) kept in sorted order: "next greater/smaller" in O(n); the deque version also drops items leaving a window — sliding-window max/min in O(n)	19.4
MRO (method resolution order)	Order Python searches classes for attributes, computed by C3 linearisation	10.5
MST (minimum spanning tree)	Cheapest set of edges connecting all nodes of a weighted graph; Kruskal's algorithm uses union-find	19.9
Multi-paradigm	Supports several programming styles: procedural, object-oriented and functional	1.2
multiprocessing	Running work in separate processes, each with its own interpreter and GIL	16.7
Mutable	Can be changed in place (`list`, `dict`, `set`)	2.2
Mutable default argument	Default like `[]` created once and shared by all calls — use `None`	7.2
Mutation testing	Deliberately changing code (`<` → `<=`) to check that some test fails	15.6
mypy	Static type checker for Python type hints	13.13
mypyc	Compiler that turns type-annotated Python modules into C extensions	17.7
Naive datetime	A `datetime` without time-zone information	14.1
Name mangling	`__attr` inside a class is renamed to `_Class__attr` to avoid subclass clashes	10.9
namedtuple	Tuple subclass with named fields	5.13
Namespace	Mapping from names to objects (module globals, a function's locals…)	7.3
Namespace package	Package directory without `__init__.py`, possibly spread over several folders	8.3
NaN	"Not a number": float value for undefined results; pandas uses it (and `NA`) for missing data; `NaN != NaN`	18.6
Narrowing	A type checker refining a union after a check like `is not None`	13.4
ndarray	NumPy's typed, contiguous, n-dimensional array	18.1
Negative infinity	`float("-inf")`, smaller than every number; `//` rounds toward it	2.5
Nested dictionary	Dict whose values are dicts — a natural fit for JSON-like hierarchical data	6.3
`NewType`	Creates a distinct type for checkers with zero runtime cost	13.10
Nominal typing	Compatibility decided by declared inheritance (class names) rather than by available methods	13.8
Non-data descriptor	Descriptor with only `__get__` (e.g. functions); the instance `__dict__` overrides it	12.4
`None`	The single "no value" object; functions without `return` give it	2.4

Term	Meaning	Where
`nonlocal`	Statement letting an inner function rebind a variable of its enclosing function	7.3
Normalization (Unicode)	Converting equivalent character sequences to one form (NFC/NFD)	4.9
`NotImplemented`	Value an operator dunder returns to say "I can't handle this type" so Python tries the other operand	10.3
numba	JIT compiler that speeds up numeric Python functions with a decorator	17.7
Numeric tower	Ordering `bool` → `int` → `float` → `complex`: mixed arithmetic converts to the wider type	2.7
NumPy / pandas / matplotlib	Libraries for numeric arrays / labelled tables / charts — the core data stack	18.1
Object	Any value in Python: has identity, type and value	17.1
Object-oriented programming (OOP)	Organising code into classes that bundle data with the methods that act on it	10.1
Open addressing	Hash-collision strategy that probes other slots in the same table	6.2
Operand	A value an operator acts on (`2` and `3` in `2 + 3`)	2.8
Operator overloading	Defining dunder methods so operators work on your types	10.3
Operator precedence	Which operators bind first (`*` before `+`); use parentheses when unsure	2.8
Optimal substructure	An optimal answer is built from optimal answers to subproblems — needed for DP and greedy	19.10
OrderedDict	Dict with `move_to_end` and order-sensitive equality; handy for LRU caches	6.9
Overflow	A value exceeding what a fixed-size type can hold; Python `int`s never overflow, floats become `inf`	2.5
Overlapping subproblems	The same smaller problems recur many times — caching them is what makes DP fast	19.10
Override	Redefining an inherited method in a subclass	10.4
Package	Directory of modules importable as a unit	8.3
Parallelism	Literally running several computations at the same instant on multiple cores	16.1
Parameter	Name in a function definition that receives an argument	7.1
`ParamSpec`	Type-hint variable capturing a function's full parameter list (for decorators)	13.7
Parametrize	`@pytest.mark.parametrize`: run one test with many inputs	15.4
Parquet	Columnar, compressed file format that keeps dtypes; fast with pandas	18.4
`partial`	`functools.partial`: new callable with some arguments pre-filled	7.6
Partition	Rearranging items around a pivot: smaller to one side, larger to the other	19.17
PATH	Environment variable listing folders the shell searches for commands such as `python`	1.4
Path compression	Union-find optimisation: point nodes straight at the root while finding it	19.9
pathlib	Object-oriented file-system paths (`Path("a") / "b.txt"`)	9.10
pdb	Python's built-in command-line debugger	15.11
PEP	Python Enhancement Proposal — design documents for the language	1.2

Term	Meaning	Where
PEP 8	Python's style guide	1.11
PEP 20 (Zen of Python)	Aphorisms describing Python's design philosophy (`import this`)	1.11
pickle	Serialises Python objects to bytes; unsafe to load from untrusted sources	9.11
Pin (version)	Fixing a dependency to one exact version (`requests==2.32.3`) for reproducible installs	8.8
pip	The standard package installer (`python -m pip install …`)	8.7
Pipeline	Chain of stages where each one's output feeds the next (often generators)	11.2
pipx	Installs Python command-line tools, each in its own isolated environment	8.7
pivot_table / melt	pandas reshaping: long → wide summary / wide → long	18.9
Placeholder	Marker filled in later: `?` in SQL, `{}` in format strings, `%s` in logging	14.11
Poetry	Tool for managing dependencies, virtual environments and packaging (alternative to uv)	8.8
Positional-only parameter	Parameter before `/` that can't be passed by keyword	7.2
Pre-emptive multitasking	The OS can pause a thread at any moment to run another — why threads need locks	16.10
Prefix sum	Running totals that give any range sum in O(1)	19.2
Priority queue	Queue that always serves the smallest (most urgent) item next — `heapq`	6.10
Process	A running program with its own memory; processes don't share Python objects	16.7
Producer–consumer	Pattern where producers put work on a queue and consumers take it	16.5
Profiling	Measuring where a program spends time or memory	17.6
Propagation (exception)	An unhandled exception travels up the call stack until some caller catches it	9.2
Propagation (logging)	A log record passes up to parent loggers' handlers; can cause duplicate output	14.4
`property`	Decorator turning methods into a managed attribute (getter/setter)	10.1
Property-based testing	Testing general properties over many generated inputs	15.9
`Protocol`	Structural interface for type checkers — match by methods, not inheritance	13.8
Pseudo-random	Numbers from a deterministic algorithm that look random; repeatable from a seed	14.7
Pure function	Result depends only on its arguments and it has no side effects — safe to cache	7.6
py-spy / scalene	Sampling profilers that measure a running program with very low overhead	17.6
pybind11 / PyO3	Tools for writing Python extension modules in C++ / Rust	17.7
Pydantic	Library validating and parsing data at runtime from type hints	13.14
PyObject	The C struct at the start of every CPython object: reference count + type pointer	17.1
PyPI	The Python Package Index, where `pip` downloads packages	8.7
pymalloc	CPython's allocator for small objects	17.2
PyPy	Alternative Python implementation with a JIT compiler	1.8

Term	Meaning	Where
`pyproject.toml`	Standard project file: metadata, dependencies, build system, tool config	8.8
pyright	Fast static type checker (also powers VS Code's Pylance)	13.13
pytest	The de-facto Python testing framework	15.4
Pythonic	Code that follows Python's idioms and style (clear, uses built-ins and comprehensions)	1.11
Queue (`queue.Queue`)	Thread-safe FIFO for passing work between threads; bounded with `maxsize`	16.5
Quickselect	Partition-based search for the k-th smallest/largest element, O(n) average	19.17
Quicksort	Divide-and-conquer sort that partitions around a pivot; O(n log n) average, O(n²) worst	19.17
Race condition	Bug where the result depends on the timing of threads/processes	16.4
Radix tree	Compressed trie where chains of single-child nodes are merged into one edge	19.13
`range`	Lazy, immutable sequence of integers (`range(start, stop, step)`, stop excluded)	3.6
Raw string	`r"…"` literal where backslashes are not escapes	4.5
Re-raise	Using a bare `raise` inside `except` to send the same exception onward	9.3
Recursion (base case, recursive case)	A function calling itself on a smaller input (the recursive case) until it reaches an input where it stops (the base case)	7.7
Recursion limit	Maximum call depth (default 1000), `sys.getrecursionlimit()`	7.7
Reentrant	Can be entered again while already in use (an `RLock`, a reusable context manager)	12.3
Reference count	Number of references to an object; freed when it reaches zero	17.2
Reference cycle	Objects that refer to each other, so reference counts never reach zero; the cyclic GC frees them	17.2
Reflected operator	`__radd__` etc., tried when the left operand doesn't support the operation	10.3
Regression test	Test added so a fixed bug can't silently return	15.1
Regular expression	Pattern language for matching text (`re` module)	4.10
Regular package	Package folder containing `__init__.py`	8.3
Relative import	Import relative to the current package (`from . import x`)	8.4
REPL	Read–eval–print loop: the interactive `>>>` prompt	1.5
`__repr__` / `__str__`	Developer-facing / user-facing text for an object	10.2
resample	pandas time-series grouping by calendar period (`"ME"`, `"W"`, `"D"`)	18.10
reveal_type	Special call that makes a type checker print the type it inferred for an expression	13.4
RLock	Re-entrant lock the same thread may acquire several times	16.11
Ruff	Very fast Rust-based linter and formatter for Python	1.11
Running median	Median kept up to date as numbers stream in, using two heaps	19.5
Salt	Random value mixed into a hash so equal inputs give different hashes	14.9
sdist	Source distribution: a `.tar.gz` of the project source that pip must build on install	8.9

Term	Meaning	Where
`secrets`	Module for cryptographically secure random tokens	14.7
`self`	The first parameter of an instance method — the object the method was called on	10.1
Semaphore	Lock allowing up to n holders at once	16.4
Sentinel	Special marker value, e.g. `None` default or an end-of-queue object	16.5
Series	pandas 1-D labelled array	18.4
set	Mutable, unordered collection of unique hashable items	6.6
Shadowing	A local name hiding an outer or built-in one (`list = [1]` hides the `list` type)	7.3
Shallow copy	New outer container sharing the same inner objects	5.8
Shebang	First line `#!/usr/bin/env python3` telling Unix which interpreter runs a script	1.5
Short-circuit evaluation	`and` / `or` stop as soon as the result is known	2.8
Side effect	Anything a function does besides returning a value: printing, writing files, mutating arguments	7.6
SIMD	"Single instruction, multiple data": CPU instructions processing several numbers at once — part of NumPy's speed	17.7
`singledispatch`	`functools` decorator choosing a function implementation by the first argument's type	7.6
Singleton	Object of which exactly one exists (`None` , `True` , `False` , `Ellipsis`) — compare with `is`	2.2
Slicing	`seq[start:stop:step]` — a new sequence (or view in NumPy)	4.4
Sliding window	Two indices bounding a contiguous range that moves in O(n)	19.2
`__slots__`	Class attribute fixing allowed instance attributes, removing `__dict__`	10.12
Small-int cache	CPython reuses the int objects −5 to 256	17.3
Soft keyword	Word that is a keyword only in context (`match` , `case` , `type` , `_`)	1.9
Space complexity	How much extra memory an algorithm needs as input grows (big-O)	19.1
Spy	Test double that records how it was called, while possibly passing calls through	15.5
SQL	Language for querying relational databases (`SELECT … FROM … WHERE …`)	14.11
SQL injection	Attack where user input is pasted into SQL text and changes the query — prevented by `?` placeholders	14.11
sqlite3	Standard-library interface to the embedded SQLite database	14.11
src layout	Project layout with code under `src/pkg/` so tests use the installed package	8.9
Stable sort	Equal keys keep their original relative order (Timsort is stable)	5.7
Stack	LIFO structure: push and pop at the same end (a Python `list`)	19.4
Standard library	Modules shipped with Python ("batteries included")	8.1
Standard output (stdout)	The stream `print` writes to — normally the terminal; `stderr` is the separate error stream	1.7
Start method	How `multiprocessing` creates children: `fork` , `spawn` or `forkserver`	16.7

Term	Meaning	Where
Static typing	Checking types before the program runs (mypy, pyright) rather than at run time	13.1
`staticmethod`	Method with no implicit first argument; a plain function in the class namespace	10.1
Steering Council	Five-person group, elected by core developers, that governs Python's development	1.2
StopIteration	Exception an iterator raises from `__next__` to signal it has no more items	11.1
str	Immutable sequence of Unicode code points	4.1
Strong typing	No silent conversion between unrelated types (`"1" + 1` fails)	2.3
Structural typing	Compatibility decided by available methods/attributes, not by class name	13.8
Structured concurrency	Tasks started in a scope must finish (or be cancelled) before the scope exits (`asyncio.TaskGroup`)	16.9
Structured logging	Log records as key–value data (often JSON) instead of free text, so tools can search them	14.4
Stub	Test double returning canned answers	15.5
Stub file (`.pyi`)	File with only type signatures for a module	13.13
Subinterpreter	Separate Python interpreter inside the same process, which can have its own GIL (3.12+)	16.6
subprocess	Module for running external commands (`subprocess.run([...])`)	14.6
Substring	A contiguous part of a string	4.3
`super()`	Proxy to the next class in the MRO, used to call overridden methods	10.4
SUT (system under test)	The code a test is exercising	15.1
Sweep line	Turning intervals into +1/−1 events sorted by time and keeping a running count	19.15
Switch interval	How long a thread may hold the GIL before being asked to release it (default 5 ms)	16.6
Syntax error	Code that breaks the language's grammar; reported before any of it runs	9.1
`sys.modules`	Cache of already-imported modules	8.2
`sys.path`	List of directories searched for imports	8.2
t-string	Template string literal `t"…"` (3.14) producing a `Template` object instead of a `str`	4.6
Tail-call optimisation	Reusing the frame when a function's last action is a call; CPython doesn't do it	7.7
Task (asyncio)	Wrapper that schedules a coroutine on the event loop to run concurrently (`create_task`)	16.9
TaskGroup	asyncio structured concurrency: runs tasks and cancels siblings on failure (3.11+)	16.9
TDD (test-driven development)	Red → green → refactor: write a failing test first	15.8
tempfile	Module creating temporary files/directories that clean themselves up	9.10
Terminal / shell	Text window where you type commands (Terminal, PowerShell) / the program interpreting them (bash, zsh)	1.5

Term	Meaning	Where
Test double	Stand-in for a real dependency in tests: dummy, stub, spy, mock, fake	15.5
Test suite	All the tests of a project, run together	15.1
TestPyPI	Separate practice copy of PyPI for trying uploads safely	8.9
Text mode	Default file mode: reads/writes `str`, decoding/encoding with the given encoding	9.9
Thread	Separate flow of execution sharing the process's memory	16.3
Thread-local data	Value where each thread sees its own copy (`threading.local()`)	16.3
`timedelta`	`datetime` type for a duration (`timedelta(days=1)`)	14.1
timeit	Module for timing small snippets accurately	17.6
Timsort	Python's stable, adaptive merge/insertion sort, O(n log n)	5.7
Token	Smallest meaningful piece of source code (name, number, operator); also a secret string such as an API key	1.8
TOML	Simple configuration file format used by `pyproject.toml`; read with `tomllib`	8.8
Top-down / bottom-up DP (tabulation)	Recursion with memoisation / tabulation: iteratively filling a table from the smallest subproblems upwards	19.10
Top-k	Finding the k largest/smallest items, typically with a size-k heap in O(n log k)	19.5
Topological sort	Ordering of a DAG's nodes so every edge points forward	19.8
tottime / cumtime	cProfile columns: time inside a function itself / including everything it calls	17.6
tracemalloc	Standard module tracing memory allocations by line	17.6
Traceback	Report of the call stack when an exception is not handled	15.10
Transaction	Group of database changes applied all together or not at all (commit / rollback)	14.11
Transpose	Flipping a matrix over its diagonal so rows become columns (`a.T`, `zip(*m)`)	19.18
Trie	Prefix tree storing strings letter by letter; insert, search and prefix checks are O(word length)	19.13
True division	`/`, which always returns a float (`7 / 2 == 3.5`)	2.5
Trusted Publishing	Uploading to PyPI from CI with short-lived credentials instead of a stored token	8.9
try / except / else / finally	Exception-handling statement; `else` on success, `finally` always	9.2
tuple	Immutable ordered sequence; hashable if its items are	5.9
Tuple packing	Commas create a tuple: `t = 1, 2`	5.9
Two pointers	Two indices moving through data to avoid nested loops	19.2
Two's complement	How computers store negative integers in binary; Python mimics it for bitwise ops (`~x == -x - 1`)	19.12
Type alias	Name for a type expression (`type Vector = list[float]`, 3.12)	13.12

Term	Meaning	Where
Type checker	Tool that reads type hints and reports mismatches without running code (mypy, pyright)	13.1
Type code	One-letter code fixing the element type of an `array.array` (`"i"`, `"d"`)	5.1
Type conversion	Changing a value's type (`int("3")`); implicit only within the numeric tower	2.7
Type inference	A checker working out a type itself from the code (`x = 1` → `int`)	13.2
`type: ignore`	Comment silencing a type-checker error on one line	13.13
TypedDict	Type hint for dicts with a fixed set of typed keys	13.9
typeshed	Collection of stub files describing the standard library and popular packages for type checkers	13.13
TypeVar	Placeholder type for generic functions/classes	13.6
ufunc	NumPy universal function operating element-wise in C (`np.sqrt`, `+`)	18.1
Unicode	Standard assigning a number (code point) to every character	4.9
Union by size	Union-find optimisation: attach the smaller tree under the larger one's root	19.9
Union type / `Optional[X]`	`A | B`: a value that may be either type; `Optional[X]` means `X | None`	13.4
Union-Find (DSU)	Structure tracking merging groups with near-O(1) find/union	19.9
Unit test	Fast test of one small piece in isolation	15.1
unittest	Standard-library xUnit-style testing framework	15.3
`unittest.mock`	Standard library for mocks and `patch`	15.5
Unpacking / star-unpacking	Assigning an iterable's items to several names (`a, b = xs`); `*` gathers "the rest" (`first, *rest = xs`) or spreads an iterable (`f(*xs)`, `[*a, *b]`)	5.10
URL	Web address (`https://host/path?query`); parse with `urllib.parse`	14.12
UTC	Coordinated Universal Time — the time-zone-free reference for storing timestamps	14.1
UTF-8	Variable-length Unicode encoding, 1–4 bytes per character; the default to use	4.9
uuid	Module generating universally unique identifiers	14.9
uv	Fast tool managing Python versions, venvs, dependencies and lock files	8.8
Validation	Checking that input has the expected shape and values before using it (pydantic at boundaries)	13.14
Variable	A name bound to an object; in Python it is a label, not a box	2.1
Vectorisation	Operating on whole arrays in compiled code instead of Python loops	18.1
Version specifier	Rule for allowed versions: `==2.1`, `>=2.1`, `<3`, `~=2.1`	8.8
View (NumPy)	Array sharing memory with another (e.g. a slice)	18.1
Virtual environment (venv)	Isolated per-project interpreter environment and package folder	8.7
Virtual machine (CPython)	The bytecode interpreter loop that executes compiled code	1.8
Wall time	Real elapsed clock time, including waiting; compare with CPU time	16.2
Walrus operator	`:=` assigns inside an expression (3.8+)	2.8

Term	Meaning	Where
Warning	Non-fatal message from the `warnings` module (e.g. `DeprecationWarning`) — not an exception by default	9.2
Weak reference	Reference that doesn't keep an object alive (`weakref`)	17.2
Wheel	Pre-built package archive (`.whl`) that installs without building	8.9
Wide / long format	Table with one column per variable or period / one row per observation; convert with `pivot_table` and `melt`	18.9
Wildcard pattern	`case _:` — matches anything without binding	3.4
`with` statement	Runs a block inside a context manager's setup and guaranteed cleanup	9.8
Worker	Thread or process in a pool that takes jobs and runs them	16.8
Wrapper	Inner function a decorator returns; it runs extra code around the original call	12.1
`yield`	Produces a value from a generator and pauses it	11.2
`yield from`	Delegates to a sub-generator/iterator	11.3
`zip`	Pairs items from several iterables, stopping at the shortest (`strict=True` to check)	3.6
zoneinfo	Standard-library IANA time zones (`ZoneInfo("Asia/Kolkata")`)	14.1

Chapter TL;DR

- Use the sheets in 21.1–21.19 as a quick reminder; follow the § references for full explanations, examples and pitfalls.
- 21.20 maps interview problem signals to patterns and lists the gotchas that cost the most points.
- 21.21 answers "what should I reach for?" in one table.
- The glossary defines 570+ terms in plain words and points to where each is taught.

Chapter 22 — Interview Question Bank

> **What this chapter is.** Python interview questions, worded the way interviewers and the popular public lists word them, each with a short answer you can say out loud and a link to the place in this book that explains it in full. It collects the questions that come up again and again in interview-experience posts, the big GitHub question collections, widely shared prep lists and data-team interview guides. Chapter 20 already answers about 45 core questions; this bank links to those answers and adds every common question Chapter 20 does not cover, including "what does this print?" puzzles, the small coding tasks Python rounds use, and NumPy/pandas basics. Versions: Python 3.14 (answers also hold for 3.12+ unless a version is named), numpy 2.x, pandas 3.0.

How to use this bank

- **Wording.** Each question is phrased the way it is usually asked. Several phrasings of one idea are merged into one question.
- **★ = asked very often.** The question appears in many of the sources listed at the end. If you have little time, drill only the ★ questions first (there are 100).
- **Answers.** Each answer is 2–6 sentences, or a short code snippet plus a sentence or two: what you would actually say in an interview. Terms are defined the first time they appear.
- **→ More.** Points to the chapter and section that explains the topic fully. "Ch 20 Q*n*" means question *n* of the Q&A in Chapter 20 (§20.3), and "gotcha *n*" means gotcha *n* in §20.2. A question with no "More" line is answered only here.
- **How to drill.** Cover the answer, say yours out loud in under a minute, then check. Mark the ones you missed and repeat them the next day. For the puzzles (Section 13), write down the output *before* reading the answer. For the coding tasks (Section 14), type the solution from memory and run it.

Contents: 1. Basics and how Python runs · 2. Data types, mutability and strings · 3. Collections · 4. Functions, arguments, scope and closures · 5. OOP, dunder methods and the MRO · 6. Iterators, generators and comprehensions · 7. Decorators, context managers and metaprogramming · 8. Exceptions and files · 9. Modules, packaging, environments and type hints · 10. Memory model, internals and performance · 11. Concurrency: GIL, free-threading, threads, processes, asyncio · 12. Standard library, testing and debugging · 13. "What does this print?" puzzles · 14. Common coding tasks in Python rounds · 15. NumPy and pandas basics.

---

1. Basics and how Python runs

Q1. ★ What is Python, and what are its key features? Python is a high-level (it hides memory management and hardware details), general-purpose programming language created by Guido van Rossum and first released in 1991. Key features: readable syntax that uses indentation for blocks; dynamic, strong typing; automatic memory management; multiple paradigms (procedural, object-oriented, functional); a large standard library ("batteries included") plus the PyPI package index; and easy integration with C/C++/Rust code. It is used for web back ends, automation and scripting, data analysis, machine learning and testing. Its main costs are slower raw execution than compiled languages and limited thread parallelism in the default build (the GIL, Q197). → *More:* Ch 1 §1.2–1.3 — What Python is, where it is used

Q2. ★ Is Python compiled or interpreted? How is Python code executed? Both, in stages. CPython (the standard implementation, written in C) parses your source into an AST (abstract syntax tree, a tree form of the code), compiles that into **bytecode** (instructions for Python's own virtual machine), then its interpreter loop executes the bytecode. Nothing is compiled ahead of time to machine code by default; an experimental JIT (just-in-time compiler) exists since 3.13 but is off unless you enable it. You can see the bytecode with `dis.dis(func)`. → *More:* Ch 20 Q1 · Ch 1 §1.8 — How CPython runs your code

Q3. What is a `.pyc` file and `__pycache__`? Why does Python create `.pyc` files if it is interpreted? A `.pyc` file is the cached bytecode of an imported module, stored in a `__pycache__` folder and named with the interpreter version (for example `utils.cpython-314.pyc`). On the next import, Python checks whether the source changed (by timestamp or hash) and, if not, loads the bytecode directly and skips the compile step, so start-up is faster. The script you run directly (`python app.py`) is compiled every time and not cached. Deleting `__pycache__` is always safe. → *More:* Ch 1 §1.8 · Ch 8 §8.2 — How `import` really works

Q4. What are CPython, PyPy, Jython and MicroPython? When would you use an alternative implementation? Python is a language specification; an *implementation* is a program that runs it. **CPython** is the reference implementation and what "Python" usually means. **PyPy** has a tracing JIT and is often several times faster on long-running pure-Python loops, but C-extension support is weaker. **MicroPython** runs on microcontrollers. **Jython** (JVM) and **IronPython** (.NET) exist but lag behind modern Python 3. Pick an alternative only for a specific need (speed of pure-Python code, embedded hardware); most libraries target CPython. → *More:* Ch 1 §1.2 · Ch 17 §17.7

Q5. ⋆ What is PEP 8? What is PEP 20? A PEP (Python Enhancement Proposal) is a design document for the language. **PEP 8** is the style guide: 4-space indentation, `snake_case` for functions and variables, `PascalCase` for classes, `UPPER_CASE` for constants, lines of about 79 characters (many teams use 88–100), imports grouped at the top, two blank lines between top-level definitions. **PEP 20**, "The Zen of Python" (`import this`), lists the design values, such as "Readability counts" and "Explicit is better than implicit". In practice, formatters and linters (black, ruff) enforce PEP 8 automatically. → *More:* Ch 1 §1.11 — PEP 8 and PEP 20

Q6. ⋆ What does "dynamically typed" and "strongly typed" mean? What is duck typing? *Dynamic*: a variable has no fixed type; the object it refers to has one, and types are checked when the code runs. *Strong*: Python does not silently convert between unrelated types, so `"1" + 1` raises `TypeError`. **Duck typing** ("if it walks like a duck and quacks like a duck…") means code relies on what an object can *do* rather than what class it is: any object with a `read()` method can be passed where a file is expected. → *More:* Ch 20 Q2 · Ch 2 §2.3 · Ch 10 §10.7

Q7. ⋆ What does "everything in Python is an object" mean? Every value — numbers, strings, functions, classes, modules, even `None` — is an object with an identity (`id()`), a type (`type()`) and a value. So functions can be stored in lists and passed as arguments, classes can be created and modified at runtime, and you can call methods on literals (`(5).bit_length()`). Variables are just names that refer to objects. → *More:* Ch 17 §17.1 — The CPython object model · Ch 2 §2.1

Q8. Why is indentation important in Python? Is Python case-sensitive? Indentation is syntax: it defines blocks (the body of `if`, `for`, `def`, `class`) instead of braces, so wrong indentation causes `IndentationError` or, worse, a different program. Use 4 spaces and never mix tabs and spaces. Python is case-sensitive: `count`, `Count` and `COUNT` are three different names. → *More:* Ch 1 §1.9 — Basic syntax

Q9. What is the difference between an expression and a statement? An **expression** produces a value (`2 + 3`, `f(x)`, `a if c else b`, a lambda body). A **statement** performs an action and has no value (`if`, `for`, `def`, `import`, `return`, assignment). That is why a `lambda` can contain only an expression. The walrus operator `:=` is an *assignment expression*: it assigns and yields the value, so it can appear inside an expression. → *More:* Ch 1 §1.9 · Ch 2 §2.8

Q10. What is the difference between `pass`, `continue` and `break`? `pass` does nothing; it is a placeholder where a statement is required (an empty function or class). `continue` skips the rest of the current loop iteration and goes to the next one. `break` leaves the innermost loop immediately (and skips the loop's `else` clause). → *More:* Ch 3 §3.8 — Loop control

Q11. What does `else` on a `for` or `while` loop do? The `else` block runs only when the loop finishes normally, that is, without hitting `break`. It is the clean way to write "search, and if not found do X" without a flag variable:

```
for n in numbers:
    if n < 0:
        print("found a negative:", n)
        break
else:
    print("no negatives")
```

→ *More:* Ch 3 §3.9 — for-else and while-else

Q12. What is the walrus operator `:=`? An assignment expression (Python 3.8+, PEP 572) that assigns a value to a name *and* returns it, so you can test and keep a value in one step: `while (line := f.readline()):` or `if (m := pattern.search(s)):`. It also avoids computing something twice in a comprehension: `[y for x in data if (y := f(x)) > 0]`. Use it where it removes duplication, not everywhere. → *More:* Ch 2 §2.8 · Ch 3

Q13. What is `match`/`case`? How is it different from a `switch` statement? Structural pattern matching (3.10+) compares a value against *patterns*, not just constants. It can destructure sequences (`case [x, y]:`), mappings (`case {"type": "click", "pos": (x, y)}:`), class instances (`case Point(x=0):`), combine alternatives (`case 401 | 403:`) and add guards (`case [x, *_] if x > 0:`). There is no fall-through, and `case _:` is the default. A bare name in a pattern *captures* (binds) the value rather than comparing with a variable; use dotted names (`Color.RED`) for constants. → *More:* Ch 3 §3.4 — match/case

Q14. What is a ternary (conditional) expression in Python? `value_if_true if condition else value_if_false`, for example `label = "even" if n % 2 == 0 else "odd"`. It is an expression, so it can be used inside function calls and comprehensions. Avoid nesting more than one. → *More:* Ch 3 §3.3

Q15. What is a docstring? How is it different from a comment? A docstring is a string literal written as the first statement of a module, class or function. Python stores it in `__doc__`, and `help()`, IDEs and documentation generators read it. A comment (`#`) is thrown away by the compiler and is for readers of the source only. Docstrings say *what* the code does and how to use it; comments explain *why* something non-obvious is done. → *More:* Ch 1 §1.10 · Ch 7 §7.8

Q16. What do `help()`, `dir()`, `type()` and `id()` do? `help(obj)` shows its documentation. `dir(obj)` lists its attribute names. `type(obj)` returns its class. `id(obj)` returns a number unique to the object while it is alive (in CPython, its memory address). Together with `vars()`, `getattr()` and the `inspect` module they are Python's **introspection** tools (examining objects at runtime). → *More:* Ch 12 §12.8 — Introspection

Q17. What is new in recent Python versions (3.12, 3.13, 3.14)? **3.12**: type-parameter syntax `def first[T](xs: list[T]) -> T` and the `type` alias statement; f-strings can reuse the same quotes and nest freely; better error messages. **3.13**: a new interactive REPL; an optional free-threaded (no-GIL) build and an experimental JIT, both off by default. **3.14**: the free-threaded build becomes officially supported but is still optional (PEP 779); annotations are evaluated lazily (PEP 649/749, new `annotationlib`); template strings `t"..."` (PEP 750); `except A, B:` without parentheses (PEP 758); a `SyntaxWarning` for `return`/`break`/`continue` leaving a `finally` block (PEP 765); multiple interpreters in the standard library (`concurrent.interpreters`, PEP 734); Zstandard compression (`compression.zstd`); `forkserver` as the default multiprocessing start method on Linux. → *More:* Ch 20 Q9 · Ch 16 §16.6–16.7 · Ch 13 §13.2

Q18. What are the main differences between Python 2 and Python 3? Still asked, though Python 2 reached end of life in January 2020. In Python 3: `print` is a function; `str` is Unicode text and `bytes` is separate; `/` is true division (`3 / 2 == 1.5`) and `//` is floor division; `range`, `map`, `zip` and `dict.keys()` return lazy objects or views instead of lists; there is a single `int` type with unlimited size; and exception syntax is `except E as e`. The answer interviewers want: "Python 2 is dead; all new code is Python 3."

Q19. What are Python's main limitations? When would you not choose Python? Pure-Python code is slow for CPU-heavy loops (often 10–100× slower than C, Java or Go); the GIL limits thread parallelism in the default build; memory use per object is high; dynamic typing lets some bugs reach runtime; and it is a poor fit for mobile apps, browsers and hard real-time systems. Mitigations: NumPy and other C-backed libraries, multiprocessing, type checkers, and rewriting hot spots in C, Cython or Rust. → *More:* Ch 17 §17.7

---

2. Data types, mutability and strings

Q20. What are the built-in data types in Python? Numbers: `int` (unlimited size), `float` (64-bit IEEE 754 double), `complex`; `bool` (a subclass of `int`); text: `str`; binary: `bytes`, `bytearray`, `memoryview`; sequences: `list`, `tuple`, `range`; mappings: `dict`; sets: `set`, `frozenset`; and `NoneType` (whose only

value is `None`). Immutable ones: `int`, `float`, `complex`, `bool`, `str`, `bytes`, `tuple`, `frozenset`, `range`. → *More:* Ch 2 §2.4 — Built-in data types

Q21. ⋆ What is the difference between mutable and immutable objects? A mutable object can be changed in place after it is created (`list`, `dict`, `set`, `bytearray`, most user classes). An immutable one cannot (`int`, `str`, `tuple`, `frozenset`); an operation that seems to change it builds a new object and rebinds the name. Consequences: only immutable (hashable) values can be dict keys or set members; mutable objects shared between names change for everyone; and immutable objects are safe to share between threads. → *More:* Ch 20 Q3 · Ch 2 §2.2

Q22. What happens when you write `b = a`? Are variables boxes? No copy is made. A variable is a *name* bound to an object; `b = a` binds a second name to the *same* object. If that object is mutable, a change through `b` is visible through `a`. To get an independent object, copy it explicitly (`a.copy()`, `list(a)`, `copy.deepcopy(a)`).

```
a = [1, 2]
b = a
b.append(3)
print(a)        # [1, 2, 3]
```

→ *More:* Ch 2 §2.1 · Ch 5 §5.8

Q23. ⋆ What is the difference between `is` and `==`? `==` compares *values* by calling `__eq__`. `is` compares *identity*: whether both names refer to the same object. Use `is` only for singletons such as `None` (`if x is None`), and `==` for everything else. `[] == []` is `True` but `[] is []` is `False`. → *More:* Ch 20 Q5 and gotchas 3–4 · Ch 2 §2.2

Q24. What is `None`? How is it different from `0`, `False` and `""`? `None` is the single instance of `NoneType` and means "no value" (a function without `return` returns it). `0`, `False` and `""` are real values of other types that happen to be *falsy*. All four are falsy, but only `None is None`. When 0 or an empty string is a valid value, test `if x is None`, not `if not x`. → *More:* Ch 2 §2.6

Q25. Which values are falsy in Python? `None`, `False`, numeric zeros (`0`, `0.0`, `0j`, `Decimal(0)`), and empty containers (`""`, `[]`, `()`, `{}`, `set()`, `range(0)`). Everything else is truthy by default. A class controls its truthiness with `__bool__`, or with `__len__` (zero length means falsy). → *More:* Ch 2 §2.6 — Booleans and truthiness

Q26. How big can an `int` be? How are floats stored? `int` has arbitrary precision: it grows as needed, so `2 ** 1000` is exact (but arithmetic on huge numbers gets slower; and since 3.11, converting an int with more than 4300 digits to `str` raises `ValueError` by default, a guard against denial-of-service attacks, adjustable with `sys.set_int_max_str_digits`). `float` is a 64-bit IEEE 754 binary double with about 15–17 significant decimal digits, so many decimal fractions cannot be stored exactly. `sys.float_info` shows the limits. → *More:* Ch 2 §2.5 — Numbers

Q27. ⋆ Why is `0.1 + 0.2 == 0.3` false? How do you compare or handle money? `0.1` and `0.2` have no exact binary representation, so their sum is `0.30000000000000004`. Compare floats with a tolerance: `math.isclose(a, b)`. For money use `decimal.Decimal` built from strings (`Decimal("0.10")`) or store integer cents. For exact rationals use `fractions.Fraction`. → *More:* Ch 20 gotcha 10 · Ch 14 §14.8

Q28. What is the difference between `/`, `//` and `%`? What about negative numbers? `/` is true division and always returns a `float` (`7 / 2 == 3.5`, `4 / 2 == 2.0`). `//` is floor division: it rounds *down* toward minus infinity (`7 // 2 == 3`, `-7 // 2 == -4`). `%` is the remainder, and its sign follows the divisor, so that `a == (a // b) * b + a % b` always holds (`-7 % 2 == 1`). `divmod(a, b)` returns both. `int(-3.5)` truncates toward zero and gives `-3`. → *More:* Ch 2 §2.8 — Operators

Q29. How does `round()` work? Why is `round(2.5) == 2`? `round` uses "round half to even" (banker's rounding): exact halves go to the nearest even number, so `round(2.5) == 2` and `round(3.5) == 4`. This avoids a bias when rounding many values. Also, `round(2.675, 2)` gives `2.67` because 2.675 is stored as slightly less than that. Use `Decimal.quantize` with `ROUND_HALF_UP` when a business rule demands "school" rounding. → *More:* Ch 2 §2.5

Q30. What do `and` and `or` return? What is short-circuit evaluation? They return one of their *operands*, not necessarily a `bool`. `a or b` returns `a` if it is truthy, otherwise `b`; `a and b` returns `a` if it is falsy, otherwise `b`. Evaluation stops as soon as the result is known (short-circuit), so `x and x.name`

is safe when `x` is `None`. Idiom: `name = user_input or "anonymous"` (careful: it also replaces `0` and `""`). → *More:* Ch 2 §2.8

Q31. How do chained comparisons like `a < b < c` work? `a < b < c` means `a < b and b < c`, with `b` evaluated only once. This works for any comparison operators, which produces surprises such as `1 == 1.0 == True` being `True`, and `x < y > z` being legal. → *More:* Ch 2 §2.8

Q32. How do you convert between types? What is implicit vs explicit conversion? Explicit conversion uses constructors: `int("42")`, `float("3.5")`, `str(10)`, `list("abc")`, `int("ff", 16)`, `int(3.9) == 3`. `int("3.5")` raises `ValueError` (go through `float` first). Implicit conversion happens only between numeric types (`1 + 2.0` gives `3.0`, `True + 1` gives `2`); Python never converts strings to numbers automatically. → *More:* Ch 2 §2.7 — Type conversion

Q33. ⋆ Are strings mutable? How do you "change" a character in a string? No. `s[0] = "x"` raises `TypeError`. Build a new string: `s = "x" + s[1:]`, `s.replace(old, new)`, or convert to a list, edit, and `"".join(...)` it back. Immutability makes strings hashable (usable as dict keys) and safe to share. → *More:* Ch 4 §4.2 — Immutability

Q34. ⋆ What is the best way to concatenate many strings? `"".join(parts)`, which computes the final size once and copies each piece once (linear time). Repeated `s += piece` in a loop may copy the growing string each time (quadratic in the worst case; CPython sometimes optimises it in place, but don't rely on that). For incremental building, append pieces to a list and join once at the end, or use `io.StringIO`. → *More:* Ch 4 §4.8 — Joining strings efficiently

Q35. ⋆ What are the ways to format strings? What are f-strings, and what are t-strings in 3.14? Three classic ways: f-strings `f"{name} is {age}"` (3.6+, fastest and most readable; support format specs such as `{x:.2f}`, `{n:,}`, `{s:>10}` and the debug form `{x=}`), `str.format()`, and the old `%` operator (still used by `logging`). **t-strings** (`t"..."`, 3.14, PEP 750) look like f-strings but produce a `string.templatelib.Template` object holding the literal parts and the interpolated values separately, so a library can escape or process the values (for example to build safe HTML or SQL) before producing the final text. → *More:* Ch 4 §4.6 — String formatting

Q36. ⋆ What is slicing? How do you reverse a string or list? `seq[start:stop:step]` returns a new sequence from `start` up to but not including `stop`; any part can be omitted, and negative indexes count from the end (`s[-1]` is the last item). `s[::-1]` reverses. Slicing never raises `IndexError` for out-of-range bounds (`[1, 2][5:]` is `[]`). On lists, slicing creates a shallow copy; on NumPy arrays, it creates a *view* (Q290). → *More:* Ch 4 §4.4 — Slicing · Ch 5 §5.5

Q37. What is the difference between `str` and `bytes`? What are encoding and decoding? `str` is a sequence of Unicode code points (abstract characters); `bytes` is a sequence of integers 0–255 (raw data). **Encoding** turns text into bytes (`"é".encode("utf-8") == b"\xc3\xa9"`), **decoding** turns bytes back into text. Files, sockets and hashes work with bytes. Always name the encoding explicitly (`open(path, encoding="utf-8")`); a wrong guess causes `UnicodeDecodeError` or garbled text. → *More:* Ch 4 §4.9 — Unicode vs bytes

Q38. What is a raw string (`r"..."`)? A raw string treats backslashes as ordinary characters, so `r"\n"` is two characters, not a newline. Use raw strings for regular expressions (`r"\d+"`) and Windows paths. A raw string cannot end with a single backslash. → *More:* Ch 4 §4.5 — Escape sequences and raw strings

Q39. What is the difference between `find()` and `index()`? Between `split()`, `rsplit()` and `partition()`? `find` returns `-1` when the substring is missing; `index` raises `ValueError`. `split(sep)` returns a list of all pieces (with no argument it splits on runs of whitespace and drops empty strings); `rsplit(sep, maxsplit)` splits from the right; `partition(sep)` always returns a 3-tuple `(before, sep, after)`, which is handy for "key=value" parsing. → *More:* Ch 4 §4.7 — String methods

Q40. What is the difference between `strip("abc")` and `removeprefix("abc")`? `strip`, `lstrip` and `rstrip` take a *set of characters* and remove any of them from the ends: `"cabbage".lstrip("abc")` gives `"ge"`. `removeprefix` and `removesuffix` (3.9+) remove an exact substring once: `"cabbage".removeprefix("cab")` gives `"bage"`. Using `lstrip` to remove a prefix is a classic bug. → *More:* Ch 4 §4.7

Q41. What is the difference between `isdigit()`, `isdecimal()`, `isnumeric()` and `isalnum()`? All test every character and return `False` for an empty string. `isdecimal` is the strictest: only characters usable to write base-10 numbers (`"123"`). `isdigit` also accepts digits like superscript `"²"`. `isnumeric` also accepts numeric characters such as `"½"` and Chinese numerals. `isalnum` is true if each character is a

letter or numeric. To check that a string parses as an int, the robust way is `try: int(s)`. → *More:* Ch 4 §4.7

Q42. What is the difference between `re.match`, `re.search`, `re.fullmatch` and `re.findall`? `match` anchors at the start of the string; `search` finds the first match anywhere; `fullmatch` requires the whole string to match; `findall` returns all non-overlapping matches as strings (or tuples of groups), and `finditer` yields match objects lazily. `re.sub` replaces, `re.split` splits. Compile patterns you reuse with `re.compile`, and use raw strings. → *More:* Ch 4 §4.10 · Ch 14 §14.3 — re, deeper

Q43. What is the difference between greedy and non-greedy matching in regex? Quantifiers `*`, `+`, `?` and `{m,n}` are greedy: they match as much as possible and then back off. Adding `?` (`*?`, `+?`) makes them lazy: as little as possible. On `"<a><b>"`, `<.*>` matches the whole string, while `<.*?>` matches `"<a>"`. → *More:* Ch 14 §14.3

Q44. What is string interning? Interning keeps one shared copy of certain strings so that identical strings are the same object, making dict lookups by those strings faster (an identity check succeeds before comparing characters). CPython interns identifiers and many compile-time constant strings automatically; `sys.intern(s)` does it explicitly. Never rely on it for correctness: compare strings with `==`. → *More:* Ch 17 §17.3 — Small-int caching and string interning · Ch 20 gotcha 4

---

3. Collections: lists, tuples, dicts, sets and `collections`

Q45. ⋆ What is the difference between a list and a tuple? When do you use each? A list is mutable and is used for a variable-length collection of similar items; a tuple is immutable and is used for a fixed-size record whose positions have meaning (`(lat, lon)`, a function returning several values). Tuples are hashable when their items are (so they can be dict keys), slightly smaller, faster to create, and signal "this will not change". → *More:* Ch 20 Q10 · Ch 5 §5.9

Q46. ⋆ How is a Python list implemented? What is the time complexity of its operations? A dynamic array of pointers to objects that over-allocates spare capacity. Index read/write, `len`, `append` and `pop()` from the end are O(1) (append is *amortised* O(1): occasionally the array grows and copies, but the average cost stays constant). `insert(0, x)`, `pop(0)`, `remove(x)`, `x in lst` and `index(x)` are O(n). Sorting is O(n log n). → *More:* Ch 20 Q11 · Ch 5 §5.3 — List internals and big-O

Q47. What is the difference between `append()`, `extend()` and `insert()`? Between `+` and `+=` on lists? `append(x)` adds `x` as one item; `extend(iterable)` adds each item of the iterable; `insert(i, x)` puts `x` before index `i` (O(n)). `a + b` builds a new list; `a += b` extends `a` *in place* (it calls `__iadd__`), so other names bound to the same list see the change. `a += "hi"` extends with `'h', 'i'`, while `a = a + "hi"` raises `TypeError`. → *More:* Ch 5 §5.4 — Adding and removing items

Q48. What is the difference between `del`, `remove()`, `pop()` and `clear()`? `del lst[i]` (or a slice) deletes by position and returns nothing; `lst.remove(x)` deletes the first item equal to `x` and raises `ValueError` if absent; `lst.pop(i=-1)` deletes by position and *returns* the item; `lst.clear()` empties the list in place. `del name` unbinds the name itself, not the object. → *More:* Ch 5 §5.4

Q49. ⋆ What is the difference between `sort()` and `sorted()`? How do you sort by multiple keys or in descending order? `lst.sort()` sorts the list in place and returns `None`; `sorted(iterable)` returns a new list and accepts any iterable. Both use Timsort, are O(n log n) and **stable** (equal items keep their order). Use `key=` to sort by a computed value and `reverse=True` for descending. For several keys, return a tuple: `sorted(people, key=lambda p: (-p.age, p.name))` sorts by age descending, then name ascending. Stability also lets you sort in two passes, least significant key first. → *More:* Ch 5 §5.7 — Sorting · Ch 19 §19.6

Q50. What is unpacking? What does `first, *rest = items` do? Unpacking assigns the items of any iterable to several names at once: `a, b = b, a` swaps; `x, y = point`. A starred name collects the leftovers into a list: `first, *rest = [1, 2, 3]` gives `first == 1`, `rest == [2, 3]`; `*_, last = items` takes the last item. The count must match, or you get `ValueError`. `*` and `**` also unpack inside literals: `[*a, *b]`, `{**d1, **d2}`. → *More:* Ch 5 §5.10 — Unpacking

Q51. ⋆ How does a dictionary work internally? What is its time complexity? A dict is a hash table: `hash(key)` chooses a slot, and collisions are resolved by open addressing (probing other slots). CPython uses a compact layout, an index table pointing into a dense array of entries, which saves

memory and keeps insertion order. Get, set, delete and `in` are O(1) on average and O(n) in the worst case (many collisions). The table is resized when it gets about two-thirds full. → *More:* Ch 20 Q12 · Ch 6 §6.2 — Hashing

Q52. ⋆ What can be used as a dictionary key? Why can't a list be a key? Any **hashable** object: one whose `__hash__` never changes during its lifetime and is consistent with `__eq__` (equal objects have equal hashes). Built-in immutables qualify (`int`, `str`, `tuple` of hashables, `frozenset`). A list is mutable: if its contents changed after insertion, its hash would change and the dict could never find it again, so lists define `__hash__ = None`. Note that `1`, `1.0` and `True` are equal and hash the same, so they are the *same* key. → *More:* Ch 20 Q13 · Ch 6 §6.2

Q53. Are dictionaries ordered? Yes. Since Python 3.7 the language guarantees that dicts keep insertion order (CPython 3.6 already did as an implementation detail). Updating an existing key keeps its position; deleting and re-inserting moves it to the end. `OrderedDict` is still useful for `move_to_end`, `popitem(last=False)` and order-sensitive equality. → *More:* Ch 6 §6.1 · Ch 6 §6.9

Q54. ⋆ How do you merge two dictionaries? `merged = d1 | d2` (3.9+) returns a new dict; `d1 |= d2` or `d1.update(d2)` updates in place; `{**d1, **d2}` works on older versions. For duplicate keys the right-hand value wins. These are shallow merges: nested dicts are not merged recursively (write a small recursive function for that, Q270). → *More:* Ch 6 §6.4 — Dictionary methods

Q55. What is the difference between `d[key]`, `d.get(key)`, `d.setdefault(key, default)` and `defaultdict`? `d[key]` raises `KeyError` if the key is missing. `d.get(key, default)` returns a default without changing the dict. `d.setdefault(key, default)` returns the existing value or inserts and returns the default. `collections.defaultdict(list)` calls the factory automatically on a missing key, ideal for grouping: `groups[k].append(v)`. A dict subclass can also define `__missing__` to decide what happens on a missing key (that is how `defaultdict` works). → *More:* Ch 6 §6.4 · Ch 6 §6.9 · Ch 20 gotcha 7

Q56. What are dictionary views (`keys()`, `values()`, `items()`)? They are live, lazy views of the dict, not copies: they reflect later changes and take no extra memory. `keys()` and `items()` support set operations (`d1.keys() & d2.keys()` gives the common keys). Changing the dict's size while iterating over a view raises `RuntimeError`; iterate over `list(d)` if you need to delete. → *More:* Ch 6 §6.4 — view objects

Q57. How do you invert a dictionary or create one from two lists? From two lists: `dict(zip(keys, values))` (add `strict=True` to fail on unequal lengths). Inverting: `{v: k for k, v in d.items()}`; if values repeat, later keys overwrite earlier ones, so group instead: `inv = defaultdict(list); for k, v in d.items(): inv[v].append(k)`. To give many keys the same value: `dict.fromkeys(keys, 0)` (never with a mutable default, see gotcha 7). → *More:* Ch 6 §6.5 — Dict and set comprehensions

Q58. ⋆ What is a set? Why is `in` faster on a set than on a list? A set is an unordered collection of unique hashable items, implemented as a hash table (a dict with keys only). `x in s` hashes `x` and checks one slot (O(1) on average), while `x in lst` scans every item (O(n)). Sets also remove duplicates (`set(items)`) and support algebra: union `|`, intersection `&`, difference `-`, symmetric difference `^`, and subset `<=`. → *More:* Ch 20 Q14 · Ch 6 §6.6–6.7

Q59. What is a `frozenset`? Can a set contain a list or another set? A `frozenset` is an immutable, hashable set, so it can be a dict key or an element of another set. A set cannot contain a list or a normal set (both are unhashable, `TypeError: unhashable type`), but it can contain tuples and frozensets. Note that `{}` is an empty *dict*; an empty set is `set()`. → *More:* Ch 6 §6.8 — frozenset

Q60. ⋆ What is the difference between a shallow copy and a deep copy? What are the ways to copy a list? A shallow copy creates a new outer container but shares the inner objects; a deep copy recursively copies everything. Shallow: `lst[:]`, `lst.copy()`, `list(lst)`, `copy.copy(lst)`. Deep: `copy.deepcopy(lst)`, which also handles cycles. With nested lists, changing an inner list through a shallow copy changes the original too. → *More:* Ch 20 Q15 and gotcha 6 · Ch 5 §5.8 — Copying

Q61. ⋆ What is in the `collections` module? `Counter` (counts hashable items, `most_common(k)`), `defaultdict` (default values for missing keys), `deque` (double-ended queue), `namedtuple` (tuple with named fields), `OrderedDict` (order-aware dict with `move_to_end`), `ChainMap` (searches several dicts as one, e.g. defaults under overrides) and `UserDict` / `UserList` / `UserString` (base classes that are easier to subclass than the built-ins). → *More:* Ch 20 Q17 · Ch 6 §6.9 — The collections module

Q62. Why use `deque` instead of a list for a queue? `list.pop(0)` and `insert(0, x)` shift every element, O(n). `collections.deque` is a doubly linked list of fixed-size blocks, so `append`, `appendleft`, `pop` and `popleft` are all O(1). `deque(maxlen=n)` keeps only the last *n* items (a sliding window or recent history). Indexing into the middle of a deque is O(n), so use it for ends, not random access. → *More:* Ch 6 §6.9 · Ch 19 §19.4 — Stacks and queues

Q63. How do you use `heapq`? How do you get a max-heap? `heapq` keeps a plain list as a binary min-heap: `heappush(h, x)` and `heappop(h)` are O(log n), `h[0]` is the smallest item, and `heapify(lst)` is O(n). For a max-heap, push negated numbers (`-x`) or tuples with negated priorities (Python 3.14 also adds public max-heap functions such as `heapq.heappush_max`, but the negation trick is what interviewers expect). `heapq.nlargest(k, xs)` and `nsmallest` find the top *k* in O(n log k). Push `(priority, counter, item)` tuples so that ties never compare the items themselves. → *More:* Ch 6 §6.10 — heapq · Ch 19 §19.5 — Heaps

Q64. What does the `bisect` module do? It does binary search on a sorted list: `bisect_left(a, x)` returns the leftmost position where `x` could be inserted to keep order, `bisect_right` the rightmost; `insort(a, x)` inserts in order (O(log n) to find the spot, but O(n) to shift elements). Since 3.10 they accept `key=`. Typical uses: grade boundaries, counting items in a range, maintaining a sorted list. → *More:* Ch 6 §6.11 — bisect · Ch 19 §19.6

Q65. What is a `namedtuple`? How does it compare with a dataclass, a dict and `TypedDict`? `namedtuple("Point", "x y")` (or `typing.NamedTuple`) creates a tuple subclass with named fields: immutable, lightweight, unpackable and indexable. A `@dataclass` is a normal class with generated methods: mutable by default, supports defaults, methods and `frozen=True`. A dict is flexible but has no fixed schema. `TypedDict` only describes the expected keys of a dict for type checkers; at runtime it is a plain dict. Use namedtuple for small immutable records, dataclass for domain objects, dicts for JSON-like data. → *More:* Ch 5 §5.13 — namedtuple · Ch 10 §10.10 · Ch 13 §13.9

Q66. What is the difference between `array.array`, `list` and a NumPy array? A `list` holds pointers to any objects. `array.array("d")` stores raw C numbers of one type compactly but has no vectorised maths. A NumPy `ndarray` stores a typed contiguous block *and* provides fast vectorised operations, broadcasting and many dimensions. Use lists for general data, NumPy for numeric work. → *More:* Ch 20 Q16 · Ch 5 §5.14

Q67. What is the time complexity of common operations on built-in types?

Operation	list	dict / set	deque
index `x[i]`	O(1)	—	O(n)
`x in c`	O(n)	O(1) avg	O(n)
add at end	O(1) amortised	O(1) avg	O(1)
add/remove at front	O(n)	—	O(1)
delete by key/value	O(n)	O(1) avg	O(n)
sort	O(n log n)	—	—
Also: `len()` is O(1) for all; `min` / `max` / `sum` are O(n); slicing `lst[a:b]` is O(b−a); string concatenation in a loop is O(n²) in the worst case.			
→ *More:* Ch 17 §17.5 — Big-O of built-in operations · Ch 19 §19.1			

Q68. Which data structure would you choose for: unique items, counting, a FIFO queue, a priority queue, an LRU cache, fast lookups by key? Unique items → `set`; counting → `Counter`; FIFO queue → `deque` (or `queue.Queue` between threads); priority queue → `heapq` (or `queue.PriorityQueue` between threads); LRU cache → `functools.lru_cache` for function results, or `OrderedDict` with `move_to_end` for a hand-made one (Q273); lookups by key → `dict`; sorted container with fast inserts → the third-party `sortedcontainers` package. → *More:* Ch 6 §6.12 — When to use what

---

4. Functions, arguments, scope and closures

Q69. ⋆ What are `*args` and `**kwargs`? In a function definition, `*args` collects extra positional arguments into a tuple and `**kwargs` collects extra keyword arguments into a dict (the names are just a convention). At a call site, `*seq` and `**mapping` unpack values into arguments. Typical use: wrappers and decorators that pass arguments through unchanged, `def wrapper(*args, **kwargs): return func(*args, **kwargs)`. → *More:* Ch 20 Q18 · Ch 7 §7.2 — Function arguments

Q70. ⋆ Does Python pass arguments by value or by reference? By object reference ("call by sharing"): the parameter becomes a new name for the caller's object. If the function *mutates* a mutable object (`lst.append(1)`), the caller sees it; if it *rebinds* the parameter (`lst = []`), the caller does not. With immutable objects such as `int` and `str`, a function therefore can never change the caller's value. → *More:* Ch 20 Q4 · Ch 7 §7.2

Q71. ⋆ Why shouldn't you use a mutable default argument such as `def f(x, items=[])`? Default values are evaluated **once**, when the `def` statement runs, and the same object is reused on every call. So appending to `items` accumulates data across calls. Fix: use `None` as the default and create the list inside: `if items is None: items = []`. The same applies to `{}`, `set()` and objects like `datetime.now()` (evaluated once at definition time). → *More:* Ch 20 gotcha 1 · Ch 7 §7.2

Q72. What do `/` and `*` mean in a function signature? What is the order of parameter kinds? Parameters before `/` are positional-only (cannot be passed by name); parameters after `*` (or after `*args`) are keyword-only (must be passed as `name=value`). Full order: positional-only, `/`, normal, `*args` or `*`, keyword-only, `**kwargs`. Positional-only protects parameter names you may want to rename later (many built-ins such as `len(obj, /)` use it); keyword-only forces readable calls (`sorted(xs, key=..., reverse=True)`).

```
def f(a, b, /, c, *, d, **rest): ...
f(1, 2, 3, d=4)          # ok
f(1, b=2, c=3, d=4)      # TypeError: missing positional argument 'b'
                         # (b is positional-only, so b=2 just lands in **rest)
```

→ *More:* Ch 7 §7.2

Q73. Can a parameter without a default follow one with a default? Not among positional parameters: `def f(a=1, b):` is a `SyntaxError`, because a positional argument could not tell which slot to fill. It is allowed for keyword-only parameters: `def f(a=1, *, b):` is fine since `b` must be passed by name. → *More:* Ch 7 §7.2

Q74. ⋆ What is a lambda function? What are its limitations? `lambda args: expression` creates a small anonymous function whose body is a single expression (no statements, no annotations). It is handy for short callbacks: `sorted(words, key=lambda w: w.lower())`. Do not assign a lambda to a name (use `def`, which gives a real name in tracebacks), and prefer `operator.itemgetter` / `attrgetter` or a named function when the logic grows. → *More:* Ch 20 Q24 · Ch 7 §7.5

Q75. What are first-class and higher-order functions? Functions are *first-class*: they are objects that can be assigned to variables, stored in data structures, passed as arguments and returned from other functions. A *higher-order function* takes or returns a function: `map`, `filter`, `sorted(key=...)`, `functools.reduce` and every decorator. This enables strategies as plain functions, callbacks and dispatch tables (`handlers = {"add": add, "sub": sub}`). → *More:* Ch 7 §7.5 — First-class and higher-order functions

Q76. ⋆ Explain `map()`, `filter()` and `reduce()`. Should you use them or comprehensions? `map(f, xs)` lazily applies `f` to each item; `filter(pred, xs)` lazily keeps items where `pred` is truthy; `functools.reduce(f, xs, init)` folds a sequence into one value (`reduce(operator.mul, xs, 1)` is a product). In Python 3, `map` and `filter` return iterators. Comprehensions are usually clearer, especially if you would need a `lambda`; `map(str, xs)` with an existing function is fine. For common reductions use `sum`, `min`, `max`, `any`, `all`, `math.prod`. → *More:* Ch 20 Q25 · Ch 11 §11.7

Q77. ⋆ What is a namespace? What is the LEGB rule? A namespace is a mapping from names to objects; each module, function call and class body has its own, plus the built-ins namespace. When Python looks up a name it searches **L**ocal (current function), **E**nclosing (outer functions, for nested functions), **G**lobal (the module), then **B**uilt-ins (`len`, `print`), and raises `NameError` if none has it. Class bodies are *not*

an enclosing scope for methods: inside a method you must write `self.x` or `ClassName.x`. → *More:* Ch 20 Q20 · Ch 7 §7.3 — Scope and namespaces

Q78. What do `global` and `nonlocal` do? Assigning to a name inside a function normally creates a local variable. `global x` makes assignments in the function rebind the module-level `x`; `nonlocal x` makes them rebind `x` in the nearest enclosing function (needed for counters in closures). Reading an outer name needs neither keyword. Mutating an outer object (`cache.append(1)`) also needs neither; only rebinding does. Prefer returning values over `global`. → *More:* Ch 7 §7.3

Q79. Why does this raise `UnboundLocalError`?

```
count = 0
def increment():
    count += 1      # UnboundLocalError
```

The compiler sees an assignment to `count` inside the function and marks `count` as local for the *whole* function body. `count += 1` then reads the local before it has a value. Fix: `global count` (or better, pass and return the value, or use a class or closure with `nonlocal`). → *More:* Ch 7 §7.3

Q80. ⋆ What is a closure? Give a use case. A closure is an inner function that remembers variables from the enclosing function's scope after the outer function has returned. Those variables live in *cells* (`inner.__closure__`). Uses: function factories (`make_multiplier(3)`), decorators, callbacks with configuration, and simple private state.

```
def make_counter():
    n = 0
    def inc():
        nonlocal n
        n += 1
        return n
    return inc
c = make_counter(); c(); print(c())   # 2
```

→ *More:* Ch 20 Q19 · Ch 7 §7.4 — Closures

Q81. ⋆ What is late binding in closures? Why does `[lambda: i for i in range(3)]` give 2, 2, 2? Closures look up a variable's value when they are *called*, not when they are created. All three lambdas share the same `i`, which is 2 when the loop ends. Fix: bind the current value as a default argument, `lambda i=i: i`, or use `functools.partial`. → *More:* Ch 20 gotcha 2 · Ch 7 §7.4

Q82. What is `functools.partial`? `partial(func, *args, **kwargs)` returns a new callable with some arguments pre-filled: `int_from_binary = partial(int, base=2)`, then `int_from_binary("101") == 5`. Unlike a lambda it is picklable (when `func` is), inspectable (`.func`, `.args`, `.keywords`) and does not suffer from late binding. Common with callbacks and `map`. → *More:* Ch 7 §7.6

Q83. How does a function return multiple values? It returns one tuple, which the caller unpacks: `return total, count` then `t, c = f()`. For more than two or three values, return a `NamedTuple` or dataclass so the fields have names. A function with no `return` (or a bare `return`) returns `None`. → *More:* Ch 7 §7.1

Q84. Does Python support function overloading? Not by signature: a second `def` with the same name simply replaces the first. Alternatives: default and keyword arguments, `*args`, `functools.singledispatch` (chooses an implementation by the type of the first argument; `singledispatchmethod` for methods), or `@classmethod` alternative constructors. `typing.overload` only describes several signatures to type checkers; you still write one implementation. → *More:* Ch 7 §7.6 · Ch 13

Q85. What is recursion in Python? What is the recursion limit? Does Python optimise tail calls? A recursive function calls itself on a smaller input until it reaches a base case. CPython limits the call depth (default 1000; `sys.getrecursionlimit()` / `setrecursionlimit()`) and raises `RecursionError` beyond it. Python does **not** do tail-call optimisation (reusing the stack frame for a call in tail position), by design, to keep full tracebacks. For deep recursion, convert to a loop with an explicit stack, or memoise with `@functools.cache` to cut repeated calls. → *More:* Ch 7 §7.7 — Recursion · Ch 19 §19.10

Q86. What are function annotations and docstrings used for at runtime? Annotations (`def f(x: int) -> str`) are metadata stored for tools; Python does not enforce them. In 3.14 they are evaluated lazily, on first access via `f.__annotations__` or `annotationlib.get_annotations(f)`. Docstrings are stored in

`f.__doc__`. Frameworks read both: FastAPI builds request validation from annotations, and `help()` shows docstrings. → *More:* Ch 7 §7.8 · Ch 13 §13.14

Q87. What does the single underscore `_` mean in Python? By convention: a throwaway variable (`for _ in range(3)`, `_, ext = os.path.splitext(p)`); a leading underscore marks a name as internal (`_helper`; also skipped by `from m import *`); in the interactive interpreter `_` holds the last result; in `match` statements `case _:` is the wildcard; and `1_000_000` uses underscores as digit separators. A trailing underscore avoids clashing with a keyword (`class_`). → *More:* Ch 10 §10.9

---

5. OOP, dunder methods and the MRO

Q88. ⋆ What is a class and an object? What is `self`? A class is a blueprint that defines attributes and methods; an object (instance) is one concrete thing built from it, with its own state. `self` is the instance the method was called on. Python passes it explicitly as the first parameter: `obj.method(1)` is just `Class.method(obj, 1)`. The name `self` is a convention, not a keyword. → *More:* Ch 10 §10.1 — Classes and instances

Q89. ⋆ What are the four pillars of OOP, and how does Python support them? Encapsulation (bundling data with the methods that use it; Python relies on naming conventions and properties rather than access modifiers), **abstraction** (exposing a simple interface; ABCs and Protocols), **inheritance** (a class reuses and extends another; Python allows multiple inheritance), and **polymorphism** (the same call works on different types; achieved by method overriding and, above all, duck typing). → *More:* Ch 10

Q90. ⋆ What is the difference between `__init__` and `__new__`? Which is called first when you create an object? `MyClass(args)` calls `type.__call__`, which first calls `__new__(cls, ...)` to **create** the instance, then, if `__new__` returned an instance of the class, calls `__init__(self, ...)` to **initialise** it. `__init__` must return `None`. Override `__new__` only for subclasses of immutable types (`int`, `str`, `tuple`), singletons/instance caches, or metaclass work. → *More:* Ch 20 Q26 · Ch 10 §10.1

Q91. ⋆ What is the difference between a class variable and an instance variable? A class variable is defined in the class body and shared by all instances; an instance variable is set on `self` (usually in `__init__`) and belongs to one object. Reading `obj.x` looks in the instance first, then the class. Assigning `obj.x = ...` always creates an instance attribute that *shadows* the class one. Danger: a mutable class variable (`items = []`) is shared, so appending through one instance changes it for all. → *More:* Ch 20 gotcha 8 · Ch 10 §10.1

Q92. ⋆ What is the difference between an instance method, `@classmethod` and `@staticmethod`? An instance method receives `self` and works with one object. A class method receives `cls` (the class it was called on, including subclasses), so it is the natural tool for alternative constructors (`Date.from_string("2026-10-14")`). A static method receives nothing automatically; it is a plain function placed in the class namespace because it logically belongs there. → *More:* Ch 20 Q27 · Ch 10 §10.1

Q93. ⋆ What does `@property` do? Why not write getters and setters? `@property` turns a method into an attribute that is computed on access; `@x.setter` and `@x.deleter` add validation on assignment and deletion. You can start with a plain public attribute and later turn it into a property without changing any calling code, so Java-style `get_x()`/`set_x()` methods are unnecessary. `functools.cached_property` computes once and stores the result on the instance. → *More:* Ch 20 Q28 · Ch 10 · Ch 12 §12.2

Q94. ⋆ How do you make a variable private in Python? What is name mangling? Python has no true private members. Convention: `_name` means "internal, don't touch from outside". A double leading underscore `__name` (without trailing underscores) triggers **name mangling**: inside class `Account`, `self.__balance` is stored as `_Account__balance`. Its purpose is to avoid accidental clashes with names in subclasses, not security; the attribute is still reachable by its mangled name. → *More:* Ch 10 §10.9 — Encapsulation conventions

Q95. What is inheritance? What types of inheritance does Python support? Inheritance lets a subclass reuse and extend a base class: `class Dog(Animal):`. Python supports single, multilevel (A → B → C), hierarchical (several children of one parent), multiple (`class C(A, B)`) and therefore hybrid/diamond inheritance. Every class ultimately inherits from `object`. A subclass overrides a method by defining it again and calls the parent's version with `super()`. → *More:* Ch 10 §10.4 — Inheritance and super()

Q96. ⋆ What does `super()` do? Why is it better than calling `Parent.method(self)`? `super()` returns a proxy that finds the *next* class in the method resolution order (MRO) of the instance's type, not simply "my parent". When every class in a hierarchy calls `super().__init__(**kwargs)`, each class's method runs exactly once, even in diamond hierarchies (cooperative multiple inheritance). Hard-coding `Parent.__init__(self)` skips classes or calls the shared base twice. → *More:* Ch 10 §10.4–10.5

Q97. ⋆ What is the MRO? How does Python resolve the diamond problem? The method resolution order is the list of classes Python searches for an attribute, available as `Class.__mro__` or `Class.mro()`. Python computes it with **C3 linearisation**: a child comes before its parents, the left-to-right order of bases is kept, and each class appears once. In the diamond `class D(B, C)` with `B(A)` and `C(A)`, the MRO is `D, B, C, A, object`, so `A` is visited once, after both `B` and `C`. → *More:* Ch 20 Q29 · Ch 10 §10.5 — Multiple inheritance and the MRO

Q98. What happens if C3 cannot build a consistent MRO? Class creation fails with `TypeError: Cannot create a consistent method resolution order (MRO)`. Example: `class X(A, B)` and `class Y(B, A)`, then `class Z(X, Y)`: X requires A before B and Y requires B before A, which cannot both hold. Fix the base order, or replace some inheritance with composition. → *More:* Ch 10 §10.5

Q99. ⋆ What is the difference between `__str__` and `__repr__`? `__repr__` is the developer-facing representation: unambiguous, ideally valid Python that recreates the object (`Point(x=1, y=2)`); the REPL, debuggers and containers (`print([p])`) use it. `__str__` is the user-facing text used by `print()`, `str()` and f-strings, and falls back to `__repr__` if not defined. Always define `__repr__`; add `__str__` only if users need a different text. → *More:* Ch 20 Q30 · Ch 10 §10.2

Q100. ⋆ What are dunder (magic) methods? Name the common ones. "Dunder" (double underscore) methods are hooks Python calls to implement syntax and built-ins: the **data model**. Construction: `__new__`, `__init__`; representation: `__repr__`, `__str__`, `__format__`; comparison: `__eq__`, `__lt__`, `__hash__`; arithmetic: `__add__`, `__radd__`, `__iadd__`; containers: `__len__`, `__getitem__`, `__setitem__`, `__contains__`, `__iter__`; truthiness: `__bool__`; calling: `__call__`; context managers: `__enter__`, `__exit__`; attribute access: `__getattr__`, `__setattr__`. → *More:* Ch 20 Q31 · Ch 10 §10.2 — The data model

Q101. How do you overload operators in Python? What is `NotImplemented`? Define the matching dunder method: `__add__` for `+`, `__mul__` for `*`, `__eq__` for `==`, `__lt__` for `<`, `__getitem__` for `[]`. If your method cannot handle the other operand's type, **return** `NotImplemented` (a special singleton, not the exception `NotImplementedError`); Python then tries the reflected method on the other operand (`__radd__`), and raises `TypeError` only if both decline. Define `__iadd__` to make `+=` mutate in place.

```
class Vec:
    def __init__(self, x, y): self.x, self.y = x, y
    def __add__(self, other):
        if not isinstance(other, Vec):
            return NotImplemented
        return Vec(self.x + other.x, self.y + other.y)
```

→ *More:* Ch 10 §10.3 — Operator overloading

Q102. If you override `__eq__`, what happens to `__hash__`? How do you make objects usable in sets? Defining `__eq__` without `__hash__` sets `__hash__` to `None`, making instances unhashable, because the default identity-based hash would break the rule "equal objects must have equal hashes". To use objects in sets and as dict keys, define `__hash__` from the same immutable fields used in `__eq__` (for example `hash((self.x, self.y))`), or use `@dataclass(frozen=True)`, which generates both. → *More:* Ch 20 Q13 · Ch 6 §6.2 · Ch 10 §10.10

Q103. How can you implement all comparison operators with little code? Define `__eq__` and one ordering method (say `__lt__`) and decorate the class with `functools.total_ordering`, which fills in `__le__`, `__gt__` and `__ge__`. Alternatively, `@dataclass(order=True)` generates all of them by comparing fields as tuples in declaration order. → *More:* Ch 10 §10.3 · Ch 12 §12.2

Q104. What does `__call__` do? It makes instances callable like functions: `obj(args)` runs `obj.__call__(args)`. Useful for objects that need state between calls (a counter, a rate limiter, a configured model), for class-based decorators, and for strategy objects. `callable(x)` checks whether something can be called. → *More:* Ch 10 §10.2 · Ch 12 §12.1

Q105. How do you make your own class behave like a container (support `len`, indexing, `in` and iteration)? Implement `__len__`, `__getitem__` (and `__setitem__`/`__delitem__` for mutability), `__contains__` and `__iter__`. With only `__getitem__` accepting integers from 0, Python can already iterate and test `in` (the legacy sequence protocol). The cleanest route is to inherit from `collections.abc.Sequence` or `MutableMapping`, implement the few abstract methods, and get the rest (`index`, `count`, `get`, `items`...) as mixin methods. → *More:* Ch 10 §10.2 · Ch 10 §10.7

Q106. What is polymorphism? What is the difference between method overloading and overriding? Polymorphism means one interface, many implementations: `shape.area()` works for any shape. **Overriding** is a subclass redefining an inherited method; Python fully supports it. **Overloading** (several methods with the same name but different parameter lists) is not supported directly: the last definition wins. Use default arguments or `singledispatch` instead (Q84). → *More:* Ch 10 §10.4

Q107. ⋆ What is an abstract base class? How do you create one? An ABC defines an interface that subclasses must implement. Inherit from `abc.ABC` and mark methods with `@abstractmethod`; trying to instantiate a class that still has abstract methods raises `TypeError`. `collections.abc` provides ready-made ABCs (`Iterable`, `Mapping`, `Sequence`) that also work with `isinstance` checks.

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self) -> float: ...
```

→ *More:* Ch 20 Q33 · Ch 10 §10.7 — ABCs, duck typing and Protocols

Q108. What is the difference between an ABC and a `Protocol`? An ABC is **nominal**: a class conforms only if it inherits from (or is registered with) the ABC, and the check happens at runtime when instantiating. A `typing.Protocol` is **structural**: any class with the right methods conforms, without inheriting, and the check is done by a static type checker (mypy, pyright). Protocols fit duck typing and third-party classes you can't modify; ABCs fit when you want shared code and enforced implementation. → *More:* Ch 20 Q33 · Ch 13 §13.8 — Protocol

Q109. ⋆ What is a dataclass? What does it generate, and what are its main options? `@dataclass` reads the class's annotated fields and generates `__init__`, `__repr__` and `__eq__`. Options: `frozen=True` (immutable and hashable), `order=True` (comparison methods), `slots=True` (uses `__slots__`), `kw_only=True` (keyword-only constructor). Use `field(default_factory=list)` for mutable defaults and `__post_init__` for validation. Since 3.13, `copy.replace(obj, x=1)` makes a modified copy of a dataclass instance. → *More:* Ch 20 Q34 · Ch 10 §10.10 — dataclasses

Q110. When would you use a dataclass, a Pydantic model or a plain class? A dataclass for internal data records where types are trusted (no runtime validation). A Pydantic model where data comes from outside (JSON from an API, config files): it validates and converts types at runtime and serialises back to JSON. A plain class when behaviour (methods, invariants) matters more than the data fields. → *More:* Ch 13 §13.14 — Runtime vs static, and pydantic

Q111. ⋆ What is `__slots__`? When would you use it? `__slots__ = ("x", "y")` replaces the per-instance `__dict__` with fixed storage for just those attributes. This saves memory (noticeable with millions of objects) and speeds up attribute access slightly, but you cannot add other attributes, and multiple inheritance with slots needs care. `@dataclass(slots=True)` generates it for you. → *More:* Ch 20 Q32 · Ch 10 §10.12 — **slots**

Q112. Composition vs inheritance: which do you prefer and why? Inheritance models an "is-a" relationship and couples the subclass tightly to the base class's internals; composition models "has-a" (an object holds other objects and delegates to them) and keeps parts replaceable and testable. The usual advice is "prefer composition over inheritance": inherit for genuine subtype relationships or framework hooks, compose everything else. → *More:* Ch 10 §10.8 — Composition vs inheritance

Q113. What is a mixin? A mixin is a small class that adds one capability (for example `JsonMixin` adding `to_json()`, or `LoggingMixin`) and is not meant to be instantiated alone. You combine it with a main base class through multiple inheritance: `class User(JsonMixin, Model):`. Mixins should be stateless or cooperative (call `super()`), and are listed before the main base. → *More:* Ch 10 §10.6 — Mixins

Q114. ⋆ How do you implement a singleton in Python? The simplest is a **module**: modules are imported once and cached, so a module-level object is a natural singleton. Other options: override `__new__` to return

a stored instance, a class decorator that caches the instance, or a metaclass. Interviewers also like the caveat: singletons are global state, which makes testing harder; dependency injection is often better.

```
class Config:
    _instance = None
    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
        return cls._instance
```

→ *More:* Ch 10 §10.13 — Pythonic design patterns

Q115. What is an `Enum` and why use it instead of constants? `enum.Enum` defines a fixed set of named members (`Color.RED`) with `.name` and `.value`. Members are singletons, iterable, compared by identity, and type checkers can verify you handled every case. Variants: `IntEnum` / `StrEnum` (also behave as int or str), `Flag` (combinable bit flags), and `auto()` for automatic values. They prevent typos and invalid values that plain string constants allow. → *More:* Ch 10 §10.11 — Enums

Q116. What is the difference between `isinstance()` and `type() ==` ? `isinstance(obj, Base)` is true for instances of `Base` *and its subclasses* (and for ABC registrations), and accepts a tuple or a union of types (`isinstance(x, int | float)`). `type(obj) is Base` matches only the exact class. Prefer `isinstance` (or better, duck typing); `issubclass` does the same check for classes. → *More:* Ch 10 §10.7

Q117. What is the difference between `__getattr__` and `__getattribute__` ? `__getattribute__` runs on **every** attribute access; overriding it is rarely needed and easily causes infinite recursion (call `object.__getattribute__(self, name)` inside). `__getattr__` runs only when normal lookup **fails**, which makes it the safe hook for proxies, delegation and lazy attributes. `__setattr__` and `__delattr__` intercept assignment and deletion. → *More:* Ch 12 §12.5 — **getattr** vs **getattribute**

Q118. What is a descriptor? An object that defines `__get__` (and optionally `__set__` / `__delete__`) and is stored as a class attribute; accessing that attribute through an instance calls those methods. A *data* descriptor (has `__set__`) takes priority over the instance `__dict__`; a *non-data* one does not. `property`, `classmethod`, `staticmethod` and ordinary functions (which become bound methods through `__get__`) are all descriptors, as are ORM fields. `__set_name__` tells a descriptor its attribute name. → *More:* Ch 20 Q35 · Ch 12 §12.4 — Descriptors

Q119. ⋆ What is a metaclass? When would you use one? A metaclass is the class of a class: it controls how classes are created (the default is `type`, so `type(int) is type`). `type(name, bases, namespace)` even creates classes dynamically. Uses: frameworks that register or transform classes at definition time (Django models, `ABCMeta`, `EnumType`). For most needs, a class decorator or `__init_subclass__` (a hook called whenever a subclass is defined) is simpler and preferred. "If you wonder whether you need a metaclass, you don't." → *More:* Ch 20 Q35 · Ch 12 §12.6 — Metaclasses and **init_subclass**

Q120. How do you access a parent class attribute that is overridden in the child? Use `super().method()` or `super().attr` inside the class; from outside, call it explicitly on the parent: `Parent.method(obj)`. For class attributes, `Parent.attr`. With multiple inheritance, remember `super()` follows the MRO, so it may reach a sibling class rather than the class you named in the bases. → *More:* Ch 10 §10.4

---

6. Iterators, generators and comprehensions

Q121. ⋆ What is the difference between an iterable, an iterator and a generator? An **iterable** can produce an iterator: it has `__iter__` (lists, strings, dicts, files, `range`). An **iterator** produces items one at a time with `__next__`, raises `StopIteration` when done, returns itself from `__iter__`, and can be consumed only once. A **generator** is an iterator created by a function containing `yield` (or by a generator expression); Python writes `__iter__` / `__next__` for you and saves the function's state between items. → *More:* Ch 20 Q22 · Ch 11 §11.1 — The iteration protocol

Q122. How does a `for` loop work internally? `for x in obj:` calls `iter(obj)` once to get an iterator, then repeatedly calls `next()` on it and binds the result to `x`, until the iterator raises `StopIteration`, which the loop catches silently and ends (running the `else` block, if any). This is why any object

implementing the protocol works in `for`, comprehensions, `list()`, `sum()` and unpacking. → *More:* Ch 3 §3.5 · Ch 11 §11.1

Q123. ⋆ What is the difference between `yield` and `return`? `return` ends the function and sends back one value. `yield` sends back a value and **pauses** the function, keeping its local variables; the next `next()` resumes right after the `yield`. A function containing `yield` returns a generator object when called and runs no code until the first `next()`. A `return value` inside a generator ends it and stores the value in `StopIteration.value`. → *More:* Ch 20 Q23 · Ch 11 §11.2 — Generators and yield

Q124. ⋆ What are the advantages of generators? Give a real use case. They are **lazy**: they produce items on demand, so memory stays constant regardless of how many items there are, the first result arrives immediately, and they can represent infinite sequences. Use cases: reading a huge log file line by line, streaming rows from a database or API page by page, and pipelines of generator stages (`read → parse → filter → aggregate`) that process data without building intermediate lists. → *More:* Ch 11 §11.4 — Lazy evaluation and memory

Q125. ⋆ What is the difference between a list comprehension and a generator expression? `[x * x for x in xs]` builds the whole list in memory immediately; `(x * x for x in xs)` returns a lazy generator that computes values one at a time and can be consumed once. Use the generator form when you only iterate once or feed a function like `sum(x * x for x in xs)` (the extra parentheses can be dropped when it is the only argument); use a list when you need indexing, `len`, or several passes. → *More:* Ch 20 §20.1 · Ch 11 §11.6

Q126. What does `yield from` do? `yield from sub` yields every value from the sub-iterator `sub`, and for generators it also forwards `send()` and `throw()` calls and returns the sub-generator's return value. It replaces `for v in sub: yield v`, and is handy for recursive generators such as flattening nested lists or walking trees. → *More:* Ch 11 §11.3 — yield from, send() and close()

Q127. What do a generator's `send()`, `throw()` and `close()` methods do? `gen.send(value)` resumes the generator and makes the paused `yield` expression evaluate to `value` (the first call must be `send(None)` or `next()`). `gen.throw(exc)` raises an exception at the paused `yield`. `gen.close()` raises `GeneratorExit` inside it so `finally` blocks run. These turn generators into simple coroutines; today `async def` is used for real coroutines. → *More:* Ch 11 §11.3

Q128. How do you write your own iterator class? Implement `__iter__` returning `self` and `__next__` returning the next value or raising `StopIteration`. For a reusable iterable, make `__iter__` return a *new* iterator (often by making `__iter__` itself a generator function).

```
class Countdown:
    def __init__(self, start): self.n = start
    def __iter__(self): return self
    def __next__(self):
        if self.n <= 0:
            raise StopIteration
        self.n -= 1
        return self.n + 1
print(list(Countdown(3)))   # [3, 2, 1]
```

→ *More:* Ch 11 §11.1

Q129. What happens if a generator raises `StopIteration` itself? Since Python 3.7 (PEP 479), a `StopIteration` that escapes from inside a generator body is converted into `RuntimeError`, because it would otherwise silently end the loop and hide bugs. Inside a generator, use `return` to finish. A common trigger is calling `next(it)` on an exhausted iterator inside a generator; use `next(it, default)` instead. → *More:* Ch 11 §11.2

Q130. Is a list comprehension faster than a `for` loop with `append`? Usually yes, modestly (often 10–30 %): the comprehension avoids looking up and calling `lst.append` on each iteration and runs a specialised loop. Since 3.12 comprehensions are also inlined (PEP 709, no hidden function call). But choose them for readability; if the body needs several statements or side effects, a plain loop is clearer. For numeric work, NumPy vectorisation beats both by far. → *More:* Ch 11 §11.6 — Comprehensions, best practices · Ch 17 §17.7

Q131. How do nested comprehensions work? When should you avoid comprehensions? Clauses read left to right like nested loops: `[(i, j) for i in range(3) for j in range(i)]` equals an outer `for i` with an inner

`for j`. Flatten: `[x for row in matrix for x in row]`; transpose: `[list(col) for col in zip(*matrix)]`. Avoid comprehensions with more than two loops or complex conditions, and never use them only for side effects (`[print(x) for x in xs]`). → *More:* Ch 5 §5.6 · Ch 11 §11.6

Q132. Which `itertools` functions should you know? `chain` (concatenate iterables), `islice` (slice any iterator), `count` / `cycle` / `repeat` (infinite), `product`, `permutations`, `combinations`, `combinations_with_replacement` (combinatorics), `accumulate` (running totals), `groupby` (groups *consecutive* equal keys, so sort first), `pairwise` (3.10, adjacent pairs), `batched` (3.12, chunks of n), `zip_longest`, `takewhile` / `dropwhile`, `starmap`, `tee` (split one iterator in several). → *More:* Ch 11 §11.5 — itertools

Q133. What do `enumerate`, `zip`, `any` and `all` do? What does `zip(strict=True)` add? `enumerate(xs, start=0)` yields `(index, item)` pairs; `zip(a, b)` yields tuples of items in parallel and stops at the shortest input; `zip(a, b, strict=True)` (3.10+) raises `ValueError` if lengths differ, catching silent data loss. `any(it)` is true if any item is truthy, `all(it)` if all are; both short-circuit, and `all([])` is `True`, `any([])` is `False`. → *More:* Ch 11 §11.7 · Ch 20 §20.1

Q134. Is `range` a generator? What does it return in Python 3? No. `range` is a lazy, immutable **sequence**: it computes items on demand, but supports `len`, indexing, slicing, `in` (O(1) for integers) and can be iterated many times. A generator supports none of those and is single-use. → *More:* Ch 3 §3.6 · Ch 20 rapid-fire round

---

7. Decorators, context managers and metaprogramming

Q135. ⋆ What is a decorator? How does it work? A decorator is a callable that takes a function (or class) and returns a replacement, usually a wrapper that adds behaviour before or after calling the original. `@decorator` above `def f` is shorthand for `f = decorator(f)`, applied once when the function is defined. Uses: logging, timing, caching, retries, access control, registering routes (`@app.get("/")`). → *More:* Ch 20 Q21 · Ch 7 §7.6 · Ch 12 §12.1 — Decorators

Q136. ⋆ Write a decorator that measures a function's execution time.

```python
import functools, time
def timed(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            print(f"{func.__name__} took {time.perf_counter() - start:.4f}s")
    return wrapper
```

`perf_counter` is the right clock for measuring durations; `*args, **kwargs` forwards any arguments; `finally` reports even if the function raises. → *More:* Ch 12 §12.1

Q137. ⋆ Why use `functools.wraps`? Without it, the decorated function is replaced by `wrapper`, so `__name__`, `__doc__`, `__module__`, `__qualname__` and annotations show the wrapper's details, which confuses debugging, logging, documentation and frameworks that look up functions by name. `@functools.wraps(func)` copies that metadata and sets `wrapper.__wrapped__ = func`, so tools (and `inspect.signature`) can reach the original. → *More:* Ch 12 §12.1

Q138. ⋆ How do you write a decorator that takes arguments? Add one more level: an outer function receives the arguments and returns the actual decorator. `@retry(times=3)` first calls `retry(times=3)`, and the result decorates the function. So the structure is factory → decorator → wrapper (see the full example in Ch 20 Q21). → *More:* Ch 20 Q21 · Ch 12 §12.1

Q139. In what order are stacked decorators applied? Bottom-up when decorating, top-down when called. `@a` over `@b` over `def f` means `f = a(b(f))`: `b` wraps first; at call time `a`'s wrapper runs first, then `b`'s, then `f`. Example: `@bold @italic def hi()` produces `"<b><i>hi</i></b>"`. → *More:* Ch 12 §12.1

Q140. Can a class be a decorator? Can you decorate a class? Yes to both. A class with `__call__` can act as a decorator: `__init__` receives the function and `__call__` runs on each call, which is convenient when the decorator keeps state (call counts). A *class decorator* receives a class and returns it

modified or replaced: `@dataclass`, `@functools.total_ordering` and registration decorators work this way. → *More:* Ch 12 §12.1

Q141. What built-in and standard-library decorators do you know? `@property`, `@classmethod`, `@staticmethod`, `@functools.cache`, `@functools.lru_cache(maxsize=...)`, `@functools.cached_property`, `@functools.wraps`, `@functools.total_ordering`, `@functools.singledispatch`, `@dataclasses.dataclass`, `@contextlib.contextmanager`, `@abc.abstractmethod`, `@typing.override` (3.12) and `@typing.overload`. Third-party favourites: `@pytest.fixture`, `@app.route` / `@app.get`. → *More:* Ch 12 §12.2 — Everyday built-in decorators

Q142. ⋆ What is the difference between `functools.cache` and `lru_cache`? What are their pitfalls? Both memoise (store results by arguments and return the stored result on a repeat call). `@cache` (3.9) is unbounded; `@lru_cache(maxsize=128)` evicts the least recently used entry when full. Pitfalls: all arguments must be hashable; cached results are shared, so mutating a returned list corrupts the cache; an unbounded cache can grow forever; and decorating a *method* caches `self` too, keeping instances alive (a memory leak). Use `cache_info()` and `cache_clear()` to inspect and reset. → *More:* Ch 7 §7.6 · Ch 12 §12.2

Q143. ⋆ What is a context manager? How does the `with` statement work? A context manager is an object that sets something up and guarantees clean-up, implementing `__enter__` and `__exit__`. `with cm as x:` calls `cm.__enter__()` and binds its return value to `x`, runs the block, then always calls `cm.__exit__(exc_type, exc, tb)`, even after an exception or `return`. If `__exit__` returns a true value, the exception is suppressed. Typical uses: files, locks, database transactions, temporary changes. → *More:* Ch 20 Q37 · Ch 9 §9.8 · Ch 12 §12.3 — Context managers

Q144. Write a context manager in two ways.

```python
class Timer:
    def __enter__(self):
        self.start = time.perf_counter(); return self
    def __exit__(self, exc_type, exc, tb):
        self.elapsed = time.perf_counter() - self.start
        return False                    # don't suppress exceptions

@contextlib.contextmanager
def timer():
    start = time.perf_counter()
    try:
        yield
    finally:
        print(f"{time.perf_counter() - start:.3f}s")
```

The generator version runs code before `yield` on enter and after it on exit; the `try/finally` makes the exit code run even if the block raises. → *More:* Ch 12 §12.3

Q145. What helpers does `contextlib` provide? `contextmanager` (generator-based managers), `asynccontextmanager`, `suppress(FileNotFoundError)` (ignore specific exceptions), `closing(obj)` (call `.close()` on exit), `nullcontext()` (a do-nothing manager for optional `with`), `redirect_stdout`, `chdir` (3.11), and `ExitStack` (enter a variable number of managers and close all of them in reverse order). → *More:* Ch 12 §12.3 · Ch 9 §9.8

Q146. What is an async context manager? An object with `async def __aenter__` and `async def __aexit__`, used with `async with` inside a coroutine, so setting up and cleaning up can await I/O (opening a network connection, acquiring an `asyncio.Lock`, starting a `TaskGroup`). `contextlib.asynccontextmanager` builds one from an async generator. → *More:* Ch 12 §12.3 · Ch 16 §16.9

Q147. ⋆ What is monkey patching? When is it acceptable? Changing a module, class or object attribute at runtime (`requests.get = fake_get`). Acceptable in tests, ideally through `unittest.mock.patch` or pytest's `monkeypatch`, which restore the original automatically; occasionally for an emergency hotfix of a third-party bug. Risky in production code: global, invisible behaviour changes that break on library upgrades. → *More:* Ch 20 Q46 · Ch 12 §12.7 — Monkey patching

Q148. What are `getattr`, `setattr`, `hasattr` and the `inspect` module used for? They access attributes by *name* at runtime: `getattr(obj, "save", None)()` for dynamic dispatch, `setattr` to set from config, `hasattr` to check (it calls `getattr` and catches `AttributeError`). `inspect` examines live objects:

`inspect.signature(f)`, `getsource`, `getmembers`, `iscoroutinefunction`. Frameworks use these for plugin systems, dependency injection and serialisation. → *More:* Ch 12 §12.8 — Introspection and dynamic code

Q149. Why are `eval()` and `exec()` dangerous? What is the safe alternative? `eval` evaluates an expression string and `exec` runs statements; given untrusted input, they run arbitrary code (delete files, read secrets), and restricting their globals is not a reliable sandbox. To parse literal data use `ast.literal_eval` (only literals: numbers, strings, lists, dicts...), `json.loads` for JSON, and explicit dispatch tables instead of building code strings. → *More:* Ch 12 §12.8

Q150. What is `__init_subclass__`? Give a use case. A class method automatically called on a base class whenever a subclass is created, receiving the new subclass (and any keyword arguments from the class line). It covers most things people used metaclasses for: registering plugins (`registry[cls.__name__] = cls`), validating that subclasses define required attributes, or configuring subclasses (`class Json(Serializer, fmt="json")`). → *More:* Ch 12 §12.6

---

8. Exceptions and files

Q151. ⋆ How do you handle exceptions? Explain `try`, `except`, `else` and `finally`. Code that may fail goes in `try`. Each `except SomeError as e:` handles matching exceptions (checked top to bottom). `else` runs only if `try` raised nothing; put the code that should run on success but should *not* be protected there. `finally` always runs, whether an exception happened, was handled, or the block returned; use it (or `with`) for clean-up. → *More:* Ch 20 Q36 · Ch 9 §9.2

Q152. ⋆ Describe the exception hierarchy. Why is a bare `except:` bad? Everything derives from `BaseException`. Directly under it are `SystemExit`, `KeyboardInterrupt`, `GeneratorExit` and `Exception`; nearly all errors (`ValueError`, `KeyError`, `OSError` ...) derive from `Exception`. A bare `except:` (or `except BaseException:`) also catches Ctrl+C and `sys.exit()`, and hides real bugs. Catch the most specific exception you can handle; if you must catch broadly, use `except Exception` and log the traceback. → *More:* Ch 20 gotcha 11 · Ch 9 §9.1 — The hierarchy

Q153. Does the order of `except` blocks matter? Yes. Python uses the first `except` clause whose type matches (including base classes). Put specific exceptions before general ones: if `except Exception` comes first, a later `except ValueError` can never run. Catch several types in one clause with a tuple: `except (KeyError, IndexError) as e:`. Since 3.14 (PEP 758) the parentheses may be dropped when there is no `as` clause: `except KeyError, IndexError:`. → *More:* Ch 9 §9.2

Q154. ⋆ How do you create a custom exception? Subclass `Exception` (or a more specific built-in such as `ValueError`), usually with a descriptive name ending in `Error`. A library typically defines one base exception and specific subclasses so callers can catch at the level they want. Add fields if callers need data.

```
class PaymentError(Exception): pass
class CardDeclined(PaymentError):
    def __init__(self, code: str):
        super().__init__(f"card declined: {code}")
        self.code = code
```

→ *More:* Ch 9 §9.3 — Raising, re-raising and custom exceptions

Q155. What is the difference between `raise`, `raise e` and `raise NewError(...) from e`? A bare `raise` inside an `except` re-raises the current exception with its original traceback (the usual way to log and propagate). `raise e` also re-raises it, adding the current line to the traceback. `raise NewError(...) from e` raises a different exception and records `e` as its explicit cause (`__cause__`), shown as "The above exception was the direct cause of...". Raising inside `except` without `from` still links them implicitly (`__context__`); `from None` hides the original. → *More:* Ch 9 §9.4 — Exception chaining and notes

Q156. What happens if both `try` and `finally` contain `return`? The `finally` block's `return` wins and silently discards the `try`'s return value, and even swallows an exception that was propagating. That is why Python 3.14 (PEP 765) emits a `SyntaxWarning` for `return`, `break` or `continue` that would leave a `finally` block. Keep `finally` for clean-up only. → *More:* Ch 9 §9.2

Q157. What are `ExceptionGroup` and `except*`? `ExceptionGroup` (3.11) bundles several unrelated exceptions raised together, for example by concurrent tasks in an `asyncio.TaskGroup`. `except* ValueError as eg:` handles the matching *part* of a group (`eg` is a sub-group) and lets the rest continue to propagate; several `except*` clauses can each run. Also new in 3.11: `e.add_note("context")` attaches extra text shown in the traceback. → *More:* Ch 9 §9.6 — ExceptionGroup and except*

Q158. What is EAFP vs LBYL? Which is more Pythonic? EAFP, "easier to ask forgiveness than permission": just try the operation and handle the exception (`try: v = d[k] except KeyError: ...`). LBYL, "look before you leap": check first (`if k in d:`). Python favours EAFP: it is often faster when failure is rare, and it avoids race conditions where the situation changes between the check and the action (a file deleted after `os.path.exists`). → *More:* Ch 9 §9.5 — EAFP vs LBYL · Ch 20 §20.1

Q159. When should you use `assert`? For internal sanity checks that should never fail if the code is correct (invariants during development and in tests). Never for validating user input or enforcing security: running Python with `-O` removes all `assert` statements. Raise a proper exception (`ValueError`, `TypeError`) for input errors. Also beware `assert (cond, "msg")`: a non-empty tuple is always true. → *More:* Ch 9 §9.7 — assert

Q160. ⋆ How do you read a very large file that does not fit in memory? Iterate over the file object, which yields one line at a time with constant memory: `with open(path, encoding="utf-8") as f: for line in f: ...`. For binary data read fixed-size chunks: `while chunk := f.read(1 << 20):`. Chain generators to process lines lazily; use `csv.reader` on the open file for CSVs, `pandas.read_csv(..., chunksize=...)` for dataframes, and `mmap` for random access. Avoid `f.read()` and `f.readlines()`, which load everything. → *More:* Ch 9 §9.9 — File I/O · Ch 11 §11.4

Q161. What are the file modes? Why should you always use `with open(...)`? `"r"` read (default), `"w"` write (truncates), `"a"` append, `"x"` create (fails if the file exists), `"+"` read and write; add `"b"` for binary (bytes) instead of text. `with` closes the file even when an exception occurs, flushing buffered writes; without it, files may stay open (leaking file handles) or lose data. Always pass `encoding="utf-8"` for text, since the default depends on the platform. → *More:* Ch 9 §9.9 · Ch 20 §20.1

Q162. ⋆ What is the difference between `json` and `pickle`? Is pickle safe? `json` writes a text format of basic types (dict, list, str, number, bool, null) that any language can read; it is safe to load. `pickle` writes a Python-only binary format that can store almost any Python object, but **loading a pickle can execute arbitrary code**, so never unpickle untrusted data. Use JSON (or formats like Parquet) for exchange and storage, pickle only between trusted Python processes. → *More:* Ch 20 Q39 · Ch 9 §9.11 — csv, json, pickle

Q163. How do you serialise an object that `json.dumps` does not support, such as a `datetime` or a dataclass? Convert it first (`dataclasses.asdict(obj)`, `dt.isoformat()`), or pass `default=` a function that converts unknown types: `json.dumps(data, default=str)` or a custom function that handles `datetime`, `Decimal` and sets. For loading, `object_hook=` turns dicts back into objects. Libraries such as Pydantic do this both ways with validation. → *More:* Ch 9 §9.11 · Ch 14 §14.9

Q164. `pathlib` vs `os.path`: which should you use? `pathlib.Path` (3.4+) represents paths as objects with operators and methods: `Path("data") / "raw" / "a.csv"`, `.exists()`, `.suffix`, `.stem`, `.read_text()`, `.glob("*.csv")`, `.mkdir(parents=True, exist_ok=True)`. It is cross-platform and more readable than string functions in `os.path`. Use `shutil` for copying and moving trees and `tempfile` for temporary files. → *More:* Ch 9 §9.10 — pathlib, shutil, tempfile

---

9. Modules, packaging, environments and type hints

Q165. ⋆ What is the difference between a module and a package? A module is a single `.py` file; its name is the file name. A package is a directory of modules (and sub-packages) imported with dotted names (`pkg.sub.mod`). A *regular* package contains an `__init__.py`, which runs on first import and can expose a public API; a directory without it is a *namespace package* that can span several directories. → *More:* Ch 20 Q7 · Ch 8 §8.3 — Packages

Q166. ⋆ What does `if __name__ == "__main__":` do? Python sets `__name__` to `"__main__"` in the file being run directly and to the module's import name when it is imported. Code under the guard therefore runs only when the file is executed as a script, not when it is imported (for example by tests). It is also

required with `multiprocessing` under the `spawn` and `forkserver` start methods, because child processes re-import the main module. → *More:* Ch 20 Q6 · Ch 8 §8.5

Q167. How does `import` work? What are `sys.path` and `sys.modules`? `import x` first checks the cache `sys.modules`; if `x` is there, it reuses it (so a module's top-level code runs once per process). Otherwise finders search the directories in `sys.path` (the script's directory, `PYTHONPATH`, the standard library, then `site-packages`), a loader creates the module object, executes its code, stores it in `sys.modules`, and binds the name. `importlib.reload(m)` re-executes a module, but existing references keep the old objects. → *More:* Ch 8 §8.2 — How import really works

Q168. What is the difference between absolute and relative imports? An absolute import names the full path from the project root (`from shop.billing import invoice`). A relative import uses dots relative to the current package (`from .billing import invoice`, `from .. import utils`) and works only inside packages, not in a file run directly as a script. PEP 8 recommends absolute imports; run modules inside packages with `python -m pkg.mod`. → *More:* Ch 8 §8.4 — Absolute vs relative imports

Q169. ⋆ What causes a circular import, and how do you fix it? Module A imports B while B (directly or indirectly) imports A. Since A is only partly executed when B runs, B's `from a import thing` fails with `ImportError: cannot import name ... (most likely due to a circular import)`. Fixes: move shared code into a third module; import the module instead of names (`import a` then use `a.thing` at call time); move the import inside the function that needs it; or use `if TYPE_CHECKING:` for imports only needed by type hints. → *More:* Ch 20 gotcha 9 · Ch 8 §8.6

Q170. What does `__all__` do? Why avoid `from module import *`? `__all__` is a list of names that `from module import *` imports (otherwise all names not starting with `_`), and it documents the public API. Star imports are discouraged: they hide where names come from, can silently overwrite existing names, and confuse linters. Import what you use explicitly. → *More:* Ch 8 §8.3

Q171. ⋆ What is a virtual environment, and why use one? A directory (`python -m venv .venv`) containing its own Python executable link and `site-packages`, so each project gets its own isolated set of package versions without conflicts or touching the system Python. Activate it (`source .venv/bin/activate`) or call its interpreter directly. Never commit the folder; commit the dependency list instead. → *More:* Ch 20 Q8 · Ch 8 §8.7 — pip and virtual environments

Q172. What is the difference between `requirements.txt` and `pyproject.toml`? What are uv and Poetry? `requirements.txt` is a plain list of packages (often pinned with `==`) that `pip install -r` installs. `pyproject.toml` (PEP 518/621) is the standard project file: metadata, dependencies with version ranges, the build backend and tool settings (ruff, pytest, mypy). Tools such as **uv** (very fast, written in Rust) and **Poetry** manage the virtual environment, resolve dependencies and write a **lock file** that pins exact versions of everything, including indirect dependencies, for reproducible installs. → *More:* Ch 8 §8.8 — requirements.txt vs pyproject.toml; uv and Poetry

Q173. What is a wheel? What does `pip install -e .` do? A wheel (`.whl`) is a pre-built distribution: a zip file installed by unpacking, with no build step, and, for packages with C code, compiled for a specific platform. An sdist is the source archive that must be built. `pip install -e .` installs your project in *editable* mode: the environment points at your source folder, so code edits take effect without reinstalling. → *More:* Ch 8 §8.9 — Project layout and publishing basics

Q174. ⋆ What are type hints? Does Python enforce them? Annotations such as `def area(r: float) -> float:` and `names: list[str] = []` that document expected types. The interpreter does **not** check them; static type checkers (mypy, pyright) and IDEs use them to find bugs before running, and libraries such as Pydantic and FastAPI read them to validate data at runtime. Typing is gradual: unannotated code is treated as `Any`. → *More:* Ch 20 Q44 · Ch 13 §13.1–13.2

Q175. What is the difference between `Optional[X]`, `X | None` and a parameter with a default? `Optional[X]` and `X | None` (3.10+ syntax, preferred) both mean "an `X` or `None`". This is about the *type*, not about being optional to pass: a parameter is optional to pass only if it has a default. `def f(x: int | None)` must still be called with an argument; `def f(x: int | None = None)` may be called without one. → *More:* Ch 13 §13.4 — Optional, Union and X | None

Q176. What is the difference between `Any` and `object` in type hints? `Any` switches type checking off: every operation on it is allowed and it is compatible with every type. `object` is the top type: any value can be assigned to it, but you can only do what every object supports, so the checker forces you

to narrow it (`isinstance`) first. Prefer `object` (or a precise type) when you mean "anything"; `Any` is an escape hatch. → *More:* Ch 13 §13.5 — Any and object

Q177. How do you write generic functions and classes? What changed in 3.12? A type variable stands for "some type, the same in each place". Since 3.12 (PEP 695) you write `def first[T](xs: list[T]) -> T:` and `class Stack[T]:` directly, and `type Pair[T] = tuple[T, T]` defines an alias. Older code uses `T = TypeVar("T")` and `Generic[T]`. Bounds and constraints: `def biggest[T: Comparable](...)` or `[T: (int, float)]`. → *More:* Ch 13 §13.6 — TypeVar and generics · Ch 13 §13.12

Q178. What are `TypedDict`, `Literal`, `Final`, `NewType`, `Self` and `@override`? `TypedDict` describes dicts with specific keys and value types; `Literal["r", "w"]` restricts a value to given constants; `Final` marks a name that must not be reassigned; `NewType("UserId", int)` creates a distinct type for the checker at no runtime cost; `Self` (3.11) is the type of the current class in methods that return `self`; `@typing.override` (3.12) asks the checker to verify the method really overrides a base-class method. → *More:* Ch 13 §13.9–13.10

Q179. How do you type a callable or a decorator? `Callable[[int, str], bool]` is a function taking an `int` and a `str` and returning `bool`; `Callable[..., T]` accepts any arguments. To type a decorator that preserves the wrapped function's signature, use `ParamSpec`: `def logged[**P, R](f: Callable[P, R]) -> Callable[P, R]:`, with the wrapper declared `(*args: P.args, **kwargs: P.kwargs) -> R`. → *More:* Ch 13 §13.7 — Callable and ParamSpec

Q180. When are annotations evaluated? What changed in Python 3.14? Up to 3.13, annotations were evaluated when the `def` or `class` ran, so a reference to a class defined later had to be quoted (`"Node"`) unless the module used `from __future__ import annotations` (which stores them as strings). In 3.14 (PEP 649/749), evaluation is deferred: annotations are computed only when accessed, so forward references usually work without quotes, and the new `annotationlib` module can retrieve them as values, forward references or strings. → *More:* Ch 13 §13.2

Q181. mypy vs pyright vs Pydantic: what does each do? mypy and pyright are **static** type checkers: they read your code without running it and report type errors (pyright powers VS Code's Pylance and is faster; mypy is the original reference checker; both are commonly run in CI). Pydantic is a **runtime** library: it validates and converts incoming data according to type hints when the code runs. They solve different problems and are often used together. → *More:* Ch 13 §13.13–13.14

---

10. Memory model, internals and performance

Q182. ⋆ How is memory managed in Python? All objects live on a private heap managed by the interpreter. CPython frees objects mainly by **reference counting** (an object is freed the moment its count drops to zero) plus a **cyclic garbage collector** for groups of objects that reference each other. Small objects (up to 512 bytes) come from **pymalloc**, an allocator that keeps pools of fixed-size blocks to avoid many system `malloc` calls. You never free memory manually. → *More:* Ch 20 Q38 · Ch 17 §17.2 — Memory management and garbage collection

Q183. ⋆ What is reference counting? What are its pros and cons? Each object stores how many references point to it; assignments, container insertions and function arguments increase it, and deleting names or leaving scope decreases it. At zero the object is destroyed immediately. Pros: predictable, prompt clean-up (files closed as soon as unused). Cons: a small cost on every reference change, it cannot free reference cycles, and it makes multi-threading harder (the reason for the GIL). `sys.getrefcount(x)` shows the count (one higher, because of its own argument). → *More:* Ch 17 §17.2

Q184. ⋆ How does Python's garbage collector work? What are generations? The `gc` module finds reference cycles that refcounting cannot free (e.g., two objects pointing at each other, or an object referring to itself). It tracks container objects and is **generational**: new objects start in the young generation, which is scanned often; objects that survive are promoted to older generations, which are scanned rarely, because most objects die young. You can call `gc.collect()`, inspect `gc.get_threshold()`, or `gc.disable()` in special cases. Version note: 3.14.0–3.14.4 shipped an *incremental* collector, and 3.14.5 went back to the 3.13-style generational one. → *More:* Ch 17 §17.2

Q185. Can Python have memory leaks? How do you find them? Yes, in practice: references kept alive unintentionally. Typical causes: ever-growing module-level lists or dicts used as caches, unbounded

`lru_cache` (especially on methods), closures or callbacks holding large objects, reference cycles through objects with external resources, and C extensions that leak. Diagnose with `tracemalloc` (compare snapshots to see which lines allocate), `gc.get_objects()` / `objgraph`, and memory profilers; fix with bounded caches and `weakref`. → *More:* Ch 20 Q38 · Ch 17 §17.2, §17.6

Q186. What is a weak reference? A reference that does not increase the object's reference count, so it does not keep the object alive. `weakref.ref(obj)` returns a callable that gives the object or `None` once it is gone; `WeakValueDictionary` and `WeakKeyDictionary` make caches whose entries vanish when objects are no longer used elsewhere; `weakref.finalize` registers clean-up. Used for caches, observers and parent pointers in trees, to avoid leaks and cycles. → *More:* Ch 17 §17.2

Q187. Where are variables stored in Python: on the stack or the heap? All objects live on the heap. Each function call gets a *frame* holding its local variables, but those slots contain only references (pointers) to heap objects. That is why returning a local list is safe: the frame disappears, but the list lives on as long as something refers to it. → *More:* Ch 17 §17.1 — The CPython object model

Q188. ⋆ Why is `a = 256; b = 256; a is b` true but the same with large numbers can be false? CPython pre-creates and caches the integers from −5 to 256, so every 256 is the same object. Larger integers are usually created separately at runtime (`int("1000") is int("1000")` is `False`), although the compiler may reuse one constant object within the same code block, so results vary between the REPL and a script. This is an implementation detail: always compare numbers with `==`. → *More:* Ch 20 gotcha 4 · Ch 17 §17.3

Q189. What are immortal objects? Since 3.12 (PEP 683), certain objects such as `None`, `True`, `False`, small integers and some interned strings are *immortal*: their reference count is fixed at a special value and never changes, so they are never freed. This avoids writes to shared objects (good for multiple interpreters and the free-threaded build). It is why `sys.getrefcount(None)` returns a huge number. → *More:* Ch 17 §17.2

Q190. What is the difference between `sys.getsizeof()` and an object's real memory use? `sys.getsizeof(x)` returns only the size of the object itself, not of the objects it refers to: a list's size covers its pointer array, not its elements. For the total, walk the references (or use `pympler.asizeof`), or measure with `tracemalloc`. Typical sizes on 64-bit CPython: an empty list about 56 bytes, a small int 28 bytes. → *More:* Ch 17 §17.1

Q191. Why is `__del__` not a good way to release resources? `__del__` (the finaliser) runs when the object is destroyed, which with reference counting is usually immediate, but not if the object is part of a cycle (it waits for the GC), not on other Python implementations like PyPy, and not reliably at interpreter shutdown. Exceptions inside it are only printed. Use context managers (`with`) or `weakref.finalize` for deterministic clean-up. → *More:* Ch 17 §17.2

Q192. How do you find out why a Python program is slow? Measure before optimising. `cProfile` (`python -m cProfile -s cumtime app.py`) shows which functions take time; `pstats` or snakeviz explore the results; `timeit` compares small snippets; `line_profiler` times individual lines; `py-spy` samples a running process without changing it; `tracemalloc` finds memory hotspots. Then optimise the top hotspot only, and measure again. → *More:* Ch 20 Q40 · Ch 17 §17.6 — Profiling

Q193. ⋆ How do you make Python code faster? In order of payoff: a better algorithm or data structure (set or dict lookups instead of list scans, a heap instead of repeated sorting); push loops into C (built-ins like `sum`, `sorted`, `str.join`, `collections`, NumPy vectorisation); avoid repeated work (caching with `functools.cache`, hoisting work out of loops); reduce Python overhead in hot loops (local variables, comprehensions); use generators for memory; then parallelism (processes) or compiled code (Cython, Numba, Rust via PyO3), or PyPy. → *More:* Ch 17 §17.7 — Optimisation techniques

Q194. Why are local variables faster than global variables? Locals are stored in a fixed array in the frame and accessed by index (`LOAD_FAST`), while globals and built-ins require dictionary lookups (`LOAD_GLOBAL`, though modern CPython caches these). In very hot loops, binding a global or attribute to a local (`append = result.append`) can still save time, but readability usually matters more; the adaptive interpreter (3.11+) has narrowed the gap. → *More:* Ch 17 §17.7

Q195. What are Cython, Numba, PyPy and PyO3, and when would you use each? Cython compiles Python-like code with C type declarations into C extensions, good for wrapping C libraries or speeding up loops. **Numba** JIT-compiles numeric functions with a decorator (`@njit`), great for NumPy-heavy loops. **PyPy** is an alternative interpreter with a JIT that speeds up pure-Python programs without code changes. **PyO3** lets

you write extensions in Rust (as many modern tools like pydantic-core and polars do). Choose by bottleneck: numeric loops → Numba; existing C code → Cython; whole pure-Python app → PyPy; new performance-critical component → Rust. → *More:* Ch 17 §17.7

Q196. Does CPython have a JIT compiler? Since 3.13, CPython can be *built* with an experimental copy-and-patch JIT that translates frequently executed bytecode traces into machine code. It is off by default (enable with `PYTHON_JIT=1` on a JIT-enabled build; 3.14's official Windows and macOS installers include it), and gains are still modest. Most of the 3.11+ speed-ups came instead from the *specialising adaptive interpreter* (PEP 659), which rewrites bytecode into faster type-specific versions as code runs. → *More:* Ch 1 §1.8 · Ch 17

---

11. Concurrency: GIL, free-threading, threads, processes and asyncio

Q197. ⋆ What is the GIL (Global Interpreter Lock)? A lock inside the default CPython build that allows only one thread at a time to execute Python bytecode. It keeps reference counting and interpreter internals simple and makes single-threaded code fast. Consequences: threads do not speed up CPU-bound pure-Python code; they do help I/O-bound code because the GIL is released while waiting on I/O (and by many C extensions such as NumPy during heavy computations). The GIL does not make your code thread-safe. → *More:* Ch 20 Q41 · Ch 16 §16.6 — The GIL

Q198. ⋆ Why does CPython have a GIL, and is it being removed? What is free-threaded Python? It was the simplest way to make reference counting and the C API safe with threads, and removing it used to slow single-threaded code. PEP 703 introduced a separate **free-threaded** build without the GIL: experimental in 3.13 (`python3.13t`) and officially supported but still optional in 3.14 (PEP 779, `python3.14t`). The default `python3.14` still has the GIL. The free-threaded build costs roughly 5–10 % single-threaded speed and more memory; C extensions must be marked compatible, or the GIL is re-enabled when they are imported. Expect it to become the default only in a later phase. → *More:* Ch 16 §16.6 — Free-threaded CPython status

Q199. How can you tell whether the GIL is enabled at runtime? `sys._is_gil_enabled()` (3.13+) returns `False` on a free-threaded build running without the GIL. `sysconfig.get_config_var("Py_GIL_DISABLED")` tells whether the interpreter was *built* free-threaded. On such a build you can force the GIL back on with `PYTHON_GIL=1` or `-X gil=1`. → *More:* Ch 16 §16.6

Q200. What is the difference between concurrency and parallelism? Concurrency is *dealing* with many tasks at once by interleaving them (one cook switching between dishes while each simmers); parallelism is *doing* many tasks at the same instant on several CPU cores (several cooks). Threads and asyncio give concurrency; processes (and free-threaded threads) give parallelism. → *More:* Ch 16 §16.1 — Concurrency vs parallelism

Q201. ⋆ When do you use threading vs multiprocessing vs asyncio? CPU-bound work (number crunching, parsing, image processing) → `multiprocessing` or `ProcessPoolExecutor`: separate processes, each with its own GIL, true parallelism. I/O-bound work with blocking libraries and moderate concurrency (tens to hundreds of requests, file I/O) → `threading` or `ThreadPoolExecutor`. I/O-bound work with very many connections and async libraries (thousands of sockets, web servers) → `asyncio`. Numeric arrays → NumPy, which runs loops in C. → *More:* Ch 20 Q42 · Ch 16 §16.10 — Choosing the right tool

Q202. ⋆ What is the difference between a process and a thread? A process has its own memory space, interpreter and GIL; starting one is slow and data must be copied between processes (pickled), but a crash in one does not corrupt others. Threads live inside one process and share its memory, so they start fast and share data easily, but shared mutable data needs locks. In CPython's default build, only one thread runs Python code at a time. → *More:* Ch 16 §16.3, §16.7

Q203. ⋆ What is a race condition? Is `counter += 1` thread-safe? A race condition is a bug where the result depends on the unpredictable timing of threads. `counter += 1` is not atomic: it reads, adds and writes back in separate bytecodes, so two threads can read the same value and one increment is lost (with or without the GIL). Protect shared updates with a `threading.Lock`: `with lock: counter += 1`, or avoid sharing by using a `queue.Queue` or per-thread results. → *More:* Ch 16 §16.4 — Race conditions and locks

Q204. What is a deadlock? How do you avoid it? What is an `RLock`? A deadlock is when threads wait forever for locks held by each other (A holds lock 1 and waits for lock 2, B holds lock 2 and waits for lock 1). Avoid it by always acquiring locks in the same global order, holding locks briefly, using timeouts (`lock.acquire(timeout=1)`), and preferring queues to shared state. An `RLock` (re-entrant lock) can be acquired again by the thread that already holds it, which is useful when a locked method calls another locked method. → *More:* Ch 16 §16.4, §16.11

Q205. What synchronisation tools does `threading` offer? `Lock` and `RLock` (mutual exclusion), `Semaphore` / `BoundedSemaphore` (allow at most *n* holders, e.g. limit connections), `Event` (one thread signals, others wait), `Condition` (wait until a state changes), `Barrier` (wait until *n* threads arrive), plus thread-safe `queue.Queue`, `LifoQueue` and `PriorityQueue`. → *More:* Ch 16 §16.4–16.5

Q206. How do you implement producer–consumer in Python? Use `queue.Queue`, which is thread-safe and blocks: producers call `q.put(item)`, consumers loop on `item = q.get()` and call `q.task_done()`; `q.join()` waits until all items are processed. Use `Queue(maxsize=n)` for back-pressure (producers wait when consumers fall behind), and a sentinel such as `None` to tell consumers to stop. For processes use `multiprocessing.Queue`; for coroutines `asyncio.Queue`. → *More:* Ch 16 §16.5 — Producer–consumer with queue.Queue

Q207. What is `concurrent.futures`? What is the difference between `submit` / `as_completed` and `map`? A high-level API with `ThreadPoolExecutor` and `ProcessPoolExecutor` (and `InterpreterPoolExecutor` in 3.14) that share one interface. `ex.submit(f, x)` returns a `Future` (a placeholder for a result); `as_completed(futures)` yields them as they finish, so you can handle results and errors one by one. `ex.map(f, xs)` returns results in input order, and an exception surfaces when you reach that item. Use the executor as a context manager so it waits and shuts down cleanly. → *More:* Ch 16 §16.8 — Executors

Q208. What are the multiprocessing start methods? What changed in 3.14? Why is `if __name__ == "__main__"` required? `fork` copies the parent process (fast, but unsafe if the parent has threads); `spawn` starts a fresh interpreter and re-imports the main module (default on Windows and macOS); `forkserver` forks children from a clean single-threaded server process. Since 3.14, `forkserver` is the default on Linux and other Unix systems except macOS. Because `spawn` and `forkserver` re-import your script in each child, start-up code must sit under `if __name__ == "__main__":`, or every child would start children again. Arguments and results are pickled, so lambdas and local functions cannot be sent. → *More:* Ch 16 §16.7 — multiprocessing

Q209. How do processes share data? By message passing: `multiprocessing.Queue` and `Pipe` (objects are pickled). For large arrays, `multiprocessing.shared_memory.SharedMemory` (3.8+) gives a raw shared buffer that NumPy arrays can wrap without copying; `Value` and `Array` share simple C types; `Manager()` provides proxied lists and dicts (convenient but slow). Prefer designs where each worker gets its input and returns its result. → *More:* Ch 16 §16.7

Q210. What are subinterpreters (`concurrent.interpreters`, Python 3.14)? PEP 734 adds a standard-library module for running several isolated interpreters inside one process, each with its own GIL, modules and state. They give multi-core parallelism with lower overhead than processes; data is shared by explicit channels (`concurrent.interpreters` queues) rather than shared objects. `concurrent.futures.InterpreterPoolExecutor` provides the familiar executor interface. Many third-party C extensions do not support them yet.

Q211. ⋆ How does asyncio work? What is the event loop? asyncio runs many **coroutines** (functions defined with `async def`) on a single thread. The **event loop** keeps a list of ready tasks and runs one until it reaches an `await` on something not ready (a socket read, a timer); the coroutine then suspends and the loop runs another ready task, resuming the first when its I/O completes (the operating system notifies the loop). This is **cooperative multitasking**: tasks give up control only at `await`, so there are no surprise interruptions, but a task that never awaits blocks everyone. → *More:* Ch 20 Q43 · Ch 16 §16.9 — asyncio

Q212. What is the difference between a coroutine, a task and a thread? Calling an `async def` function returns a **coroutine** object that does nothing until awaited or scheduled. A **task** (`asyncio.create_task(coro)`) wraps a coroutine and schedules it to run concurrently on the event loop. A **thread** is an operating-system unit of execution, preemptively scheduled (the OS can switch it at any moment), with its own stack, costing much more memory than a task; asyncio can run hundreds of thousands of tasks on one thread. → *More:* Ch 16 §16.9

Q213. How do you run several coroutines concurrently? `gather` vs `TaskGroup` vs `wait`? `await asyncio.gather(c1, c2)` runs them concurrently and returns results in order; by default the first exception is propagated while the other tasks keep running (`return_exceptions=True` returns exceptions as values). `async with asyncio.TaskGroup() as tg: tg.create_task(...)` (3.11+) is the modern structured choice: if one task fails, the others are cancelled and the errors are raised as an `ExceptionGroup`. `asyncio.wait` gives low-level control (`FIRST_COMPLETED`). Timeouts: `async with asyncio.timeout(5):` (3.11) or `asyncio.wait_for`. → *More:* Ch 16 §16.9

Q214. ⋆ What happens if you call a blocking function like `time.sleep()` or `requests.get()` inside a coroutine? How do you fix it? It blocks the whole event loop: no other task runs until it returns, so the async program becomes sequential. Use async equivalents (`await asyncio.sleep()`, `httpx.AsyncClient`, `aiohttp`, async database drivers), or move blocking or CPU-heavy calls off the loop with `await asyncio.to_thread(func, *args)` (or `loop.run_in_executor` with a process pool for CPU work). → *More:* Ch 16 §16.9, §16.11 — Common pitfalls

Q215. How do you limit concurrency in asyncio, for example to at most 10 requests at a time? Use an `asyncio.Semaphore(10)`, and make each task `async with sem:` around the request; or start a fixed number of worker tasks reading from an `asyncio.Queue`. Both prevent overloading the remote server or running out of sockets.

```
sem = asyncio.Semaphore(10)
async def fetch(client, url):
    async with sem:
        return await client.get(url)
```

→ *More:* Ch 16 §16.9

Q216. What are daemon threads? A daemon thread (`Thread(target=f, daemon=True)`) does not keep the program alive: when only daemon threads remain, the interpreter exits and kills them abruptly, without running their `finally` blocks. Use them for background helpers (heartbeats, monitoring) that can be stopped at any point, never for work that must finish or flush data. Non-daemon threads should be `join()`ed. → *More:* Ch 16 §16.3 — Threads

Q217. What is the difference between `threading.local` and `contextvars`? `threading.local()` gives each thread its own copy of attributes. `contextvars.ContextVar` gives each *context* its own value; asyncio copies the context into every task, so it works correctly for coroutines on one thread (request IDs, current user, database session in async web frameworks), where thread-locals would be shared by all tasks. → *More:* Ch 16 §16.9

Q218. Are Python's built-in data structures thread-safe? Individual operations on built-ins such as `list.append`, `dict[key] = v` and `deque.append` are atomic in CPython (protected by the GIL, and by per-object locks in the free-threaded build), but this is an implementation detail and a *sequence* of operations (check-then-set, read-modify-write) is not atomic. Use locks for compound updates, `queue.Queue` for hand-offs between threads, and never rely on atomicity for correctness logic. → *More:* Ch 16 §16.3–16.4

---

12. Standard library, testing and debugging

Q219. Which standard-library modules do you use most? A good answer names a range with a use for each: `collections`, `itertools`, `functools` (data and functional tools); `pathlib`, `os`, `shutil`, `sys` (files and environment); `json`, `csv`, `sqlite3`, `tomllib` (data formats); `datetime` and `zoneinfo` (time); `re` (regex); `logging`, `argparse` (applications); `subprocess` (run programs); `dataclasses`, `enum`, `typing`; `concurrent.futures`, `asyncio`; `unittest`/`unittest.mock`; `random`, `secrets`, `hashlib`, `uuid`. → *More:* Ch 14 — A Tour of the Standard Library

Q220. What is the difference between naive and aware `datetime` objects? How do you handle time zones? A naive `datetime` has no time zone (`tzinfo is None`), so it is ambiguous; an aware one carries a time zone. Rules: store and compute in UTC (`datetime.now(timezone.utc)`), convert to local time only for display with `zoneinfo.ZoneInfo("Asia/Kolkata")` (3.9+), and never mix naive and aware values (comparisons raise

`TypeError`). `datetime.utcnow()` is deprecated because it returns a naive value. Use `timedelta` for arithmetic, `isoformat()`/`fromisoformat()` for strings. → *More:* Ch 14 §14.1 — datetime and zoneinfo

Q221. `random` vs `secrets`: which do you use for tokens and passwords? `random` is a fast pseudo-random generator (Mersenne Twister) that is predictable once its state is known; fine for simulations, games, sampling and shuffling (`random.seed` for reproducibility). For anything security-related (passwords, reset tokens, API keys) use `secrets`: `secrets.token_urlsafe(32)`, `secrets.choice`, `secrets.compare_digest` for constant-time comparison. → *More:* Ch 14 §14.7 — random vs secrets

Q222. Why use `logging` instead of `print`? How should you set it up? `logging` gives severity levels (DEBUG, INFO, WARNING, ERROR, CRITICAL), timestamps and source information, configurable destinations (console, files, external systems) and filtering, all without code changes; `print` gives none of that. Best practice: in each module `logger = logging.getLogger(__name__)`; configure handlers once in the application entry point (`logging.basicConfig` or `dictConfig`); use lazy formatting `logger.info("user %s", uid)`; use `logger.exception(...)` inside `except` to include the traceback; libraries should not configure logging. → *More:* Ch 14 §14.4 — logging · Ch 15 §15.12

Q223. How do you run an external command from Python safely? `subprocess.run(["git", "status"], capture_output=True, text=True, check=True, timeout=30)`. Pass the command as a list so no shell is involved; avoid `shell=True` with any user input, because it allows shell injection (`; rm -rf /`). `check=True` raises `CalledProcessError` on a non-zero exit code. `os.system` is obsolete. → *More:* Ch 14 §14.6 — os, sys, subprocess, shutil

Q224. What is the difference between `os`, `sys` and `shutil`? `os` talks to the operating system: environment variables (`os.environ`), processes, low-level file operations, `os.walk`. `sys` talks to the interpreter: `sys.argv`, `sys.path`, `sys.exit`, `sys.stdin/stdout`, `sys.version`, `sys.getrecursionlimit`. `shutil` provides high-level file operations: copying and moving trees, deleting directories, archives, `shutil.which`. → *More:* Ch 14 §14.6

Q225. How do you read command-line arguments? `sys.argv` is the raw list (the script name is `argv[0]`). For real tools use `argparse`: declare positional and optional arguments with types, defaults, choices and help text, and it parses, validates and generates `--help` automatically. Third-party alternatives: `click` and `typer` (which builds a CLI from type hints). → *More:* Ch 14 §14.5 — argparse

Q226. ⋆ What is the difference between `unittest` and `pytest`? Which do you prefer? `unittest` is in the standard library and uses classes inheriting `TestCase` with `self.assertEqual`-style methods and `setUp`/`tearDown`. `pytest` (third-party, the de-facto standard) uses plain functions and the `assert` statement (with detailed failure messages), fixtures for setup, `parametrize`, a rich plugin ecosystem, and it also runs `unittest` tests. Most teams prefer pytest for less boilerplate. → *More:* Ch 20 Q45 · Ch 15 §15.3–15.4

Q227. What are pytest fixtures? What are fixture scopes and `conftest.py`? A fixture is a function decorated with `@pytest.fixture` that provides a test with data or a resource; a test requests it by naming it as a parameter. Code after a `yield` in the fixture runs as teardown. Scope controls how often it is created: `function` (default), `class`, `module`, `package`, `session` (e.g. one database container per run). Fixtures in `conftest.py` are shared with all tests in that directory and below, without imports. Built-ins include `tmp_path`, `monkeypatch`, `capsys`. → *More:* Ch 15 §15.4 — pytest

Q228. What does `@pytest.mark.parametrize` do? It runs the same test function once for each set of arguments, reporting each as a separate test: `@pytest.mark.parametrize("a, b, expected", [(1, 2, 3), (0, 0, 0), (-1, 1, 0)])`. This replaces copy-pasted tests and loops inside a test (where the first failure hides the rest). Use `pytest.param(..., id="name")` for readable names. → *More:* Ch 15 §15.4

Q229. ⋆ What is mocking? Where should you patch? A mock is a fake object that replaces a real dependency (network, database, clock) so a test is fast and deterministic, and that records how it was called (`mock.assert_called_once_with(...)`). `unittest.mock.patch("target")` temporarily replaces a name. **Patch where the name is looked up**, not where it is defined: if `app/service.py` does `from requests import get`, patch `"app.service.get"`, not `"requests.get"`. Use `autospec=True` so the mock enforces the real signature. → *More:* Ch 20 Q46 · Ch 15 §15.5 — Test doubles and unittest.mock

Q230. What is the difference between `Mock` and `MagicMock`? What are stubs, fakes and spies? `Mock` creates attributes and methods on demand; `MagicMock` additionally supports dunder methods (`len(m)`, iteration, `with m:`); `AsyncMock` returns awaitables. Vocabulary: a *stub* returns canned answers, a *fake*

is a working lightweight implementation (an in-memory repository), a *spy* records calls to a real object, and a *mock* additionally verifies expected interactions. → *More:* Ch 15 §15.5

Q231. What is code coverage? Is 100 % coverage the goal? Coverage (`coverage.py`, `pytest --cov`) measures which lines (and branches, with `--branch`) ran during tests. It shows untested code, but high coverage does not prove tests check the right things: a test without assertions still "covers" lines. Aim for high coverage of important logic (commonly 80–90 %), check branches, and judge tests by whether they would catch real bugs. → *More:* Ch 15 §15.6 — Coverage

Q232. What is TDD? What is property-based testing? Test-driven development: write a failing test (red), write the minimum code to pass it (green), then clean up (refactor), in small cycles. Property-based testing (the Hypothesis library) states a rule that must always hold, such as "decoding an encoded value returns the original" or "sorting is idempotent", and the library generates hundreds of inputs, including edge cases, and shrinks any failure to a minimal example. → *More:* Ch 15 §15.8–15.9

Q233. How do you debug Python code? Read the traceback from the bottom (the exception and the line that raised it) upward through the call chain. Reproduce with a small test. Use `breakpoint()` (3.7+) to drop into `pdb` (`n` next, `s` step into, `c` continue, `p expr` print, `w` where, `u`/`d` move up/down the stack), or an IDE debugger; `python -m pdb -c continue script.py` enters the debugger on a crash, and 3.14 can attach to a running process with `python -m pdb -p PID`. Add logging for problems in production. → *More:* Ch 15 §15.10–15.11 — Tracebacks and pdb

Q234. What is a doctest? Examples written in a docstring in REPL form (`>>> add(2, 3)` followed by `5`) that the `doctest` module runs and compares with the shown output. They keep documentation examples correct, but are not a replacement for a real test suite. Run with `python -m doctest module.py -v` or `pytest --doctest-modules`. → *More:* Ch 15 §15.7 — doctest

---

13. "What does this print?" puzzles

Write down your answer before reading ours. All outputs were checked on CPython 3.14.8 (3.11 prints the same; only the traceback layout differs).

Q235. ⋆ What does this print?

```
a = [[]] * 3
a[0].append(1)
print(a)
```

`[[1], [1], [1]]`. `* 3` copies the *reference* three times, so all three slots point to one inner list. Create independent lists with `[[] for _ in range(3)]`. → *More:* Ch 5 §5.8 — the nested-list trap

Q236. ⋆ What does this print?

```
def add(x, items=[]):
    items.append(x)
    return items
print(add(1), add(2))
```

`[1, 2] [1, 2]`. The default list is created once and shared by both calls, and both calls return that same list, so by the time `print` runs it holds both values. Use `items=None`. → *More:* Ch 20 gotcha 1

Q237. ⋆ What does this print?

```
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])
```

`[2, 2, 2]`. Late binding: each lambda looks up `i` when called, after the loop ended with `i == 2`. Fix with `lambda i=i: i`. → *More:* Ch 20 gotcha 2

Q238. What does this print?

```
t = (1, [2])
try:
    t[1] += [3]
except TypeError as e:
    print(e)
print(t)
```

`'tuple' object does not support item assignment`, then `(1, [2, 3])`. `+=` first extends the list in place (succeeds), then tries to store the result back into the tuple slot (fails). The error is raised, yet the list has already changed. → *More:* Ch 5 §5.9

Q239. What does this print?

```
print(0.1 + 0.2 == 0.3, 0.1 * 3)
print(round(2.5), round(3.5))
```

`False 0.30000000000000004`, then `2 4`. Binary floating point cannot store 0.1 exactly; `round` uses round-half-to-even. → *More:* Ch 20 gotcha 10 · Ch 2 §2.5

Q240. What does this print?

```
print(-7 // 2, -7 % 2, int(-3.5), 2 ** 3 ** 2)
```

`-4 1 -3 512`. `//` floors toward minus infinity; `%` takes the sign of the divisor; `int()` truncates toward zero; `**` is right-associative (`2 ** 9`). → *More:* Ch 2 §2.8

Q241. ⋆ What does this print?

```
d = {1: "a", True: "b", 1.0: "c"}
print(d)
```

`{1: 'c'}`. `1 == True == 1.0` and they hash equally, so they are one key. The first key object (`1`) is kept and the value is overwritten twice. → *More:* Ch 6 §6.2

Q242. What does this print?

```
x = 10
def f():
    print(x)
    x = 5
f()
```

`UnboundLocalError: cannot access local variable 'x' where it is not associated with a value`. The assignment makes `x` local throughout `f`, so the `print` reads an unassigned local. → *More:* Ch 7 §7.3

Q243. What does this print?

```
g = (n for n in range(3))
print(list(g), list(g))
```

`[0, 1, 2] []`. A generator is an iterator and can be consumed only once. → *More:* Ch 11 §11.1

Q244. What does this print?

```
a = [1, 2, 3]
b = a
b += [4]
b = b + [5]
print(a, b)
```

`[1, 2, 3, 4] [1, 2, 3, 4, 5]`. `+=` mutates the shared list in place; `b + [5]` creates a new list and rebinds only `b`. → *More:* Ch 5 §5.4

Q245. ⋆ What does this print?

```
nums = [1, 2, 3, 4]
for n in nums:
    nums.remove(n)
print(nums)
```

`[2, 4]`. Removing an item shifts the rest left while the loop's internal index keeps advancing, so every other item is skipped. Iterate over a copy (`for n in nums[:]`) or build a new list with a comprehension. → *More:* Ch 20 gotcha 5

Q246. ⋆ What does this print?

```
class A:
    items = []
p, q = A(), A()
p.items.append(1)
print(q.items)

class C:
    n = 0
c = C()
c.n += 1
print(C.n, c.n)
```

`[1]`, then `0 1`. `items` is one class-level list shared by all instances. `c.n += 1` reads the class value 0, then *assigns* `c.n = 1` on the instance, shadowing the class attribute without changing it. → *More:* Ch 20 gotcha 8

Q247. What does this print?

```
class A:
    def hi(self): return "A"
class B(A):
    def hi(self): return "B" + super().hi()
class C(A):
    def hi(self): return "C" + super().hi()
class D(B, C):
    pass
print(D().hi(), [k.__name__ for k in D.__mro__])
```

`BCA ['D', 'B', 'C', 'A', 'object']`. `super()` in `B` follows the MRO of the instance's class (`D`), so it calls `C.hi`, not `A.hi`. → *More:* Ch 10 §10.5

Q248. What does this print?

```
def f():
    try:
        return "try"
    finally:
        print("finally runs")
print(f())
```

`finally runs`, then `try`. `finally` executes before the function actually returns, and here it does not override the value. (If `finally` itself had a `return`, its value would win, and 3.14 would emit a `SyntaxWarning`.) → *More:* Ch 9 §9.2

Q249. What does this print?

```
print(["a" "b", "c"])
print(type((1)), type((1,)))
```

`['ab', 'c']`, then `<class 'int'> <class 'tuple'>`. Adjacent string literals are concatenated at compile time (a missing comma becomes a silent bug). The comma, not the parentheses, makes a tuple. → *More:* Ch 5 §5.9

Q250. What does this print?

```
for i in range(3):
    pass
print(i)
print([x for x in range(3)], "x" in dir())
```

`2`, then `[0, 1, 2] False` (when run as a fresh script). A `for` loop variable survives after the loop in the enclosing scope, but a comprehension's variable is local to the comprehension and does not leak (in Python 3). → *More:* Ch 3 §3.6

Q251. What does this print?

```
print(1 < 2 < 3, 1 == 1.0 == True, (1 < 2) < 3)
print(all([]), any([]), bool("False"), bool([0]))
```

`True True True`, then `True False True True`. Chained comparisons combine with `and`; `True < 3` compares `1 < 3`. `all` of nothing is vacuously true; any non-empty string or list is truthy. → *More:* Ch 2 §2.6, §2.8

Q252. What does this print?

```
print(None or 0 or "x", 0 and 5, 3 and 5, [] or {})
```

`x 0 5 {}`. `or` returns the first truthy operand or the last one; `and` returns the first falsy operand or the last one. → *More:* Ch 2 §2.8

Q253. What does this print?

```
nums = [3, 1, 2]
result = nums.sort()
print(result, nums, sorted("bca"))
```

`None [1, 2, 3] ['a', 'b', 'c']`. In-place methods return `None`; `sorted` returns a new list and on a string gives a list of characters. → *More:* Ch 20 gotcha 13

Q254. What does this print?

```
s = "hello"
print(s[::-1], s[1:-1], s[-3:], s[10:], s[::2])
```

`olleh ell llo hlo` (the empty `s[10:]` produces two spaces in the output). Out-of-range slices return empty sequences instead of raising. → *More:* Ch 4 §4.4

Q255. What does this print?

```
import copy
m = [[1], [2]]
shallow = copy.copy(m)
shallow[0].append(9)
shallow.append([3])
print(m)
```

`[[1, 9], [2]]`. The shallow copy has its own outer list (so appending `[3]` does not affect `m`) but shares the inner lists. → *More:* Ch 5 §5.8

Q256. What does this print?

```
d = {"a": 1}
for k in d:
    d["b"] = 2
```

`RuntimeError: dictionary changed size during iteration`. Iterate over `list(d)` when you need to add or delete keys inside the loop. → *More:* Ch 6 §6.4

Q257. What does this print?

```
print([] == [], [] is [], "a" * 0, [0] * 3, [1, 2] * 2)
```

`True False [0, 0, 0] [1, 2, 1, 2]`. Two separate empty lists are equal but not identical; multiplying a sequence by 0 gives an empty one. → *More:* Ch 2 §2.2

Q258. What does this print?

```
def outer():
    n = 0
    def inc():
        nonlocal n
        n += 1
        return n
    return inc
c1, c2 = outer(), outer()
c1(); c1()
print(c1(), c2())
```

`3 1`. Each call to `outer` creates a new closure with its own `n`. → *More:* Ch 7 §7.4

Q259. What does this print?

```
print(f"{3.14159:.2f}|{42:05d}|{'hi':>5}|{1234567:,}|{0.25:.0%}")
```

`3.14|00042| hi|1,234,567|25%`. Format specs: precision, zero-padding with width, right alignment, thousands separator, percentage. → *More:* Ch 4 §4.6

Q260. What does this print?

```
x = [1, 2, 3]
y = x[:]
z = x
print(x == y, x is y, x is z)
print(list(zip([1, 2, 3], "ab")))
```

`True False True`, then `[(1, 'a'), (2, 'b')]`. A slice is a new list; `zip` stops at the shortest input (silently dropping `3`; use `strict=True` to catch that). → *More:* Ch 11 §11.7

---

14. Common coding tasks in Python rounds

These are the short tasks Python rounds use to see whether you write idiomatic code. Algorithm-heavy problems (two pointers, graphs, DP and so on) are covered pattern by pattern in Chapter 19; this section gives the Python-specific ones. Say the time complexity after each answer.

Q261. ⋆ Check whether a string is a palindrome (ignoring case and non-alphanumeric characters).

```
def is_palindrome(s: str) -> bool:
    cleaned = [c.lower() for c in s if c.isalnum()]
    return cleaned == cleaned[::-1]
```

O(n) time and space. Mention the O(1)-space two-pointer version (indices from both ends, skipping non-alphanumerics) as a follow-up. → *More:* Ch 19 §19.2 — Two pointers

Q262. ⋆ Check whether two strings are anagrams; group a list of words into anagrams.

```
from collections import Counter, defaultdict
def is_anagram(a: str, b: str) -> bool:
    return Counter(a) == Counter(b)

def group_anagrams(words: list[str]) -> list[list[str]]:
    groups = defaultdict(list)
    for w in words:
        groups["".join(sorted(w))].append(w)
    return list(groups.values())
```

`is_anagram` is O(n); grouping is O(n · k log k) for n words of length k (sorted letters as the key). → *More:* Ch 19 §19.3 — Hashing

Q263. ⋆ Count the frequency of each word in a text and return the top *k*.

```python
import re
from collections import Counter
def top_words(text: str, k: int) -> list[tuple[str, int]]:
    words = re.findall(r"[a-z']+", text.lower())
    return Counter(words).most_common(k)
```

O(n) to count, and `most_common(k)` uses a heap, O(m log k) for m distinct words. → *More:* Ch 6 §6.9

Q264. Find the first non-repeating character in a string.

```python
from collections import Counter
def first_unique(s: str) -> str | None:
    counts = Counter(s)
    return next((c for c in s if counts[c] == 1), None)
```

Two passes, O(n). `next(generator, default)` is the idiomatic "first match or default".

Q265. ⋆ Remove duplicates from a list while preserving order.

```python
def dedupe(items):
    return list(dict.fromkeys(items))      # dicts keep insertion order
```

O(n) for hashable items. `list(set(items))` loses the order. For unhashable items, keep a `seen` list or convert each item to a hashable key. → *More:* Ch 6 §6.1

Q266. Find the duplicates in a list.

```python
from collections import Counter
def duplicates(items):
    return sorted(x for x, n in Counter(items).items() if n > 1)
```

O(n log n) because of the sort (O(n) without it).

Q267. ⋆ Two Sum: return the indices of two numbers that add up to a target.

```python
def two_sum(nums: list[int], target: int) -> tuple[int, int] | None:
    seen: dict[int, int] = {}
    for i, n in enumerate(nums):
        if target - n in seen:
            return seen[target - n], i
        seen[n] = i
    return None
```

One pass, O(n) time and space, using a dict from value to index. → *More:* Ch 19 §19.3

Q268. ⋆ Flatten an arbitrarily nested list.

```python
def flatten(items):
    for x in items:
        if isinstance(x, list):
            yield from flatten(x)
        else:
            yield x
print(list(flatten([1, [2, [3, 4]], 5])))   # [1, 2, 3, 4, 5]
```

A recursive generator with `yield from`. For one level only: `[x for sub in items for x in sub]` or `itertools.chain.from_iterable`. Very deep nesting can hit the recursion limit; an explicit stack avoids it. → *More:* Ch 11 §11.3

Q269. Flatten a nested dictionary into dotted keys.

```python
def flatten_dict(d: dict, parent: str = "", sep: str = ".") -> dict:
    out = {}
    for k, v in d.items():
        key = f"{parent}{sep}{k}" if parent else k
        if isinstance(v, dict):
            out.update(flatten_dict(v, key, sep))
        else:
            out[key] = v
    return out
# {"a": {"b": 1, "c": {"d": 2}}} -> {"a.b": 1, "a.c.d": 2}
```

Q270. Deep-merge two nested dictionaries.

```python
def deep_merge(a: dict, b: dict) -> dict:
    out = dict(a)
    for k, v in b.items():
        if isinstance(v, dict) and isinstance(out.get(k), dict):
            out[k] = deep_merge(out[k], v)
        else:
            out[k] = v
    return out
```

Values from `b` win, nested dicts are merged recursively, and the inputs are not modified (useful for layered configuration).

Q271. Sort a dictionary by value; get the key with the maximum value.

```python
scores = {"ann": 82, "bob": 95, "cy": 82}
by_value = dict(sorted(scores.items(), key=lambda kv: kv[1], reverse=True))
best = max(scores, key=scores.get)          # 'bob'
```

O(n log n) to sort, O(n) for the max. `operator.itemgetter(1)` can replace the lambda. → *More:* Ch 5 §5.7

Q272. ⋆ Write a Fibonacci generator and a memoised Fibonacci function.

```python
from functools import cache
def fib_gen():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

@cache
def fib(n: int) -> int:
    return n if n < 2 else fib(n - 1) + fib(n - 2)
```

`itertools.islice(fib_gen(), 10)` takes the first ten. Plain recursion without the cache is $O(2^n)$; with it, O(n) (but recursion depth limits large n, so the iterative version is better there). → *More:* Ch 19 §19.10 — Dynamic programming

Q273. ⋆ Implement an LRU cache class.

```python
from collections import OrderedDict
class LRUCache:
    def __init__(self, capacity: int):
        self.capacity, self.data = capacity, OrderedDict()
    def get(self, key):
        if key not in self.data:
            return -1
        self.data.move_to_end(key)          # mark as most recently used
        return self.data[key]
    def put(self, key, value):
        self.data[key] = value
        self.data.move_to_end(key)
        if len(self.data) > self.capacity:
            self.data.popitem(last=False)   # evict least recently used
```

Both operations are O(1). Interviewers may ask for the version with a dict plus a hand-written doubly linked list; the logic is the same. → *More:* Ch 6 §6.9 — the classic LRU cache

Q274. Write a retry decorator, or a memoise decorator, from scratch.

```python
import functools
def memoize(func):
    cache = {}
    @functools.wraps(func)
    def wrapper(*args):
        if args not in cache:
            cache[args] = func(*args)
        return cache[args]
    return wrapper
```

The cache lives in the closure; arguments must be hashable. For a retry decorator with arguments, see Ch 20 Q21; mention exponential back-off (`time.sleep(base * 2 ** attempt)`) and retrying only specific exceptions. → *More:* Ch 20 Q21 · Ch 12 §12.1

Q275. Split a list into chunks of size *n*.

```python
def chunks(items, n):
    return [items[i:i + n] for i in range(0, len(items), n)]
# Python 3.12+: list(itertools.batched(items, n)) gives tuples, and works on any iterable
```

→ *More:* Ch 11 §11.5

Q276. ⋆ Validate balanced brackets.

```python
def balanced(s: str) -> bool:
    pairs = {")": "(", "]": "[", "}": "{"}
    stack = []
    for ch in s:
        if ch in "([{":
            stack.append(ch)
        elif ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
    return not stack
```

A list used as a stack; O(n). → *More:* Ch 19 §19.4 — Stacks and queues

Q277. Transpose a matrix and rotate it 90 degrees clockwise.

```python
m = [[1, 2, 3], [4, 5, 6]]
transposed = [list(row) for row in zip(*m)]        # [[1, 4], [2, 5], [3, 6]]
rotated = [list(row) for row in zip(*m[::-1])]     # [[4, 1], [5, 2], [6, 3]]
```

`zip(*m)` unpacks the rows as arguments, so it groups the i-th elements together. → *More:* Ch 11 §11.7

Q278. Find the missing number in a list containing 0..n with one number missing.

```python
def missing(nums: list[int]) -> int:
    n = len(nums)
    return n * (n + 1) // 2 - sum(nums)
```

O(n) time, O(1) extra space. An XOR of all indices and values also works and cannot overflow (not a concern for Python ints, but worth saying for other languages). → *More:* Ch 19 §19.12 — Bit tricks and math

Q279. Merge two sorted lists.

```python
def merge(a: list[int], b: list[int]) -> list[int]:
    i = j = 0
    out = []
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            out.append(a[i]); i += 1
        else:
            out.append(b[j]); j += 1
    return out + a[i:] + b[j:]
```

O(n + m). Library shortcut: `list(heapq.merge(a, b))`, which also merges many sorted iterables lazily. → *More:* Ch 19 §19.17

Q280. Read a log file and report the 3 most frequent IP addresses.

```python
from collections import Counter
def top_ips(path: str, k: int = 3) -> list[tuple[str, int]]:
    with open(path, encoding="utf-8") as f:
        ips = (line.split(maxsplit=1)[0] for line in f if line.strip())
        return Counter(ips).most_common(k)
```

Streams the file with a generator, so memory stays constant for huge logs (only the counter grows). Mention `re` for stricter parsing and handling malformed lines. → *More:* Ch 9 §9.9

Q281. Implement your own `range`-like generator and your own version of `zip`.

```python
def my_range(start, stop=None, step=1):
    if stop is None:
        start, stop = 0, start
    while (step > 0 and start < stop) or (step < 0 and start > stop):
        yield start
        start += step

def my_zip(*iterables):
    iterators = [iter(it) for it in iterables]
    while iterators:
        row = []
        for it in iterators:
            try:
                row.append(next(it))
            except StopIteration:
                return            # stop at the shortest input
        yield tuple(row)
```

Talking point: writing `yield tuple(next(it) for it in iterators)` instead looks shorter but is wrong. The `StopIteration` escapes from inside a generator expression and is turned into `RuntimeError` (PEP 479, Q129). → *More:* Ch 11 §11.1–11.2

Q282. Write a thread-safe counter, then sum a list of URLs' sizes concurrently.

```python
import threading
from concurrent.futures import ThreadPoolExecutor
class SafeCounter:
    def __init__(self):
        self._value, self._lock = 0, threading.Lock()
    def increment(self):
        with self._lock:
            self._value += 1

def total_size(urls, fetch):              # fetch(url) -> bytes
    with ThreadPoolExecutor(max_workers=8) as ex:
        return sum(len(body) for body in ex.map(fetch, urls))
```

Threads suit network I/O; the lock makes the read-modify-write atomic. → *More:* Ch 16 §16.4, §16.8

Q283. Fetch many URLs concurrently with asyncio, at most 10 at a time.

```python
import asyncio
async def fetch_all(urls, fetch):         # fetch is an async function
    sem = asyncio.Semaphore(10)
    async def one(url):
        async with sem:
            return await fetch(url)
    async with asyncio.TaskGroup() as tg:
        tasks = [tg.create_task(one(u)) for u in urls]
    return [t.result() for t in tasks]
```

`TaskGroup` cancels the rest if one fails; the semaphore caps concurrency. Results come back in input order. → *More:* Ch 16 §16.9

Q284. Implement a stack and a queue class.

```python
from collections import deque
class Stack:
    def __init__(self): self._items = []
    def push(self, x): self._items.append(x)
    def pop(self): return self._items.pop()        # IndexError if empty
    def peek(self): return self._items[-1]
    def __len__(self): return len(self._items)

class Queue:
    def __init__(self): self._items = deque()
    def enqueue(self, x): self._items.append(x)
    def dequeue(self): return self._items.popleft()
```

All operations O(1). Using `list.pop(0)` for the queue would be O(n). → *More:* Ch 19 §19.4

Q285. Reverse the words in a sentence; capitalise each word.

```python
s = "  the quick   brown fox "
print(" ".join(reversed(s.split())))     # 'fox brown quick the'
print(" ".join(w.capitalize() for w in s.split()))
```

`split()` with no argument handles repeated spaces. `str.title()` also capitalises but mishandles apostrophes ("don'T"). → *More:* Ch 4 §4.7–4.8

Q286. Rotate a list by *k* positions.

```python
def rotate_right(xs: list, k: int) -> list:
    if not xs:
        return xs
    k %= len(xs)
    return xs[-k:] + xs[:-k]
```

O(n). In place: `deque.rotate(k)`, or the three-reversal trick for an array. → *More:* Ch 19 §19.2

Q287. Implement binary search.

```python
def binary_search(a: list[int], target: int) -> int:
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == target:
            return mid
        if a[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return -1
```

O(log n) on a sorted list. In practice: `i = bisect.bisect_left(a, target)` then check `i < len(a) and a[i] == target`. → *More:* Ch 19 §19.6 — Binary search and bisect

---

15. NumPy and pandas basics

Asked in data-analyst, data-engineer, data-scientist and ML interviews, and often as a warm-up in general Python rounds at data-heavy companies.

Q288. Why is a NumPy array faster than a Python list? A NumPy `ndarray` stores numbers of one fixed type (its **dtype**, such as `int64` or `float64`) in one contiguous block of memory, without per-element Python objects. Operations on whole arrays run as compiled C loops (**vectorisation**), often 10–100× faster than Python loops, and use SIMD CPU instructions and less memory. Lists hold pointers to separate objects of any type, so every operation goes through the interpreter. → *More:* Ch 18 §18.1 — numpy arrays

Q289. What is broadcasting? NumPy's rule for combining arrays of different shapes without copying data. Shapes are compared from the right; two dimensions are compatible if they are equal or one of them is 1, and size-1 (or missing) dimensions are stretched virtually. So a `(3, 4)` array minus a `(4,)` array subtracts the row vector from every row, and `(3, 1)` with `(1, 4)` gives `(3, 4)`. Incompatible shapes raise `ValueError: operands could not be broadcast together`. → *More:* Ch 18 §18.2 — Broadcasting

Q290. What is the difference between a view and a copy in NumPy? Basic slicing (`a[1:3]`, `a[:, 0]`) and `reshape` (when possible) return **views**: new array objects that share the original data, so writing to them changes the original. Fancy indexing with lists or arrays (`a[[0, 2]]`) and boolean masks (`a[a > 0]`) return **copies**. Use `.copy()` when you need independence; `np.shares_memory(a, b)` checks. → *More:* Ch 18 §18.1

Q291. What does `axis` mean in NumPy and pandas? The axis is the dimension that an operation runs *along*, and which disappears from the result. For a 2-D array, `a.sum(axis=0)` sums down the rows and gives one value per column; `axis=1` sums across columns and gives one value per row. In pandas, `axis=0` (the default) means "per column" for aggregations, and `axis=1` means "per row" (and refers to columns in `drop`). → *More:* Ch 18 §18.1

Q292. How do you create arrays and random numbers the modern way? `np.array(list)`, `np.zeros((3, 4))`, `np.ones`, `np.arange(0, 10, 2)`, `np.linspace(0, 1, 5)`, `np.eye(3)`. For random numbers use a Generator: `rng = np.random.default_rng(seed=42)`, then `rng.random(5)`, `rng.integers(0, 10, size=3)`, `rng.normal(size=(2, 2))`. The old global `np.random.seed` / `np.random.rand` API is legacy. → *More:* Ch 18 §18.3 — Linear algebra and random numbers

Q293. What is the difference between a pandas Series and a DataFrame? How does pandas relate to NumPy? A `Series` is a one-dimensional labelled array (values plus an index). A `DataFrame` is a two-dimensional table: a set of Series (columns) sharing one row index, where each column can have its own dtype. pandas builds on NumPy (and optionally Apache Arrow) for storage and adds labels, automatic alignment on labels, missing-data handling, grouping, joins and time series. → *More:* Ch 18 §18.4 — Series and DataFrame

Q294. ⋆ What is the difference between `loc` and `iloc`? `loc` selects by **label** (index values and column names) and its slices *include* the end label: `df.loc["a":"c", ["price", "qty"]]`. `iloc` selects by integer **position** and its slices exclude the end like normal Python: `df.iloc[0:3, 0:2]`. Both accept boolean masks (`df.loc[df["qty"] > 5, "price"]`). `at` and `iat` are faster versions for a single cell. → *More:* Ch 18 §18.5 — Selecting data: loc, iloc and filters

Q295. How do you filter rows with multiple conditions? Combine boolean masks with `&` (and), `|` (or) and `~` (not), with **parentheses around each condition**, because these operators bind tighter than comparisons: `df[(df["age"] > 30) & (df["city"] == "Pune")]`. Python's `and` / `or` raise "The truth value of a Series is ambiguous". Alternatives: `df.query("age > 30 and city == 'Pune'")`, `df["city"].isin([...])`, `df["age"].between(20, 30)`. → *More:* Ch 18 §18.5

Q296. ⋆ How do you handle missing values in pandas? Detect with `df.isna()` and `df.isna().sum()` (count per column). Remove with `df.dropna()` (options `subset=`, `thresh=`). Fill with `df.fillna(value)` (a constant, a column mean or median, or a dict per column), or `ffill()` / `bfill()` for time series, or `interpolate()`. Choose based on why data is missing; sometimes add an indicator column. `NaN` is a float, so an integer column with missing values becomes float unless you use the nullable `Int64` dtype. → *More:* Ch 18 §18.6 — Cleaning: missing data and new columns

Q297. ⋆ Explain `groupby`. What is the difference between `agg`, `transform` and `apply` after a groupby? `groupby` follows split–apply–combine: split rows into groups by key, apply a function to each group,

combine the results. `agg` reduces each group to one row (`df.groupby("city")["sales"].agg(["sum", "mean"])`, or named aggregations). `transform` returns a result the same length as the original, aligned to each row, which is ideal for adding group statistics as a column (`df["share"] = df["sales"] / df.groupby("city")["sales"].transform("sum")`). `apply` runs arbitrary Python per group: flexible but slow. → *More:* Ch 18 §18.7 — groupby

Q298. ⋆ What is the difference between `merge`, `join` and `concat`? `pd.merge(left, right, on="key", how="inner")` is a SQL-style join on columns (`how` = `inner`, `left`, `right`, `outer`, `cross`). `df.join(other)` is a convenience that joins on the index. `pd.concat([a, b])` stacks frames vertically (`axis=0`) or side by side (`axis=1`), aligning on labels. Watch for duplicate keys multiplying rows; `validate="one_to_one"` or `"many_to_one"` checks your assumption, and `indicator=True` shows where each row came from. → *More:* Ch 18 §18.8 — Combining tables

Q299. What is the difference between `pivot`, `pivot_table` and `melt`? `pivot` reshapes long data to wide (index, columns, values) and fails if any index/column pair repeats. `pivot_table` does the same but aggregates duplicates (`aggfunc="sum"`), like a spreadsheet pivot, with optional totals (`margins=True`). `melt` goes the other way: wide to long, turning columns into rows. `stack`/`unstack` do similar reshaping using the index levels. → *More:* Ch 18 §18.9 — Reshaping

Q300. ⋆ Why is `apply` (or `iterrows`) slow, and what should you use instead? `apply(func, axis=1)` and `iterrows` call a Python function once per row, so they run at interpreter speed and build Python objects for each row. Use vectorised operations instead: column arithmetic (`df["a"] * df["b"]`), `np.where` or `np.select` for conditional logic, the `.str` and `.dt` accessors, `map` with a dict for lookups, and built-in group aggregations. If you must loop, `itertuples` is much faster than `iterrows`. → *More:* Ch 18 §18.11 — apply vs vectorised, and performance tips

Q301. What was `SettingWithCopyWarning`, and what changed in pandas 3.0 (Copy-on-Write)? In older pandas, chained indexing such as `df[df["x"] > 0]["y"] = 1` might modify a temporary copy instead of `df`, so pandas warned. In pandas 3.0, Copy-on-Write is always on: every indexing result *behaves* as a copy (memory is shared until one side is modified), so chained assignment never updates the original. `SettingWithCopyWarning` is gone; instead pandas emits a `ChainedAssignmentError` warning when it sees a chained assignment, which does nothing. Always assign in one step: `df.loc[df["x"] > 0, "y"] = 1`. pandas 3.0 also made `str` the default dtype for text columns. → *More:* Ch 18 §18.5 — Setting values and Copy-on-Write

Q302. How do you work with a CSV file that is too big for memory, or speed up pandas on large data? Read only what you need (`usecols=`, `dtype=` with smaller types, `category` for repeated strings, `parse_dates=`); process in pieces with `chunksize=` and aggregate per chunk; store data as Parquet (columnar, compressed, keeps dtypes) instead of CSV; and vectorise everything. Beyond one machine's memory, consider DuckDB, Polars (multi-threaded, lazy queries) or Dask/Spark. → *More:* Ch 18 §18.11

Q303. How do you find and remove duplicates, and count values, in pandas? `df.duplicated(subset=["email"])` returns a boolean mask, `df.drop_duplicates(subset=["email"], keep="last")` removes them. `df["city"].value_counts()` counts occurrences (`normalize=True` for proportions), and `df["city"].nunique()` counts distinct values. → *More:* Ch 18 §18.6

Q304. How do you work with dates in pandas? Parse with `pd.to_datetime(df["date"], format="%Y-%m-%d")`, then use the `.dt` accessor (`.dt.year`, `.dt.day_name()`, `.dt.to_period("M")`). With a `DatetimeIndex`, `resample("ME").sum()` aggregates by calendar period (pandas 3 uses `"ME"` for month end and `"h"` for hour), `rolling(7).mean()` gives moving averages, and `shift(1)` compares with the previous period. Use `tz_localize`/`tz_convert` for time zones. → *More:* Ch 18 §18.10 — Dates and times

Q305. How do you create a new column based on a condition? For two outcomes: `df["size"] = np.where(df["qty"] > 10, "big", "small")`. For several: `np.select([cond1, cond2], ["a", "b"], default="c")`. For binning numbers: `pd.cut(df["age"], bins=[0, 18, 65, 120], labels=["child", "adult", "senior"])`. For a mapping: `df["code"].map(lookup_dict)`. All are vectorised; avoid `apply` with an `if` per row. → *More:* Ch 18 §18.6

Sources of the question lists

Question wording and frequency were collected from these public lists and posts (titles only; all answers above are written for this book):

- **GitHub collections:** Devinterview-io, *python-interview-questions* ("100 Core Python Interview Questions"); baliyanvinay, *Python-Interview-Preparation* ("questions really asked in a Python interview", with coding questions); learning-zone, *python-basics* (Python interview questions by topic); softwaredeveloperhub01, *Python-Interview-Questions-and-Answers*; narayan09, *Python-Interview-Questions* (OOP); digitechit07, *Python-Interview-Questions-52*; pythonsheets.com, *Interview* notes.
- **Interview-experience posts and hiring guides:** LeetCode Discuss, *My 14 Coding Interview Experiences | 13 Rejections* (includes Python questions such as how list memory allocation works); Glassdoor question pages for "Python Developer", "Junior Python Developer", "Python Software Engineer" and "Senior Python Developer" (recurring: list vs tuple, decorators with parameters, dict comprehensions, GIL, memory management, class and abstract methods, generators, context managers, "what is the output" questions); InterviewKickstart, *Advanced Python Interview Questions*; GoLinuxCloud, *Python Developer Interview Questions*; Boundev, *Python interview questions hiring guide*; NovelVista and Naukri Python interview question lists.
- **Shared prep lists:** Scribd-hosted lists ("Top 25 Python Interview Questions", "25 Must-Know Python Interview Questions", "150+ Python Interview Questions") and Medium, *Top 100+ Python Coding Questions for Interview*, used to confirm which questions repeat across sources. (No public Notion page turned up in searches; these shared lists cover the same ground.)
- **LeetCode study lists (for Section 14):** LeetCode "Top Interview 150" and Blind 75 problems that Python rounds commonly use as warm-ups (Two Sum, Valid Anagram, Group Anagrams, Valid Parentheses, Missing Number, LRU Cache, Binary Search, Merge Two Sorted Lists, Rotate Array).
- **Data-stack questions (Section 15):** DataCamp, *Top Python pandas Interview Questions*; AskPython, *50 Essential Pandas Interview Questions*; GoLinuxCloud, *Pandas Interview Questions*; Educative, *pandas loc vs iloc*.
- **Version facts** were checked against the official *What's New in Python 3.12 / 3.13 / 3.14* pages, the Python 3.14.5 release notes (garbage collector revert), the PEPs named in the answers, and the pandas 3.0 release notes; outputs in Section 13 and the solutions in Section 14 were run on CPython 3.14.8, and the Section 15 snippets on numpy 2.4 and pandas 3.0.